

Einführung

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)

Material: Thomas Nagler

LMU München

Organisatorisches

- **Personen:**

- Dr. Fabian Scheipl (VL)
- Jana Gauß (Ü)

- **Zeiten & Räume:**

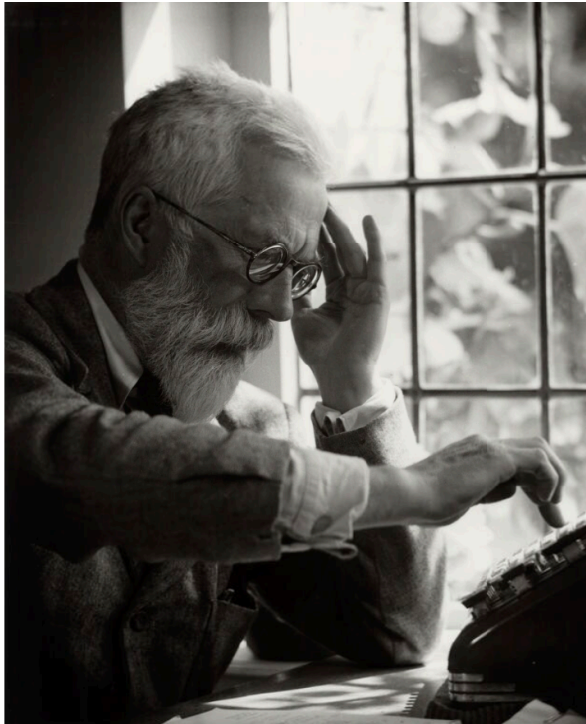
VL	Do 10:15-12:00	Hgb. – A 214
Übung 1	Di 14:15-15:45	Prof.-Huber-Pl. 2 – W 201
Übung 2	Mi 12:15-13:45	Hgb. - M 114

- **Termine WS 26/27:** wöchentlich Do, **15.10.2026 – 4.2.2027** (Weihnachtspause: letzte VL 17.12.2026, weiter am 7.1.2027)

Motivation

Inhalt: Statistik früher & heute

Statistik früher



Statistik heute



Motivation

- In der Praxis ist Statistik / ML immer *computational*.
- In diesem Kurs lernen wir *mathematische Konzepte*, die bei der praktischen Lösung von Problemen helfen.
- Diese Konzepte werden benötigt, um *Computeralgorithmen* zu entwickeln, verstehen und analysieren.
 - ⇒ fortgeschrittene (numerische) Lineare Algebra
 - ⇒ fortgeschrittene Analysis

Warum so viel Mathe? Ein Beispiel

Aufgabe: Trainiere ein lineares Modell $y_i \approx \beta_1 x_{i1} + \dots + \beta_p x_{ip}$ mit $p = 10.000$ Features auf $n = 100.000$ Beobachtungen.

$\implies$ **Kleinste-Quadrate-Problem:**

$$\min_{\beta \in \mathbb{R}^p} \left(\sum_{i=1}^n \left(y_i - \sum_{j=1}^p x_{ij} \beta_j \right)^2 \right) = \min_{\beta \in \mathbb{R}^p} \| \mathbf{X} \boldsymbol{\beta} - \mathbf{y} \|_2^2$$

Naive Lösung (falls $\text{rang}(\mathbf{X}) = p$): $\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$

Probleme in der Praxis:

1. **Kondition:** $\mathbf{X}^\top \mathbf{X}$ kann sehr schlecht “konditioniert” sein $\implies$ numerische Instabilität, große Fehler
2. **Komplexität:** Matrixinversion kostet $O(p^3)$ Operationen $\implies$ für $p = 10.000$: ca. 10^{12} FLOPs!
3. **Speicher:** $\mathbf{X}^\top \mathbf{X}$ ist $p \times p$ Matrix $\implies$ für $p = 10.000$: ca. 800 MB nur für diese Matrix

Bessere Lösungen: QR-Zerlegung, SVD, iterative Verfahren $\rightarrow$ dieser Kurs

Themen

- Kondition und Komplexität
 - Was sind gute Algorithmen?
- Numerische Lineare Algebra
 - Matrix- & Tensorrechnung
 - Berechne (KQ-)Lösungen für $\mathbf{Ax} \approx \mathbf{b}$ etc.
- Fortgeschrittene Differenzialrechnung / Analysis
 - Höherdimensionale, verallgemeinerte Ableitungen
 - Wie kann man komplizierte Fkt. einfach approximieren?
- Numerische Optimierung
 - Wie finde ich Minima/Maxima von komplizierten Funktionen?
 - z.B. Training neuronaler Netze: Millionen Parameter, nicht-konvexe Verlustfunktionen → smarte Algorithmen nötig (SGD & Co.)

Fünf Beispiele

Beispiel 1: Kondition

Betrachte das lineare Gleichungssystem $Ax = b$ mit

$$A = \begin{pmatrix} 1 & 1 \\ 1 & 1.0001 \end{pmatrix}, \quad b = \begin{pmatrix} 2 \\ 2.0001 \end{pmatrix}$$

Exakte Lösung: $x = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$

Gestörtes Problem (z.B. durch Rundungsfehler): $Ax = \tilde{b}$ mit

$$\tilde{b} = \begin{pmatrix} 2 \\ 2.0002 \end{pmatrix} \quad (\text{nur } 0.005\% \text{ Fehler!})$$

Lösung des gestörten Problems: $\tilde{x} = \begin{pmatrix} 0 \\ 2 \end{pmatrix}$

$\implies$ **100% Fehler** in x_1 durch 0.005% Fehler in den Daten!

$\implies$ Das Problem ist (furchtbar) **schlecht konditioniert**.

Beispiel 2: Komplexität

Matrixmultiplikation $C = AB$ für $n \times n$ Matrizen:

Algorithmus	Rechenops. für $n = 1000$	Komplexität
Naive Methode	$\approx 10^9$ Ops	$O(n^3)$
Strassen	$\approx 6 \times 10^8$ Ops	$O(n^{2.807})$
Optimiert	$\approx 2 \times 10^8$ Ops	$O(n^{2.373})$

Für $n = 10.000$:

- Naiv: $\approx 10^{12}$ Ops
- Optimiert: $\approx 2 \times 10^{10}$ Ops

$\implies$ **Faktor 50 schneller** durch besseren Algorithmus (...potentiell)!

Beispiel 3: Höherdimensionale Ableitungen

Beispiel A1: Gradient Descent für neuronale Netze

Verlustfunktion: $L : \mathbb{R}^{p \times K} \rightarrow \mathbb{R}$ (Matrix-Argument $\mathbf{W}$!)

Zum Beispiel: **MSE**

$$L(\mathbf{W}) = \frac{1}{2} \|\mathbf{XW} - \mathbf{Y}\|_F^2$$

mit $\mathbf{X} \in \mathbb{R}^{N \times p}$, $\mathbf{W} \in \mathbb{R}^{p \times K}$, $\mathbf{Y} \in \mathbb{R}^{N \times K}$

Frage: Wie berechnen wir $\frac{\partial L}{\partial \mathbf{W}}$?

Naive Methode: Berechne $\frac{\partial L}{\partial W_{ij}}$ für alle i, j einzeln

$\implies p \times K$ Ableitungen!

Für $p = 1000$, $K = 10$: **10.000 Ableitungen!**

Bessere Methode: Matrix-Kalkül

$$\frac{\partial L}{\partial \mathbf{W}} = \mathbf{X}^\top (\mathbf{XW} - \mathbf{Y})$$

Eine Zeile statt 10.000! Und: **effizient implementierbar**.

$\implies$ Ableitungen von Matrix/Tensor-Funktionen essentiell für Deep Learning, Optimierung, etc.

Beispiel 4: Funktionsapproximation: Taylor-Reihen

Ziel: Approximiere komplizierte Funktion $f(x)$ durch ein Polynom in der Nähe von x_0

Taylor-Approximation 2. Ordnung:

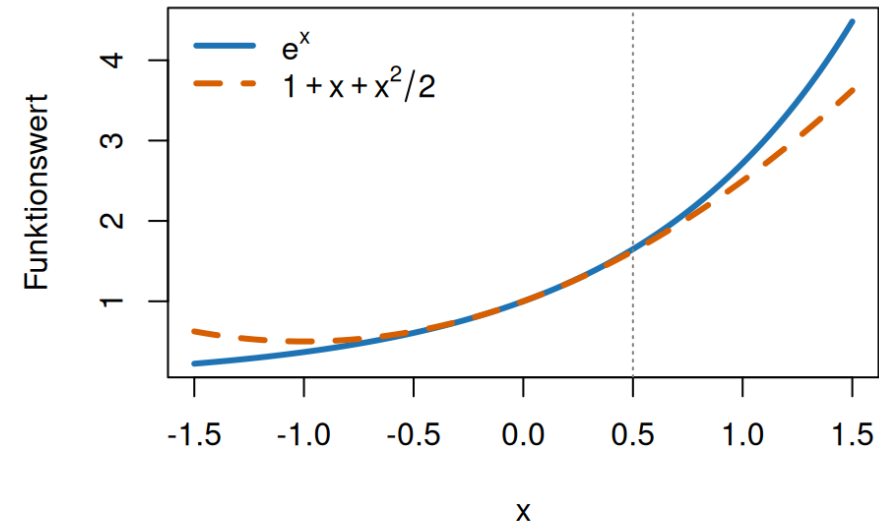
$$f(x) \approx f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2}(x - x_0)^2$$

Konkretes Beispiel: $f(x) = e^x$ um $x_0 = 0$

$$f(0) = f'(0) = f''(0) = 1 \implies e^x \approx 1 + x + \frac{x^2}{2}$$

Fehler für $x = 0.5$:

- Exakt: $e^{0.5} = 1.6487\dots$
- Approximation: $1 + 0.5 + 0.125 = 1.625$
- Relativer Fehler: $< 2\%$



Beispiel 5: Optimierung in Machine Learning

Typisches Problem: Training eines neuronalen Netzes

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^d} L(\boldsymbol{\theta}), \quad L(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N \ell(f(\mathbf{x}_i; \boldsymbol{\theta}), y_i)$$

Modell	Parameter d	Daten N
Logistische Regression (MNIST, 10 Klassen)	$\sim 8 \times 10^3$	6×10^4
ResNet-50	$\sim 2.5 \times 10^7$	$> 10^6$
GPT-3	$\sim 1.75 \times 10^{11}$	$\sim 3 \times 10^{11}$ Tokens

Herausforderungen:

1. **Nicht-konvex:** Viele lokale Minima / Sattelpunkte der Verlustfunktion
2. **Hochdimensional:** Naive Methoden scheitern
3. **Stochastisch:** Nur Zugriff auf Minibatches

$\implies$ Brauchen **smarte Algorithmen:** Stochastic Gradient Descent, Momentum, etc.

Vorkenntnisse & Selbsttest

Benötigte Vorkenntnisse

- Lineare Algebra (VL Lineare Algebra I):
 - v.a. Matrixrechnung, Eigenwerte und -vektoren
- Analysis (VL Analysis I):
 - v.a. Stetigkeit & Differenzierbarkeit, Ableitungen
- R-Programmierung (VL Einführung in R)

Empfehlung:

- Intuition für LA z.B. mit *3blue1brown* o.ä. aufbauen!
- Skript **aktiv** lesen & benutzen um Grundbegriffe zu wiederholen

(Minimaler) Selbsttest: Vorkenntnisse evtl. ausreichend?

Lineare Algebra - Sie sollten wissen:

1. Was ist der *Rang* einer Matrix? Wie berechne ich ihn?
2. Was bedeutet es, wenn eine Matrix $\mathbf{A}$ *invertierbar* ist?
3. Was sind *Eigenwerte/Eigenvektoren* einer Matrix? Wie finde ich sie?
4. Was ist die geometrische Bedeutung einer Matrix-Vektor-Multiplikation $\mathbf{A}\mathbf{x}$?
5. Wann bilden Vektoren $\mathbf{v}_1, \dots, \mathbf{v}_n$ eine *Basis* des $\mathbb{R}^n$? Was ist ihr *Span*?

(Minimaler) Selbsttest: Vorkenntnisse evtl. ausreichend?

Analysis - Sie sollten wissen:

1. Was bedeutet es, wenn eine Funktion $f : \mathbb{R} \rightarrow \mathbb{R}$ *stetig* ist?
2. Was ist die *Ableitung* $f'(x_0)$ geometrisch?
3. Was ist ein lokales vs. globales Minimum?

Algorithmen und Komplexität

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)

Material: Thomas Nagler

LMU München

Verwendete Vorkenntnisse

- **Grundlegende Analysis** (aus Analysis I)
 - Folgen und Grenzwerte
 - Grundbegriffe der Konvergenz
- **Grundlegende Lineare Algebra** (aus Lineare Algebra I)
 - Matrixoperationen (Addition, Multiplikation)
 - Verständnis des Rechenaufwands von Operationen

Numerische Probleme und Algorithmen

Numerische Probleme

- Ein numerisches Problem hat als Lösung eine oder mehrere Zahlen.
 - Wert eines Integrals
 - Lösung eines LGS
 - Basis-Koeffizienten in Funktionsapproximation
- Allgemein:

Gegeben Problem f mit Input x , berechne Lösung $f(x)$.

Live-Demo: Rechnen Computer richtig?

 Erst raten, dann ausführen!

Was geben diese Codezeilen aus?

```
1 1.0 - 1.0
```

```
[1] 0
```

```
1 1.0 - 0.9 - 0.1
```

```
[1] -2.78e-17
```

```
1 100 * 0.58 == 58
```

```
[1] FALSE
```

```
1 x <- seq(1, 2e16, length = 10^5)
2 sum(x) - sum(rev(x))
```

```
[1] -262144
```

Numerische Probleme: Schwierigkeiten

- Computer sind so gut wie nie exakt.
- Sie können
 - ... nur endliche und endlich viele Zahlen darstellen.
 - $\pi = 3.141592653589793238462643383279 \dots$ 😞
 - ... nur endlich viele Berechnungen durchführen.
 - $e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}$ 😞
- Deshalb ist es wichtig, *wie* wir Dinge berechnen!

Numerische Probleme: Schwierigkeiten

- Computer können nur endlich viele Zahlen darstellen.
- Das führt zu *Rundungsfehlern*, die sich *akkumulieren* können.
- Die *Reihenfolge* unserer Rechenoperationen spielt eine Rolle.
- *Fehler* einzelner Operationen sind oft vernachlässigbar, aber sie *akkumulieren*.
- **Empfehlung:** Besonders große/kleine Zahlen vermeiden.

Beispiel: Katastrophale Auslöschung

Problem: Berechne die Varianz von $\mathbf{x}$ mit der Formel

$$\text{Var}(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n x_i^2 - \left(\frac{1}{n} \sum_{i=1}^n x_i \right)^2$$

Daten: $\mathbf{x} = (4 + 10^9, 7 + 10^9, 13 + 10^9, 16 + 10^9)$

Berechnung:

```
1 x <- c(4, 7, 13, 16) + 1e9
2 mean((x - mean(x))^2) # Varianz (mit 1/n)
```

```
[1] 22.5
```

```
1 mean(x^2) - mean(x)^2 # !! auch Varianz (mit 1/n) ?!
```

```
[1] 0
```

- **Ursache:** Subtraktion zweier fast gleich großer Zahlen ($\approx 10^{18}$).

```
1 mean(x^2)
```

```
[1] 1e+18
```

```
1 mean(x)^2
```

```
[1] 1e+18
```

Beispiel: Assoziativität verletzt

Problem: Gleitkomma-Addition ist nicht assoziativ!

Konkretes Beispiel: Seien $x = 10^{30}$, $y = -10^{30}$, $z = 1$.

$$(x + y) + z = (10^{30} + (-10^{30})) + 1 = 0 + 1 = 1$$

$$x + (y + z) = 10^{30} + ((-10^{30}) + 1) \stackrel{!!}{\approx} 10^{30} + (-10^{30}) = 0$$

- Die zweite Berechnung ergibt 0, weil 1 zu klein ist gegenüber 10^{30} .

$\implies$ Reihenfolge der Summe beeinflusst Ergebnis!!

Algorithmen

Def.: Algorithmus

Ein **Algorithmus** ist ein Verfahren $\tilde{f} = \tilde{f}_s \circ \dots \circ \tilde{f}_1$, das für Inputs $\mathbf{x}$ eine mögliche Lösung $\tilde{f}(\mathbf{x})$ des Problems $f(\mathbf{x})$ berechnet.

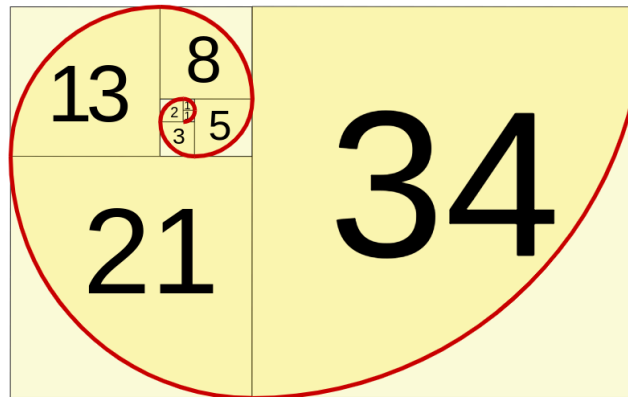
Arten von Algorithmen

- **exakt**: es gilt $f(\mathbf{x}) = \tilde{f}(\mathbf{x})$ (bis auf Rundungsfehler)
vs. **approximativ**: es gilt lediglich $f(\mathbf{x}) \approx \tilde{f}(\mathbf{x})$.
- **iterativ**: gleiche Vorschrift wird mehrfach ausgeführt
vs. **direkt**: Vorschrift wird nur einmal ausgeführt.
- **probabilistisch/randomisiert**: beruht auf Zufall
vs. **deterministisch**: keine Zufallselemente.

Beispiel: Fibonacci-Zahlen

- Problem: Berechne die ersten N Fibonacci-Zahlen

$$x_1 = 0, \quad x_2 = 1, \quad x_{n+1} = x_n + x_{n-1} \quad (n \geq 2).$$



- Algorithmus:

$$\tilde{f} : \mathbb{N} \rightarrow \mathbb{N}^n$$

$$\tilde{f}(n) = (\tilde{f}_n \circ \dots \circ \tilde{f}_3 \circ \tilde{f}_2 \circ \tilde{f}_1)(n)$$

$$\tilde{f}_1(n) = 0, \quad \tilde{f}_2(x) = (x, 1),$$

$$\tilde{f}_{i+1}(x_1, \dots, x_i) = (x_1, \dots, x_i, x_i + x_{i-1}) \quad (i \geq 2).$$

Fibonacci: Implementierung

- Implementierung des Algorithmus:

```
1 fibonacci <- function(n) {  
2   x <- numeric(n) # Vektor (0, ..., 0) der Länge n  
3   if (n > 1) {  
4     x[2] = 1  
5   }  
6   if (n > 2) {  
7     for (i in 2:(n-1)) {  
8       x[i+1] <- x[i] + x[i - 1]  
9     }  
10  }  
11  x  
12 }
```

- **Beispiel** ($n = 6$; schrittweise Rechnung s. Anhang):

```
1 fibonacci(6)
```

```
[1] 0 1 1 2 3 5
```

Quiz

Der Algorithmus ist ...

1. exakt ?
2. iterativ ?
3. probabilistisch ?

Algorithmenarten in ML / Statistik

- **Exakt/direkt:**

- LGS-Lösung via Gauß-Elimination (später...)
- Matrixmultiplikation

- **Approximativ:**

- Monte-Carlo-Integration (schätzt Integrale durch Stichproben)
- Neuronale Netze (optimieren bis zu einem Toleranzlevel)

- **Iterativ:**

- Gradientenabstieg in Machine Learning
- Newton-Verfahren zum Nullstellenfinden

- **Probabilistisch:**

- Approximative Singulärwertzerlegung, Matrix Sketching
- MCMC-Methoden in Bayesianischer Statistik

Oft kombiniert: ML-Algorithmen sind oft iterativ + approximativ + probabilistisch!

Analogie

Problem:

“Kochte Schweinebraten wie bei Oma.”

Algorithmus $\leftrightarrow$ Kochrezept

Computer $\leftrightarrow$ Koch (*der sich ans Rezept hält, aber Fehler macht*)

Gute Algorithmen

Wir wollen Algorithmen, die

- möglichst exakte Ergebnisse liefern,
- schnell sind,
- wenig Speicher brauchen.

Aufwand und Komplexität

Aufwand und Komplexität

- Der Computer führt einen Algorithmus durch elementare Operationen aus: Rechenoperationen $f_i \in \{+, -, \cdot, /\}$, aber auch Vergleiche, Zuweisungen und Speicherzugriffe — jede mit (etwa) konstantem Aufwand.
- Der **Zeitaufwand** des Algorithmus ist die Anzahl dieser Operationen.
 - höherer Aufwand $\approx$ längere Laufzeit
- Der **Speicheraufwand** ist (in etwa) die Anzahl der (zwischen-)gespeicherten Zahlen.
 - höherer Aufwand $\rightarrow$ mehr Speicherplatz

Beispiel: Matrix-Vektor-Multiplikation

Problem: Berechne $\mathbf{y} = \mathbf{A}\mathbf{x}$ für $\mathbf{A} \in \mathbb{R}^{3 \times 2}$, $\mathbf{x} \in \mathbb{R}^2$.

Konkret: $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}, \mathbf{x} = \begin{pmatrix} 7 \\ 8 \end{pmatrix}$

Berechnung:

$$y_1 = 1 \cdot 7 + 2 \cdot 8 = 7 + 16 = 23 \quad (2 \text{ Mult.}, 1 \text{ Add.})$$

$$y_2 = 3 \cdot 7 + 4 \cdot 8 = 21 + 32 = 53 \quad (2 \text{ Mult.}, 1 \text{ Add.})$$

$$y_3 = 5 \cdot 7 + 6 \cdot 8 = 35 + 48 = 83 \quad (2 \text{ Mult.}, 1 \text{ Add.})$$

- **Zeitaufwand:** $3 \times 2 = 6$ Multiplikationen + $3 \times 1 = 3$ Additionen = 9 Operationen
- *Allgemein:* Für $\mathbf{A} \in \mathbb{R}^{n \times d}$: nd Multiplikationen + $n(d - 1)$ Additionen $\approx 2nd$ Operationen
- **Speicheraufwand:** nd (für $\mathbf{A}$) + d (für $\mathbf{x}$) + n (für $\mathbf{y}$) = $nd + n + d$ Zahlen

Quiz: Zeit- und Speicheraufwand

Sei $A \in \mathbb{R}^{n \times d}$, $x, y \in \mathbb{R}^d$,

$$f(A, x, y) = A(x - y) = \sum_{i=1}^d (x_i - y_i) A_{.i}.$$

```
1 f <- rep(0, nrow(A))
2 for (i in 1:ncol(A)) {
3   f <- f + (x[i] - y[i]) * A[, i]
4 }
```

a) Wie viele elementare Rechenoperationen werden benötigt?

1. $2n + d$
2. $2dn + d$
3. dn^2
4. $2(n + d)$

b) Für wie viele Gleitkommazahlen brauchen wir Speicherplatz?

1. $3n + d$
2. $4nd$
3. $2(n + d)$
4. $n + nd + 2d$

Quiz: Lösung

a) Zeitaufwand: $2nd + d$ Operationen

Aufschlüsselung pro Iteration ($i = 1, \dots, d$):

1. **Subtraktion:** $(x_i - y_i) \rightarrow 1$ Operation

2. **Skalierung:** $(x_i - y_i) \cdot \mathbf{A}_{.i} \rightarrow n$ Multiplikationen (1 Element/Zeile f. $\mathbf{A}_{.i}$)

3. **Addition:** $\mathbf{f} \leftarrow \mathbf{f} + \dots \rightarrow n$ Additionen

Pro Iteration: $1 + n + n = 2n + 1$ Operationen

Gesamt: $d \times (2n + 1) = 2nd + d$ Operationen

b) **Speicheraufwand:** $n + nd + 2d$ Gleitkommazahlen

- n (für $\mathbf{f}$) + nd (für $\mathbf{A}$) + $2d$ (für $\mathbf{x}$ und $\mathbf{y}$)

Komplexität

- Die **exakte Anzahl** an Operationen und gespeicherten Zahlen zu kennen, ist **nicht wichtig**.
- Wir sind hauptsächlich daran interessiert **wie schnell der Aufwand** mit der Größe des Problems wächst (**“skaliert”**).
- Das nennen wir *(Laufzeit-/Speicher-)Komplexität des Algorithmus*.

Komplexität

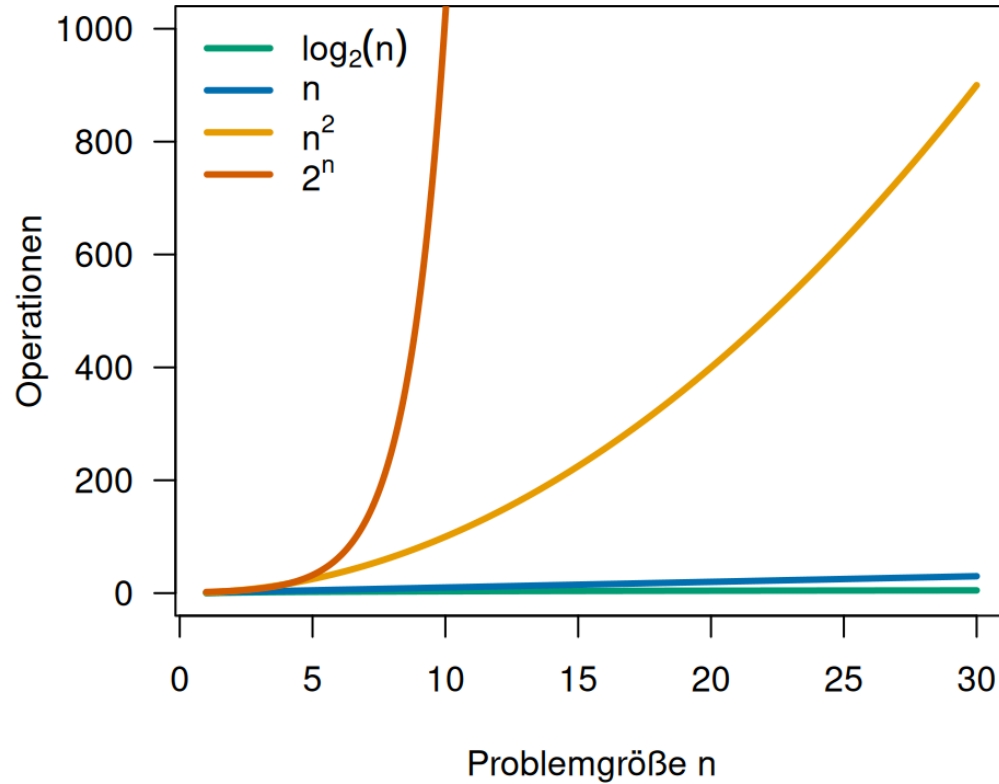
Beispiel:

- Ein Algorithmus benötigt $4n^3 + 16n^2 + 239$ Operationen.
- Für großes n ist $16n^2 + 239$ gegenüber $4n^3$ vernachlässigbar.

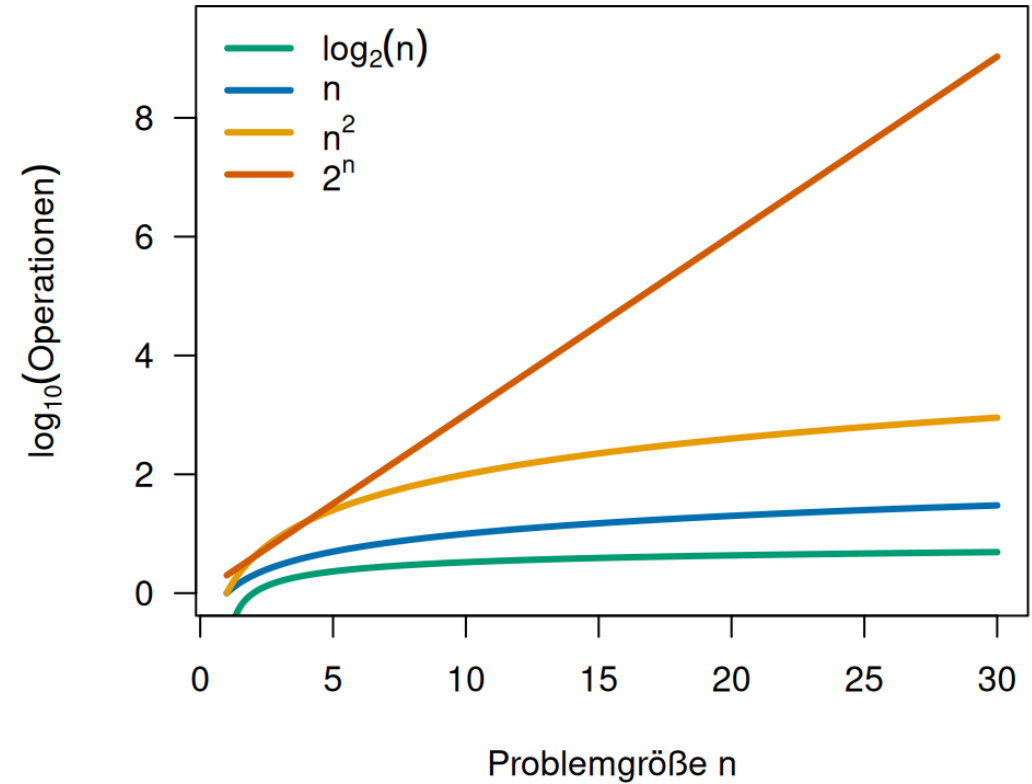
⇒ Der Algorithmus skaliert wie $4n^3$,
die Komplexität ist *kubischer* **Ordnung**: " $O(n^3)$ ".

Komplexitätsklassen

Lineare Skala



Logarithmische Skala



Komplexitätsklassen

Wie verschiedene Komplexitäten skalieren:

Interpretation:

- $O(1)$ (konstant):
Anzahl Operationen ist unabhängig von n
- $O(\log_b n)$ (logarithmisch):
eine Operation mehr falls sich n ver- b -facht.
- $O(n)$ (linear):
verdoppelt sich n , verdoppeln sich Operationen
- $O(n^2)$ (quadratisch):
verdoppelt sich n , vervierfachen sich Operationen
- $O(2^n)$ (exponentiell):
verdoppelt sich n , exponentieren sich Operationen

Vorsicht:

$O()$ -Notation ignoriert konstante Faktoren und Terme – z.B.

- $1000n + 10000$ Operationen sind immer noch $O(n)$
- aber (viel) langsamer als n^2 Operationen ($O(n^2)$) für $n < 1010!$

Landausymbole

Def.: Landausymbole

Seien a_n, b_n Folgen mit $b_n \neq 0$.

- $a_n = o(b_n)$ („ a_n ist **klein-o** von b_n “), wenn

$$\lim_{n \rightarrow \infty} \frac{a_n}{b_n} = 0.$$

- $a_n = O(b_n)$ („ a_n ist **groß-O** von b_n “), wenn

$$\limsup_{n \rightarrow \infty} \left| \frac{a_n}{b_n} \right| < \infty.$$

Interpretation:

- $a_n = o(b_n)$: a_n **wächst langsamer als** b_n , a_n wird gegenüber b_n vernachlässigbar klein.
- $a_n = O(b_n)$: a_n **wächst höchstens so schnell wie** b_n , b_n ist gegenüber a_n nicht vernachlässigbar.
- Zusatz:

$$a_n = \Theta(b_n)$$

falls $a_n = O(b_n)$ und $b_n = O(a_n)$, d.h. a_n **wächst genauso schnell wie** b_n .

Landausymbole: Rechenbeispiel

Beispiel: Zeige, dass $3n^2 + 5n = O(n^2)$

$$\begin{aligned}\lim_{n \rightarrow \infty} \frac{3n^2 + 5n}{n^2} &= \lim_{n \rightarrow \infty} \left(3 + \frac{5}{n} \right) = 3 < \infty \\ \implies 3n^2 + 5n &= O(n^2) \quad \checkmark\end{aligned}$$

Weitere Rechenbeispiele: siehe *Anhang*

Landausymbole

- Meist suchen wir eine möglichst einfache Folge b_n , sodass $a_n = O(b_n)$ aber nicht $a_n = o(b_n)$.

Lemma: Ein paar Rechenregeln

Seien $a_n, b_n > 0$ (z.B. Anzahl von Operationen).

1. Sei $f_n = O(a_n)$ und $g_n = O(b_n)$. Dann gilt

$$f_n + g_n = O(a_n + b_n) \quad \text{und} \quad f_n \cdot g_n = O(a_n \cdot b_n).$$

2. Sei $f_n = O(a_n)$ und $g_n = o(b_n)$. Dann gilt

$$f_n \cdot g_n = o(a_n \cdot b_n)$$

3. Sei $f_n = O(a_n + b_n)$ mit $a_n = O(b_n)$. Dann gilt

$$f_n = O(b_n).$$

Rechenregeln: Anwendungsbeispiele

Beispiel Addition und Multiplikation:

Gegeben: $f_n = 3n^2 = O(n^2)$ und $g_n = 5n = O(n)$

$$\begin{aligned}\text{Addition: } f_n + g_n &= 3n^2 + 5n = O(n^2 + n) = O(n^2) \\ &\quad (\text{wegen Regel 3: } n = O(n^2))\end{aligned}$$

$$\text{Multiplikation: } f_n \cdot g_n = (3n^2) \cdot (5n) = 15n^3 = O(n^2 \cdot n) = O(n^3)$$

Beispiel Vereinfachung:

Gegeben: Algorithmus benötigt $4n^3 + 16n^2 + 239$ Operationen.

$$\begin{aligned}4n^3 + 16n^2 + 239 &= O(n^3 + n^2 + 1) \\ &= O(n^3) \quad (\text{da } n^2 = O(n^3) \text{ und } 1 = O(n^3))\end{aligned}$$

$\implies$ **Wir können komplexe Ausdrücke auf dominanten Term reduzieren!**

Fibonacci: Komplexitätsanalyse 1

Iterativer Algorithmus:

```
1 fibonacci <- function(n) {  
2   x <- numeric(n)  
3   if (n > 1) {  
4     x[2] = 1  
5   }  
6   if (n > 2) {  
7     for (i in 2:(n-1)) {  
8       x[i+1] <- x[i] + x[i - 1]  
9     }  
10  }  
11  x  
12 }
```

Zeitkomplexität:

- Initialisierung: $x \leftarrow \text{numeric}(n) \rightarrow O(n)$
- Schleife: $n - 2$ Iterationen mit je konstant vielen Operationen (1 Addition + Indexrechnung) $\rightarrow O(n)$
- **Gesamt:** $O(n)$ (linear)

Speicherkomplexität:

- Vektor x mit n Elementen $\rightarrow O(n)$

Fibonacci: Komplexitätsanalyse 2

Naive Rekursion:

```
1 fib_recursive <- function(n) {  
2   if (n <= 1) {  
3     return(n)  
4   }  
5   return(  
6     fib_recursive(n-1) +  
7     fib_recursive(n-2)  
8   )  
9 }
```

- **Zeitkomplexität:**
jede Iteration macht 2 Aufrufe, die wiederum je 2 Aufrufe machen, usw.
 $\implies$ insgesamt $O(2^n)$, also exponentiell!
katastrophal langsam
- **Speicherkomplexität:**
 $O(n)$ (Rekursionstiefe)

Live: Laufzeit in Sekunden für $n = 30$ –

```
1 system.time(fibonacci(30))["elapsed"]
```

elapsed
0.024

```
1 system.time(fib_recursive(30))["elapsed"]
```

elapsed
4.27

$\implies$ Iterative Lösung ist dramatisch effizienter ($\times 180$) – und liefert dabei sogar *alle* ersten n Zahlen, während die Rekursion nur F_n berechnet!

Self-Check: Landausymbole

Welche Aussagen sind wahr?

1. $2n = O(n^2)$

2. $7/n = o(1)$

3. $8n^3 + 7n^2 + n = O(n)$

4. $n^{-3} + n^{-2} = O(n^{-2})$

Zusammenfassung

- Für numerische *Probleme* f mit Input x wollen wir eine Lösung $f(x)$ berechnen.
- Gute *Algorithmen* lösen das *Problem* mit niedrigem Aufwand.
- Wir betrachten deshalb ihre *Laufzeit-/Speicher-Komplexitäten*.
- Diese analysieren wir anhand ihrer *Ordnung* mit *Landausymbolen*.

Nächste Vorlesung

- *Kondition* und *Stabilität*.

Anhang

Fibonacci: Schrittweise Berechnung

Berechne die ersten 6 Fibonacci-Zahlen ($n = 6$):

$$\begin{aligned}\tilde{f}_1(6) &= 0 \\ \tilde{f}_2(\tilde{f}_1(6)) &= \tilde{f}_2(0) = (0, 1) \\ \tilde{f}_3(0, 1) &= (0, 1, 1) \\ \tilde{f}_4(0, 1, 1) &= (0, 1, 1, 2) \\ \tilde{f}_5(0, 1, 1, 2) &= (0, 1, 1, 2, 3) \\ \tilde{f}_6(0, 1, 1, 2, 3) &= (0, 1, 1, 2, 3, 5)\end{aligned}$$

Ergebnis: $\tilde{f}(6) = (0, 1, 1, 2, 3, 5)$

↩ zurück: „Fibonacci: Implementierung“

Landausymbole: Weitere Rechenbeispiele

Beispiel 2: Zeige, dass $5n = o(n^2)$

$$\lim_{n \rightarrow \infty} \frac{5n}{n^2} = \lim_{n \rightarrow \infty} \frac{5}{n} = 0 \implies 5n = o(n^2) \quad \checkmark$$

Beispiel 3: Ist $n^2 = O(n)$? **Nein!**

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{n^2}{n} &= \lim_{n \rightarrow \infty} n = \infty \quad (\text{divergiert!}) \\ \implies n^2 &\neq O(n) \end{aligned}$$

↩ zurück: „Landausymbole: Rechenbeispiel“

Matrix-Spur und Matrixnormen

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)

Material: Thomas Nagler

LMU München

Verwendete Vorkenntnisse

- Normen als Fehlermaße
- Konditionszahlen
- **Lineare Algebra**
 - Matrixoperationen (Addition, Multiplikation, Transposition)
 - Eigenwerte und Eigenvektoren
 - Orthogonalität
 - Skalarprodukte
 - Grundlegende Vektornormen (Euklidische Norm $\|\boldsymbol{x}\|_2$, p -Normen)
- **Analysis**
 - Grenzwerte und Suprema
 - Maximum/Minimum von Funktionen

Die Spur einer Matrix

Matrix-Trace (Spur)

Def.: Spur

Die **Spur** (“trace”) einer quadratischen Matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ ist die Summe ihrer Diagonalelemente:

$$\text{tr}(\mathbf{A}) := \sum_{i=1}^n a_{ii}$$

Bemerkungen:

- Nur für quadratische Matrizen definiert
- Skalare Invariante der Matrix

Beispiele:

- $\text{tr}(\mathbf{I}_n) = n$ $\text{tr} \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} = 1 + 4 = 5$
- $\text{tr} \begin{pmatrix} 2 & -1 & 0 \\ 0 & 3 & 5 \\ 1 & 0 & -2 \end{pmatrix} = 2 + 3 + (-2) = 3$

Grundlegende Eigenschaften der Spur

Theorem: Eigenschaften der Spur

Für Matrizen $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$ und $c \in \mathbb{R}$:

1. *Additiv:*

$$\text{tr}(\mathbf{A} + \mathbf{B}) = \text{tr}(\mathbf{A}) + \text{tr}(\mathbf{B})$$

2. *Homogen:*

$$\text{tr}(c\mathbf{A}) = c \cdot \text{tr}(\mathbf{A})$$

3. *Zyklisch:*

$$\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$$

4. *Ähnlichkeitsinvariant:*

$$\text{tr}(\mathbf{PAP}^{-1}) = \text{tr}(\mathbf{A}) \text{ (für invertierbares } \mathbf{P} \in \mathbb{R}^{n \times n})$$

5. *Transpositionsinvariant:*

$$\text{tr}(\mathbf{A}^\top) = \text{tr}(\mathbf{A})$$

Wichtigste Eigenschaft:

Zyklische Vertauschung:

$$\text{tr}(\mathbf{ABC}) = \text{tr}(\mathbf{BCA}) = \text{tr}(\mathbf{CAB})$$

Vorsicht:

$\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$, aber $\text{tr}(\mathbf{AB}) \neq \text{tr}(\mathbf{A}) \cdot \text{tr}(\mathbf{B})$!

Spur und Eigenwerte

Theorem: Spur als Summe der Eigenwerte

Für eine Matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ mit Eigenwerten $\lambda_1, \dots, \lambda_n$:

$$\text{tr}(\mathbf{A}) = \sum_{i=1}^n \lambda_i$$

Beweisskizze:

siehe [Anhang](#)

Kernidee:

Für diagonalisierbare $\mathbf{A} = \mathbf{P}\mathbf{D}\mathbf{P}^{-1}$ folgt die Aussage direkt aus der Ähnlichkeitsinvarianz (4).

Frobenius-Norm

Def.: Frobenius-Norm

Die **Frobenius-Norm** einer Matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ ist

$$\|\mathbf{A}\|_F := \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^2}$$

Bemerkungen:

- Analog zur euklidischen Vektornorm
- Auch als **Hilbert-Schmidt-Norm** bekannt
- Misst sowas wie die “Größe” der Matrix

Spur und Frobenius-Norm

Verbindung zur Frobenius-Norm:

Für eine Matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$:

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^2} = \sqrt{\text{tr}(\mathbf{A}^\top \mathbf{A})}$$

Beweis:

$$\mathbf{A}^\top \mathbf{A} = \begin{pmatrix} \sum_{i=1}^m a_{i1}^2 & \sum_{i=1}^m a_{i1}a_{i2} & \cdots \\ \sum_{i=1}^m a_{i2}a_{i1} & \sum_{i=1}^m a_{i2}^2 & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix}$$
$$\Rightarrow \text{tr}(\mathbf{A}^\top \mathbf{A}) = \sum_{j=1}^n \sum_{i=1}^m a_{ij}^2$$

Spur = Summe der **Diagonalelemente**; die **Nicht-Diagonalelemente** fallen weg.

Matrixnormen - Definition und Beispiele

Motivation

Warum brauchen wir **Matrixnormen**?

- Größe einer Matrix messen
- **Fehler** in Berechnungen quantifizieren
- **Konvergenz** von Algorithmen analysieren
- **Konditionierung** von Problemen verstehen

Problem mit der Frobenius-Norm:

- Ignoriert die Eigenschaften der **lineare Transformation** $x \mapsto Ax$, die A repräsentiert
- Behandelt Matrix wie langen Vektor:
invariant gegen Anordnung der Einträge
- misst nur die *Summe* der quadrierten Bildlängen der Einheitsvektoren

$$\|A\|_F^2 = \sum_j \|Ae_j\|_2^2$$

— nicht, wie stark A einen Vektor *maximal* verzerren kann

Frage:

Wie können wir Normen definieren, die die Transformationseigenschaften berücksichtigen?

Matrixnorm

Def.: Matrixnorm

Eine Funktion $\|\cdot\| : \mathbb{R}^{m \times n} \rightarrow [0, \infty)$ ist eine **Matrixnorm**, wenn für alle $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m \times n}$ und $c \in \mathbb{R}$ gilt:

1. *Definitheit*:

$$\|\mathbf{A}\| = 0 \iff \mathbf{A} = \mathbf{0}$$

2. *Absolute Homogenität*:

$$\|c\mathbf{A}\| = |c| \cdot \|\mathbf{A}\|$$

3. *Dreiecksungleichung*:

$$\|\mathbf{A} + \mathbf{B}\| \leq \|\mathbf{A}\| + \|\mathbf{B}\|$$

Bemerkung:

- Selbe Axiome wie für **Vektornormen**
- Eine **Vektornorm** ist eine Matrixnorm auf $\mathbb{R}^{n \times 1}$

Aus Vektornormen abgeleitete Matrixnormen

Methode: Vektorisierung

1. Schreibe Matrix als langen Vektor

$$\text{vec}(\mathbf{A}) = \begin{pmatrix} a_{11} \\ \vdots \\ a_{m1} \\ \vdots \\ a_{1n} \\ \vdots \\ a_{mn} \end{pmatrix} \in \mathbb{R}^{mn}$$

2. Wende Vektornorm an

Beispiele:

- **Frobenius-Norm:**

$$\|\mathbf{A}\|_F = \|\text{vec}(\mathbf{A})\|_2$$

- **Summennorm:**

$$\|\mathbf{A}\|_S = \|\text{vec}(\mathbf{A})\|_1 = \sum_{i,j} |a_{ij}|$$

- **Maximumsnorm:**

$$\|\mathbf{A}\|_M = \|\text{vec}(\mathbf{A})\|_\infty = \max_{i,j} |a_{ij}|$$

Problem mit element-weisen Normen

Verlust der Transformationsinterpretation:

Beispiel: Betrachte $\mathbf{A} = \mathbf{I}_n$ (Identitätsmatrix)

- $\|\mathbf{I}_n\|_F = \sqrt{n}$ (wächst mit Dimension!)
- $\|\mathbf{I}_n\|_S = n$
- $\|\mathbf{I}_n\|_M = 1$

Problem: Die Transformation $\mathbf{x} \mapsto \mathbf{I}_n \mathbf{x} = \mathbf{x}$ ändert nichts, aber Frobenius- und Summennorm wachsen mit n !

Lösung: *Operatornormen*, die die Transformation direkt messen

Beispiel: Problematische Frobenius-Norm

$$\mathbf{A}_1 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{A}_2 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad \mathbf{A}_3 = \begin{pmatrix} \sqrt{2} & 0 \\ 0 & 0 \end{pmatrix}$$

Frobenius-Norm:

$$\|\mathbf{A}_1\|_F = \sqrt{1^2 + 1^2} = \|\mathbf{A}_2\|_F = \sqrt{1^2 + 1^2} = \|\mathbf{A}_3\|_F = \sqrt{(\sqrt{2})^2} = \sqrt{2}$$

$\implies$ Selbe Frobenius-Norm, **Transformationen völlig unterschiedlich:**

Mit $\mathbf{x} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$:

- $\mathbf{A}_1 \mathbf{x} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ (Identität, keine Änderung)
- $\mathbf{A}_2 \mathbf{x} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$ (Koordinatenvertauschung: Spiegelung an $x_1 = x_2$)
- $\mathbf{A}_3 \mathbf{x} = \begin{pmatrix} \sqrt{2} \\ 0 \end{pmatrix}$ (Streckung der Projektion auf erste Komponente um $\sqrt{2}$)

Operatornormen

Operatornormen

Def.: Operatornorm

Sei $\|\cdot\|_V$ eine *Vektornorm* (definiert auf $\mathbb{R}^n$ und $\mathbb{R}^m$). Die **Operatornorm** von $\mathbf{A} \in \mathbb{R}^{m \times n}$ ist:

$$\|\mathbf{A}\|_V := \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|\mathbf{A}\mathbf{x}\|_V}{\|\mathbf{x}\|_V} = \max_{\|\mathbf{x}\|_V=1} \|\mathbf{A}\mathbf{x}\|_V$$

Interpretation:

- Misst maximale **Streckung** durch $\mathbf{A}$ (im Sinne der Vektornorm)
 - große Norm $\rightarrow$ starke “Streckung”
 - kleine Norm $\rightarrow$ schwache “Streckung” / starke “Stauchung”
- *Streckungsfaktor* von $\mathbf{x}$ unter $\mathbf{A}$: $\frac{\|\mathbf{A}\mathbf{x}\|_V}{\|\mathbf{x}\|_V}$

Visualisierung: Operatornorm

$$A = \begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix}$$

```
1 (A <- matrix(c(2, 1, 0, 1), nrow = 2, byrow = TRUE))
```

```
      [,1] [,2]  
[1,]    2    1  
[2,]    0    1
```

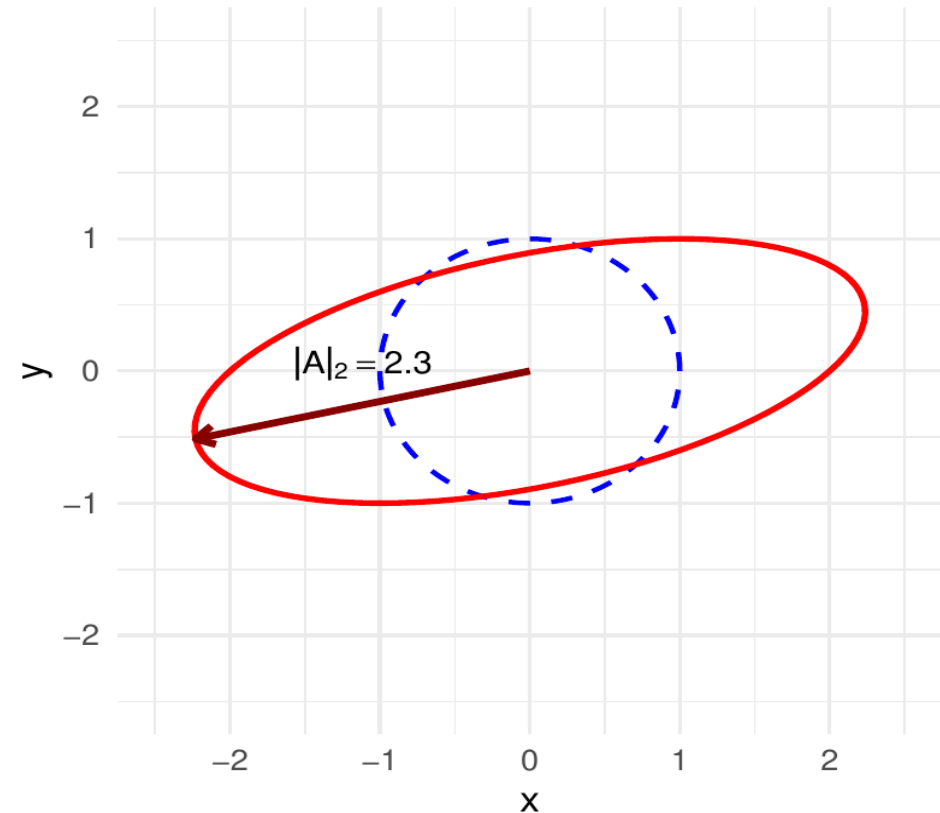
```
1 (AtA <- crossprod(A))
```

```
      [,1] [,2]  
[1,]    4    2  
[2,]    2    2
```

```
1 eigen(AtA)$values |> sqrt()
```

```
[1] 2.288 0.874
```

Operatornorm = maximale Streckung
Blau: Einheitskreis, Rot: Transformiertes Bild



Wichtige Operatornormen

Basierend auf verschiedenen p -Normen

$$\|\mathbf{v}\|_p = \left(\sum_i |v_i|^p \right)^{1/p} \quad \text{für } 1 \leq p < \infty$$

bzw. $\|\mathbf{v}\|_\infty = \max_i |v_i|$:

Spektralnrm ($p = 2$):

$$\|\mathbf{A}\|_2 = \max_{\|\mathbf{x}\|_2=1} \|\mathbf{Ax}\|_2 = \sqrt{\lambda_{\max}(\mathbf{A}^\top \mathbf{A})}$$

Spaltensummennorm ($p = 1$):

$$\|\mathbf{A}\|_1 = \max_{\|\mathbf{x}\|_1=1} \|\mathbf{Ax}\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^m |a_{ij}|$$

Zeilensummennorm ($p = \infty$):

$$\|\mathbf{A}\|_\infty = \max_{\|\mathbf{x}\|_\infty=1} \|\mathbf{Ax}\|_\infty = \max_{1 \leq i \leq m} \sum_{j=1}^n |a_{ij}|$$

Einschub: Orthogonalmatrizen

Def.: Orthogonalmatrix

Eine Matrix $Q \in \mathbb{R}^{n \times n}$ heißt **Orthogonalmatrix**, wenn ihre Spalten *orthonormale Vektoren* sind, also $Q^\top Q = I$.

Nützliche Eigenschaften

- Geometrisch:
 Q beschreibt eine *Rotation und/oder Spiegelung* in $\mathbb{R}^n$.
- für Orthogonalmatrizen $Q_1, \dots, Q_n$ ist auch $Q = Q_1 \cdots Q_n$ **orthogonal**.
- Normerhaltend:
 $\|Qx\|_2 = \sqrt{x^\top Q^\top Q x} = \sqrt{x^\top x} = \|x\|_2$ für alle $x \in \mathbb{R}^n$.
- Unitär:
Es gilt $|\lambda_1(Q)| = \dots = |\lambda_n(Q)| = 1$.
- $Q^{-1} = Q^\top$ ist wieder orthogonal, also $\|Q\|_2 = \|Q^{-1}\|_2 = 1$
 $\implies \kappa_2(Q) = \|Q\|_2 \cdot \|Q^{-1}\|_2 = 1$
 $\implies$ Multiplikation mit Q oder Q^{-1} ist immer gut konditioniert.
 $\implies$ Multiplikation mit Q oder Q^{-1} verschlechtert nicht die Stabilität von Algorithmen.

Spektralnrm und Spektralzerlegung

Theorem

Für $\mathbf{A} \in \mathbb{R}^{m \times n}$ mit Rang r und Spektralzerlegung $\mathbf{A}^\top \mathbf{A} = \mathbf{P} \mathbf{\Lambda} \mathbf{P}^\top$, wobei $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_r, 0, \dots, 0)$ mit $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_r > 0$:

$$\|\mathbf{A}\|_2 = \sqrt{\lambda_1} = \sqrt{\lambda_{\max}(\mathbf{A}^\top \mathbf{A})}$$

Beweis:

siehe **Anhang**

Interpretation:

- Wurzel des größten Eigenwerts von $\mathbf{A}^\top \mathbf{A}$ = maximaler Streckungsfaktor
- intuitiver über **Singulärwerte** / **SVD** (später...)

Beispiele für Operatornormen

Beispiel 1:

$$\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$$

Spaltensummennorm:

$$\|\mathbf{A}\|_1 = \max\{|2| + |0|, |1| + |3|\} = 4$$

Zeilensummennorm:

$$\|\mathbf{A}\|_\infty = \max\{|2| + |1|, |0| + |3|\} = 3$$

Spektralnrm: $\|\mathbf{A}\|_2 = \sqrt{\lambda_{\max}(\mathbf{A}^\top \mathbf{A})} \approx 3.26$

Beispiel 2: Identitätsmatrix $\mathbf{I}_n$

- $\|\mathbf{I}_n\|_1 = \|\mathbf{I}_n\|_2 = \|\mathbf{I}_n\|_\infty = 1 \checkmark$
- Macht Sinn: $\mathbf{I}_n$ verändert nichts!

Schattennormen

Schatten-p-Normen

Def.: Schatten-p-Normen

Die **Schatten-p-Norm** ($1 \leq p \leq \infty$) einer Matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, deren Kreuzprodukt $\mathbf{A}^\top \mathbf{A}$ die Eigenwerte $\boldsymbol{\lambda} = (\lambda_1, \dots, \lambda_n)$ hat, ist:

$$\|\mathbf{A}\|_{S,p} := \|\boldsymbol{\lambda}^{1/2}\|_p = \left(\sum_{i=1}^n \lambda_i^{p/2} \right)^{1/p}$$

Wichtige Spezialfälle:

- $p = \infty$: $\|\mathbf{A}\|_{S,\infty} = \sqrt{\lambda_{\max}} = \|\mathbf{A}\|_2$ (Spektralnrm)
- $p = 2$: $\|\mathbf{A}\|_{S,2} = \|\boldsymbol{\lambda}^{1/2}\|_2 = \|\mathbf{A}\|_F$ (Frobenius-Norm!)
- $p = 1$: $\|\mathbf{A}\|_{S,1} = \|\mathbf{A}\|_* = \sum_{i=1}^r \sqrt{\lambda_i}$ (Nuklearnorm)

Überraschung: Frobenius-Norm ist eine Schatten-Norm!

Schatten-Normen

- Benannt nach dem Mathematiker **Robert Schatten** (1911-1977)
- Schatten-Normen sind **unitär invariant**, d.h. invariant unter orthogonalen Transformationen:
Für orthogonale Matrizen P, Q gilt $\|PAQ\|_{S,p} = \|A\|_{S,p}$
Beweis: $(PAQ)^\top (PAQ) = Q^\top A^\top A Q$ und $A^\top A$ haben dieselben Eigenwerte.
- **Warum ist $\|A\|_{S,2} = \|A\|_F$?**
Wie bereits gezeigt gilt $\|A\|_F^2 = \text{tr}(A^\top A)$; mit der Spektralzerlegung $A^\top A = P\Lambda P^\top$ folgt

$$\|A\|_F^2 = \text{tr}(P\Lambda P^\top) = \text{tr}(\Lambda) = \sum_{i=1}^r \lambda_i = \|A\|_{S,2}^2$$

- $(P, P^\top)$ heben sich weg: zyklische Vertauschung und $P^\top P = I$)
- **Anwendung:** Schatten-Normen sind ideal für **Approximationsprobleme**, da sie invariant unter Koordinatentransformationen sind.

Eigenschaften von Matrixnormen

Normenäquivalenz

Theorem: Alle Matrixnormen sind äquivalent

Für je zwei Matrixnormen $\|\cdot\|_a$ und $\|\cdot\|_b$ auf $\mathbb{R}^{m \times n}$ (bei festen m, n) existieren Konstanten $c, C > 0$ mit:

$$c\|\mathbf{A}\|_a \leq \|\mathbf{A}\|_b \leq C\|\mathbf{A}\|_a$$

Explizite Konstanten (Beispiele):

- $\|\mathbf{A}\|_2 \leq \|\mathbf{A}\|_F \leq \sqrt{\min(m, n)} \|\mathbf{A}\|_2$
- $\|\mathbf{A}\|_F \leq \|\mathbf{A}\|_* \leq \sqrt{\min(m, n)} \|\mathbf{A}\|_F$
- $\frac{1}{\sqrt{n}} \|\mathbf{A}\|_\infty \leq \|\mathbf{A}\|_2 \leq \sqrt{m} \|\mathbf{A}\|_\infty$

Wichtig:

- **Äquivalenz $\neq$ Gleichheit**
- Verschiedene Normen können Matrizen **unterschiedlich ordnen**
- Jede Norm hat ihre eigene **Interpretation**

Submultiplikativität

Definition: Submultiplikative Norm

Eine Matrixnorm $\| \cdot \|$ heißt **submultiplikativ**, wenn für alle A, B mit passenden Dimensionen gilt:

$$\|AB\| \leq \|A\| \cdot \|B\|$$

- Alle **Operatornormen** und **Schatten-Normen** sind submultiplikativ
- Die **Maximumsnorm** nicht (Gegenbeispiel: siehe **Anhang**)

Warum ist das wichtig?

- Kontrolliert **Fehlerfortpflanzung** in Matrixprodukten
- Ermöglicht **Konvergenzanalyse** iterativer Verfahren
- Verbindet Normen mit **Spektralradius**

Verträglichkeit von Normen

Def.: Verträgliche Norm

Eine Matrixnorm $\| \cdot \|$ ist **verträglich** mit einer Vektornorm $\| \cdot \|_V$, wenn:

$$\| \mathbf{A} \mathbf{x} \|_V \leq \| \mathbf{A} \| \cdot \| \mathbf{x} \|_V$$

Verträglichkeiten:

1. **Jede submultiplikative Norm** ist mit sich selbst verträglich
2. **Jede Operatornorm** ist mit ihrer induzierenden Vektornorm verträglich
3. **Frobenius-Norm** ist mit der euklidischen Norm verträglich

Anwendung:

Verträglichkeit ermöglicht **Fehlerabschätzungen**: Wenn $\| \mathbf{e} \|_V$ klein ist, dann ist auch $\| \mathbf{A} \mathbf{e} \|_V$ kontrolliert.

Matrixnormen und Konditionierung

Konditionszahl einer (invertierbaren) Matrix:

$$\kappa(\mathbf{A}) = \|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\|$$

Für Spektralnorm:

$$\kappa_2(\mathbf{A}) = \|\mathbf{A}\|_2 \cdot \|\mathbf{A}^{-1}\|_2 = \frac{\lambda_{\max}(\mathbf{A}^\top \mathbf{A})^{1/2}}{\lambda_{\min}(\mathbf{A}^\top \mathbf{A})^{1/2}}$$

Interpretation:

- $\kappa \approx 1$: Gut konditioniert
- $\kappa \gg 1$: Schlecht konditioniert
- $\kappa = \infty$: Singulär

Verbindung zu *Spektralzerlegung* / **SVD**: Konditionierung wird durch das Verhältnis von größtem zu kleinstem Singulärwert bestimmt!

Normen in der Fehleranalyse

Rückwärts-Fehleranalyse:

Problem:

Löse $\mathbf{A}\mathbf{x} = \mathbf{b}$, erhalte $\tilde{\mathbf{x}}$

Frage:

Für welches gestörte Problem ist $\tilde{\mathbf{x}}$ die exakte Lösung?

Antwort:

$(\mathbf{A} + \Delta\mathbf{A})\tilde{\mathbf{x}} = \mathbf{b}$ mit $\Delta\mathbf{A} = -\mathbf{r}\tilde{\mathbf{x}}^\top / \|\tilde{\mathbf{x}}\|^2$ für $\mathbf{r} = \mathbf{A}\tilde{\mathbf{x}} - \mathbf{b}$

Fehlerabschätzung:

$$\frac{\|\Delta\mathbf{A}\|_2}{\|\mathbf{A}\|_2} = \frac{\|\mathbf{r}\|_2}{\|\mathbf{A}\|_2 \cdot \|\tilde{\mathbf{x}}\|_2}$$

Interpretation:

Normen quantifizieren die **relative Störung** des Problems!

Zusammenfassung

Zusammenfassung

- **Spur:**

$$\text{tr}(\mathbf{A}) = \sum_{i=1}^n a_{ii} = \sum_{i=1}^n \lambda_i$$

linear, zyklisch ($\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$), ähnlichkeitsinvariant

- **Drei Familien von Matrixnormen:**

1. *Element-weise Normen* ($\|\mathbf{A}\|_F, \|\mathbf{A}\|_S, \|\mathbf{A}\|_M$): einfach zu berechnen, verlieren aber die Transformationsinterpretation
2. *Operatornormen* ($\|\mathbf{A}\|_1, \|\mathbf{A}\|_2, \|\mathbf{A}\|_\infty$): messen maximale Streckung, **submultiplikativ**, **verträglich**
3. *Schatten-Normen* ($\|\mathbf{A}\|_{S,p}$): basieren auf Eigen-/Singulärwerten, **unitär invariant**

- **Verbindungen:**

- **Frobenius-Norm** ist element-weise Norm **und** Schatten-Norm:

$$\|\mathbf{A}\|_F = \sqrt{\text{tr}(\mathbf{A}^\top \mathbf{A})} = \sqrt{\sum_i \lambda_i} = \|\mathbf{A}\|_{S,2}$$

- **Spektralnorm** $\|\mathbf{A}\|_2 = \sqrt{\lambda_{\max}(\mathbf{A}^\top \mathbf{A})}$ verbindet Operatornormen mit Spektralzerlegung / **SVD**
- **Konditionszahl**: $\kappa(\mathbf{A}) = \|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\|$

Self-Check

Quiz - Trace Eigenschaften

Welche Aussagen sind korrekt?

1. $\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{A}) \cdot \text{tr}(\mathbf{B})$
2. $\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$
3. $\text{tr}(\mathbf{A}^\top) = -\text{tr}(\mathbf{A})$
4. $\text{tr}(\mathbf{I}_n) = n$
5. Wenn $\mathbf{A}$ invertierbar ist, dann $\text{tr}(\mathbf{A}^{-1}) = 1/\text{tr}(\mathbf{A})$

Quiz - Normen berechnen

Gegeben: $A = \begin{pmatrix} 1 & 2 \\ 3 & 0 \end{pmatrix}$

Berechne:

1. $\|A\|_1$ (Spaltensummennorm)
2. $\|A\|_\infty$ (Zeilensummennorm)
3. $\|A\|_F$ (Frobenius-Norm)

Anhang

Beweisskizze: Spur als Summe der Eigenwerte

Beweisskizze: (für diagonalisierbare $\mathbf{A}$)

Sei $\mathbf{A} = \mathbf{P}\mathbf{D}\mathbf{P}^{-1}$ eine Diagonalisierung, also $\mathbf{D} = \text{diag}(\lambda_1, \dots, \lambda_n)$

$$\implies \text{tr}(\mathbf{A}) = \text{tr}(\mathbf{P}\mathbf{D}\mathbf{P}^{-1}) \stackrel{(4)}{=} \text{tr}(\mathbf{D}) = \sum_{i=1}^n \lambda_i$$

($\mathbf{P}, \mathbf{P}^{-1}$ heben sich per Ähnlichkeitsinvarianz (4) weg.)

Beweis für allgemeine $\mathbf{A}$ über charakteristisches Polynom.

↔ zurück: „Spur und Eigenwerte“

Beweis: Spektralnorm und Spektralzerlegung

Sei $\mathbf{A} \in \mathbb{R}^{m \times n}$ mit Rang r und Spektralzerlegung $\mathbf{A}^\top \mathbf{A} = \mathbf{P} \mathbf{\Lambda} \mathbf{P}^\top$, wobei $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_r, 0, \dots, 0)$ mit $\lambda_1 \geq \dots \geq \lambda_r > 0$.

Beweis (Skizze):

$$\|\mathbf{A}\mathbf{x}\|_2^2 = \mathbf{x}^\top \mathbf{A}^\top \mathbf{A} \mathbf{x} = \mathbf{x}^\top \mathbf{P} \mathbf{\Lambda} \mathbf{P}^\top \mathbf{x} = \|\mathbf{\Lambda}^{1/2} \mathbf{P}^\top \mathbf{x}\|_2^2$$

Weil $\mathbf{P}$ *orthogonal* ist, durchläuft $\mathbf{P}^\top \mathbf{x}$ die Einheitssphäre wenn $\mathbf{x}$ dies tut, also

$$\max_{\|\mathbf{x}\|_2=1} \|\mathbf{\Lambda}^{1/2} \mathbf{P}^\top \mathbf{x}\|_2^2 = \max_{\|\mathbf{y}\|_2=1} \|\mathbf{\Lambda}^{1/2} \mathbf{y}\|_2^2 = \lambda_1,$$

$$\text{da } \|\mathbf{\Lambda}^{1/2} \mathbf{y}\|_2^2 = \sum_{i=1}^r \lambda_i y_i^2 \leq \lambda_1 \sum_{i=1}^r y_i^2 \leq \lambda_1,$$

mit Gleichheit für $\mathbf{y} = \mathbf{e}_1$. ■

↔ zurück: „Spektralnorm und Spektralzerlegung“

Gegenbeispiel: Maximumsnorm ist nicht submultiplikativ

Beispiel: $\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$

$$\mathbf{A}^2 = \begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix}$$

$$\|\mathbf{A}\|_M = 1, \quad \|\mathbf{A}^2\|_M = 2$$

$$\|\mathbf{A}^2\|_M = 2 > 1 = \|\mathbf{A}\|_M \cdot \|\mathbf{A}\|_M$$

Reparatur: Gesamtnorm

$$\|\mathbf{A}\|_G = \sqrt{mn} \|\mathbf{A}\|_M$$

Diese ist submultiplikativ!

↔ zurück: „Submultiplikativität“

Ausblick: Normen in Machine Learning

Ridge Regression (L2-Regularisierung):

$$\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2^2 + \lambda \|\mathbf{x}\|_2^2$$

LASSO (L1-Regularisierung):

$$\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2^2 + \lambda \|\mathbf{x}\|_1$$

Nuclear Norm Regularisierung (Matrix Completion):

$$\min_{\mathbf{X}} \|\mathbf{P}_\Omega(\mathbf{X}) - \mathbf{P}_\Omega(\mathbf{M})\|_F^2 + \lambda \|\mathbf{X}\|_*$$

($\mathbf{P}_\Omega$: behält nur die beobachteten Einträge Ω , Rest = 0)

$\implies$ Verschiedene Normen führen zu verschiedenen Lösungscharakteristika!

Numerische Fehleranalyse

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)

Material: Thomas Nagler

LMU München

Verwendete Vorkenntnisse

- Algorithmuskonzept
- Komplexitätsanalyse
- Verständnis, dass Algorithmen (meist) *approximative* Lösungen berechnen
- **Lineare Algebra I**
 - Vektor- und Matrixnormen
 - Normeigenschaften: Definitheit, Homogenität, Dreiecksungleichung
 - Matrixoperationen und Matrix-Vektor-Produkte
- **Analysis I**
 - Grenzwerte und Konvergenz
 - Stetigkeit
 - Grundlagen Taylor-Approximation

Fehleranalyse

Fehleranalyse

- Wenn ein Computer etwas speichert/rechnet, macht er Fehler.
- Der *absolute Fehler* ist dabei

$$\text{absoluter Fehler} = \text{fehlerhafter Wert} - \text{tatsächlicher Wert}.$$

- Es ist üblicher, Fehler *relativ* zu betrachten:

$$\text{relativer Fehler} = \frac{\text{fehlerhafter Wert} - \text{tatsächlicher Wert}}{\text{tatsächlicher Wert}}$$

- Vorteil: Keine Einheiten, Fehler in %.
 - Nachteil: tatsächlicher Wert kann nicht 0 sein.
- Es gilt dann

$$\text{fehlerhafter Wert} = \text{tatsächlicher Wert} \times (1 + \text{relativer Fehler})$$

Fehlermaß

Def.: Fehlermaß

Sei V ein Vektorraum mit (Semi-)Norm $\|\cdot\|$ und $\tilde{v} \in V$ eine Approximation zu $v \in V$. Dann ist

- der **absolute Fehler**

$$\Delta_v = \tilde{v} - v,$$

- der **relative Fehler** (nur definiert falls $\|v\| > 0$)

$$\delta_v = \frac{\|\tilde{v} - v\|}{\|v\|} = \frac{\|\Delta_v\|}{\|v\|}.$$

- Ist $\|\Delta_v\| \leq \varepsilon$ bzw. $\delta_v \leq \varepsilon$, dann nennen wir ε eine absolute bzw. relative **Fehlerschranke**.

Fehlermaß

Lemma: Fehlerschranken

Es gilt

$$\|z\|(1 - \delta_z) \leq \|\tilde{z}\| \leq \|z\|(1 + \delta_z).$$

Beweis:

$$\delta_z = \frac{\|\tilde{z} - z\|}{\|z\|}$$

$$\|\tilde{z} - z\| = \delta_z \|z\|$$

$$||\tilde{z}\| - \|z\|| \leq \delta_z \|z\|$$

$$-\delta_z \|z\| \leq \|\tilde{z}\| - \|z\| \leq \delta_z \|z\|$$

$$\|z\| - \delta_z \|z\| \leq \|\tilde{z}\| \leq \|z\| + \delta_z \|z\|.$$

[umgekehrte Dreiecksungl.]

[Definition von Betrag]



Beispiel: Fehlermaße für Vektoren

Sei $\mathbf{v} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$ und $\tilde{\mathbf{v}} = \begin{pmatrix} 3.2 \\ 4.3 \end{pmatrix}$ mit der Euklidischen Norm $\|\cdot\|_2$.

- **Absoluter Fehler:**

$$\Delta_{\mathbf{v}} = \tilde{\mathbf{v}} - \mathbf{v} = \begin{pmatrix} 3.2 \\ 4.3 \end{pmatrix} - \begin{pmatrix} 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 0.2 \\ 0.3 \end{pmatrix}$$

- **Norm des absoluten Fehlers:**

$$\|\Delta_{\mathbf{v}}\|_2 = \sqrt{0.2^2 + 0.3^2} = \sqrt{0.04 + 0.09} = \sqrt{0.13} \approx 0.361$$

- **Relativer Fehler:**

$$\delta_{\mathbf{v}} = \frac{\|\Delta_{\mathbf{v}}\|_2}{\|\mathbf{v}\|_2} = \frac{\sqrt{0.13}}{\sqrt{3^2 + 4^2}} = \frac{\sqrt{0.13}}{5} \approx \frac{0.361}{5} \approx 0.072 = 7.2\%$$

- **Fehlerschranke:** Mit $\varepsilon = 0.1$ gilt $\delta_{\mathbf{v}} \approx 0.072 < 0.1$, also ist $\varepsilon = 0.1$ eine relative Fehlerschranke.

Fehlerzerlegung

- Sei
 - f ein Problem mit idealem Input $\boldsymbol{x}$
 - $\tilde{f}$ ein zugehöriger Algorithmus
 - und $\tilde{\boldsymbol{x}}$ der tatsächliche Input.
- Ein Algorithmus folgt dem Schema:
 - Input $\tilde{\boldsymbol{x}} \longrightarrow$ Algorithmus $\tilde{f} \longrightarrow$ Ergebnis $\tilde{f}(\tilde{\boldsymbol{x}})$
- Ein Fehler im Ergebnis $\tilde{f}(\tilde{\boldsymbol{x}}) \neq f(\boldsymbol{x})$ kann entstehen durch
 - Fehler im Input $\tilde{\boldsymbol{x}}$ (durch Messen/Speichern von $\boldsymbol{x}$)
 - unvermeidbar
 - Fehler im Algorithmus $\tilde{f}$ (durch Rundung/Approximation)
 - können wir beeinflussen

Fehlerzerlegung

$$\begin{aligned}\text{Gesamtfehler} &= \tilde{f}(\tilde{x}) - f(x) \\ &= \underbrace{\tilde{f}(\tilde{x}) - f(\tilde{x})}_{\substack{\text{Fehler im Algorithmus} \\ \text{(Maschinenarithmetik)}}} + \underbrace{f(\tilde{x}) - f(x)}_{\substack{\text{Folgefehler aus Input} \\ \text{(exakte Arithmetik)}}}\end{aligned}$$

- Erster Term misst nur den Fehler des Algorithmus (bei gegebenem/fixen Input), nicht den Inputfehler.

Ist er “klein”, nennen wir den Algorithmus $\tilde{f}$ **stabil**.

- Zweiter Term hängt nicht vom Algorithmus $\tilde{f}$ ab.

Ist er “klein”, nennen wir das Problem f an der Stelle x **gut konditioniert**.

Beispiel: Fehlerzerlegung

- Es gilt $e^x := \sum_{n=0}^{\infty} \frac{x^n}{n!}$
- Angenommen, wir wollen $f(\pi) = e^\pi$ berechnen und verwenden dazu die Approximationen $\tilde{\pi} = 3$ und $\tilde{f}(x) = \sum_{n=0}^2 x^n/n!$

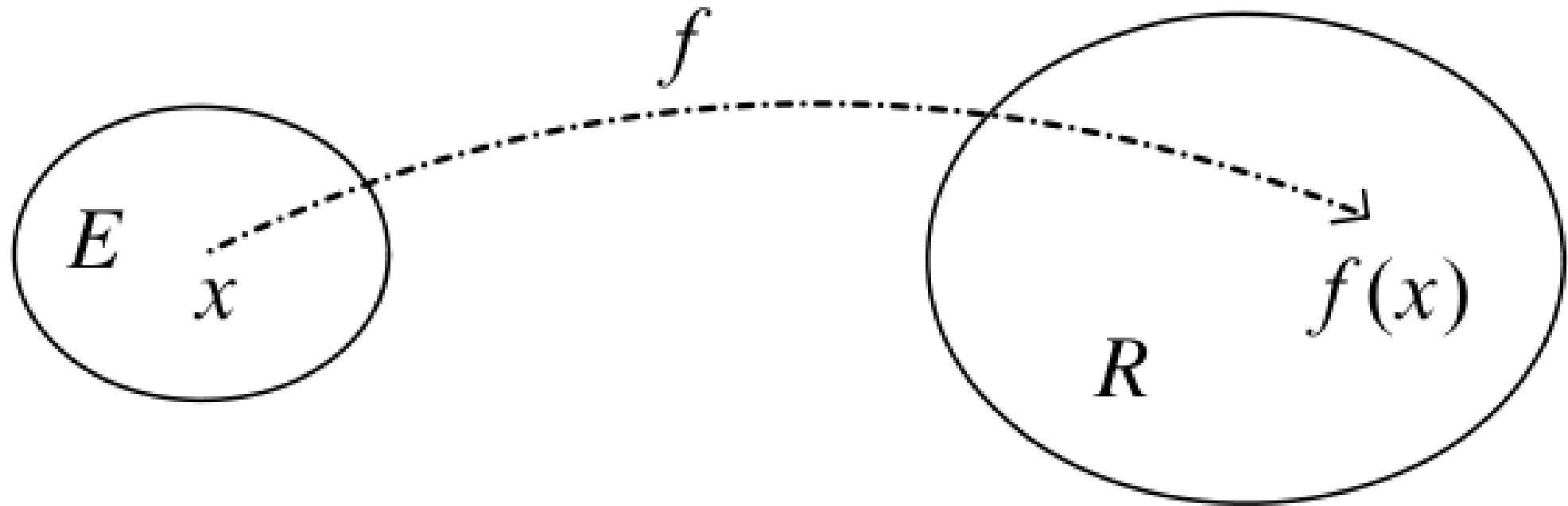
Zerlege den Gesamtfehler in die zwei erwähnten Teile.

Lösung:

$$\begin{aligned}\tilde{f}(3) - f(\pi) &= [\tilde{f}(3) - f(3)] + [f(3) - f(\pi)] \\ &= \text{Fehler im Algorithmus} + \text{Folgefehler aus Input} \\ &= \left[-\sum_{n=3}^{\infty} 3^n/n! \right] + [e^3 - e^\pi] \\ &\approx \left[\left(1 + \frac{3}{1} + \frac{9}{2}\right) - e^3 \right] + [20.1 - 23.1] \\ &\approx [-11.6] + [-3]\end{aligned}$$

Kondition

Kondition



- Inputfehler betrachten wir als *unvermeidbar*.
- Manche Probleme sind schwer zu lösen, weil sie *zu sensitiv bzgl. des Inputs* sind:
→ Outputmenge $R = f(E)$ ist *groß* gegenüber der Menge fehlerhafter Inputs
$$E = \left\{ \tilde{x} : \frac{\|\tilde{x} - x\|}{\|x\|} \leq \delta \right\}.$$
- Wir nennen das Problem dann **schlecht konditioniert**.

Kondition: Beispiel

Beispiel:

- Sei $f(x) = x^{-1}$
- Sei $x = 10^{-17}$, $\tilde{x} = x + \varepsilon$ mit $\varepsilon \in (-10^{-17}, 10^{-17})$.
- $\implies f(\tilde{x}) \in (1/2 \cdot 10^{17}, \infty)$ und

$$\left| \frac{f(\tilde{x}) - f(x)}{f(x)} \right| \in [0, \infty).$$

Achtung:

Die Störung ist hier *relativ zu x* nicht klein (bis zu 100%).

Bei relativ *kleinen* Störungen ist $x \mapsto 1/x$ dagegen harmlos: der relative Outputfehler ist dann ungefähr so groß wie der relative Inputfehler.

Es kommt also darauf an, *welche* Art von Inputfehler unvermeidbar ist.

Kondition

- Im Folgenden betrachten wir Abbildungen $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$.
- $\|\cdot\|$ bezeichnet dabei eine beliebige Vektornorm (bzw. die dadurch induzierte **Operatornorm** für Matrizen).

Def.: Konditionszahl

Sei f ein Problem mit Input $\mathbf{x}$.

- Die **absolute Konditionszahl** ist die kleinste Zahl $\kappa_{abs} = \kappa_{abs}(f, \mathbf{x})$ mit

$$\|f(\tilde{\mathbf{x}}) - f(\mathbf{x})\| \leq \kappa_{abs} \|\tilde{\mathbf{x}} - \mathbf{x}\|, \quad \text{für } \tilde{\mathbf{x}} \rightarrow \mathbf{x}.$$

- Die **relative Konditionszahl** (für $\|\mathbf{x}\| > 0$, $\|f(\mathbf{x})\| > 0$) ist die kleinste Zahl $\kappa_{rel} = \kappa_{rel}(f, \mathbf{x})$ mit

$$\frac{\|f(\tilde{\mathbf{x}}) - f(\mathbf{x})\|}{\|f(\mathbf{x})\|} \leq \kappa_{rel} \frac{\|\tilde{\mathbf{x}} - \mathbf{x}\|}{\|\mathbf{x}\|}, \quad \text{für } \tilde{\mathbf{x}} \rightarrow \mathbf{x}.$$

Interpretation

$$\|f(\tilde{\mathbf{x}}) - f(\mathbf{x})\| \leq \kappa_{abs} \|\tilde{\mathbf{x}} - \mathbf{x}\|, \quad \frac{\|f(\tilde{\mathbf{x}}) - f(\mathbf{x})\|}{\|f(\mathbf{x})\|} \leq \kappa_{rel} \frac{\|\tilde{\mathbf{x}} - \mathbf{x}\|}{\|\mathbf{x}\|}.$$

- *gut konditioniert* ($\kappa \lesssim 1$):
Inputfehler führt zu ähnlich großem oder kleinerem Outputfehler.
- *schlecht konditioniert* ($\kappa \gg 1$):
Inputfehler können zu viel größeren Outputfehlern führen.
- *schlecht gestellt* ($\kappa = \infty$):
Der Fehlerverstärkungsfaktor ist unbeschränkt — beliebig kleine Inputfehler können (relativ zu ihrer Größe) beliebig stark verstärkt werden.

Beispiel

- Sei $f(\boldsymbol{x}) = \boldsymbol{A}^{-1}\boldsymbol{x} = \boldsymbol{y}$.
- Absolute Konditionszahl:

$$\begin{aligned}\|f(\tilde{\boldsymbol{x}}) - f(\boldsymbol{x})\| &= \|\boldsymbol{A}^{-1}(\tilde{\boldsymbol{x}} - \boldsymbol{x})\| \leq \|\boldsymbol{A}^{-1}\| \|\tilde{\boldsymbol{x}} - \boldsymbol{x}\| \\ \implies \kappa_{abs} &= \|\boldsymbol{A}^{-1}\|\end{aligned}$$

- Relative Konditionszahl: Sei $\boldsymbol{h} = \tilde{\boldsymbol{x}} - \boldsymbol{x}$, dann

$$\begin{aligned}\frac{\|\boldsymbol{A}^{-1}\boldsymbol{h}\| / \|\boldsymbol{A}^{-1}\boldsymbol{x}\|}{\|\boldsymbol{h}\| / \|\boldsymbol{x}\|} &= \underbrace{\frac{\|\boldsymbol{A}^{-1}\boldsymbol{h}\|}{\|\boldsymbol{h}\|}}_{\leq \|\boldsymbol{A}^{-1}\|} \cdot \frac{\|\boldsymbol{x}\|}{\|\boldsymbol{A}^{-1}\boldsymbol{x}\|} \\ \implies \kappa_{rel} &= \|\boldsymbol{A}^{-1}\| \frac{\|\boldsymbol{x}\|}{\|\boldsymbol{A}^{-1}\boldsymbol{x}\|} = \kappa_{abs} \frac{\|\boldsymbol{A}\boldsymbol{y}\|}{\|\boldsymbol{y}\|} \leq \kappa_{abs} \|\boldsymbol{A}\| = \|\boldsymbol{A}^{-1}\| \|\boldsymbol{A}\|\end{aligned}$$

(Gleichheit im ersten Faktor, weil $\boldsymbol{h}$ entlang einer die Operatornorm maximierenden Richtung gewählt werden darf.)

- $\kappa(\boldsymbol{A}) = \|\boldsymbol{A}^{-1}\| \|\boldsymbol{A}\|$ wird in der linearen Algebra auch *Konditionszahl von $\boldsymbol{A}$* genannt – sie ist die von $\boldsymbol{x}$ unabhängige obere Schranke für κ_{rel} , nicht κ_{rel} selbst.
- Für die Operatornorm $\|\cdot\|_2$ (*Spektralnorm*) gilt

$$\kappa_2(\boldsymbol{A}) = \frac{\sqrt{\lambda_{\max}(\boldsymbol{A}^\top \boldsymbol{A})}}{\sqrt{\lambda_{\min}(\boldsymbol{A}^\top \boldsymbol{A})}}.$$

Quiz

Sei

$$\mathbf{A} = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 0.1 \end{pmatrix}.$$

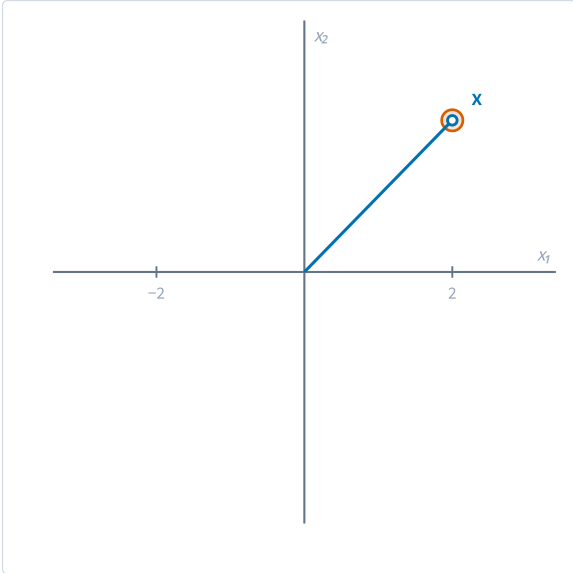
Was ist die Konditionszahl der Matrix bezüglich der 2-Norm?

- 1. 2
- 2. 20
- 3. 5
- 4. 50

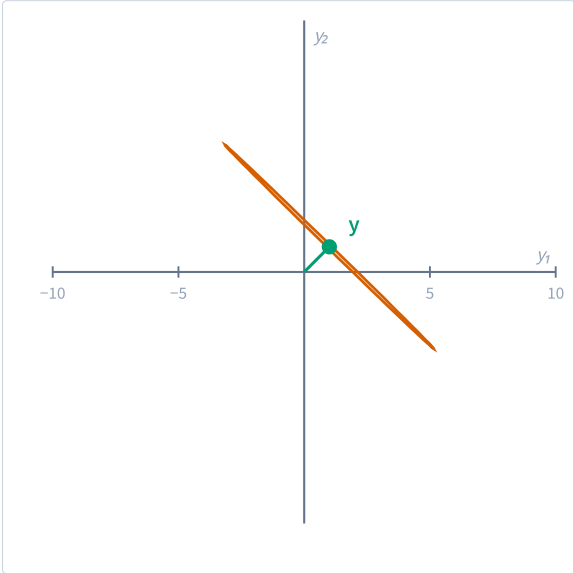
Interaktive Demo: Kondition

► Ziehen wir $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$ im linken Feld im Kreis herum und beobachten, wie sich die rote Ellipse rechts dabei streckt und staucht.

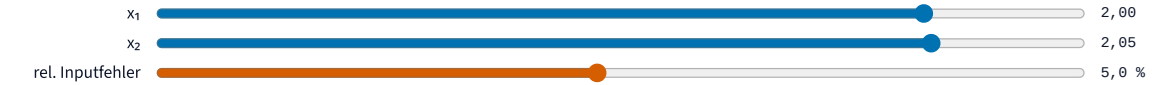
Inputebene: rechte Seite $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$ und die plausiblen Inputs



Lösungsebene: $\begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ und das Bild des Kreises



Drehung ($\kappa = 1$) mäßig gedehnt **schlecht konditioniert** fast singular



rel. Inputfehler

rel. Outputfehler

$A = \begin{pmatrix} 1,00 & 1,00; & 1,00 & 1,05 \end{pmatrix}$
 $y = A^{-1}x = \begin{pmatrix} 1,00; & 1,00 \end{pmatrix}$
Schranke $\kappa(A) = \sigma_{\max}/\sigma_{\min} = 82,04$ · ausgeschöpft zu 100 %

$\sigma_{\max} = 2,025, \sigma_{\min} = 0,0247$
 $\kappa_{\text{rel}}(f, x) = 82,03$

✗ **Die ungünstige rechte Seite.** $\kappa_{\text{rel}}(f, \begin{pmatrix} 1 \\ 0 \end{pmatrix}) = 82,0$ schöpft 100 % der Schranke $\kappa(A) = 82,0$ aus. Bemerkung 4.2.7 sagt genau das: Die Schranke ist kein Pessimismus, sie wird für ungünstige rechte Seiten angenommen. Aus 5,0 % Inputfehler werden hier 410 % Outputfehler.

Aufgabe

Sei $f(\mathbf{x}) = x_1 + x_2 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}^\top \mathbf{x} = \mathbf{1}^\top \mathbf{x}$.

1. Was sind die Konditionszahlen?
2. Für welche $\mathbf{x}$ ist das Problem schlecht konditioniert?

Lösung:

- Bezüglich der euklidischen Norm gilt (Cauchy-Schwarz)

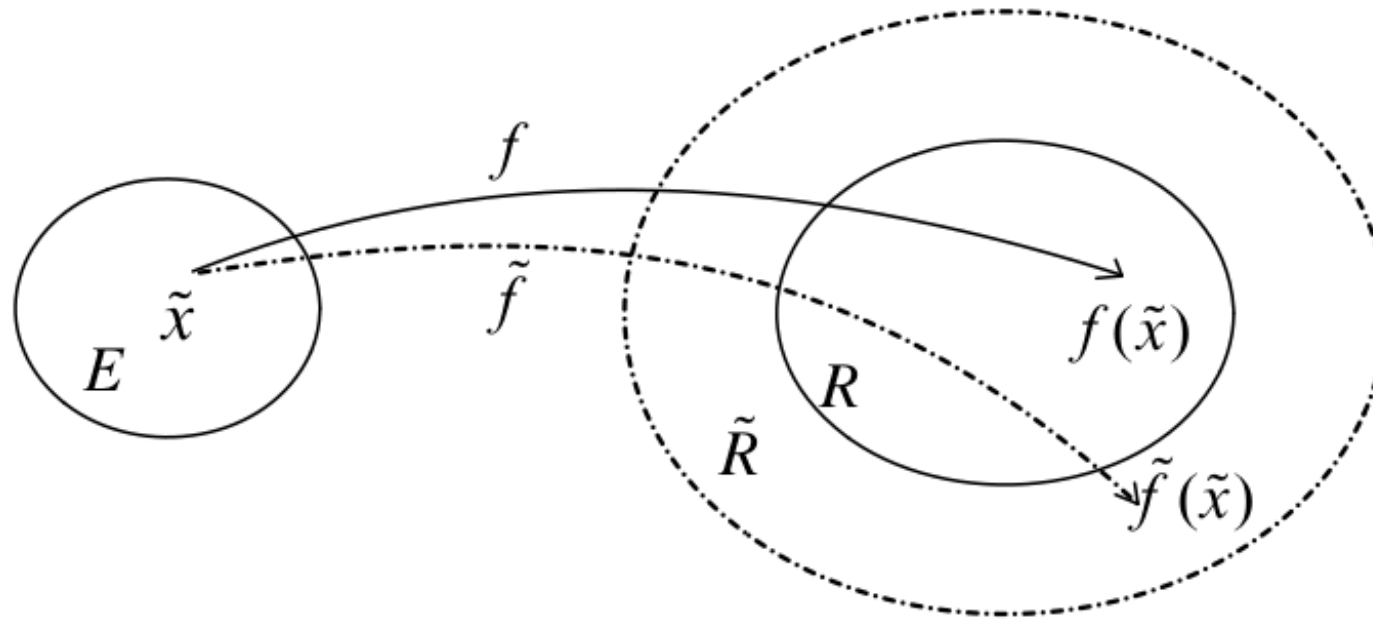
$$|f(\tilde{\mathbf{x}}) - f(\mathbf{x})| = |\mathbf{1}^\top (\tilde{\mathbf{x}} - \mathbf{x})| \leq \|\mathbf{1}\|_2 \|\tilde{\mathbf{x}} - \mathbf{x}\|_2 = \sqrt{2} \|\tilde{\mathbf{x}} - \mathbf{x}\|_2,$$

mit Gleichheit falls $(\tilde{\mathbf{x}} - \mathbf{x}) \parallel \mathbf{1}$, deshalb $\kappa_{abs} = \sqrt{2}$ und

$$\kappa_{rel} = \kappa_{abs} \frac{\|\mathbf{x}\|_2}{|x_1 + x_2|} = \sqrt{2} \frac{\|\mathbf{x}\|_2}{|x_1 + x_2|}.$$

- Für $0 < |x_1 + x_2| \ll \|\mathbf{x}\|_2$ ist $\kappa_{rel} \gg 1$ und das Problem somit schlecht konditioniert

Stabilität von Algorithmen



$$\begin{aligned}\text{Gesamtfehler} &= \tilde{f}(\tilde{x}) - f(x) \\ &= \underbrace{\tilde{f}(\tilde{x}) - f(\tilde{x})}_{\text{Fehler im Algorithmus}} + \underbrace{f(\tilde{x}) - f(x)}_{\text{Folgefehler aus Input}}\end{aligned}$$

- Analog zur Kondition können wir auch den algorithmischen Fehler $\tilde{f}(\tilde{x}) - f(\tilde{x})$ untersuchen.
- Ist er “klein”, nennen wir $\tilde{f}$ **stabil**.

Beispiel: Stochastic Gradient Descent

Optimierung der Verlustfunktion $L(\theta)$ mit $\theta_{k+1} = \theta_k - \alpha \hat{\nabla} L(\theta_k)$

Stabiler Algorithmus:

- Kleine Lernrate α (z.B. 0.001)
- (Numerische) Fehler akkumulieren langsam
- $\rightarrow$ Konvergiert zuverlässig zu einem Optimum

Instabiler Algorithmus:

- Zu große Lernrate α (z.B. 1.0)
- Verstärkt Rundung-/Schätzfehler des Gradienten $\hat{\nabla} L(\theta_k)$
- $\rightarrow$ Oszilliert oder divergiert

Das ist eine *Analogie* zum Konzept der **Stabilität** — hier mischen sich numerische Fehler und die Konvergenzdynamik des Verfahrens:

- Problem f : Finde Minimum von $L(\theta)$
- Algorithmus $\tilde{f}$: Stochastic Gradient Descent mit endlicher Präzision
- Algorithmischer Fehler: Akkumuliert Rundung-/Schätzfehler über k

Stabilität: Wichtigste Erkenntnis

Theorem: Stabilität zusammengesetzter Algorithmen

Sei f ein Problem mit Input $\mathbf{x}$, wobei $f(\mathbf{x}) = (h \circ g)(\mathbf{x})$ für zwei Teilprobleme g und h .
Dann gilt für einen Algorithmus $\tilde{f} = \tilde{h} \circ \tilde{g}$:

$$\frac{\|\tilde{f}(\mathbf{x}) - f(\mathbf{x})\|}{\|f(\mathbf{x})\|} \leq \frac{\|\tilde{h}(\tilde{\mathbf{y}}) - h(\tilde{\mathbf{y}})\|}{\|h(\mathbf{y})\|} + \kappa_{rel}(h, g(\mathbf{x})) \frac{\|\tilde{g}(\mathbf{x}) - g(\mathbf{x})\|}{\|g(\mathbf{x})\|},$$

wobei $\mathbf{y} = g(\mathbf{x})$, $\tilde{\mathbf{y}} = \tilde{g}(\mathbf{x})$.

- Ein schlecht konditioniertes Teilproblem h (an der Stelle $\mathbf{y} = g(\mathbf{x})$) verstärkt Fehler in $\tilde{g}$, wäre also ein *Indiz* für die Instabilität von $\tilde{f}$.
- Deshalb sollten wir **schlecht konditionierte Schritte immer so früh wie möglich** im Algorithmus ausführen.

Stabilität: Wichtigste Erkenntnis

Beweis:

$$\begin{aligned}\frac{\|\tilde{f}(\mathbf{x}) - f(\mathbf{x})\|}{\|f(\mathbf{x})\|} &= \frac{\|\tilde{h}(\tilde{g}(\mathbf{x})) - h(g(\mathbf{x}))\|}{\|h(g(\mathbf{x}))\|} \\&= \frac{\|\tilde{h}(\tilde{g}(\mathbf{x})) - h(\tilde{g}(\mathbf{x})) + h(\tilde{g}(\mathbf{x})) - h(g(\mathbf{x}))\|}{\|h(g(\mathbf{x}))\|} \\&\leq \frac{\|\tilde{h}(\tilde{g}(\mathbf{x})) - h(\tilde{g}(\mathbf{x}))\|}{\|h(g(\mathbf{x}))\|} + \frac{\|h(\tilde{g}(\mathbf{x})) - h(g(\mathbf{x}))\|}{\|h(g(\mathbf{x}))\|} \\&\leq \frac{\|\tilde{h}(\tilde{\mathbf{y}}) - h(\tilde{\mathbf{y}})\|}{\|h(\mathbf{y})\|} + \kappa_{rel}(h, g(\mathbf{x})) \frac{\|\tilde{g}(\mathbf{x}) - g(\mathbf{x})\|}{\|g(\mathbf{x})\|}.\end{aligned}$$

Fehler von $\tilde{h}$ selbst + durch h verstärkter Fehler von $\tilde{g}$ –

Idee: Einfügen von $0 = -h(\tilde{g}(\mathbf{x})) + h(\tilde{g}(\mathbf{x}))$, dann Dreiecksungleichung.

Beispiel: Stabilität der Varianz

Problem: Berechne Stichprobenvarianz

$$f(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2.$$

Algorithmen:

$$\tilde{f}_1(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2, \quad \tilde{f}_2(\mathbf{x}) = \left(\frac{1}{n} \sum_{i=1}^n x_i^2 \right) - \bar{x}^2$$

- Bei $\tilde{f}_2$ berechnen wir zum Schluss die Differenz $h(a, b) = a - b$ mit

$$a = \frac{1}{n} \sum_{i=1}^n x_i^2 \quad \text{und} \quad b = \bar{x}^2.$$

Beispiel: Stabilität der Varianz

- Die Konditionszahl ist dabei

$$\kappa_{rel}(h, (a, b)) = \sqrt{2} \frac{\sqrt{a^2 + b^2}}{|a - b|}.$$

- Falls a, b groß sind, aber $a \approx b$ gilt, ist $\kappa_{rel}(h, (a, b))$ groß.
- In diesem Fall kann h also vorangegangene Fehler massiv verstärken!

```
1 set.seed(5)
2 x <- rnorm(10^3, mean = 0, sd = 1) + 10^10
3 mean((x - mean(x))^2)
```

```
[1] 1.02
```

```
1 mean(x^2) - mean(x)^2
```

```
[1] 16384
```

Übersicht

- **Absoluter Fehler**: $\Delta_v = \tilde{v} - v$
- **Relativer Fehler**: $\delta_v = \frac{\|\tilde{v} - v\|}{\|v\|}$
- **Fehlerzerlegung**:
Gesamtfehler = **algorithmische Fehler** ($\tilde{f}(\tilde{x}) - f(\tilde{x})$) + **Input-Folgefehler** ($f(\tilde{x}) - f(x)$)
- **Kondition**: Sensitivität eines Problems bzgl. **Inputfehlern**
- **Konditionszahlen**: Absolut und relativ ($\kappa_{abs}, \kappa_{rel}$)
- **Konditionszahl einer Matrix**: $\kappa(\mathbf{A}) = \|\mathbf{A}\| \|\mathbf{A}^{-1}\|$
- **Stabilität**:
Ein Algorithmus ist stabil, wenn der **algorithmische Fehler** klein ist.
- **Stabilität zusammengesetzter Algorithmen**:
Fehlerfortpflanzung über Algorithmus-Teilschritte

Wrap-up

Heute gelernt

- Die *Kondition* eines Problems beschreibt dessen **Sensitivität** bzgl. Inputfehlern.
- Die *Stabilität* eines Algorithmus beschreibt dessen **Robustheit** bzgl. Fehler.
- **Schlecht konditionierte Teilschritte möglichst früh ausführen!**

Nächste Vorlesung

- Numerische Lösung von linearen Gleichungssystemen (LGS)

Gleichungssysteme und Matrixzerlegungen

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)

Material: Thomas Nagler

LMU München

Verwendete Vorkenntnisse

- **Algorithmen:** Komplexität, Algorithmustypen (direkte Methoden)
- **Fehleranalyse:** Stabilität, Kondition von Problemen
- Matrixnormen, Orthogonalmatrizen
- **Lineare Algebra I**
 - Lösung linearer Gleichungssysteme (grundlegende Methoden)
 - Dreiecksmatrizen, Symmetrische Matrizen, Positiv definite Matrizen
 - Matrixinverse (wir lernen, dass wir sie *NICHT* berechnen...!)

Opener-Quiz: Was wissen Sie noch?

Frage 1: Sei $\mathbf{U} \in \mathbb{R}^{n \times n}$ eine obere Dreiecksmatrix mit $u_{ii} \neq 0$. Mit welchem Aufwand lässt sich $\mathbf{U}\mathbf{x} = \mathbf{c}$ lösen?

1. $O(n)$
2. $O(n^2)$
3. $O(n^3)$
4. Gar nicht — dazu muss man erst $\mathbf{U}^{-1}$ berechnen.

Frage 2: Was bedeutet “ $\mathbf{A} \in \mathbb{R}^{n \times n}$ ist *positiv definit*”?

1. Alle Einträge von $\mathbf{A}$ sind positiv.
2. $\det(\mathbf{A}) > 0$.
3. $\mathbf{x}^\top \mathbf{A} \mathbf{x} > 0$ für alle $\mathbf{x} \neq \mathbf{0}$.
4. Alle Eigenwerte von $\mathbf{A}$ sind verschieden.

Numerische lineare Algebra

- Wir kümmern uns nun um numerische Methoden für die lineare Algebra:
 - Matrizenrechnung
 - Lineare Gleichungssysteme
 - Kleinste-Quadrate-Probleme
 - Eigenwerte
 - ...
- Dabei achten wir insbesondere auf **Kondition** und **Komplexität**.

Matrizenrechnung

Matrizenrechnung

- Die algorithmische Berechnung folgt für gewöhnlich den direkten mathematischen Formeln

$$\mathbf{A} + \mathbf{B} = \begin{pmatrix} a_{11} + b_{11} & \cdots & a_{1m} + b_{1m} \\ \vdots & & \vdots \\ a_{n1} + b_{n1} & \cdots & a_{nm} + b_{nm} \end{pmatrix},$$

$$\mathbf{AB} = \begin{pmatrix} \sum_{i=1}^m a_{1i}b_{i1} & \cdots & \sum_{i=1}^m a_{1i}b_{ik} \\ \vdots & & \vdots \\ \sum_{i=1}^m a_{ni}b_{i1} & \cdots & \sum_{i=1}^m a_{ni}b_{ik} \end{pmatrix}.$$

Quiz

Was ist die algorithmische Komplexität für die Addition von $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times m}$?

1. $O(nm)$
2. $O(n + m)$
3. $O(n^2m^2)$

Quiz

Was ist die algorithmische Komplexität für die Multiplikation von $\mathbf{A} \in \mathbb{R}^{n \times m}$, $\mathbf{B} \in \mathbb{R}^{m \times k}$?

1. $O(nk)$
2. $O(nmk)$
3. $O(nm + k)$

Matrixmultiplikation

- Für die Multiplikation von zwei $(n \times n)$ Matrizen gibt es Algorithmen mit besserer Komplexität.
- Für $A, B \in \mathbb{R}^{n \times n}$:
 - Strassen (1969): $\approx O(n^{2.8})$
 - Alman et al. (2024): $\approx O(n^{2.372})$ (bester bekannter Exponent)



Quelle: Titelbild *Nature* **610** (6. Oktober 2022) zu AlphaTensor (Fawzi et al., 2022).

- Die Konstante, die hinter $O(\cdot)$ versteckt wird, ist aber meist so groß, dass die Algorithmen kaum verwendet werden.
- Wir rechnen der Einfachheit halber immer mit $O(nmk)$.

Lineare Gleichungssysteme

Lineare Gleichungssysteme

- Wir beschäftigen uns nun mit der Lösung eines LGS:

$$\boldsymbol{x} : \boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$$

- Das ist nur sinnvoll, wenn es (genau) eine Lösung gibt. Also nehmen wir an, dass $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ invertierbar ist.
- Die Lösung kann als $\boldsymbol{x} = \boldsymbol{A}^{-1}\boldsymbol{b}$ geschrieben werden.
- Das numerische Invertieren einer Matrix ist aber
 - relativ instabil und
 - oft langsamer als direktes Lösen des LGS.

Deshalb:

Invertiere niemals eine Matrix!

Demo:

- LGS mit schlecht konditionierter Matrix mit $\kappa_2(\boldsymbol{A}) \approx 10^{14}$
- wahre Lösung $\boldsymbol{x} = (1, \dots, 1)^\top$:

```
1 n <- 11 # Hilbert-Matrix:
2 A <- outer(1:n, 1:n,
3           function(i, j) 1 / (i + j - 1))
4 x <- rep(1, n)
5 b <- A %*% x
6
7 range((solve(A) %*% b) - x) # Fehler über Inverse
8 range(solve(A, b) - x)      # Fehler direkte Lösung
```

```
[1] -0.195  0.288
[1] -0.000260  0.000332
```

$\implies$ maximaler Fehler über $\boldsymbol{A}^{-1}$: ≈ 0.3 ;
bei direktem Lösen: $\approx 3 \cdot 10^{-4}$
— ca. **1000× genauer** (und schneller).

Lineare Gleichungssysteme

Wir erinnern uns an das Gauss'sche Eliminationsverfahren:

1. Bringe $(\mathbf{A} \mid \mathbf{b})$ in Zeilenstufenform $(\mathbf{U} \mid \mathbf{c})$ mit oberer Dreiecksmatrix $\mathbf{U}$.
2. Löse das System

$$\begin{array}{ccccccccc} u_{11}x_1 & + & u_{12}x_2 & + & \dots & + & u_{1n}x_n & = & c_1 \\ & & u_{22}x_2 & + & \dots & + & u_{2n}x_n & = & c_2 \\ & & & & \ddots & & \vdots & & \vdots \\ & & & & & & u_{nn}x_n & = & c_n \end{array}$$

durch Rückwärtssubstitution

$$x_n = c_n/u_{nn}, \quad x_i = \left(c_i - \sum_{j=i+1}^n u_{ij}x_j \right) / u_{ii} \quad (i \leq n-1).$$

Quiz

Was ist die algorithmische Komplexität der Rückwärtssubstitution?

1. $O(1)$
2. $O(n)$
3. $O(n^2)$
4. $O(n^3)$

Warum Matrixzerlegungen?

Grundidee: “Teile und herrsche” für Matrizen

Analogie: Primfaktorzerlegung von Zahlen

- $84 = 2^2 \cdot 3 \cdot 7$
- Vereinfacht Rechnung: $84 \cdot 126 = (2^2 \cdot 3 \cdot 7) \cdot (2 \cdot 3^2 \cdot 7) = 2^3 \cdot 3^3 \cdot 7^2$

Für Matrizen: $A = BC$ mit “einfachen” B, C

- **Dreiecksmatrizen:** Lösen in $O(n^2)$ statt $O(n^3)$
- **Orthogonale Matrizen:** Numerisch stabil, einfach zu invertieren
- **Diagonalmatrizen:** Triviale Operationen

Vorteile von Zerlegungen:

1. Effiziente Lösung mehrerer LGS mit gleicher Matrix
2. Numerische Stabilität verbessern
3. Struktur und Eigenschaften von A erkennen
4. Grundlage für viele weitere Algorithmen (Spektralzerlegung & SVD, ...)

Die LU-Zerlegung

Intuition: LU-Zerlegung

Idee: Die Gauss-Elimination zerlegt A in:

- L (Lower): speichert alle Eliminationsschritte
- U (Upper): die resultierende obere Dreiecksmatrix

Vorteil: Sobald wir $A = LU$ einmal berechnet haben, können wir $Ax = b_i$ für jedes b_i effizient lösen:

$$Ax = b \implies LUx = b \implies \begin{array}{l} 1. \text{ Löse } Ly = b \\ 2. \text{ Löse } Ux = y \end{array}$$

Die LU-Zerlegung

- Im ersten Schritt des Eliminationsverfahrens bringen wir $\mathbf{A} \in \mathbb{R}^{n \times n}$ in Zeilenstufenform.
- Dazu gehen wir von links nach rechts durch die Spalten und entfernen alle Einträge unter der Diagonalen.
- Ist das aktuelle System $(\mathbf{M} \mid \mathbf{r})$, erreichen wir das für die k -te Spalte durch die Multiplikation einer Matrix $\mathbf{L}_k$ von links, wobei

$$\mathbf{L}_k = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -l_{(k+1)k} & 1 & \\ & & \vdots & & \ddots \\ & & -l_{nk} & & & 1 \end{pmatrix}$$

mit $l_{ik} = m_{ik}/m_{kk}$

Die LU-Zerlegung

- Insgesamt ist also $(U \mid c)$ gegeben durch $U = L_{n-1} \cdots L_1 A$ und $c = L_{n-1} \cdots L_1 b$.
- Es gilt dann $A = LU$, wobei $L = L_1^{-1} \cdots L_{n-1}^{-1}$.

Die Inverse zu L_k macht deren Operation rückgängig (Vorzeichen kippen):

Jedes L_k^{-1} ist eine untere Dreiecksmatrix, also auch $L = L_1^{-1} \cdots L_{n-1}^{-1}$ – genauer:

$$L_k^{-1} = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & l_{(k+1)k} & 1 & \\ & & \vdots & & \ddots \\ & & l_{nk} & & & 1 \end{pmatrix}$$

$$L = \begin{pmatrix} 1 & & & & \\ l_{21} & \ddots & & & \\ l_{31} & l_{32} & 1 & & \\ \vdots & & \ddots & \ddots & \\ l_{n1} & \cdots & \cdots & l_{n(n-1)} & 1 \end{pmatrix}$$

$\implies L$ sammelt einfach alle Eliminationskoeffizienten l_{ik} ein.

Beispiel: LU-Zerlegung für 3×3 -Matrix

$$\text{Sei } A = \begin{pmatrix} 2 & 1 & -1 \\ 4 & -6 & 0 \\ -2 & 7 & 2 \end{pmatrix}.$$

Farben: zu eliminieren, Pivot, Eliminationskoeffizienten

Schritt 1: Eliminiere Spalte 1 mit L_1 :

$$l_{21} = 4/2 = 2, \quad l_{31} = -2/2 = -1$$

$$L_1 = \begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ -(-1) & 0 & 1 \end{pmatrix},$$

$$L_1 A = \begin{pmatrix} 2 & 1 & -1 \\ 0 & -8 & 2 \\ 0 & 8 & 1 \end{pmatrix}$$

Schritt 2: Eliminiere Spalte 2 mit L_2 :

$$l_{32} = 8/(-8) = -1$$

$$L_2 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -(-1) & 1 \end{pmatrix},$$

$$L_2 L_1 A = \begin{pmatrix} 2 & 1 & -1 \\ 0 & -8 & 2 \\ 0 & 0 & 3 \end{pmatrix} = U$$

Ergebnis: $A = LU$ mit

$$L = L_1^{-1} L_2^{-1} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & -1 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 2 & 1 & -1 \\ 0 & -8 & 2 \\ 0 & 0 & 3 \end{pmatrix}$$

Die LU-Zerlegung zur Lösung von LGS

Theorem: LU-Zerlegung

Jede invertierbare Matrix A hat – ggf. erst nach Zeilenvertauschungen $A \rightarrow PA$ mit einer Permutationsmatrix P ($\rightarrow$ *Pivotierung*, s.u.) – eine Darstellung

$$A = LU \quad \text{bzw.} \quad PA = LU$$

mit

- einer unteren Dreiecksmatrix L ,
 - einer oberen Dreiecksmatrix U .
-
- Eine Zerlegung ohne Pivotierung (ohne Zeilenvertauschungen, also mit $P = I$) erhalten wir genau dann, wenn bei der Gauß-Elimination kein Pivotelement null wird. Wird ein Pivot null, müssen wir vorher Zeilen vertauschen.
 - Da L und U Dreiecksmatrizen sind, können wir das LGS $Ax = b$ bzw. $LUx = b$ jetzt leicht lösen:
 1. Löse $Ly = b$ (bzw. $Ly = Pb$) durch Vorwärtssubstitution.
 2. Löse $Ux = y$ durch Rückwärtssubstitution.
 - **Beachte:** Die Zerlegung $A = LU$ ist für alle b gleich. Wir müssen sie also nur einmal ausführen.

Beispiel: Lösen mit LU-Zerlegung (Fortsetzung)

Mit L, U von vorher und $b = \begin{pmatrix} 5 \\ -2 \\ 9 \end{pmatrix}$.

Schritt 1: Vorwärtssubstitution für $Ly = b$:

$$\begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & -1 & 1 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} 5 \\ -2 \\ 9 \end{pmatrix} \implies \begin{array}{lcl} y_1 & = & 5 \\ y_2 & = & -2 - 2 \cdot 5 = -12 \\ y_3 & = & 9 - (-1) \cdot 5 - (-1) \cdot (-12) = 9 + 5 - 12 = 2 \end{array}$$

Schritt 2: Rückwärtssubstitution für $Ux = y$:

$$\begin{pmatrix} 2 & 1 & -1 \\ 0 & -8 & 2 \\ 0 & 0 & 3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 5 \\ -12 \\ 2 \end{pmatrix} \implies \begin{array}{lcl} x_3 & = & 2/3 \\ x_2 & = & (-12 - 2 \cdot (2/3))/(-8) = 5/3 \\ x_1 & = & (5 - 1 \cdot (5/3) - (-1) \cdot (2/3))/2 = 2 \end{array}$$

Pivotierte LU-Zerlegung

- Die Methode scheitert, falls im k -ten Schritt $m_{kk} = 0$.
- Das lässt sich vermeiden, indem wir Zeilen von $\mathbf{A}$ tauschen: → **Pivotierung**
 - löst $\mathbf{PA} = \mathbf{LU}$ mit Permutationsmatrix $\mathbf{P}$
- *pivoting* verbessert Stabilität auch schon für $m_{kk} \approx 0$.

Quiz

Im k -ten Schritt zum Finden von $\mathbf{L}$ und $\mathbf{U}$ multiplizieren wir

$$\mathbf{M} = \begin{pmatrix} * & \dots & * & \dots & * \\ & \ddots & \vdots & \ddots & \vdots \\ & & * & \dots & * \\ & & \vdots & \ddots & \vdots \\ & & * & \dots & * \end{pmatrix} \text{ mit } \mathbf{L}_k = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & -l_{(k+1)k} & & 1 & \\ & \vdots & & & \ddots \\ & -l_{nk} & & & & 1 \end{pmatrix}.$$

Was ist die Komplexität dieses Schritts?

1. $O(n^2)$
2. $O(n^3)$
3. $O((n - k)^2)$
4. $O(k^3)$

Komplexität

- Die Berechnung der LU-Zerlegung hat also die Komplexität

$$\sum_{k=1}^{n-1} O((n-k)^2) = \sum_{k=1}^{n-1} O(n^2) = O(n^3).$$

- Das müssen wir nur einmal ausführen und können damit mehrere LGS $\mathbf{Ax} = \mathbf{b}_j, j = 1, \dots, J$, lösen.
- Für Vorwärts- und Rückwärtssubstitution brauchen wir je $O(n^2)$.
- Insgesamt ist die Laufzeit also $O(n^3 + Jn^2)$.
- Die Laufzeit kann verbessert werden, wenn $\mathbf{A}$ besondere Eigenschaften erfüllt, z.B.:
 - symmetrisch: $\mathbf{A} = \mathbf{A}^\top$,
 - positiv definit: $\mathbf{x}^\top \mathbf{Ax} > 0$ für alle $\mathbf{x} \neq \mathbf{0}$,
 - Bandeigenschaft: $a_{ij} = 0$ für alle $|i - j| > \beta$,
 - dünn besetzt (engl.: *sparse*): $a_{ij} = 0$ für die allermeisten (i, j) .
- Für all das gibt es hervorragende Implementierungen. Man schreibt diese Algorithmen selten selbst.

Cholesky-Zerlegung

SPD-Matrizen

Ein Spezialfall ist in der Statistik besonders wichtig:

SPD-Matrizen sind

- **s**ymmetrisch und
- **p**ositiv **d**efinit.

Def.: SPD-Matrix

Eine Matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ heißt **SPD**, wenn

- $\mathbf{A} = \mathbf{A}^\top$ (*symmetrisch*) und
 - $\mathbf{x}^\top \mathbf{A} \mathbf{x} > 0 \ \forall \ \mathbf{x} \in \mathbb{R}^n \setminus \{\mathbf{0}\}$ (*positiv-definit*).
-
- Symmetrisches $\mathbf{A}$ ist **positiv-semidefinit** (PSD) $\iff \mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0 \ \forall \ \mathbf{x} \in \mathbb{R}^n$

SPD- und PSD-Matrizen in Statistik und ML

1. Kovarianzmatrizen: $\Sigma = \mathbb{E}[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top]$

sind immer PSD, und SPD, wenn $\mathbf{x}$ nicht auf einer Untermenge von $\mathbb{R}^p$ mit Dimension $< p$ liegt.

- Multivariate Normalverteilungen $\sim \mathcal{N}_p(\boldsymbol{\mu}, \Sigma)$, Hauptkomponentenanalyse (PCA)

2. Gram-Matrizen: $\mathbf{K} = \mathbf{X}^\top \mathbf{X}$

- Normalgleichungen $\mathbf{X}^\top \mathbf{X} \boldsymbol{\beta} = \mathbf{X}^\top \mathbf{y}$ für **Kleinste-Quadrate**-Probleme; Kernel-Matrizen (SVMs, Gauss-Prozesse, ...)

3. Hesse-Matrizen: $\mathbf{H} = \left[\frac{\partial^2}{\partial x_i \partial x_j} f(\mathbf{x}) \right]_{i,j}$

- Fisher-Information; **Konvexität** & **Newton-Verfahren** in der Optimierung

Warum so wichtig?

SPD-Matrizen garantieren:

- Existenz und — bei positiver Diagonale von $\mathbf{L}$ — Eindeutigkeit der Cholesky-Zerlegung
- Numerische Stabilität der Cholesky-Zerlegung (auch ohne Pivotierung)
- Alle Eigenwerte sind (reell und) positiv $\implies$ voller Rang

Cholesky-Zerlegung

- Für SPD-Matrizen kann man $\mathbf{L}$ und $\mathbf{U}$ so wählen, dass $\mathbf{U} = \mathbf{L}^\top$.

Theorem: Cholesky-Zerlegung

Jede SPD-Matrix $\mathbf{A}$ kann in $\mathbf{A} = \mathbf{L}\mathbf{L}^\top$ zerlegt werden, wobei $\mathbf{L}$ eine untere Dreiecksmatrix mit positiver Diagonale ($l_{ii} > 0$) ist. Diese Zerlegung ist eindeutig.

- Der Rechen- und Speicheraufwand ist hier etwa halb so groß wie für die volle LU-Zerlegung.

(Beweis per Induktion $\rightarrow$ *Anhang*)

Beispiel: Cholesky-Zerlegung

Berechne Cholesky-Zerlegung von $\mathbf{A} = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$.

Ansatz: $\mathbf{A} = \mathbf{L}\mathbf{L}^\top$ mit $\mathbf{L} = \begin{pmatrix} l_{11} & 0 \\ l_{21} & l_{22} \end{pmatrix}$

Ausmultiplizieren:

$$\mathbf{L}\mathbf{L}^\top = \begin{pmatrix} l_{11} & 0 \\ l_{21} & l_{22} \end{pmatrix} \begin{pmatrix} l_{11} & l_{21} \\ 0 & l_{22} \end{pmatrix} = \begin{pmatrix} l_{11}^2 & l_{11}l_{21} \\ l_{11}l_{21} & l_{21}^2 + l_{22}^2 \end{pmatrix} = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$$

Koeffizientenvergleich:

$$\begin{aligned} l_{11}^2 &= 4 & \implies & l_{11} = 2 \\ l_{11}l_{21} &= 2 & \implies & l_{21} = 2/2 = 1 \\ l_{21}^2 + l_{22}^2 &= 3 & \implies & l_{22} = \sqrt{3-1} = \sqrt{2} \end{aligned}$$

Ergebnis:

$$\mathbf{L} = \begin{pmatrix} 2 & 0 \\ 1 & \sqrt{2} \end{pmatrix} \implies \mathbf{L}\mathbf{L}^\top = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$$

Cholesky-Zerlegung

Beispielanwendung:

- Sei $\mathbf{x} \in \mathbb{R}^d$ ein Zufallsvektor mit $\mathbb{E}[\mathbf{x}] = \mathbf{0}$ und $\text{Var}(\mathbf{x}) = \mathbf{I}_d = \mathbb{E}[\mathbf{x}\mathbf{x}^\top]$.
- Sei $\mathbf{\Sigma} \in \mathbb{R}^{d \times d}$ eine SPD Matrix und $\mathbf{\Sigma} = \mathbf{L}\mathbf{L}^\top$.
- Dann hat $\mathbf{y} = \mathbf{L}\mathbf{x}$ die Kovarianz

$$\text{Var}(\mathbf{y}) = \mathbb{E}[\mathbf{y}\mathbf{y}^\top] = \mathbb{E}[\mathbf{L}\mathbf{x}\mathbf{x}^\top\mathbf{L}^\top] = \mathbf{L}\mathbb{E}[\mathbf{x}\mathbf{x}^\top]\mathbf{L}^\top = \mathbf{L}\mathbf{L}^\top = \mathbf{\Sigma}.$$

(das **mittlere Stück** fällt weg: $\mathbb{E}[\mathbf{x}\mathbf{x}^\top] = \mathbf{I}_d$ nach Voraussetzung)

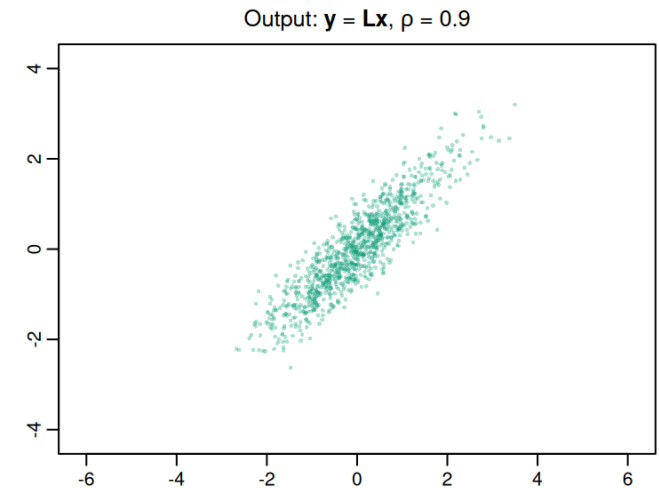
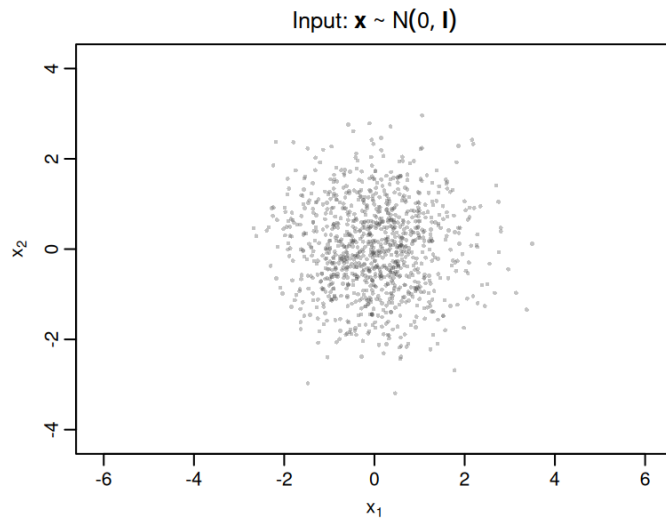
- Dies wird z.B. verwendet, um aus $\mathcal{N}(\boldsymbol{\mu}, \mathbf{\Sigma})$ zu simulieren:
für $\mathbf{x} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$ ist $\boldsymbol{\mu} + \mathbf{L}\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \mathbf{\Sigma})$.

Demo: Aus $\mathcal{N}(\mathbf{0}, \Sigma)$ simulieren

```
1 set.seed(7)
2 Sigma <- matrix(c(1, 0.9,
3 0.9, 1), 2, 2)
4 # !! chol() liefert die obere Dreiecksmatrix R mit Sigma = R'R !!
5 L <- t(chol(Sigma))
6 x <- matrix(rnorm(2 * 1000), nrow = 2)
7 y <- L %*% x
8 round(cov(t(y)), 2)
```

```
      [,1] [,2]
[1,] 1.00 0.89
[2,] 0.89 0.99
```

$$\mathbf{L} = \begin{pmatrix} 1 & 0 \\ 0.9 & \sqrt{1 - 0.9^2} \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0.9 & 0.436 \end{pmatrix}, \quad \Sigma = \mathbf{L}\mathbf{L}^\top.$$



$\implies \mathbf{L}$ verformt Gitter und Punktwolke auf dieselbe Weise; die empirische Kovarianz von $\mathbf{y}$ reproduziert Σ .

Vergleich: LU und Cholesky

LU-Zerlegung

- **Ohne Pivotierung instabil** bei kleinen Pivotelementen
 - Division durch sehr kleine Zahlen führt zu numerischer Instabilität
- **Mit partieller Pivotierung** praktisch stabil (Higham, 2020)
- **Kondition:** Fehler $\approx \kappa(\mathbf{A}) \cdot \epsilon_{\text{Maschine}}$

Cholesky-Zerlegung

- **Natürlich stabil** ohne Pivotierung für **SPD-Matrizen**
- **Effizienz:** $\approx 2\times$ schneller als LU (halber Aufwand)
- **Robustheit:** In exakter Arithmetik bricht das Verfahren für **jede** nicht positiv definite Matrix ab (Wurzel aus einer nicht-positiven Zahl); bei echten SPD-Matrizen passiert das nur durch Rundungsfehler und nur bei **extrem** schlechter Kondition
 - Der Abbruch ist ein nützlicher *Diagnosehinweis*: $\mathbf{A}$ ist nicht SPD oder numerisch grenzwertig
- **Pivotierte Variante:** Anwendung auch für *semi*-positiv-definite Matrizen! (Details → **Anhang**)

Fazit

- **LU:** Universell einsetzbar, benötigt Pivotierung für Stabilität
- **Cholesky:** optimal für **SPD-Matrizen** (Statistik!), numerisch überlegen

Self-Check & Zusammenfassung

Wrap-up

Heute gelernt

- **Invertiere niemals eine Matrix!**
- *LU*- und *Cholesky*-Zerlegungen

Nächste Vorlesung

- Singulärwertzerlegung (*SVD*)

Self-Check

Frage 1: Sie müssen viele LGS $\mathbf{Ax} = \mathbf{b}_j$ mit derselben SPD-Matrix $\mathbf{A}$ lösen. Bestes Vorgehen?

1. Für jedes $\mathbf{b}_j$ die Gauss-Elimination komplett neu durchführen.
2. Einmal $\mathbf{A}^{-1}$ berechnen, dann jeweils $\mathbf{A}^{-1}\mathbf{b}_j$ multiplizieren.
3. Einmal Cholesky $\mathbf{A} = \mathbf{LL}^\top$, dann je zweimal Substitution.
4. Einmal pivotierte LU, da sie für SPD-Matrizen stabiler als Cholesky ist.

Frage 2: Wann braucht die LU-Zerlegung Pivotierung?

1. Nur wenn $\mathbf{A}$ singulär ist.
2. Wenn ein Pivotelement m_{kk} null oder nahe null ist.
3. Nie — sie verbessert nur die Laufzeit.
4. Nur für symmetrische Matrizen.

Frage 3: $\mathbf{A} = \mathbf{LU}$ liegt bereits vor. Kosten, um ein LGS $\mathbf{Ax} = \mathbf{b}$ zu lösen?

1. $O(n)$
2. $O(n^2)$
3. $O(n^3)$
4. $O(n^2 + n^3)$

Anhang

Cholesky-Zerlegung (Induktionsbeweis)

- $n = 1$: Weil $A \in \mathbb{R}$ SPD ist, gilt $A > 0$. Dann ist $A = LL^\top$ mit $L = \sqrt{A}$.

- $n > 1$:

- Sei $A = \begin{pmatrix} a & \mathbf{c}^\top \\ \mathbf{c} & B \end{pmatrix}$ und die Hypothese gelte für $n - 1$.

- Setze $\mathbf{l} = \frac{\mathbf{c}}{\sqrt{a}}$. (A SPD $\implies a > 0$, da $a = \mathbf{e}_1^\top A \mathbf{e}_1$.)

- Außerdem ist für jedes $\mathbf{x} \in \mathbb{R}^{n-1} \setminus \{\mathbf{0}\}$:

$$\mathbf{x}^\top (B - \mathbf{u}^\top) \mathbf{x} = \begin{pmatrix} -\frac{\mathbf{c}^\top \mathbf{x}}{a} \\ \mathbf{x} \end{pmatrix}^\top A \begin{pmatrix} -\frac{\mathbf{c}^\top \mathbf{x}}{a} \\ \mathbf{x} \end{pmatrix} > 0$$

also ist $(B - \mathbf{u}^\top)$ SPD und $(B - \mathbf{u}^\top) = \tilde{L}\tilde{L}^\top$ (Induktionshypothese).

- Wähle nun

$$L = \begin{pmatrix} \sqrt{a} & \mathbf{0}^\top \\ \mathbf{l} & \tilde{L} \end{pmatrix} \implies LL^\top = \begin{pmatrix} a & \mathbf{c}^\top \\ \mathbf{c} & \mathbf{u}^\top + \tilde{L}\tilde{L}^\top \end{pmatrix} = A \quad \checkmark$$

- Eindeutigkeit: $l_{11} = \sqrt{a}$ und $\mathbf{l} = \mathbf{c}/\sqrt{a}$ sind erzwungen, $\tilde{L}$ ist eindeutig nach Induktionsannahme.

↔ zurück: „Cholesky-Zerlegung“

Cholesky mit Pivotierung

- Notwendig falls $\mathbf{A}$ singular oder numerisch rangdefizient
 - für positiv **semi**-definite Matrizen ($\mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0$ statt > 0):
- **Complete pivoting:** $\mathbf{P}^\top \mathbf{A} \mathbf{P} = \mathbf{L} \mathbf{L}^\top$
 - Permutationsmatrix $\mathbf{P}$ (zeilen- und spaltenvertauschend)
 - **Rang-aufdeckend:** Nullspalten in $\mathbf{L}$ zeigen Rangabfall von $\mathbf{A}$
- **Stabilität:** Auch für semidefinite Matrizen stabil (Higham, 1990)
- **Anwendungen** in ML/Statistik:
 - Kernel-Matrizen (Gauss-Prozesse, SVMs, ...)
 - Niedrig-rangige approximative Matrixzerlegung: $O(k^2 n)$ statt $O(n^3)$
 - Gram-Matrizen $\mathbf{X}^\top \mathbf{X}$ bei $p > n$ (mehr Features als Daten)

↔ zurück: „Vergleich: LU und Cholesky“

Singulärwertzerlegung (SVD)

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)

Material: Thomas Nagler

LMU München

Verwendete Vorkenntnisse

- **Matrixnormen** und ihre Eigenschaften, speziell:
 - **Spektralnorm** $\|\mathbf{A}\|_2 = \sqrt{\lambda_{\max}(\mathbf{A}^\top \mathbf{A})}$
 - **Orthogonalmatrizen** $\mathbf{Q}$ mit $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$
- Matrixzerlegungskonzepte
- **Lineare Algebra**
 - Eigenwerte und Eigenvektoren: $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$
 - Rang einer Matrix
 - Fundamentale Unterräume: $\text{col}(\mathbf{A})$, $\text{ker}(\mathbf{A})$, $\text{col}(\mathbf{A}^\top)$
 - Projektionen auf Unterräume

Opener-Quiz: Was wissen Sie noch?

Frage 1: Sei $\mathbf{A} \in \mathbb{R}^{n \times n}$ symmetrisch ($\mathbf{A} = \mathbf{A}^\top$). Welche Aussage gilt **immer**?

1. Alle Eigenwerte von $\mathbf{A}$ sind ≥ 0 .
2. $\mathbf{A}$ ist orthogonal diagonalisierbar: $\mathbf{A} = \mathbf{Q}\mathbf{D}\mathbf{Q}^\top$ mit reellen Eigenwerten.
3. $\mathbf{A}$ ist invertierbar.
4. $\mathbf{A}$ hat n verschiedene Eigenwerte.

Frage 2: Was ist der Rang von $\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 0 & 0 \end{pmatrix}$?

1. 0
2. 1
3. 2
4. 3

Motivation

Diagonalisierung ist limitiert

- Eigenwertzerlegung funktioniert nur für **quadratische** Matrizen – und auch dort nicht immer:
 $\begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ hat nur den Eigenwert 0 mit eindimensionalem Eigenraum, also keine Eigenvektorbasis
- Orthogonale Diagonalisierung bisher nur für **symmetrische** Matrizen
- **Problem:**
Die meisten Matrizen sind weder quadratisch noch symmetrisch.

Was wir wollen: Eine Zerlegung $A = PDQ^{-1}$

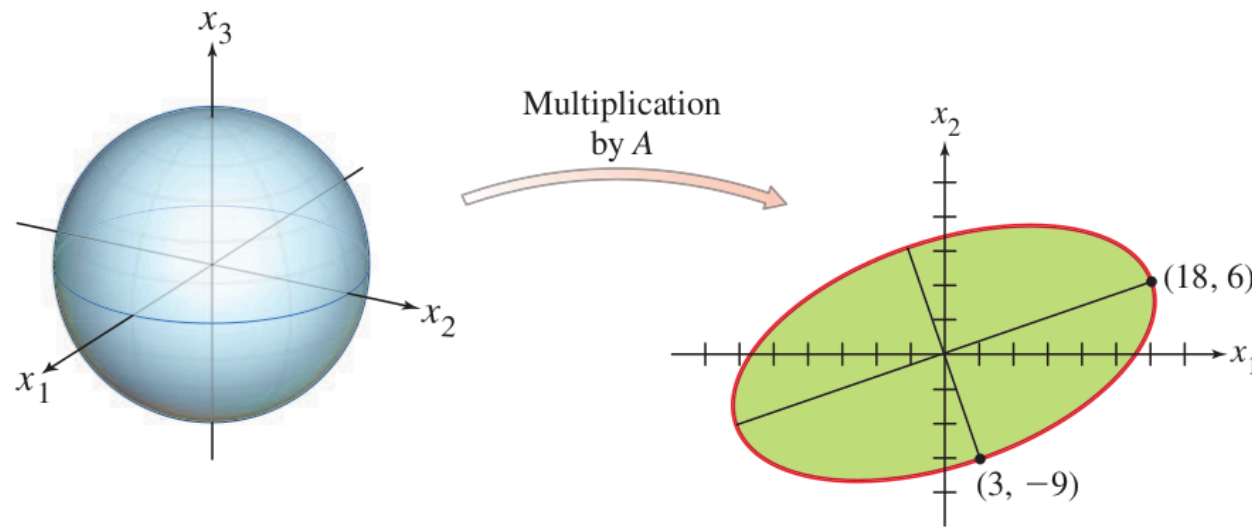
- für *beliebige* $m \times n$ Matrizen, so dass
- D “diagonal” ist und
- P und Q **Orthogonalmatrizen** sind

Geometrische Intuition

Die Einheitssphäre als Testfall:

- Betrachte alle Vektoren $\mathbf{x}$ mit $\|\mathbf{x}\| = 1$
- Nach Transformation: $\mathbf{A}\mathbf{x}$ hat verschiedene Längen
- **Frage:** Welche Richtung $\mathbf{A}\mathbf{x}$ wurde am stärksten (zweitstärksten, drittstärksten, ...) gestreckt?

Beispiel: $\mathbf{A} \in \mathbb{R}^{2 \times 3}$



- Einheitssphäre in $\mathbb{R}^3 \rightarrow$ ausgefüllte Ellipse in $\mathbb{R}^2$; stärkste Streckung: Hauptachse, zweitstärkste: Nebenachse; Richtungen in $\ker(\mathbf{A})$ werden auf $\mathbf{0}$ gestaucht

► Interaktive Visualisierung im Skript

Die Grundidee

Maximierungsproblem:

$$\max_{\|x\|=1} \|Ax\|$$

Trick: Maximiere $\|Ax\|^2$ statt $\|Ax\|$

$$\|Ax\|^2 = (Ax)^\top (Ax) = x^\top (A^\top A)x$$

Erkenntnis: Dies ist eine quadratische Form bezüglich $A^\top A$!

Im Folgenden immer: $\|\cdot\| \equiv \|\cdot\|_2$!

SVD: Singulärwerte und Singulärvektoren

Die Matrix $\mathbf{A}^\top \mathbf{A}$

Theorem: Eigenschaften von $\mathbf{A}^\top \mathbf{A}$

Für $\mathbf{A} \in \mathbb{R}^{m \times n}$ gilt:

1. $\mathbf{A}^\top \mathbf{A}$ ist symmetrisch: $(\mathbf{A}^\top \mathbf{A})^\top = \mathbf{A}^\top \mathbf{A}$
2. $\mathbf{A}^\top \mathbf{A}$ ist positiv semidefinit: $\mathbf{x}^\top (\mathbf{A}^\top \mathbf{A}) \mathbf{x} \geq 0$ für alle $\mathbf{x} \in \mathbb{R}^n$; insbesondere sind alle Eigenwerte $\lambda_i \geq 0$

Beweis für (2): $\mathbf{x}^\top (\mathbf{A}^\top \mathbf{A}) \mathbf{x} = \|\mathbf{Ax}\|_2^2 \geq 0$; für jeden normierten Eigenvektor $\mathbf{v}_i$ ($\|\mathbf{v}_i\| = 1$) gilt damit:

$$0 \leq \|\mathbf{Av}_i\|_2^2 = \mathbf{v}_i^\top (\mathbf{A}^\top \mathbf{A}) \mathbf{v}_i = \mathbf{v}_i^\top (\lambda_i \mathbf{v}_i) = \lambda_i$$

Folgerung: $\mathbf{A}^\top \mathbf{A}$ ist orthogonal diagonalisierbar!

Bsp: Eigenschaften von $\mathbf{A}^\top \mathbf{A}$

Sei $\mathbf{A} = (\mathbf{a}_1 \quad \mathbf{a}_2) = \begin{pmatrix} 1 & 2 \\ 2 & 1 \\ 1 & 0 \end{pmatrix} \in \mathbb{R}^{3 \times 2}$

$$\begin{aligned} \mathbf{A}^\top \mathbf{A} &= \begin{pmatrix} \mathbf{a}_1^\top \\ \mathbf{a}_2^\top \end{pmatrix} (\mathbf{a}_1 \quad \mathbf{a}_2) = \begin{pmatrix} \mathbf{a}_1^\top \mathbf{a}_1 & \mathbf{a}_1^\top \mathbf{a}_2 \\ \mathbf{a}_2^\top \mathbf{a}_1 & \mathbf{a}_2^\top \mathbf{a}_2 \end{pmatrix} \\ &= \begin{pmatrix} 1 & 2 & 1 \\ 2 & 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 2 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 6 & 4 \\ 4 & 5 \end{pmatrix} \end{aligned}$$

Symmetrie: ✓

Positiv semidefinit:

$$\det(\mathbf{A}^\top \mathbf{A} - \lambda \mathbf{I}) = (6 - \lambda)(5 - \lambda) - 16 = \lambda^2 - 11\lambda + 14 = 0$$

$$\lambda_{1,2} = \frac{11 \pm \sqrt{11^2 - 4 \cdot 1 \cdot 14}}{2} = \frac{11 \pm \sqrt{65}}{2}$$

$$\implies \lambda_1 \approx 9.53, \quad \lambda_2 \approx 1.47$$

$\lambda_1, \lambda_2 > 0 \implies \mathbf{A}^\top \mathbf{A}$ hier sogar positiv *definit* – mehr als das vom Theorem garantierte “semidefinit”, da $\mathbf{A}$ vollen Spaltenrang hat ($\text{rang}(\mathbf{A}) = n = 2$)

Definition: Singulärwerte

Seien $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$ die Eigenwerte von $\mathbf{A}^\top \mathbf{A}$.

Def.: Singulärwerte

Die **Singulärwerte** von $\mathbf{A}$ sind:

$$\sigma_i = \sqrt{\lambda_i}, \quad i = 1, 2, \dots, n$$

Interpretation:

- $\sigma_1 = \max_{\|x\|=1} \|\mathbf{A}x\|$ (maximale Streckung)
- σ_n entspricht der minimalen Streckung / maximalen Stauchung
- $\sigma_i = 0 \iff \lambda_i = 0$

Bsp: Singulärwerte

Matrix $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 2 & 1 \\ 1 & 0 \end{pmatrix}$: $\mathbf{A}^\top \mathbf{A}$ mit $\boldsymbol{\lambda} \approx (9.53, 1.47)$:

Singulärwerte:

$$\sigma_1 = \sqrt{\lambda_1} = \sqrt{9.53} \approx 3.09$$

$$\sigma_2 = \sqrt{\lambda_2} = \sqrt{1.47} \approx 1.21$$

Interpretation:

- $\sigma_1 \approx 3.09$: Maximale Streckung der Matrix $\mathbf{A}$ ($\mathbf{v}_1$ wird um Faktor 3.1 gestreckt)
- $\sigma_2 \approx 1.21$: Minimale Streckung ($\mathbf{v}_2$ wird um Faktor 1.2 gestreckt)
- Verhältnis $\frac{\sigma_{\max}}{\sigma_{\min}} \approx 2.55$ zeigt, wie unterschiedlich die Streckung je nach Richtung ist (*anisotrop*).

Definition: Singulärvektoren

Def.: Rechte und linke Singulärvektoren

Rechte Singulärvektoren $\mathbf{v}_i$:

- Normierte Eigenvektoren von $\mathbf{A}^\top \mathbf{A}$
- $\mathbf{A}^\top \mathbf{A} \mathbf{v}_i = \lambda_i \mathbf{v}_i = \sigma_i^2 \mathbf{v}_i$

Linke Singulärvektoren $\mathbf{u}_i$:

- Definiert durch: $\mathbf{u}_i = \frac{\mathbf{A} \mathbf{v}_i}{\sigma_i}$ (für $\sigma_i > 0$)

Wichtige Eigenschaften:

- $\{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ ist orthonormal
- $\{\mathbf{u}_1, \dots, \mathbf{u}_r\}$ ist orthonormal ($r = \text{rang}(\mathbf{A})$)

Beweis der Orthonormalität: siehe [Anhang](#)

Bsp: Singulärvektoren

$$\text{Matrix } \mathbf{A} = \begin{pmatrix} 1 & 2 \\ 2 & 1 \\ 1 & 0 \end{pmatrix} : \quad \mathbf{A}^\top \mathbf{A} = \begin{pmatrix} 6 & 4 \\ 4 & 5 \end{pmatrix} \text{ mit } \boldsymbol{\lambda} \approx (9.53, 1.47):$$

Rechte Singulärvektoren (normierte Eigenvektoren von $\mathbf{A}^\top \mathbf{A}$):

$$\begin{pmatrix} 6 & 4 \\ 4 & 5 \end{pmatrix} \mathbf{v}_1 = 9.53 \mathbf{v}_1 \implies \mathbf{v}_1 \approx \begin{pmatrix} -0.75 \\ -0.662 \end{pmatrix}, \quad \mathbf{v}_2 \approx \begin{pmatrix} -0.662 \\ 0.75 \end{pmatrix} \text{ (normiert, orthogonal zu } \mathbf{v}_1)$$

Linke Singulärvektoren ($\mathbf{u}_i = \mathbf{A}\mathbf{v}_i/\sigma_i$), eine Rechnung exemplarisch:

$$\mathbf{u}_1 = \frac{\mathbf{A}\mathbf{v}_1}{\sigma_1} \approx \begin{pmatrix} 1 & 2 \\ 2 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} -0.75 \\ -0.662 \end{pmatrix} / 3.09 = \begin{pmatrix} -0.672 \\ -0.7 \\ -0.243 \end{pmatrix}, \quad \text{analog: } \mathbf{u}_2 \approx \begin{pmatrix} 0.691 \\ -0.474 \\ -0.546 \end{pmatrix}$$

Check: $\mathbf{u}_1^\top \mathbf{u}_2 \approx 0$ und $\|\mathbf{u}_1\| \approx 1$ ✓ (vollständige Rechnung: siehe **Anhang**)

Live-Demo: SVD in R

Stimmt unsere Handrechnung? **Vorhersage:** Was liefert `svd(A)`?

```
1 A <- matrix(c(1, 2,  
2               2, 1,  
3               1, 0), nrow = 3, byrow = TRUE)  
4 svd(A)
```

```
$d  
[1] 3.09 1.21
```

```
$u  
      [,1] [,2]  
[1,] -0.672 0.691  
[2,] -0.700 -0.474  
[3,] -0.243 -0.546
```

```
$v  
      [,1] [,2]  
[1,] -0.750 -0.662  
[2,] -0.662 0.750
```

- $d \approx (3.09, 1.21)$: wie von Hand berechnet ✓
- u, v : Singulärvektoren – Vorzeichen nicht eindeutig ($\pm u_i, \pm v_i$ ebenfalls gültig)!

Fundamentale Unterräume

Theorem: Charakterisierung der Unterräume

Angenommen $\mathbf{A}$ hat r Singulärwerte ungleich Null. Dann:

1. $\{\mathbf{v}_1, \dots, \mathbf{v}_r\}$ ist Orthonormalbasis für $\text{col}(\mathbf{A}^\top)$ (Zeilenraum)
2. $\{\mathbf{A}\mathbf{v}_1, \dots, \mathbf{A}\mathbf{v}_r\}$ ist Orthogonalbasis für $\text{col}(\mathbf{A})$ (Spaltenraum) – normiert ($\|\mathbf{A}\mathbf{v}_i\| = \sigma_i$) sind das die $\{\mathbf{u}_1, \dots, \mathbf{u}_r\}$
3. $\{\mathbf{v}_{r+1}, \dots, \mathbf{v}_n\}$ ist Orthonormalbasis für $\ker(\mathbf{A})$ (Nullraum)
4. $\text{rang}(\mathbf{A}) = r$

Interpretation: SVD gibt uns explizite Orthonormalbasen für alle Fundamentalräume! (für den linken Nullraum $\ker(\mathbf{A}^\top)$: erweiterte $\mathbf{u}_{r+1}, \dots, \mathbf{u}_m \rightarrow$ SVD-Hauptsatz & Beispiel)

Rechenbeispiel: Fundamentale Unterräume (1)

Für $\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 0 & 0 \end{pmatrix}$ mit $\mathbf{A}^\top \mathbf{A} = \begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix}$ und $\text{rang}(\mathbf{A}) = 1$:

- **Singulärwerte:**

$$\lambda_1(\mathbf{A}^\top \mathbf{A}) = 4, \lambda_2(\mathbf{A}^\top \mathbf{A}) = 0$$

$\implies \sigma_1(\mathbf{A}) = 2$: Maximale Streckung durch $\mathbf{A}$ um Faktor 2 in Richtung $\mathbf{v}_1$; Faktor 0 in Richtung $\mathbf{v}_2$

- **Zeilenraum: Rechte Singulärvektoren**

$$\mathbf{v}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad \mathbf{v}_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

$$\implies \text{col}(\mathbf{A}^\top) = \text{span}\{\mathbf{v}_1\} = \text{span}\left\{ \begin{pmatrix} 1 \\ 1 \end{pmatrix} \right\}$$

- **Spaltenraum: Linke Singulärvektoren** $\mathbf{u}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}, \quad \mathbf{u}_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} -1 \\ 1 \\ 0 \end{pmatrix}, \quad \mathbf{u}_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$

($\mathbf{u}_2, \mathbf{u}_3$: orthonormale Ergänzung, da $\sigma_2 = 0$; $\rightarrow$ SVD-Hauptsatz)

$$\implies \text{col}(\mathbf{A}) = \text{span}\{\mathbf{u}_1\} = \text{span}\left\{ \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} \right\}$$

Rechenbeispiel: Fundamentale Unterräume (2)

Weiter mit $\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 0 & 0 \end{pmatrix}$, $\mathbf{v}_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$, $\mathbf{u}_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} -1 \\ 1 \\ 0 \end{pmatrix}$, $\mathbf{u}_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$:

(Rechter) Nullraum: $\ker(\mathbf{A}) = \text{span}\{\mathbf{v}_2\} = \text{span}$
 $\left\{ \begin{pmatrix} 1 \\ -1 \end{pmatrix} \right\}$ Verifikation: $\mathbf{A} \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} \quad \checkmark$

Linker Nullraum: $\ker(\mathbf{A}^\top) = \text{span}\{\mathbf{u}_2, \mathbf{u}_3\} = \text{span}$
 $\left\{ \begin{pmatrix} -1 \\ 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \right\}$ Verifikation:
 $\mathbf{A}^\top \begin{pmatrix} -1 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix} \quad \checkmark$

Der SVD-Hauptsatz

Theorem: Singulärwertzerlegung

Sei $\mathbf{A} \in \mathbb{R}^{m \times n}$ mit $\text{rang}(\mathbf{A}) = r$. Dann existiert eine Zerlegung:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$$

wobei:

- $\mathbf{V} \in \mathbb{R}^{n \times n}$: Orthogonalmatrix (rechte Singulärvektoren in Spalten)
- $\mathbf{U} \in \mathbb{R}^{m \times m}$: Orthogonalmatrix (linke Singulärvektoren erweitert zu Basis)
- $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$: "Diagonalmatrix" mit Singulärwerten

Beweis (Konstruktionsskizze): siehe [Anhang](#)

Die Struktur von Σ

Form der Matrix Σ :

$$\Sigma = \begin{pmatrix} \text{diag}(\sigma_1, \dots, \sigma_r) & 0_{r \times (n-r)} \\ 0_{(m-r) \times r} & 0_{(m-r) \times (n-r)} \end{pmatrix}$$

Beispiele:

A quadratisch ($m = n = 3, r = 2$):

$$\Sigma = \begin{pmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

A rechteckig ($m = 2, n = 3, r = 2$):

$$\Sigma = \begin{pmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \end{pmatrix}$$

Geometrische Interpretation

Transformation: $x \mapsto Ax = U\Sigma V^\top x$

Drei Schritte:

1. $V^\top$:

Koordinatentransformation (Rotation/Spiegelung) zu “optimalen” Richtungen

2. Σ :

“Streckung” entlang dieser Achsen um Faktoren σ_i , Nullsetzen für $i > r$

3. U :

Koordinatentransformation (Rotation/Spiegelung) zurück in den Zielraum

$\implies$ **SVD zerlegt *jede* lineare Transformation von $\mathbb{R}^n$ nach $\mathbb{R}^m$ in “Drehen-Strecken-Drehen”.**

Reduzierte SVD und Pseudoinverse

Reduzierte SVD

Problem: Viele Nulleinträge in $\Sigma \in \mathbb{R}^{m \times n}$, sobald $\text{rang}(\mathbf{A}) = r < m$ oder $r < n$ ($\mathbf{A}$ rangdefizient oder nicht quadratisch):

- Σ enthält $(m - r)$ Nullzeilen **und** $(n - r)$ Nullspalten
- Entsprechende Spalten in $\mathbf{U}$ bzw. $\mathbf{V}$ werden mit 0 multipliziert

Idee: Partitioniere $\mathbf{U} = (\mathbf{U}_r \mid \mathbf{U}_{m-r})$, $\mathbf{V} = (\mathbf{V}_r \mid \mathbf{V}_{n-r})$ mit $\mathbf{U}_r \in \mathbb{R}^{m \times r}$, $\mathbf{V}_r \in \mathbb{R}^{n \times r}$ und behalte nur die ersten r Singulärwerte und -vektoren:

Def.: Reduzierte SVD

Die **reduzierte SVD** von $\mathbf{A}$ mit $\text{rang}(\mathbf{A}) = r$ ist

$$\mathbf{A} = \mathbf{U}_r \Sigma_r \mathbf{V}_r^\top$$

wobei:

- $\mathbf{U}_r \in \mathbb{R}^{m \times r}$: Erste r linke Singulärvektoren
- $\mathbf{V}_r \in \mathbb{R}^{n \times r}$: Erste r rechte Singulärvektoren
- $\Sigma_r \in \mathbb{R}^{r \times r}$: $\text{diag}(\sigma_1, \dots, \sigma_r)$

Vorteile:

Moore-Penrose Pseudoinverse

Motivation: Was wenn $\mathbf{A}$ nicht quadratisch/invertierbar?

Def.: Moore-Penrose Pseudoinverse

Für jede Matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ mit Rang $r \geq 1$ und *reduzierter SVD* $\mathbf{A} = \mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}_r^\top$ ist die **Moore-Penrose Pseudoinverse**:

$$\mathbf{A}^+ := \mathbf{V}_r \mathbf{\Sigma}_r^{-1} \mathbf{U}_r^\top \in \mathbb{R}^{n \times m}$$

Bemerkung:

$\mathbf{\Sigma}_r^{-1} = \text{diag}(1/\sigma_1, \dots, 1/\sigma_r)$; für $\mathbf{A} = \mathbf{0}$ (also $r = 0$) setzt man $\mathbf{A}^+ = \mathbf{0} \in \mathbb{R}^{n \times m}$

Eigenschaften der Pseudoinverse

Theorem: Eigenschaften von $\mathbf{A}^+$

1. $\mathbf{A}\mathbf{A}^+\mathbf{x} = \text{proj}_{\text{col}(\mathbf{A})}\mathbf{x}$ für alle $\mathbf{x} \in \mathbb{R}^m$
2. $\mathbf{A}^+\mathbf{A}\mathbf{x} = \text{proj}_{\text{col}(\mathbf{A}^\top)}\mathbf{x}$ für alle $\mathbf{x} \in \mathbb{R}^n$
3. $\mathbf{A}\mathbf{A}^+\mathbf{A} = \mathbf{A}$ (Konsistenz)
4. $\mathbf{A}^+\mathbf{A}\mathbf{A}^+ = \mathbf{A}^+$ (Konsistenz)

Warum Projektion?

$\mathbf{A}\mathbf{A}^+ = \mathbf{U}_r \Sigma_r \mathbf{V}_r^\top \mathbf{V}_r \Sigma_r^{-1} \mathbf{U}_r^\top = \mathbf{U}_r \mathbf{U}_r^\top$, und die Spalten von $\mathbf{U}_r$ sind eine Orthonormalbasis von $\text{col}(\mathbf{A})$.

Spezialfälle:

- Wenn $\text{rang}(\mathbf{A}) = m$: $\mathbf{A}\mathbf{A}^+ = \mathbf{I}_m$
- Wenn $\text{rang}(\mathbf{A}) = n$: $\mathbf{A}^+\mathbf{A} = \mathbf{I}_n$
- Wenn $\mathbf{A}$ quadratisch und invertierbar: $\mathbf{A}^+ = \mathbf{A}^{-1}$

Anwendungen

Spektralnrm und SVD

Bisher: $\|\mathbf{A}\|_2 = \sqrt{\lambda_{\max}(\mathbf{A}^\top \mathbf{A})}$

Aus Definition SVD ist bekannt, dass $\lambda_j(\mathbf{A}^\top \mathbf{A}) = \sigma_j(\mathbf{A})^2$

Also:

$$\|\mathbf{A}\|_2 = \sigma_{\max}(\mathbf{A})$$

Interpretation: Größter Singulärwert = maximale Streckung

Die SVD als Summe von Rang-1 Matrizen

SVD in Summenform:

$$\mathbf{A} = \mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}_r^\top = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$$

Interpretation:

- Jeder Term $\mathbf{u}_i \mathbf{v}_i^\top$ ist eine **Rang-1 Matrix** (mit $\|\mathbf{u}_i \mathbf{v}_i^\top\|_F = 1$)
- σ_i gewichtet die Bedeutung des i -ten Terms
- Große $\sigma_i \rightarrow$ wichtige Richtungen
- Kleine $\sigma_i \rightarrow$ weniger wichtige Richtungen

Idee für Approximation: Verwende nur die wichtigsten Terme!

Rang-k Approximation

Low-Rank Approximation:

$$\mathbf{A}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top \quad (k < r)$$

Theorem: Optimale Approximation

$\mathbf{A}_k$ ist beste Rang- k Approximation von $\mathbf{A}$ bezüglich der Frobenius- und Spektral-Normen, d.h. $\mathbf{A}_k$ löst beide Approximationsprobleme:

$$\mathbf{A}_k \in \arg \min_{\substack{\mathbf{B} \in \mathbb{R}^{m \times n} \\ \text{rang}(\mathbf{B}) \leq k}} \|\mathbf{A} - \mathbf{B}\|_F \quad \text{und} \quad \mathbf{A}_k \in \arg \min_{\substack{\mathbf{B} \in \mathbb{R}^{m \times n} \\ \text{rang}(\mathbf{B}) \leq k}} \|\mathbf{A} - \mathbf{B}\|_2$$

Approximationsfehler:

- $\|\mathbf{A} - \mathbf{A}_k\|_F = \sqrt{\sum_{i=k+1}^r \sigma_i^2}$
- $\|\mathbf{A} - \mathbf{A}_k\|_2 = \sigma_{k+1}$

Interpretation: Fehler bestimmt durch *weggelassene* Singulärwerte

Nicht-Eindeutigkeit in Spektralnorm: Für $\mathbf{A} = \text{diag}(3, 2)$, $k = 1$ und $\mathbf{B}_b = \text{diag}(b, 0)$ gilt $\|\mathbf{A} - \mathbf{B}_b\|_2 = \max\{|3 - b|, 2\}$. Damit sind alle $b \in [1, 5]$ optimal; die abgeschnittene SVD entspricht $b = 3$. In Frobenius-Norm ist nur $b = 3$ optimal, denn der Fehler ist $\sqrt{(3 - b)^2 + 2^2}$.

Wahl der optimalen Rang-k Approximation

Kriterien für die Wahl von k :

1. Relativer Fehler (Spektralnorm):

$$\frac{\|\mathbf{A} - \mathbf{A}_k\|_2}{\|\mathbf{A}\|_2} = \frac{\sigma_{k+1}}{\sigma_1}$$

2. Relativer Fehler (Frobenius-Norm):

$$\frac{\|\mathbf{A} - \mathbf{A}_k\|_F}{\|\mathbf{A}\|_F} = \frac{\sqrt{\sum_{i=k+1}^r \sigma_i^2}}{\sqrt{\sum_{i=1}^r \sigma_i^2}}$$

3. “Energieerhaltung”:

$$\frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2} \geq p \quad (p \cdot 100\% \text{ der "Energie"})$$

Praktische Heuristik: k am “Ellenbogen” im $\log(\sigma_i)$ -Plot wählen – dort, wo die Singulärwerte abknicken.
(Typische Verläufe: siehe *Anhang*)

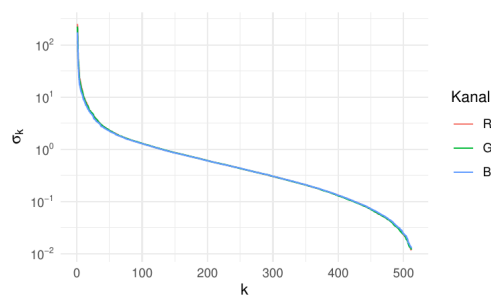
Anwendung: Datenkompression

Bildkompression – Originalbild: 659×512 Pixel-Matrix (pro Farbkanal)

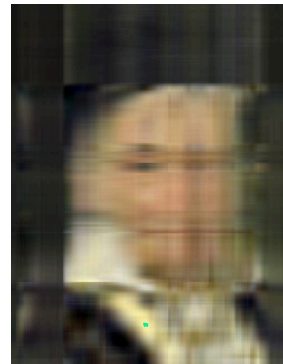
- Volle Speicherung: $659 \times 512 = 337.408$ Zahlen; **Rang-50 SVD**:
 $(659 \times 50) + 50 + (50 \times 512) = 58.600$ Zahlen $\implies$ *Speicherersparnis*: $\approx 83\%$ (Speicheranteil $\approx 17\%$)!
- **Vorgehen**: SVD berechnen, k anhand der Singulärwerte wählen, nur U_k , Σ_k , V_k speichern.



Original



Singulärwerte pro Kanal



$k = 5$



$k = 20$



$k = 50$



$k = 200$

Bildquelle: C. A. Jensen, Porträt C. F. Gauß (1840), gemeinfrei (Wikimedia Commons).

Interaktive Demo: Bildkompression



Fallback: ► [App direkt öffnen](#) — Screenshot im Anhang.

Anwendung: Empfehlungssysteme

Collaborative Filtering / Recommender Systems

Problem: Netflix-Szenario: m Nutzer, n Filme, aber Daten sehr *sparse*: meiste Nutzer haben meiste Filme nicht bewertet

$$\mathbf{R} = \begin{pmatrix} 5 & ? & 1 & ? & 4 \\ ? & 3 & ? & 4 & ? \\ 2 & 1 & ? & ? & 5 \\ ? & 5 & 4 & 3 & ? \end{pmatrix} \in \mathbb{R}^{m \times n}$$

Idee: Nutzer-Film-Präferenzen liegen in einem *niedrigdimensionalen Raum* (Genres und Nutzerklassen).

SVD-Ansatz:

1. Ergänze fehlende Einträge (z.B. durch Spaltenmittelwerte)
2. Berechne Rang- k Approximation: $\mathbf{R}_k = \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^\top$
3. Nutze $\mathbf{R}_k$ für Vorhersagen fehlender Ratings

Interpretation: $\mathbf{v}_j$ = “gelernte” Film-Genres, $\mathbf{u}_i$ = Nutzer-Präferenzen in diesen Genre-Dimensionen, σ_i = Wichtigkeit der Dimension (*mehr: siehe [Anhang](#)*)

Wann ist SVD besonders nützlich?

Ideale Situationen für SVD:

1. *Niedrig-Rang Strukturen*: $\text{rang}(\mathbf{A}) \ll \min(m, n)$
2. *Schnell abfallende Singulärwerte*: Wenige dominieren
3. *Überbestimmte Systeme*: $m > n$, möglicherweise inkonsistent
4. *Datenanalyse*: Hauptkomponentenanalyse (PCA) ohne Kreuzprodukte zu berechnen.
5. *Numerische Robustheit*: Alternative zu Normalgleichungen

Rechenaufwand:

- **SVD**: $O(\min(m^2n, mn^2))$, für $m \gg n$ etwa $4mn^2$
- *Normalgleichungen*: $O(mn^2 + n^3)$, für $m \gg n$ etwa mn^2 .
- Für $m \gg n$ sind Normalgleichungen schneller, aber SVD ist robuster $\rightarrow$ trade-off zwischen Geschwindigkeit und Stabilität

SVD vs. Eigenwertzerlegung

Eigenschaft	Eigenwertzerlegung	SVD
<i>Matrix-Typ</i>	Quadratisch	Beliebig $m \times n$
<i>Orthogonale Zerlegung</i>	Nur symmetrisch ($\rightarrow$ Spektralzerlegung)	Immer
<i>Numerische Stabilität</i>	Kann instabil sein	Stabil
<i>Existenz</i>	Nur für diagonalisierbare A	Immer
<i>Geometrische Interpretation</i>	Hauptachsen (symmetrisch)	“Optimale” Koordinaten

Zusammenfassung & Self-Check

Zusammenfassung

Kernkonzepte:

1. SVD erweitert Eigenwertzerlegung auf beliebige $m \times n$ Matrizen
2. Singulärwerte vs. Eigenwerte: $\sigma_i(\mathbf{A}) = \sqrt{\lambda_i(\mathbf{A}^\top \mathbf{A})}$
3. Hauptsatz: $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ mit orthogonalen $\mathbf{U}$ ($m \times m$), $\mathbf{V}$ ($n \times n$), diagonalem $\mathbf{\Sigma}$ ($m \times n$)
4. Reduzierte SVD: $\mathbf{A} = \mathbf{U}_r\mathbf{\Sigma}_r\mathbf{V}_r^\top$ für Effizienz ($r = \text{rang}(\mathbf{A})$)
5. Pseudoinverse: $\mathbf{A}^+ = \mathbf{V}_r\mathbf{\Sigma}_r^{-1}\mathbf{U}_r^\top$

Anwendungen: Lösung von **Kleinste-Quadrate-Problemen**, optimale **Low-Rank Approximation** (Datenkompression, -analyse)

Praktische Hinweise:

- SVD **nicht** über $\mathbf{A}^\top \mathbf{A}$ berechnen (numerisch instabil) – spezielle Algorithmen (z.B. Golub-Reinsch) nutzen: in R `svd(A)`
- *truncated SVD* für sehr große Matrizen: `irlba::irlba(A, nv = k)` (iterativ, Lanczos) bzw. `irlba::svdr(A, k)` (randomisiert)

Self-Check

Welche Aussagen sind korrekt?

1. Wenn $\mathbf{A} \in \mathbb{R}^{m \times n}$, dann ist $\mathbf{A}^\top \mathbf{A} \in \mathbb{R}^{m \times m}$
2. Wenn $\mathbf{A} \in \mathbb{R}^{m \times n}$, dann ist $\mathbf{A}^\top \mathbf{A}$ symmetrisch
3. Die Eigenwerte von $\mathbf{A}^\top \mathbf{A}$ sind die Singulärwerte von $\mathbf{A}$
4. Wenn $\mathbf{A} \in \mathbb{R}^{n \times n}$, dann ist $\mathbf{A}$ immer diagonalisierbar
5. $\mathbf{A}^\top \mathbf{A}$ ist immer orthogonal diagonalisierbar

Anhang

Orthogonalität der Singulärvektoren

Warum sind die $\sigma_i \mathbf{u}_i = \mathbf{A} \mathbf{v}_i$ orthogonal?

Für $i \neq j$:

$$\begin{aligned} (\mathbf{A} \mathbf{v}_i)^\top (\mathbf{A} \mathbf{v}_j) &= \mathbf{v}_i^\top (\mathbf{A}^\top \mathbf{A}) \mathbf{v}_j \\ &= \mathbf{v}_i^\top (\lambda_j \mathbf{v}_j) \\ &= \lambda_j (\mathbf{v}_i^\top \mathbf{v}_j) \\ &= \lambda_j \cdot 0 = 0 \end{aligned}$$

Für $i = j$:

$$\|\mathbf{A} \mathbf{v}_i\|^2 = \mathbf{v}_i^\top (\mathbf{A}^\top \mathbf{A}) \mathbf{v}_i = \lambda_i \|\mathbf{v}_i\|^2 = \lambda_i = \sigma_i^2$$

$\implies$ Für $i, j \leq r$ (also $\sigma_i, \sigma_j > 0$) ist $\mathbf{u}_i^\top \mathbf{u}_j = \frac{(\mathbf{A} \mathbf{v}_i)^\top (\mathbf{A} \mathbf{v}_j)}{\sigma_i \sigma_j} = 0$ für $i \neq j$ bzw. $= 1$ für $i = j$: Die $\mathbf{u}_1, \dots, \mathbf{u}_r$ sind orthonormal! (Die übrigen $\mathbf{u}_{r+1}, \dots, \mathbf{u}_m$ werden als orthonormale Ergänzung gewählt.)

$\hookleftarrow$ zurück: „Definition: Singulärvektoren“

Bsp: Singulärvektoren – vollständige Rechnung (1)

Matrix $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 2 & 1 \\ 1 & 0 \end{pmatrix}$: $\mathbf{A}^\top \mathbf{A}$ mit $\boldsymbol{\lambda} \approx (9.53, 1.47)$:

Rechte Singulärvektoren (Eigenvektoren von $\mathbf{A}^\top \mathbf{A}$):

Für $\lambda_1 \approx 9.53$:

$$\begin{pmatrix} 6 & 4 \\ 4 & 5 \end{pmatrix} \begin{pmatrix} v_1^{(1)} \\ v_2^{(1)} \end{pmatrix} = 9.53 \begin{pmatrix} v_1^{(1)} \\ v_2^{(1)} \end{pmatrix}$$

Lösung: $\mathbf{v}_1 \approx \begin{pmatrix} -0.75 \\ -0.662 \end{pmatrix}$

Für $\lambda_2 \approx 1.47$: $\mathbf{v}_2 \approx \begin{pmatrix} -0.662 \\ 0.75 \end{pmatrix}$ (normiert, orthogonal zu $\mathbf{v}_1$)

Bsp: Singulärvektoren – vollständige Rechnung (2)

Linke Singulärvektoren:

$$\mathbf{u}_1 = \frac{\mathbf{A}\mathbf{v}_1}{\sigma_1} \approx \begin{pmatrix} 1 & 2 \\ 2 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} -0.75 \\ -0.662 \end{pmatrix} / 3.09 = \begin{pmatrix} -0.672 \\ -0.7 \\ -0.243 \end{pmatrix}, \quad \text{analog: } \mathbf{u}_2 = \frac{\mathbf{A}\mathbf{v}_2}{\sigma_2} \approx \begin{pmatrix} 0.691 \\ -0.474 \\ -0.546 \end{pmatrix}$$

Verifikation Orthonormalität:

- $\mathbf{u}_1^\top \mathbf{u}_2 \approx (-0.672)(0.691) + (-0.7)(-0.474) + (-0.243)(-0.546) = -0.464 + 0.332 + 0.133 \approx 0 \quad \checkmark$
- $\|\mathbf{u}_1\| \approx \sqrt{0.672^2 + 0.7^2 + 0.243^2} = \sqrt{0.452 + 0.49 + 0.059} \approx 1 \quad \checkmark$

↔ zurück: „Bsp: Singulärvektoren“

Beweis des SVD-Theorems (Skizze)

Konstruktion der Zerlegung:

1. *Rechte Singulärvektoren*: Eigenvektoren von $\mathbf{A}^\top \mathbf{A}$

$$\mathbf{V} = (\mathbf{v}_1 \mid \mathbf{v}_2 \mid \cdots \mid \mathbf{v}_n)$$

2. *Linke Singulärvektoren*: $\mathbf{u}_i = \mathbf{A}\mathbf{v}_i / \sigma_i$ für $i \leq r$

$$\mathbf{U}_r = (\mathbf{u}_1 \mid \mathbf{u}_2 \mid \cdots \mid \mathbf{u}_r)$$

3. *Vervollständigung*: Erweitere $\mathbf{U}_r$ zu Orthogonalbasis des $\mathbb{R}^m$

4. *Verifikation*: $\mathbf{A}\mathbf{v}_i = \sigma_i \mathbf{u}_i$ für $i \leq r$, $\mathbf{A}\mathbf{v}_i = \mathbf{0}$ für $i > r$

↔ zurück: „Der SVD-Hauptsatz“

Typische Singulärwert-Verläufe

1. Schneller Abfall: $\sigma_i \approx c \cdot e^{-\alpha i}$

- Gut geeignet für Low-Rank Approximation
- Wenige dominante Singulärwerte

2. Linearer Abfall: $\sigma_i \approx c - \alpha i$

- Moderate Approximation möglich
- Mittlerer Kompressionsfaktor

3. Langsamer Abfall: $\sigma_i \approx c \cdot i^{-\alpha}$

- Schwierige Approximation
- Viele wichtige Komponenten

↔ zurück: „Wahl der optimalen Rang-k Approximation“

Empfehlungssysteme: Details

Interpretation der Faktoren:

- **Rechte Singulärvektoren** $\mathbf{v}_j$: “gelernte” Film-Genres
 - $\mathbf{v}_1$ könnte “Action vs. Drama” kodieren, $\mathbf{v}_2$ “Komödie vs. Thriller”, etc
- **Linke Singulärvektoren** $\mathbf{u}_i$: Nutzer-Präferenzen in diesen Genre-Dimensionen
- **Singulärwerte** σ_i : Wichtigkeit dieser Genre-Dimension

Vorteile:

- Reduziert Speicher: $k + (m + n) \cdot k \ll m \cdot n$ für $k \ll \min(m, n)$
- “Glättet” die Daten (Low-Rank = *Regularisierung*)
- Kann auch bei sehr spärlichen Daten funktionieren

Modernere Varianten:

- *Matrix Completion*: Low-Rank-Faktorisierung nur über die *beobachteten* Einträge; mit Nichtnegativitäts-Nebenbedingung: *Non-negative Matrix Factorization* (NMF)

↔ zurück: „Anwendung: Empfehlungssysteme“

Methode der Kleinsten Quadrate

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)

Material: Thomas Nagler

LMU München

Verwendete Vorkenntnisse

- **Kondition** & Konditionszahl $\kappa(\mathbf{A})$
- **Matrixnormen** (besonders $\|\cdot\|_2$) und **Orthogonalmatrizen**
- Matrixzerlegungen: **LU-** und **Cholesky-Zerlegung**
- **SVD**: $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$, **Pseudoinverse** $\mathbf{A}^+$, Reduzierte SVD
- **Lineare Algebra**
 - **Normalengleichungen**: $\mathbf{A}^\top \mathbf{A} \mathbf{x} = \mathbf{A}^\top \mathbf{b}$
 - (Orthogonale) Projektionen auf Unterräume
- **Statistik**
 - Lineares Regressionsmodell $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$
 - Methode der Kleinsten Quadrate (OLS)

Opener-Quiz: Was wissen Sie noch?

Frage 1: Sei $\mathbf{A} \in \mathbb{R}^{m \times n}$ mit $\text{rang}(\mathbf{A}) = n$. Die Kleinste-Quadrate-Lösung $\hat{\mathbf{x}}$ von $\min_{\mathbf{x}} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2$ erfüllt ...

1. $\mathbf{A}\hat{\mathbf{x}} = \mathbf{b}$
2. $\mathbf{A}^\top \mathbf{A}\hat{\mathbf{x}} = \mathbf{A}^\top \mathbf{b}$
3. $\mathbf{A}\mathbf{A}^\top \hat{\mathbf{x}} = \mathbf{A}\mathbf{b}$
4. $\hat{\mathbf{x}} = \mathbf{A}^\top \mathbf{b}$

Frage 2: Sei $\hat{\mathbf{b}} = \text{proj}_{\text{col}(\mathbf{A})} \mathbf{b}$ die orthogonale Projektion von $\mathbf{b}$ auf den Spaltenraum von $\mathbf{A}$. Welche Aussage gilt **immer**?

1. $\hat{\mathbf{b}} = \mathbf{b}$
2. $\mathbf{b} - \hat{\mathbf{b}} \perp \text{col}(\mathbf{A})$
3. $\|\hat{\mathbf{b}}\|_2 \geq \|\mathbf{b}\|_2$
4. $\hat{\mathbf{b}} = \mathbf{0}$

Recap: Kleinste Quadrate

Recap: Kleinste Quadrate

Kleinste-Quadrate-Problem (KQ-Problem)

- Gegeben sei Matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ und Vektor $\mathbf{b} \in \mathbb{R}^m$.
- Wir wollen einen Vektor $\hat{\mathbf{x}} \in \mathbb{R}^n$ finden, sodass $\|\mathbf{A}\hat{\mathbf{x}} - \mathbf{b}\|$ minimal ist.

Anmerkung:

- Hat das LGS eine Lösung $\mathbf{x}^*$, dann gilt $\|\mathbf{A}\mathbf{x}^* - \mathbf{b}\| = 0$ und $\mathbf{x}^*$ ist automatisch auch eine Lösung des Kleinste-Quadrate-Problems.
- Interessant ist die Problemstellung insbesondere für “Gleichungssysteme” $\mathbf{A}\mathbf{x}^* \approx \mathbf{b}$, die keine *exakte* Lösung besitzen.

Motivation

KQ-Methode *fundamentalst* in Statistik und Machine Learning:

- **Beispiel: Lineare Regression**

- **Problem:** Vorhersage von Immobilienpreisen basierend auf Wohnfläche
- **Daten:** n Beobachtungen (x_i, y_i) mit x_i = Fläche in m^2 , y_i = Preis in €
- **Modell:** $y_i = \beta_0 + \beta_1 x_i + \epsilon_i$ mit Fehlerterm ϵ_i
- **Matrix-Form:** $\mathbf{b} = \mathbf{A}\boldsymbol{\beta} + \boldsymbol{\epsilon}$ mit

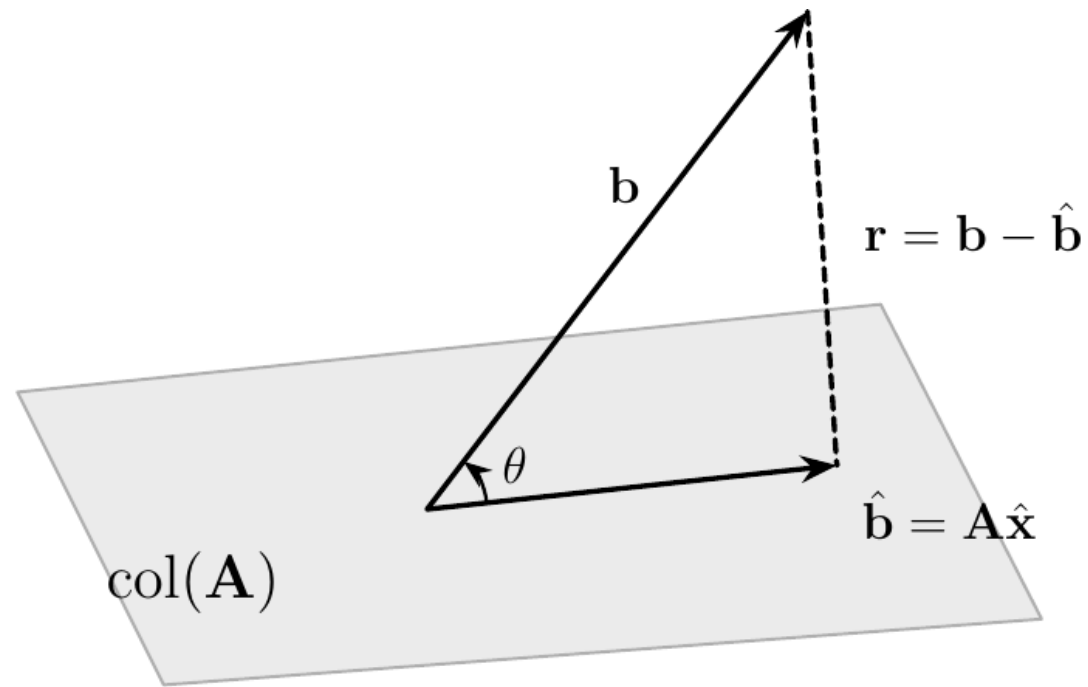
$$\mathbf{A} = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, \quad \boldsymbol{\beta} = \begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix}$$

- **Lösung:** $\hat{\boldsymbol{\beta}} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{b}$ minimiert $\sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$

- **Weitere Anwendungen:**

- **Generalisierte Lineare Modelle, Additive Modelle:** approximative Lösungen über iterative KQ-Probleme
- **Deep Learning:** Training neuronaler Netze (minimiert oft KQ-Fehler)
- **Computer Vision:** Kamerakalibration, 3D-Rekonstruktion aus 2D-Bildern
- **Signal Processing:** Rauschreduktion/De-Noising, Signalschätzung
- **Geodäsie:** GPS-Positionsbestimmung
-

Recap: Kleinste Quadrate



Theorem

Die Lösungsmenge des KQ-Problems gleicht der Lösungsmenge des LGS

$$A\hat{\mathbf{x}} = \hat{\mathbf{b}} \quad \text{mit } \hat{\mathbf{b}} = \text{proj}_{\text{col}(A)} \mathbf{b}.$$

Recap: Kleinste Quadrate

Theorem

Die Lösungsmenge des KQ-Problems gleicht der Lösungsmenge der Normalgleichungen

$$\mathbf{A}^\top \mathbf{A} \hat{\mathbf{x}} = \mathbf{A}^\top \mathbf{b}.$$

Theorem

Falls $\text{rang}(\mathbf{A}) = n$, ist $\mathbf{A}^\top \mathbf{A}$ invertierbar und das KQ-Problem hat die eindeutige Lösung

$$\hat{\mathbf{x}} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{b} =: \mathbf{A}^+ \mathbf{b}.$$

Quiz

Welche der folgenden Aussagen sind wahr? (*mehrere Antworten möglich*)

1. $\mathbf{A}\mathbf{A}^+ = \mathbf{I}_m$
2. $\mathbf{A}\mathbf{A}^+\mathbf{b} = \text{proj}_{\text{col}(\mathbf{A})}\mathbf{b}$
3. $\mathbf{r} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}} \perp \text{col}(\mathbf{A})$
4. $\mathbf{b} = \text{proj}_V\mathbf{b} + \text{proj}_{V^\perp}\mathbf{b}$ für jeden Untervektorraum V

Kondition des KQ-Problems

Kondition

Vorbemerkungen

- Wir untersuchen nun die **Kondition** von KQ-Problemen $\mathbf{A}\mathbf{x} \approx \mathbf{b}$ mit $\mathbf{A} \in \mathbb{R}^{m \times n}$ und $\mathbf{b} \in \mathbb{R}^m$.
- Wir beschränken uns auf den Fall $m \geq n$ und $\text{rang}(\mathbf{A}) = n$
 $\implies$ eindeutige Lösung existiert.
- Im folgenden $\|\cdot\| = \|\cdot\|_2$ (natürliche Norm für KQ-Probleme).
- Für nicht notwendigerweise quadratische Matrizen definieren wir

$$\kappa(\mathbf{A}) = \begin{cases} \infty, & \text{rang}(\mathbf{A}) < n, \\ \|\mathbf{A}\| \|\mathbf{A}^+\|, & \text{sonst.} \end{cases}$$

Kondition bezüglich $\mathbf{b}$

Theorem

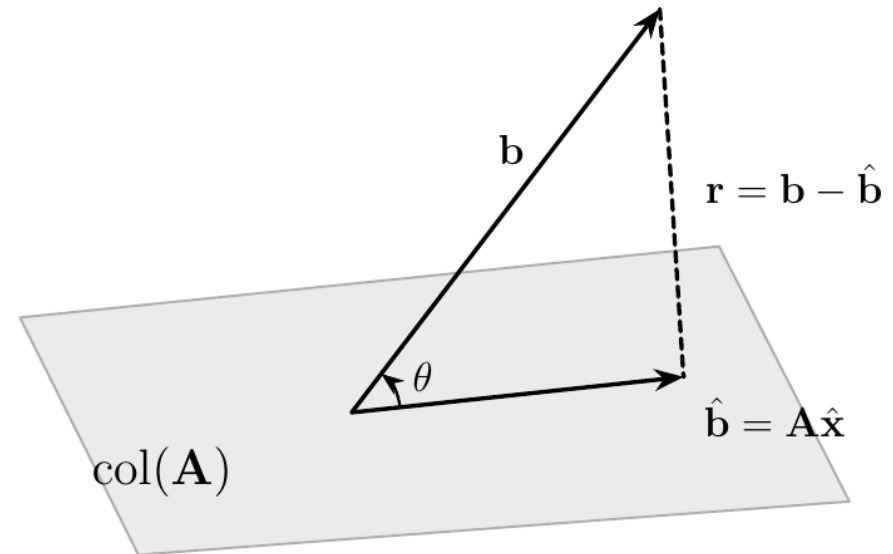
Sei $\mathbf{A} \in \mathbb{R}^{m \times n}$ fix und f das Problem mit Lösung $f(\mathbf{b}) = \mathbf{A}^+ \mathbf{b}$. Dann gilt

$$\frac{\|f(\tilde{\mathbf{b}}) - f(\mathbf{b})\|}{\|f(\mathbf{b})\|} \leq \kappa \frac{\|\tilde{\mathbf{b}} - \mathbf{b}\|}{\|\mathbf{b}\|} \quad \text{mit} \quad \kappa = \kappa_2(\mathbf{A}) \frac{\|\mathbf{b}\|}{\|\text{proj}_{\text{col}(\mathbf{A})} \mathbf{b}\|}.$$

Beweis: siehe **Anhang**

Kondition bezüglich $\mathbf{b}$

$$\kappa = \kappa_2(\mathbf{A}) \frac{\|\mathbf{b}\|}{\|\text{proj}_{\text{col}(\mathbf{A})}\mathbf{b}\|}$$



Interpretation

- Erinnerung:

$$\|\mathbf{b}\|^2 = \|\text{proj}_{\text{col}(\mathbf{A})}\mathbf{b}\|^2 + \|\text{proj}_{\text{col}(\mathbf{A})^\perp}\mathbf{b}\|^2 \geq \|\text{proj}_{\text{col}(\mathbf{A})}\mathbf{b}\|^2.$$

- **Frage:** Was sind best und worst case für κ ?

Kondition bezüglich $\mathbf{A}$

Theorem

Sei $\mathbf{b} \in \mathbb{R}^m$ fix und f das Problem mit Lösung $f(\mathbf{A}) = \mathbf{A}^+ \mathbf{b}$. Dann gilt für kleine Störungen $\tilde{\mathbf{A}} \rightarrow \mathbf{A}$ (in erster Ordnung)

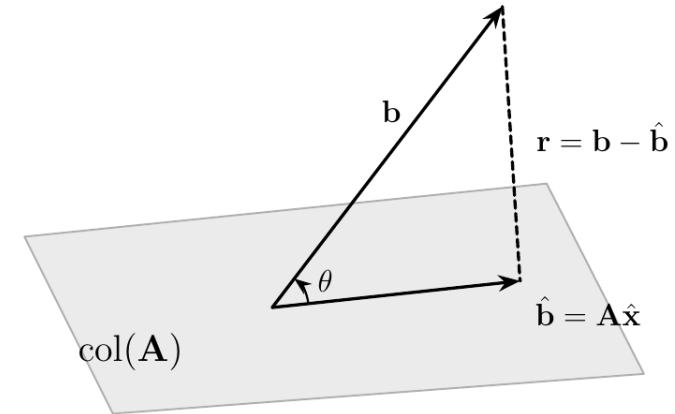
$$\frac{\|f(\tilde{\mathbf{A}}) - f(\mathbf{A})\|}{\|f(\mathbf{A})\|} \lesssim \kappa \frac{\|\tilde{\mathbf{A}} - \mathbf{A}\|}{\|\mathbf{A}\|}$$

mit $\kappa = \kappa_2(\mathbf{A}) + \kappa_2(\mathbf{A})^2 \frac{\|\mathbf{r}\|}{\|\text{proj}_{\text{col}(\mathbf{A})} \mathbf{b}\|}.$

Beweis: Sehr aufwändige und langweilige Rechnung.

Kondition bezüglich $\mathbf{A}$

$$\kappa = \kappa_2(\mathbf{A}) + \kappa_2(\mathbf{A})^2 \frac{\|\mathbf{r}\|}{\|\text{proj}_{\text{col}(\mathbf{A})}\mathbf{b}\|}$$



Interpretation

- Best case: $\mathbf{r} = \mathbf{0} \Rightarrow \mathbf{Ax} = \mathbf{b}$ ist lösbar $\Rightarrow \kappa = \kappa_2(\mathbf{A})$.
- Worst case: $\text{proj}_{\text{col}(\mathbf{A})}\mathbf{b} = \mathbf{0}$, aber $\mathbf{r} \neq \mathbf{0}$
 - $\Rightarrow \mathbf{Ax} = \mathbf{b}$ ist nicht lösbar und $\mathbf{b} \perp \text{col}(\mathbf{A})$
 - $\Rightarrow \kappa = \infty$.
- Normalerweise: $\mathbf{r} \neq \mathbf{0}$ und $\text{proj}_{\text{col}(\mathbf{A})}\mathbf{b} \neq \mathbf{0} \Rightarrow \kappa = O(\kappa_2(\mathbf{A})^2)$.

Lösungsverfahren

Frage

Um die Lösung $\hat{x} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{b}$ zu berechnen, sei folgende Funktion gegeben:

```
1 kq_solve <- function(A, b) {  
2   solve(t(A) %*% A) %*% t(A) %*% b  
3 }
```

Aufgabe: Finde den Fehler in der Funktion.

Invertiere niemals eine Matrix!

`solve(M)` berechnet die volle Inverse — teurer *und* instabiler, als das LGS $\mathbf{M}\mathbf{x} = \mathbf{c}$ direkt zu lösen.

besser: `solve(M, c)`.

Cholesky-Zerlegung

- Die Normalengleichung $\mathbf{A}^\top \mathbf{A} \mathbf{x} = \mathbf{A}^\top \mathbf{b}$ gibt uns ein einfaches Lösungsverfahren:
 1. Berechne $\mathbf{M} = \mathbf{A}^\top \mathbf{A}$ und $\mathbf{c} = \mathbf{A}^\top \mathbf{b}$.
 2. Berechne Cholesky-Zerlegung $\mathbf{M} = \mathbf{L} \mathbf{L}^\top$.
 3. Löse $\mathbf{L} \mathbf{y} = \mathbf{c}$ durch Vorwärtssubstitution.
 4. Löse $\mathbf{L}^\top \mathbf{x} = \mathbf{y}$ durch Rückwärtssubstitution.
- Das Verfahren ist effizient, aber möglicherweise instabil:
 - Fehler in $\mathbf{A}^\top \mathbf{A}$ beeinflussen alle folgenden Schritte.
 - Man kann zeigen, dass der relative algorithmische Fehler typischerweise von der Größenordnung $\kappa_2(\mathbf{A})^2 \epsilon_{mach}^1$ ist.
 - Vergleiche mit Best-Case-Kondition $O(\kappa_2(\mathbf{A}))$.

Alternative Lösungswege

- Wir würden gerne $\mathbf{A}^\top \mathbf{A}$ vermeiden und direkt auf $\mathbf{A}$ operieren.
- Erinnerung: KQ-Lösung zu $\mathbf{A}\mathbf{x} = \mathbf{b}$ ist

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x}} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2$$

- Wir wollen das Problem mit einer Matrix $\mathbf{Q}$ zu einem “einfacheren” Problem $\mathbf{Q}\mathbf{A}\mathbf{x} \approx \mathbf{Q}\mathbf{b}$ transformieren ohne die Lösung zu ändern.

$\implies \mathbf{Q}$ soll **Orthogonalmatrix** sein, denn für diese gilt

$$\|\mathbf{Q}\mathbf{A}\mathbf{x} - \mathbf{Q}\mathbf{b}\| = \|\mathbf{Q}(\mathbf{A}\mathbf{x} - \mathbf{b})\| = \|\mathbf{A}\mathbf{x} - \mathbf{b}\|.$$

Erinnerung: **Orthogonalmatrix** $\mathbf{Q}$

- $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}, \mathbf{Q}^{-1} = \mathbf{Q}^\top$ (“Rotation/Spiegelung”);
- Produkte von Orthogonalmatrizen sind orthogonal.
- Normerhaltend für $\|\cdot\|_2$: $\|\mathbf{Q}\mathbf{z}\|_2 = \sqrt{\mathbf{z}^\top \mathbf{Q}^\top \mathbf{Q} \mathbf{z}} = \sqrt{\mathbf{z}^\top \mathbf{z}} = \|\mathbf{z}\|_2$ für alle $\mathbf{z} \in \mathbb{R}^m$.
- Alle Singulärwerte $\sigma_i(\mathbf{Q}) = 1$ (da $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$) $\Rightarrow \kappa_2(\mathbf{Q}) = \frac{\sigma_{\max}}{\sigma_{\min}} = 1$
 $\implies$ Multiplikation mit $\mathbf{Q}$ ist immer gut konditioniert.

QR-Zerlegung

Def.: QR-Zerlegung

Sei $\mathbf{A} \in \mathbb{R}^{m \times n}$ mit $m \geq n$ und $\mathbf{Q} \in \mathbb{R}^{m \times m}$ eine *Orthogonalmatrix*, sodass

$$\mathbf{Q}^\top \mathbf{A} = \begin{pmatrix} \mathbf{R} \\ \mathbf{0}_{(m-n) \times n} \end{pmatrix},$$

wobei $\mathbf{R} \in \mathbb{R}^{n \times n}$ eine *obere Dreiecksmatrix* ist. Dann nennen wir

$$\mathbf{A} = \mathbf{Q} \begin{pmatrix} \mathbf{R} \\ \mathbf{0} \end{pmatrix}$$

eine **QR-Zerlegung** von $\mathbf{A}$.

Anmerkung:

- Es gibt immer eine QR-Zerlegung von $\mathbf{A}$.
- Die Zerlegung ist nicht eindeutig.

Beispiel: QR-Zerlegung

Gegeben:

$$\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 0 & 0 \\ 0 & 2 \end{pmatrix}$$

Gesucht:

$$\mathbf{Q}^\top \mathbf{A} = \begin{pmatrix} \mathbf{R} \\ \mathbf{0}_{1 \times 2} \end{pmatrix} \text{ mit } \mathbf{Q} \in \mathbb{R}^{3 \times 3} \text{ und } \mathbf{R} \in \mathbb{R}^{2 \times 2}$$

Methode:

Gram-Schmidt-Orthonormalisierung auf den Spalten von $\mathbf{A}$

Schritt 1: Erste Spalte $\mathbf{a}_1 = (1 \ 0 \ 0)^\top$

- Normalisierung: $\mathbf{q}_1 = \frac{\mathbf{a}_1}{\|\mathbf{a}_1\|} = (1 \ 0 \ 0)^\top \implies R_{11} = \|\mathbf{a}_1\| = 1$

Schritt 2: Zweite Spalte $\mathbf{a}_2 = (2 \ 0 \ 2)^\top$

- Projektion auf $\mathbf{q}_1$: $R_{12} = \mathbf{q}_1^\top \mathbf{a}_2 = 2$
- Orthogonalisierung: $\tilde{\mathbf{q}}_2 = \mathbf{a}_2 - R_{12}\mathbf{q}_1 = (2 \ 0 \ 2)^\top - (2 \ 0 \ 0)^\top = (0 \ 0 \ 2)^\top$
- Normalisierung: $\mathbf{q}_2 = \frac{\tilde{\mathbf{q}}_2}{\|\tilde{\mathbf{q}}_2\|} = (0 \ 0 \ 1)^\top, R_{22} = \|\tilde{\mathbf{q}}_2\| = 2$

Schritt 3: Vervollständige zu Orthonormalbasis von $\mathbb{R}^3$: wähle $\mathbf{q}_3 \perp \mathbf{q}_1, \mathbf{q}_2$, z.B. $\mathbf{q}_3 = (0 \ 1 \ 0)^\top$

Ergebnis: $\mathbf{Q} = (\mathbf{q}_1 \quad \mathbf{q}_2 \quad \mathbf{q}_3) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}, \quad \mathbf{R} = \begin{pmatrix} 1 & 2 \\ 0 & 2 \end{pmatrix}$ (*Verifikation* $\rightarrow$ Anhang)

QR-Zerlegung: Idee

Theorem

Sei $\mathbf{A} \in \mathbb{R}^{m \times n}$ mit $m \geq n$, $\mathbf{b} \in \mathbb{R}^m$ und $\mathbf{Q} \in \mathbb{R}^{m \times m}$ eine Orthogonalmatrix, sodass

$$\mathbf{Q}^\top \mathbf{A} = \begin{pmatrix} \mathbf{R} \\ \mathbf{0}_{(m-n) \times n} \end{pmatrix} \quad \text{und} \quad \mathbf{Q}^\top \mathbf{b} = \begin{pmatrix} \mathbf{c}_1 \\ \mathbf{c}_2 \end{pmatrix}$$

mit $\mathbf{R} \in \mathbb{R}^{n \times n}$ invertierbar. Dann gilt

$$\hat{\mathbf{x}} = \mathbf{R}^{-1} \mathbf{c}_1 = \arg \min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2.$$

Numerisch bilden wir $\mathbf{R}^{-1}$ **nicht** explizit, sondern lösen das Dreieckssystem $\mathbf{R}\hat{\mathbf{x}} = \mathbf{c}_1$ durch Rückwärtssubstitution.

Beweis:

- Es gilt $\|\mathbf{Ax} - \mathbf{b}\|^2 = \|\mathbf{Q}^\top \mathbf{Ax} - \mathbf{Q}^\top \mathbf{b}\|^2 = \|\mathbf{Rx} - \mathbf{c}_1\|^2 + \|\mathbf{c}_2\|^2$.
- Weil der zweite Term nicht von $\mathbf{x}$ abhängt, gilt

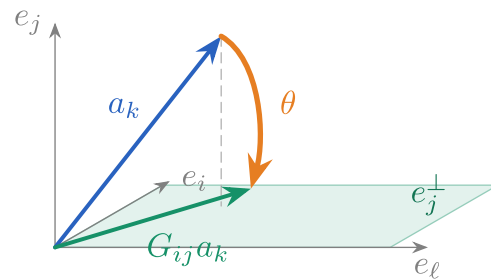
$$\arg \min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\| = \arg \min_{\mathbf{x}} \|\mathbf{Rx} - \mathbf{c}_1\|.$$

- $\hat{\mathbf{x}} = \mathbf{R}^{-1} \mathbf{c}_1$ ist Lösung, weil $\|\mathbf{R}\hat{\mathbf{x}} - \mathbf{c}_1\| = \|\mathbf{RR}^{-1} \mathbf{c}_1 - \mathbf{c}_1\| = 0$.

Konstruktion von Q

- Wie beim Gauß-Verfahren erzeugen wir R durch Nullen unter der Diagonalen. Dazu multiplizieren wir A von links mit **orthogonalen** Transformationen; ihr Produkt ist $Q^\top$.
- In Spalte k sollen die $m - k$ Einträge unter a_{kk} null werden:
- Givens-Rotation:**
- Householder-Reflexion:**

Givens-Rotation

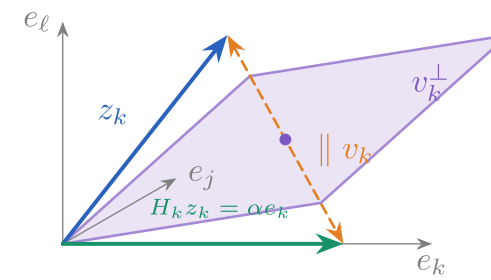


$G_{ij}(\theta)$ dreht die Projektion von a_k auf $\text{span}(e_i, e_j)$ auf e_i .
So wird **ein** ausgewählter Eintrag a_{jk} null.

Bei üblicher Drehrichtung gilt

$$\theta = -\text{atan } 2(A_{jk}, A_{ik}).$$

Householder-Reflexion



Definiere den nach oben mit Nullen aufgefüllten Rest der k -ten Spalte von A : $z_k := \sum_{\ell=k}^m A_{\ell k} e_\ell$.
Spiegelebene $v_k^\perp$ halbiert den Winkel zwischen z_k und e_k .
Nach Reflexion gilt $a_{\ell k}^{\text{neu}} = 0$ **für alle** $\ell > k$, Einträge mit $\ell < k$ bleiben unverändert.

$$\text{Wähle } \alpha = \pm \|z_k\|_2, \quad z_k \neq \alpha e_k, \quad v_k = \frac{z_k - \alpha e_k}{\|z_k - \alpha e_k\|_2},$$

$$H_k = I - 2v_k v_k^\top, \quad H_k z_k = \alpha e_k.$$

Pseudoinverse und Kleinste Quadrate

SVD-Lösung:

$$\mathbf{x} = \mathbf{A}^+ \mathbf{b} = \mathbf{V}_r \mathbf{\Sigma}_r^{-1} \mathbf{U}_r^\top \mathbf{b}$$

Warum funktioniert das?

$$\mathbf{A}\mathbf{x} = \mathbf{A}\mathbf{A}^+ \mathbf{b} = \text{proj}_{\text{col}(\mathbf{A})} \mathbf{b}$$

Vorteile gegenüber Normalengleichungen:

- Arbeitet direkt mit $\mathbf{A}$, nicht $\mathbf{A}^\top \mathbf{A} \rightarrow$ numerisch stabiler
- Funktioniert auch für rangdefiziente Matrizen ($\text{rang}(\mathbf{A}) < n$) – liefert dann die KQ-Lösung mit minimaler Norm:

$$\mathbf{A}^+ \mathbf{b} = \arg \min_{\substack{\mathbf{x} \in \mathbb{R}^n \\ \mathbf{A}^\top \mathbf{A} \mathbf{x} = \mathbf{A}^\top \mathbf{b}}} \|\mathbf{x}\|_2$$

- Explizite Konstruktion der Lösung, klare geometrische Interpretation

Demo: Ein KQ-Problem, drei Verfahren

Lineare Regression mit **fast kollinearen** Spalten — die wahre Lösung ist bekannt:

```
1 set.seed(1)
2 m <- 100
3 x <- runif(m)
4 A <- cbind(1, x, x + 1e-6 * runif(m)) # 3. Spalte fast identisch zur 2.
5 beta_wahr <- c(1, 2, 3)
6 b <- A %*% beta_wahr # exakte Daten: Loesung = beta_wahr
7 kappa(A, exact = TRUE)
```

[1] 6592188

- $\kappa_2(\mathbf{A}) \approx 7 \cdot 10^6 \implies \kappa_2(\mathbf{A}^\top \mathbf{A}) = \kappa_2(\mathbf{A})^2 \approx 4 \cdot 10^{13}$

Vorhersage: Doppelte Gleitkommazahlen haben ≈ 16 Dezimalstellen. Wie viele *korrekte* Stellen von β erwarten Sie

- für die Normalgleichungen (Fehler $\sim \kappa_2(\mathbf{A})^2 \epsilon_{mach}$)?
- für QR/SVD (Fehler $\sim \kappa_2(\mathbf{A}) \epsilon_{mach}$)?

Demo: Ergebnisse

```
1 beta_ne <- solve(t(A) %*% A, t(A) %*% b) # Normalgleichungen (ohne Inverse!)
2 beta_qr <- qr.solve(A, b)               # QR-Zerlegung
3 s       <- svd(A)
4 beta_svd <- s$v %*% ((t(s$u) %*% b) / s$d) # SVD: A+ b
5 fehler   <- function(beta_hat) max(abs(beta_hat - beta_wahr))
6 sapply(list("Normalengl." = beta_ne, QR = beta_qr, SVD = beta_svd), fehler)
```

Normalengl.	QR	SVD
8.77e-03	1.19e-11	1.55e-10

- **Normalgleichungen:**

Fehler $\approx 10^{-2}$ —

passt zu $\kappa_2(\mathbf{A})^2 \epsilon_{mach} \approx 4 \cdot 10^{13} \cdot 2 \cdot 10^{-16} \approx 10^{-2}$

⇒ nur noch **2–3 korrekte Stellen**, obwohl wir die Matrixinversion vermieden haben:
die *Quadrierung der Konditionszahl* ist das Problem, nicht (nur) `solve(M)`.

- **QR/SVD:**

Fehler $\approx 10^{-10}$ — passt zu $\kappa_2(\mathbf{A}) \epsilon_{mach} \approx 10^{-9}$

⇒ **ca. 10 korrekte Stellen** statt nur 2–3 bei den Normalgleichungen.

Vergleich der Lösungsverfahren

Verfahren	Ansatz	Komplexität ($m \gg n$)	Stabilität
Cholesky auf $\mathbf{A}^\top \mathbf{A}$	Normalengleichungen	$O(mn^2)$, kleinste Konstante ($\approx mn^2$)	Fehler $O(\kappa_2(\mathbf{A})^2 \epsilon_{mach})$ – potenziell instabil
QR-Zerlegung	löse $\mathbf{R}\hat{\mathbf{x}} = \mathbf{c}_1$	$O(mn^2)$ ($\approx 2mn^2$, Householder)	stabil – Standardwahl
SVD	$\hat{\mathbf{x}} = \mathbf{A}^+ \mathbf{b} = \mathbf{V}_r \mathbf{\Sigma}_r^{-1} \mathbf{U}_r^\top \mathbf{b}$	$O(mn^2)$, größte Konstante	am stabilsten, auch für $\text{rang}(\mathbf{A}) < n$

- Für $m \gg n$ haben alle drei Verfahren dieselbe Komplexitätsordnung $O(mn^2)$ – die Konstanten (und die Stabilität!) unterscheiden sich aber deutlich: Normalengleichungen + Cholesky brauchen etwa halb so viele Operationen wie Householder-QR, die SVD ein Mehrfaches davon.
- SVD besonders sinnvoll, wenn $\mathbf{A}$ fast singulär ist ($\sigma_{\min}(\mathbf{A}) \approx 0 \Rightarrow \kappa(\mathbf{A}) \gg 1$): die $r - k$ kleinsten Singulärwerte auf 0 setzen $\rightarrow$ **Rang-k-Approximation** als Regularisierung.

Self-Check & Zusammenfassung

Wrap-up

Heute gelernt

- **Kondition des KQ-Problems:**
bzgl. $\mathbf{b}$ abhängig vom Winkel zwischen $\mathbf{b}$ und $\text{col}(\mathbf{A})$,
bzgl. $\mathbf{A}$ typischerweise $O(\kappa_2(\mathbf{A})^2)$.
- $\kappa_2(\mathbf{A}^\top \mathbf{A}) = \kappa_2(\mathbf{A})^2 \implies$ **Normalgleichungen** können instabil sein – und: **Invertiere niemals eine Matrix!**
- Drei Lösungsverfahren:
 - **Cholesky** (schnell, heikel),
 - **QR-Zerlegung** (Standardwahl),
 - **SVD** (am stabilsten, auch für rangdefiziente $\mathbf{A}$).

Nächste Vorlesung

- Probabilistische Methoden
- Iterative Methoden

Self-Check (1/2)

Frage 1: Welches Verfahren ist für KQ-Probleme mit $\text{rang}(\mathbf{A}) = n$ die übliche **Standardwahl** (guter Kompromiss aus Stabilität und Geschwindigkeit)?

1. Cholesky auf $\mathbf{A}^\top \mathbf{A}$
2. QR-Zerlegung
3. SVD
4. Explizite Inversion: `solve(t(A) %*% A) %*% t(A) %*% b`

Frage 2: Für $m \gg n$ gilt für die Komplexität von Cholesky, QR und SVD:

1. Alle drei haben dieselbe Ordnung $O(mn^2 + n^3)$, aber deutlich verschiedene Konstanten.
2. Cholesky ist $O(mn)$, QR ist $O(mn^2)$, SVD ist $O(m^2n^2)$.
3. Die SVD hat exponentielle Laufzeit.
4. QR ist immer am langsamsten.

Self-Check (2/2)

Frage 3: Die Kondition des KQ-Problems bezüglich $\mathbf{b}$ explodiert ($\kappa \rightarrow \infty$) genau dann, wenn ...

1. $\mathbf{b} \in \text{col}(\mathbf{A})$.
2. $\mathbf{b} \perp \text{col}(\mathbf{A})$, d.h. $\text{proj}_{\text{col}(\mathbf{A})}\mathbf{b} = \mathbf{0}$.
3. das Residuum $\mathbf{r} = \mathbf{0}$ ist.
4. $\mathbf{A}$ eine Orthogonalmatrix ist.

Frage 4: Warum sind die Normalengleichungen numerisch heikel, selbst wenn man die explizite Inversion vermeidet?

1. $\mathbf{A}^\top \mathbf{A}$ ist nie invertierbar.
2. Vorwärts-/Rückwärtssubstitution ist instabil.
3. Der Übergang zu $\mathbf{A}^\top \mathbf{A}$ quadriert die Konditionszahl: $\kappa_2(\mathbf{A}^\top \mathbf{A}) = \kappa_2(\mathbf{A})^2$.
4. Die Cholesky-Zerlegung existiert für SPD-Matrizen nicht.

Anhang

Beweis: Kondition bezüglich $\mathbf{b}$

Behauptung: $\frac{\|f(\tilde{\mathbf{b}}) - f(\mathbf{b})\|}{\|f(\mathbf{b})\|} \leq \kappa_2(\mathbf{A}) \frac{\|\mathbf{b}\|}{\|\text{proj}_{\text{col}(\mathbf{A})}\mathbf{b}\|} \cdot \frac{\|\tilde{\mathbf{b}} - \mathbf{b}\|}{\|\mathbf{b}\|}$ für $f(\mathbf{b}) = \mathbf{A}^+\mathbf{b}$.

$$\begin{aligned} \frac{\|\mathbf{A}^+\tilde{\mathbf{b}} - \mathbf{A}^+\mathbf{b}\|}{\|\mathbf{A}^+\mathbf{b}\|} &= \frac{\|\mathbf{A}^+(\tilde{\mathbf{b}} - \mathbf{b})\|}{\|\mathbf{A}^+\mathbf{b}\|} \\ &\leq \frac{\|\mathbf{A}^+\| \|\tilde{\mathbf{b}} - \mathbf{b}\|}{\|\mathbf{A}^+\mathbf{b}\|} && [\text{Submultiplikativitat}] \\ &= \frac{\kappa_2(\mathbf{A})}{\|\mathbf{A}\| \cdot \|\mathbf{A}^+\mathbf{b}\|} \|\tilde{\mathbf{b}} - \mathbf{b}\| && [\text{Def. von } \kappa_2(\mathbf{A})] \\ &\leq \frac{\kappa_2(\mathbf{A})}{\|\mathbf{A}\mathbf{A}^+\mathbf{b}\|} \|\tilde{\mathbf{b}} - \mathbf{b}\| && [\text{Submultiplikativitat}] \end{aligned}$$

und mit $\mathbf{A}\mathbf{A}^+\mathbf{b} = \text{proj}_{\text{col}(\mathbf{A})}\mathbf{b}$ sowie Erweitern mit $\|\mathbf{b}\|/\|\mathbf{b}\|$ folgt die Behauptung. ■

↔ zurück: „Kondition bezüglich $\mathbf{b}$ “

Beispiel: QR-Zerlegung – Verifikation

Aus dem Hauptteil: $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 0 & 0 \\ 0 & 2 \end{pmatrix}$, berechnet via Gram-Schmidt:

- $\mathbf{q}_1 = (1 \ 0 \ 0)^\top$; $\mathbf{q}_2 = (0 \ 0 \ 1)^\top$; $\mathbf{q}_3 = (0 \ 1 \ 0)^\top$
- $R_{11} = 1$; $R_{12} = 2$; $R_{22} = 2$

Verifikation:

$$\mathbf{Q} \begin{pmatrix} \mathbf{R} \\ \mathbf{0} \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 0 & 2 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 2 \\ 0 & 0 \\ 0 & 2 \end{pmatrix} = \mathbf{A} \checkmark$$

und $\mathbf{Q}$ ist orthogonal: $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}_3$ (Spalten sind orthonormale Vektoren). $\checkmark$

↩ zurück: „Beispiel: QR-Zerlegung“

Numerische Lineare Algebra: Iteration & Zufall

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)

Material: Thomas Nagler

LMU München

Verwendete Vorkenntnisse

- Komplexitätsanalyse / $O()$ -Notation
- Stabilität und Konditionierung
- Orthogonalmatrizen
- Spektralnorm $\| \cdot \|_2$
- LGS: Problem, Direkte Algorithmen
- **Lineare Algebra:**
 - Eigenwerte und Eigenvektoren
 - Ähnlichkeitstransformationen: $B = Q A Q^{-1}$
- **Wahrscheinlichkeitstheorie:** Zufallsvektoren
- **Analysis I:**
 - Konvergenz von Folgen
 - Geometrische Reihe $\sum_k p^k$ für $|p| < 1$

Opener-Quiz: Was wissen Sie noch?

Frage 1: Sei $A \in \mathbb{R}^{n \times n}$ und $v \neq 0$ ein Eigenvektor mit $Av = \lambda v$. Welche Aussage gilt dann **immer**?

1. $A^k v = \lambda^k v$ für alle $k \in \mathbb{N}$
2. Es muss $\|v\| = 1$ gelten
3. $\lambda > 0$
4. v ist der einzige Eigenvektor zu λ

Frage 2: Sei $|p| < 1$. Was ist $\sum_{k=0}^{\infty} p^k$?

1. $\frac{1}{1-p}$
2. $\frac{p}{1-p}$
3. $\frac{1}{p-1}$
4. Die Reihe divergiert.

Überblick

- Numerische lineare Algebra ist ein sehr großes Feld.
- Wir haben bisher die wichtigsten Lösungsmethoden gesehen.
- Die Kernidee dabei war *Matrixzerlegung* (*LU*, *QR*, *Cholesky*, ...).
- Heute sehen wir in kürzerer Form zwei weitere wichtige Kernideen:
 - *Iterative Methoden*
 - verkürzen *Laufzeit* auf Kosten von Genauigkeit.
 - *Probabilistische Methoden*
 - ersetzen große Probleme durch kleinere mit *höchstwahrscheinlich* sehr ähnlicher Lösung.
- An beiden wird in der numerischen linearen Algebra aktiv geforscht.

Eigenwertprobleme

Lösungsverfahren für Eigenwertprobleme

- *Direkte Methoden existieren:*
 - Charakteristisches Polynom: $\det(\mathbf{A} - \lambda \mathbf{I}) = 0 \rightarrow$ Nullstellensuche
 - Problem:
Polynom vom Grad n für $n \times n$ Matrix $\rightarrow$ keine direkte Lösung für $n \geq 5$
 - Numerisch instabil und teuer: $O(n^4)$ oder schlechter
- *Wann sind direkte Methoden unzureichend?*
 - Große Matrizen ($n > 1000$): $O(n^3)$ oder schlechter
 - Dünnbesetzte Matrizen: Direkte Methoden zerstören *sparsity*-Struktur
 - Oft brauchen wir nur wenige Eigenwerte (z.B. die größten $k \ll n$)
- *Warum iterativ?*
 - Schneller wenn nur wenige Eigenwerte benötigt
 - Nutzen *sparsity*-Struktur aus (nur Matrix-Vektor-Multiplikationen!)
 - Trade-off “Genauigkeit vs. Aufwand” ist steuerbar

Potenzmethode

Die **Potenzmethode** (nach Mises) liefert über Iteration den größten (separierten!) Eigenwert λ_1 und Eigenvektor $\mathbf{v}_1$ einer Matrix $\mathbf{A}$:

Idee:

Mit $\mathbf{x}^{(k)} = \frac{\mathbf{A}\mathbf{x}^{(k-1)}}{\|\mathbf{A}\mathbf{x}^{(k-1)}\|}$ gilt

$$\lambda^{(k)} = \|\mathbf{A}\mathbf{x}^{(k)}\| \xrightarrow{k \rightarrow \infty} |\lambda_1| \text{ und } \mathbf{x}^{(k)} \xrightarrow{k \rightarrow \infty} \pm \mathbf{v}_1$$

(Die Norm liefert nur den Betrag $|\lambda_1|$ – das Vorzeichen von λ_1 liefert sie nicht; für negatives λ_1 oszilliert $\mathbf{x}^{(k)}$ zwischen $\pm \mathbf{v}_1$.)

Intuition:

- Für diagonalisierbare $\mathbf{A}$ ist jeder Startvektor Linearkombination der Eigenvektoren $\mathbf{v}_i$ von $\mathbf{A}$:
 $\mathbf{x}^{(0)} = \sum_i^n c_i \mathbf{v}_i$
- Jede Iteration verstärkt die Komponente von $\mathbf{x}^{(k)}$ in Richtung $\mathbf{v}_1$ mehr als die anderen Komponenten

Algebraisch:

Wegen $\mathbf{A}\mathbf{v} = \lambda\mathbf{v} \implies \mathbf{A}^k\mathbf{v} = \lambda^k\mathbf{v}$ gilt auch:

- $\mathbf{A}^k \mathbf{x}^{(0)} = \lambda_1^k \left(\mathbf{c}_1 \mathbf{v}_1 + \sum_{i=2}^n \mathbf{c}_i \left(\frac{\lambda_i}{\lambda_1} \right)^k \mathbf{v}_i \right) \approx \lambda_1^k \mathbf{c}_1 \mathbf{v}_1$ für $k \rightarrow \infty$ weil

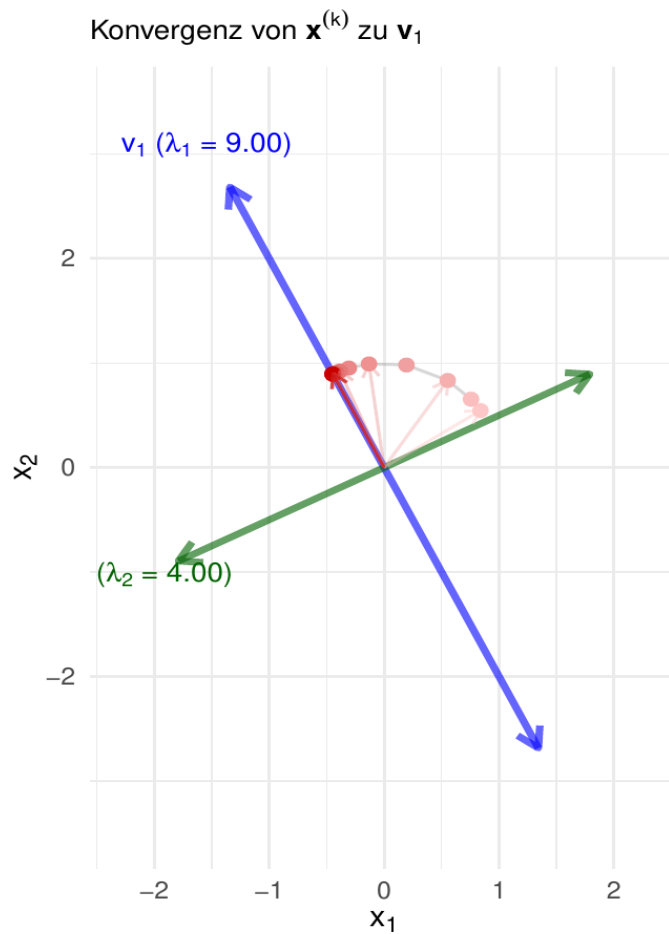
$$\left(\frac{\lambda_i}{\lambda_1} \right)^k \xrightarrow{k \rightarrow \infty} 0 \text{ für } i \geq 2$$

- $\frac{\mathbf{A}^k \mathbf{x}^{(0)}}{\|\mathbf{A}^k \mathbf{x}^{(0)}\|} \approx \frac{\lambda_1^k \mathbf{c}_1 \mathbf{v}_1}{|\lambda_1^k \mathbf{c}_1| \|\mathbf{v}_1\|} = \pm \mathbf{v}_1$

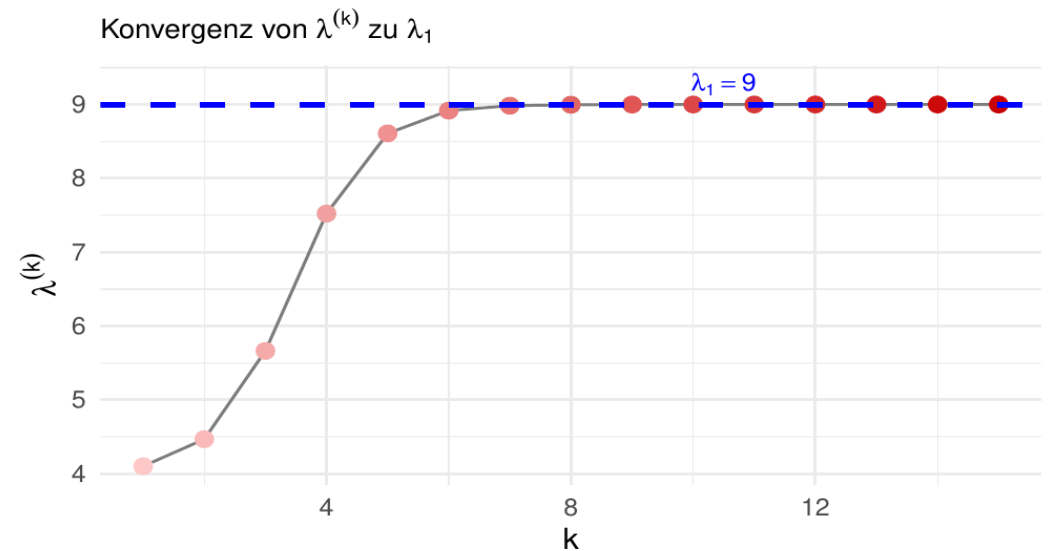
Potenzmethode: Beispiel

$$A = \begin{pmatrix} 5 & -2 \\ -2 & 8 \end{pmatrix} \implies \lambda_1 = 9, \lambda_2 = 4; \mathbf{v}_1 = \frac{1}{\sqrt{5}} \begin{pmatrix} -1 \\ 2 \end{pmatrix}, \mathbf{v}_2 = \frac{1}{\sqrt{5}} \begin{pmatrix} 2 \\ 1 \end{pmatrix} \text{ mit } \mathbf{x}^{(0)} = \begin{pmatrix} 2 \\ 1.3 \end{pmatrix}$$

:



k	Winkel zu v[1]
0	83.54°
1	75.71°
2	60.18°
3	37.79°
5	8.71°
10	0.15°
15	0.00°



Quiz

Für welche $\boldsymbol{x}^{(0)}$ versagt die Potenzmethode?

1. $\boldsymbol{x}^{(0)} = c\boldsymbol{v}_1$

2. $\boldsymbol{x}^{(0)} = c\boldsymbol{v}_2$

3. $\boldsymbol{x}^{(0)} = c(\boldsymbol{v}_1 + \boldsymbol{v}_2)$

Iterative Berechnungsverfahren

- Wir wollen (alle) Eigenwerte/-vektoren von $\mathbf{A} \in \mathbb{R}^{n \times n}$ finden.
- Idee: Überführe $\mathbf{A}$ in spezielle Matrix $\mathbf{B}$, so dass das Problem leichter wird.

Theorem

Sei $\mathbf{B}$ eine zu $\mathbf{A}$ ähnliche Matrix (d.h. $\mathbf{B} = \mathbf{Q}\mathbf{A}\mathbf{Q}^{-1}$). Dann gilt:

- $\mathbf{A}$ und $\mathbf{B}$ haben die gleichen Eigenwerte (und damit auch gleiche Determinante und Spur).
- Ist $\mathbf{y}$ Eigenvektor von $\mathbf{B}$, so ist $\mathbf{x} = \mathbf{Q}^{-1}\mathbf{y}$ Eigenvektor von $\mathbf{A}$ zum gleichen Eigenwert.

Beweis:

$$\mathbf{B}\mathbf{y} = \lambda\mathbf{y} \implies \mathbf{Q}\mathbf{A}\mathbf{Q}^{-1}\mathbf{y} = \lambda\mathbf{y} \implies \mathbf{A}\mathbf{Q}^{-1}\mathbf{y} = \lambda\mathbf{Q}^{-1}\mathbf{y} \implies \mathbf{A}\mathbf{x} = \lambda\mathbf{x}.$$

Beispiel

$$\mathbf{A} = \begin{pmatrix} 5 & -2 \\ -2 & 8 \end{pmatrix}, \quad \mathbf{Q} = \frac{1}{\sqrt{5}} \begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} \text{ mit } \mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$$

- $\mathbf{Q}$ ist **orthogonal**, also $\mathbf{Q}^{-1} = \mathbf{Q}^\top$: das Beispiel spezialisiert die Ähnlichkeitstransformation $\mathbf{Q}\mathbf{A}\mathbf{Q}^{-1}$ auf *orthogonale* Ähnlichkeit $\mathbf{Q}\mathbf{A}\mathbf{Q}^\top$.

$$\mathbf{B} = \mathbf{Q}(\mathbf{A}\mathbf{Q}^\top) = \frac{1}{5} \begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} \begin{pmatrix} 8 & 9 \\ 4 & -18 \end{pmatrix} = \begin{pmatrix} 4 & 0 \\ 0 & 9 \end{pmatrix}$$

(Rechnung $\rightarrow$ Anhang)

- $\mathbf{B}$ diagonal $\implies$ Eigenwerte direkt ablesbar: 4 und 9
- **Der Punkt:**
Ähnliche Matrizen haben dieselben Eigenwerte – $\mathbf{A}$ hat also ebenfalls die Eigenwerte 4 und 9, obwohl man sie an $\mathbf{A}$ nicht ablesen kann.

Verifikation:

$$\det(\mathbf{A} - \lambda \mathbf{I}) = (5 - \lambda)(8 - \lambda) - 4 = \lambda^2 - 13\lambda + 36 = (\lambda - 4)(\lambda - 9) = 0 \quad \checkmark$$

Verfahren

Zur (iterativen) Berechnung einer zu $\mathbf{A}$ ähnlichen Matrix $\mathbf{B}^{(k)}$ deren Eigenwerte einfach zu bestimmen sind gibt es verschiedenste Ideen/Methoden:

- **QR-Iteration** führt zu (approximativer) Dreiecksmatrix $\mathbf{B}$ (*Schur-Zerlegung*).
 - ähnlich für **LU**- und **Cholesky**-Iteration
- *Lanczos-Iteration* führt zu tridiagonalem $\mathbf{B}$ ($b_{ij} = 0$ für $|i - j| > 1$) und kann auf die “wichtigsten” Eigenwerte eingeschränkt werden. Effizienter für große, dünn besetzte Matrizen.
- In der Praxis brauchen wir zusätzliche numerische Tricks, um diese Methoden **stabil** und akzeptabel schnell zu machen: *Shifted QR*, *Implicitly Restarted Lanczos*

QR-Iteration

Def.: QR-Iteration

Die **QR-Iteration** nutzt die *QR-Zerlegung* zur Iteration:

1. Starte mit $\mathbf{A}^{(0)} = \mathbf{A}$
2. Für $k = 1, 2, \dots$:
 - Berechne QR-Zerlegung: $\mathbf{A}^{(k-1)} = \mathbf{Q}^{(k)} \mathbf{R}^{(k)}$
 - Setze $\mathbf{A}^{(k)} = (\mathbf{Q}^{(k)})^\top \mathbf{A}^{(k-1)} \mathbf{Q}^{(k)} = \mathbf{R}^{(k)} \mathbf{Q}^{(k)}$

- $\mathbf{A}^{(k)}$ sind ähnlich zu $\mathbf{A}$:

$$\mathbf{A}^{(k)} = \mathbf{R}^{(k)} \mathbf{Q}^{(k)} = (\mathbf{Q}^{(k)})^{-1} \mathbf{A}^{(k-1)} \mathbf{Q}^{(k)} = (\mathbf{Q}^{(1)} \dots \mathbf{Q}^{(k)})^{-1} \mathbf{A} (\mathbf{Q}^{(1)} \dots \mathbf{Q}^{(k)})$$

- Für $k \rightarrow \infty$ konvergiert $\mathbf{A}^{(k)}$ gegen eine obere Dreiecksmatrix (!) $\rightarrow$ Eigenwerte auf der Diagonalen (für symmetrisches $\mathbf{A}$ mit separierten $|\lambda_i|$; allgemein: Schur-Form)
- Berechnung der Eigenvektoren $\mathbf{V}$ über Produkt $\mathbf{Q}_k := \prod_{j=1}^k \mathbf{Q}^{(j)}$ (für symmetrische $\mathbf{A}$).

Beweis, dass $\mathbf{A}^k = \mathbf{Q}_k \mathbf{R}_k$: siehe **Anhang**

QR-Iteration: Interpretation

- Die QR-Iteration berechnet iterativ die **QR-Zerlegung** von A^k :

$$A^k = Q_k R_k \text{ mit } Q_k := Q^{(1)} \dots Q^{(k)}$$

(Beweis $\rightarrow$ **Anhang**)

- Spalten von Q_k sind orthonormalisierte Versionen von $A^k e_1, A^k e_2, \dots, A^k e_n$
 $\rightarrow$ entspricht **simultaner Potenzmethode** auf **allen** Einheitsvektoren!

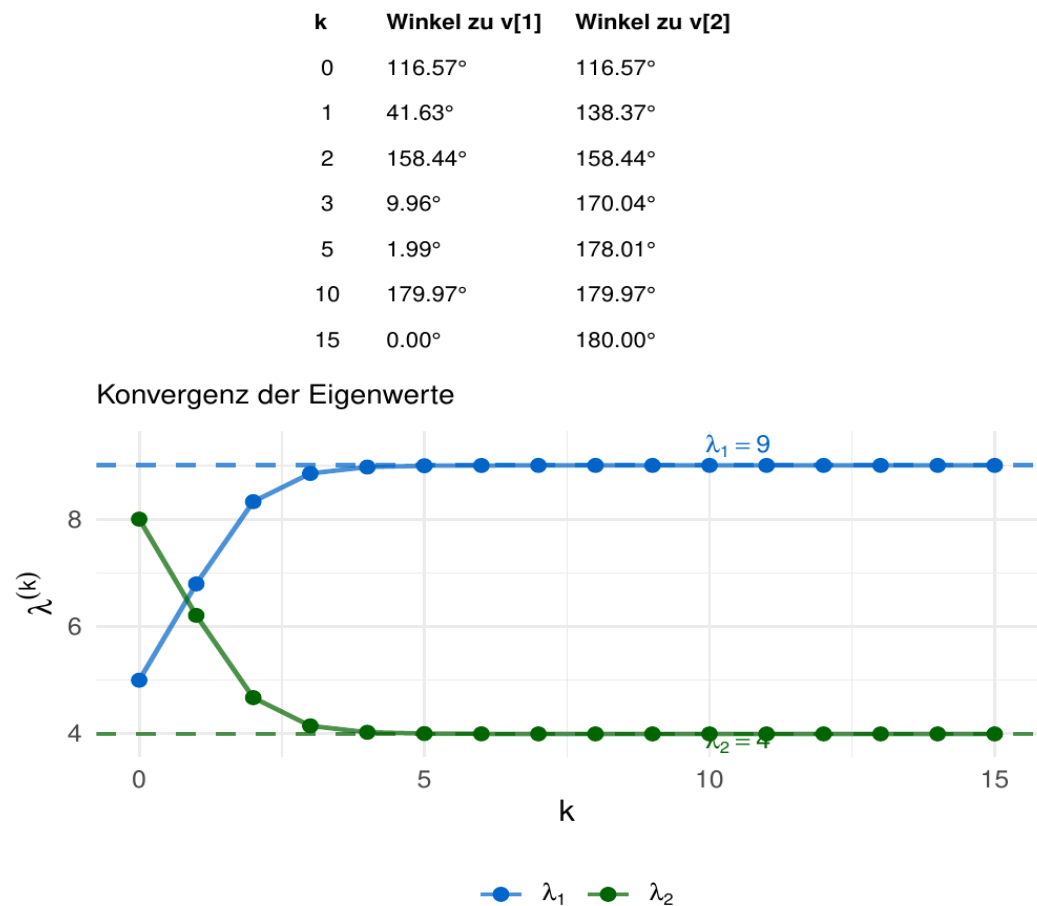
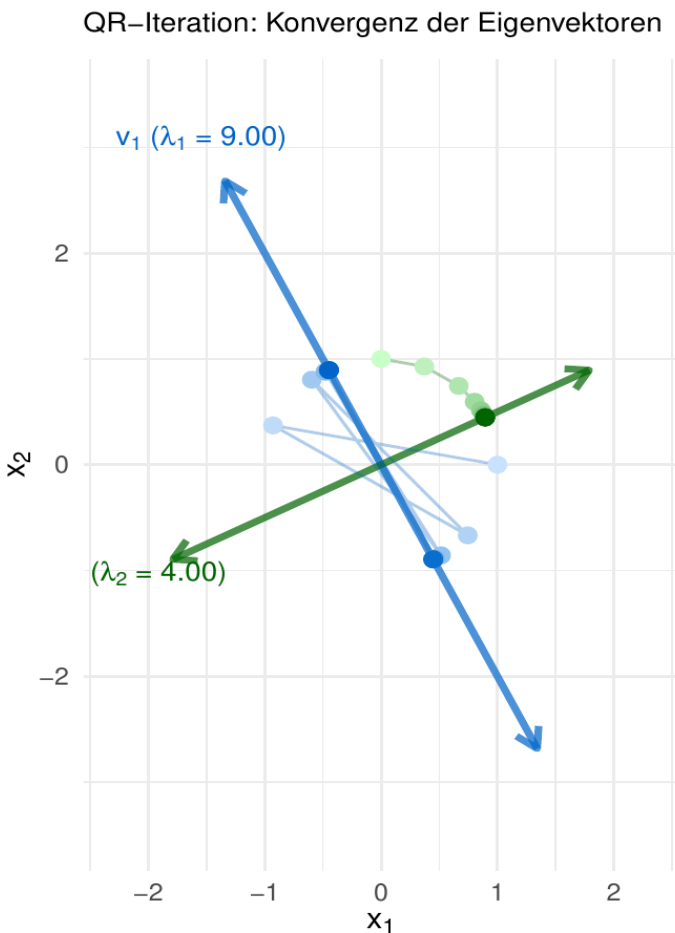
$$\implies Q_k \xrightarrow{k \rightarrow \infty} V$$

- $A^{(k)} = Q_k^\top A Q_k$ konvergiert wegen $Q_k \xrightarrow{k \rightarrow \infty} V$ gegen eine obere Dreiecksmatrix mit Eigenwerten auf der Diagonale (**Schur-Zerlegung**);
für **symmetrische** $A = V \Lambda V^\top$ (mit **orthogonalem** V) sogar gegen die Diagonalmatrix Λ :

$$Q_k^\top A Q_k \xrightarrow{k \rightarrow \infty} V^\top A V = V^\top V \Lambda V^\top V = \Lambda$$

QR-Iteration: Beispiel

$$A = \begin{pmatrix} 5 & -2 \\ -2 & 8 \end{pmatrix} \implies \lambda_1 = 9, \lambda_2 = 4; \mathbf{v}_1 = \frac{1}{\sqrt{5}} \begin{pmatrix} -1 \\ 2 \end{pmatrix}, \mathbf{v}_2 = \frac{1}{\sqrt{5}} \begin{pmatrix} 2 \\ 1 \end{pmatrix}$$



Bemerkungen

Konditionierung von Eigenwertproblemen:

- Eigenwertprobleme sind *gut konditioniert*, wenn die Matrix nahezu *symmetrisch* ist.
- Eigenvektorprobleme sind *schlecht konditioniert*, wenn *Eigenwerte nahe beieinander* liegen.

Laufzeit:

- Jede QR-Iteration ist $O(n^3)$
- Insgesamt werden oft nur wenige Iterationen benötigt (abhängig von Eigenwertverteilung).
- Iterative *SVD*-Verfahren (z.B. *Lanczos Bi-Diagonalization*; `{irlba}`) sind noch schneller für große, dünn besetzte Matrizen

Anwendung: Google PageRank

Problem:

Rangordnung von $n \approx 10^{10}$ Webseiten im Internet bilden.

Modell:

Jede Seite i hat einen “Wichtigkeits-Score” x_i , der die Wichtigkeit der Seiten verlinkt auf i gewichtet:

$$x_i = \sum_{j:j \rightarrow i} \frac{x_j}{d_j} \iff \mathbf{x} = \mathbf{A}\mathbf{x} \text{ mit } \mathbf{A} = \left[\frac{I(j \rightarrow i)}{d_j} \right]_{i,j} \implies \text{Eigenwertproblem mit } \lambda = 1!$$

(“ $j \rightarrow i$ ” heißt “Seite j verlinkt auf i ”, d_j sei Anzahl ausgehender Links von j)

(Setzt voraus, dass jede Seite mindestens einen ausgehenden Link hat — sonst ist $d_j = 0$ und die Spalte j gar nicht definiert; solche Spalten füllt man gleichverteilt auf.)

Frage: Warum kann Google keine $10^{10} \times 10^{10}$ -Matrix invertieren — und warum funktioniert die Eigenwert-Berechnung trotzdem?

- Direkte Methoden unmöglich: $\mathbf{A}$ dicht gespeichert wären 10^{20} Einträge (≈ 800 Exabyte), Inversion/Zerlegung $O(n^3) = 10^{30}$ Operationen.
- Aber: $\mathbf{A}$ ist *extrem dünn besetzt* (jede Seite hat nur ~ 10 Links) $\implies$ Speicherbedarf $O(n)$ statt $O(n^2)$
- **Potenzmethode:** $\mathbf{x}^{(k+1)} = \mathbf{A}\mathbf{x}^{(k)} / \|\mathbf{A}\mathbf{x}^{(k)}\|$ braucht nur Matrix-Vektor-Produkte: eine Iteration ist $O(n)$, Konvergenz in ~ 50 Iterationen
- **Gesamt:** $O(n)$ statt $O(n^3)$ — macht Google überhaupt erst möglich!

Anwendung: Hauptkomponentenanalyse (PCA)

Berechnung von Eigenwerten/-vektoren für Datenanalysen essentiell:

- **Problem:** Gegeben ist eine mittelwert-zentrierte Datenmatrix $\mathbf{X} \in \mathbb{R}^{n \times p}$ (n Beobachtungen, p Variablen)
- **Ziel:** Finde Richtungen maximaler Varianz zur Dimensionsreduktion
- **Lösung:** Berechne die Kovarianzmatrix $\mathbf{\Sigma} = \frac{1}{n-1} \mathbf{X}^\top \mathbf{X} \in \mathbb{R}^{p \times p}$
 - Eigenvektoren $\mathbf{v}_j$ von $\mathbf{\Sigma}$ sind die *Hauptachsen* bzw. *Loading-Richtungen* maximaler Varianz in $\mathbb{R}^p$
 - Die *Hauptkomponenten* (Scores) sind $\mathbf{z}_j = \mathbf{X} \mathbf{v}_j$; die Eigenwerte geben ihre Varianz an
- In der Praxis noch schneller/effizienter über *SVD* von $\mathbf{X}$ (z.B. `{irlba}: irlba()` (iterativ), `svdr()` (iterativ & probabilistisch))

Warum iterative Methoden?

- Moderne Anwendungen: $p = 10^4$ bis 10^6 (z.B. Gen-Expression, Bildanalyse, ...)
- Oft nur die k größten Eigenwerte benötigt ($k \ll p$) - vgl. *Bildkompression* durch *Reduzierte SVD*
- Lanczos-Iteration: Berechnet nur die k größten Eigenwerte in $O(kp^2)$ statt $O(p^3)$ für alle
 - Bei $p = 100.000$ und $k = 10$ ist Lanczos ~ 10.000 mal schneller!

Anwendung: Approximative SVD

Bildkompression durch **Reduzierte SVD**: X ist ein Graustufen-Foto als 2500×3300 -Matrix (*Setup-Code, Fehlervergleich & Bildpanel* → *Anhang*):

```
1 r <- 50
2 microbenchmark::microbenchmark(
3   svd_full    = svd(X),                # exakt: alle Singulaerwerte
4   svd_irlba   = irlba::irlba(X, nv = r, nu = r), # iterativ (Lanczos): nur r
5   svd_random  = irlba::svdr(X, k = r),      # iterativ & randomisiert
6   times = 5)
```

Unit: seconds

	expr	min	lq	mean	median	uq	max	neval	cld
svd_full		84.14	92.85	107.77	112.60	114.01	135.23	5	a
svd_irlba		1.98	2.09	2.85	2.13	3.46	4.58	5	b
svd_random		5.79	7.00	9.36	10.44	10.77	12.80	5	b

```
1 X_svdr <- with(irlba::svdr(X, k = r), u %*% diag(d) %*% t(v))
2 norm(X - X_svdr, "F") / norm(X, "F") # rel. Fehler der Rang-50-Approximation
```

[1] 0.129

→ iterative/randomisierte Verfahren sind hier $\sim 40\times$ bzw. $\sim 11\times$ schneller — bei praktisch identischem Ergebnis (Vergleich mit exakter Rang-50-SVD → *Anhang*).

Iterative Eigenwertverfahren: Vergleich

Methode	Komplexität / Iteration	Anwendung	Output
Potenz- methode	$O(\text{nnz}(\mathbf{A}))$	Größter Eigenwert/-vektor	1 Eigenpaar
(shifted) QR	$O(n^2)$ - $O(n^3)$	Alle Eigenwerte	n Eigenwerte
Lanczos	$O(m \text{ nnz}(\mathbf{A}) + m^2)$ für m EW	Wenige Extremal-EW, <i>sparse</i> $\mathbf{A}$	$m \ll n$ Eigenpaare

- Konvergenzrate **Potenzmethode**: $|\lambda_2/\lambda_1|$
- **QR-Iteration** mit Shifts oft mit quadratischer Konvergenz, am stabilsten (ohne Shifts nur linear, mit Rate $|\lambda_i/\lambda_j|$).
- Anwendung:
 - *sparse* Matrizen: Lanczos klar überlegen
 - Alle Eigenwerte, Speicher sekundär: (shifted) QR-Iteration
 - Wenige Eigenwerte: Potenzmethode (für 1), Lanczos (für $m \ll n$)

$\text{nnz}(\cdot)$ = Anzahl Nicht-Null-Einträge

Iterative Lösungsverfahren

Iterative Lösungsverfahren: Intuition

- Zur Lösung von **LGS** und **KQ-Problemen** haben wir bisher direkte Methoden gesehen — **iterative Verfahren** (meist **Fixpunktiterationen**) sind oft viel schneller, wenn man etwas Genauigkeit opfert.

Wie nähern wir uns iterativ der Lösung?

- **Residuum:**
 $\mathbf{r}^{(k)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(k)}$ misst, wie stark $\mathbf{x}^{(k)}$ die Gleichung $\mathbf{A}\mathbf{x} = \mathbf{b}$ verletzt
- Falls $\mathbf{A}\mathbf{x}^{(k)} = \mathbf{b}$, dann ist $\mathbf{r}^{(k)} = \mathbf{0}$ und wir haben die Lösung gefunden
- **Korrektur:**
Solange $\mathbf{r}^{(k)} \neq \mathbf{0}$, korrigieren wir unsere Schätzung:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \underbrace{\mathbf{C}\mathbf{r}^{(k)}}_{\text{Korrekturschritt}}$$

- $\mathbf{C}$ bestimmt, wie stark wir wo in Richtung des Residuums korrigieren
- **Analogie:** *Gradient Descent* - wir bewegen uns Schritt für Schritt zur Lösung

Warum nicht direkt lösen?

→ Bei großen und/oder dünnbesetzten Matrizen sind iterative Methoden oft um Größenordnungen schneller als direkte Verfahren!

Iterative Lösungsverfahren für LGS

- Sei $\mathbf{A} \in \mathbb{R}^{n \times n}$, $\mathbf{b} \in \mathbb{R}^n$ und $\mathbf{x}$ die Lösung des LGS $\mathbf{Ax} = \mathbf{b}$.
- Für einen beliebigen Startwert $\mathbf{x}^{(0)} \in \mathbb{R}^n$ und $\mathbf{C} \in \mathbb{R}^{n \times n}$, definiere die Iteration

$$\mathbf{x}^{(k)} = \mathbf{x}^{(k-1)} + \mathbf{C}(\mathbf{b} - \mathbf{Ax}^{(k-1)}), \quad k = 1, 2, \dots$$

Theorem

Sei $\mathbf{x}$ die eindeutige Lösung des LGS $\mathbf{Ax} = \mathbf{b}$.

Falls $\rho := \|\mathbf{I}_n - \mathbf{CA}\|_2 < 1$ (Spektralnorm), gilt für alle $k \geq 1$:

$$\|\mathbf{x}^{(k)} - \mathbf{x}\|_2 \leq \rho^k \|\mathbf{x}^{(0)} - \mathbf{x}\|_2.$$

- Insbesondere: $\lim_{k \rightarrow \infty} \mathbf{x}^{(k)} = \mathbf{x}$ (Konvergenz).

Beweis: siehe **Anhang**

Quiz

Sei

$$\mathbf{x}^{(k)} = (\mathbf{I} - \mathbf{C}\mathbf{A})\mathbf{x}^{(k-1)} + \mathbf{C}\mathbf{b}$$

mit $\mathbf{C} = \gamma \mathbf{I}_n$ für ein $\gamma > 0$. Was ist die Laufzeitkomplexität eines Iterationsschrittes?

1. $O(1)$
2. $O(n)$
3. $O(n^2)$
4. $O(n^3)$

Lösung: 3. – Mit $\mathbf{C} = \gamma \mathbf{I}$ ist $\mathbf{x}^{(k)} = \mathbf{x}^{(k-1)} - \gamma(\mathbf{A}\mathbf{x}^{(k-1)}) + \gamma\mathbf{b}$: pro Schritt eine dichte Matrix-Vektor-Multiplikation $\mathbf{A}\mathbf{x}^{(k-1)}$, also $O(n^2)$; Skalierung mit γ und die zwei Vektoradditionen sind nur $O(n)$.

Iterative Lösungsverfahren für LGS: Wahl von C

- Die Frage ist nun, wie wir C so wählen, dass:
 - $\rho = \|I - CA\|_2 < 1$ möglichst klein ist (optimal: $C \approx A^{-1}$),
 - $x^{(k)} = (I - CA)x^{(k-1)} + Cb$ einfach zu berechnen ist.

Beliebte Wahlen für C mit Iterationsaufwand $O(n^2)$:

- *Richardson*-Iteration: $C = \gamma I_n$; konvergiert z.B. für SPD A , falls $0 < \gamma < 2/\lambda_{\max}(A)$.
- *Jacobi*-Iteration: $C = \text{diag}(A_{11}, \dots, A_{nn})^{-1}$.
- *Gauss-Seidel*-Iteration: C^{-1} gleich unterem Dreieck von A .

Gesamtaufwand:

- Wegen $\|x^{(k)} - x\| \leq \rho^k \|x^{(0)} - x\|$ reichen $k = O(\log(1/\epsilon))$ Schritte für Genauigkeit ϵ ($\rightarrow$ Self-Check).
- Damit brauchen wir $O(n^2 \log(1/\epsilon))$ Operationen für $\|x^{(k)} - x\| \leq \epsilon$.
- Vergleiche: $O(n^3)$ für exakte Lösungen durch **LU** oder **QR**.

Analoge iterative Verfahren gibt es auch für **KQ**-Probleme; in seltenen Fällen sind sie sogar *exakt*. Meist gilt: Trade-off zwischen **Laufzeit** und Genauigkeit – steuerbar über die Iterationszahl.

Bsp: Richardson-Iteration

Gegeben: $\mathbf{A} = \begin{pmatrix} 4 & 1 \\ 1 & 3 \end{pmatrix}$, $\mathbf{b} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$. Wahre Lösung: $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b} = \frac{1}{11} \begin{pmatrix} 1 \\ 7 \end{pmatrix} \approx \begin{pmatrix} 0.091 \\ 0.636 \end{pmatrix}$.

Richardson mit $\gamma = 0.25$, Startwert $\mathbf{x}^{(0)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$:

Iteration 0: $\mathbf{r}^{(0)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(0)} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, $\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + 0.25\mathbf{r}^{(0)} = \begin{pmatrix} 0.25 \\ 0.5 \end{pmatrix}$

Iteration 1: $\mathbf{A}\mathbf{x}^{(1)} = \begin{pmatrix} 1.5 \\ 1.75 \end{pmatrix}$, $\mathbf{r}^{(1)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(1)} = \begin{pmatrix} -0.5 \\ 0.25 \end{pmatrix}$, $\mathbf{x}^{(2)} = \mathbf{x}^{(1)} + 0.25\mathbf{r}^{(1)} = \begin{pmatrix} 0.125 \\ 0.563 \end{pmatrix}$

Iteration 2: $\mathbf{A}\mathbf{x}^{(2)} = \begin{pmatrix} 1.063 \\ 1.813 \end{pmatrix}$, $\mathbf{r}^{(2)} = \begin{pmatrix} -0.063 \\ 0.188 \end{pmatrix}$, $\mathbf{x}^{(3)} = \begin{pmatrix} 0.109 \\ 0.609 \end{pmatrix}$

Iteration 3: $\mathbf{x}^{(4)} = \begin{pmatrix} 0.098 \\ 0.625 \end{pmatrix}$

Fehler: $\|\mathbf{x}^{(k)} - \mathbf{x}\|_2: 0.643 \rightarrow 0.210 \rightarrow 0.081 \rightarrow 0.033 \rightarrow 0.013 \rightarrow \dots$

Konvergenzrate: $\rho = \|\mathbf{I} - \gamma\mathbf{A}\|_2 \approx 0.41 \implies$ Fehler sollte ca. um Faktor 2.5 pro Iteration sinken.

Probabilistische Methoden

Zufall mit Absicht

- Oft sind Probleme so groß, dass wir gerne Genauigkeit für bessere *Komplexität* tauschen.
- Heute wird das zunehmend durch *Randomisierung* erreicht.
- **Hauptidee:**
 - Ersetze großes Problem durch ein
 - zufällig erzeugtes, *kleines* Problem,
 - das *höchstwahrscheinlich* eine *ähnliche Lösung* hat.

Idee: Matrix-Sketching

Geometrische Fakten in hoch-dimensionalem Raum $\mathbb{R}^n$ mit $n \gg 1$:

Für großes n sind zwei zufällige Vektoren $\mathbf{s}_1, \mathbf{s}_2 \in \mathbb{R}^n$ mit hoher Wahrscheinlichkeit fast orthogonal.

$\implies$ geeignet zufällig gezogene Matrix $\mathbf{S} \in \mathbb{R}^{m \times n}$ mit $m \ll n$ erhält mit hoher Wahrscheinlichkeit

- Abstände: $\|\mathbf{S}\mathbf{x} - \mathbf{S}\mathbf{y}\|_2 \approx \|\mathbf{x} - \mathbf{y}\|_2$
- Winkel: $\angle(\mathbf{S}\mathbf{x}, \mathbf{S}\mathbf{y}) \approx \angle(\mathbf{x}, \mathbf{y})$

$\implies$ Eine **zufällige Projektion** $\mathbf{S}$ in einen niedrigdimensionaleren Raum kann Geometrie oft näherungsweise erhalten.

Beispiel: Sketching in R

```
1 set.seed(1312)
2 n <- 10000          # zwei beliebige n-dimensionale Vektoren:
3 x <- runif(n); y <- runif(n)
4 m <- 50             # m << n !!
5 S <- rnorm(m * n, sd = sqrt(1 / m)) |> matrix(nrow = m, ncol = n)
6 Sx <- S %%% x; Sy <- S %%% y
```

Wir ersetzen $n = 10000$ Koordinaten durch $m = 50$ zufällige Projektionen.

Vorhersage:

Um wie viel Prozent ändern sich Distanz $\|\mathbf{x} - \mathbf{y}\|_2$ und Winkel $\angle(\mathbf{x}, \mathbf{y})$ dabei wohl — eher $\sim 50\%$, $\sim 3\%$ oder $< 0.1\%$?

```
1 angle <- function(x, y) (sum(x * y) / (norm(x, "2") * norm(y, "2"))) |> acos()
2 c(norm(x - y, "2"), norm(Sx - Sy, "2")) ## Distanzen ||x - y||, ||Sx - Sy||
3 c(angle(x, y), angle(Sx, Sy))          ## Winkel (rad)
```

```
[1] 41.0 42.2
[1] 0.728 0.735
```

→ nur $\approx 3\%$ [$\approx 1\%$] Abweichung in Distanz [Winkel] — obwohl 99.5% der Dimensionen wegfallen!

(Eine einzelne Ziehung, und eine recht glückliche: für Gauss-Skizzen streut die relative Abweichung der Distanz mit $\approx 1/\sqrt{2m} = 10\%$.)

Zufällige Einbettung eines festen Vektors

Theorem

Sei

- $\mathbf{S} \in \mathbb{R}^{m \times n}$ eine Matrix mit i.i.d. Zeilen $\mathbf{s}_i^\top$ (wobei $\mathbf{s}_i \in \mathbb{R}^n$),
- $\mathbb{E}[\mathbf{s}_i \mathbf{s}_i^\top] = \mathbf{I}_n/m$ und $\mathbb{E}[(\mathbf{s}_i^\top \mathbf{x})^4] \leq K \|\mathbf{x}\|^4/m^2$ für alle $\mathbf{x} \in \mathbb{R}^n$.

Dann gilt für jeden festen Vektor $\mathbf{x} \in \mathbb{R}^n$ mit einer Wahrscheinlichkeit von mindestens $1 - \delta$

$$(1 - \epsilon) \|\mathbf{x}\|^2 \leq \|\mathbf{S}\mathbf{x}\|^2 \leq (1 + \epsilon) \|\mathbf{x}\|^2,$$

wobei $\delta > 0$ und $\epsilon = \sqrt{K/\delta m}$.

Beispiele:

- *Gauss*: $\mathbf{s}_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \mathbf{I}_n/m)$.
- *Rademacher*: s_{ij} i.i.d. mit $\mathbb{P}(s_{ij} = 1/\sqrt{m}) = \mathbb{P}(s_{ij} = -1/\sqrt{m}) = 1/2$.
- *Subsampling*: $\mathbf{s}_i = \sqrt{n/m} \mathbf{e}_{K_i}$ skaliertes Einheitsvektor mit $K_i \sim \text{Unif}\{1, \dots, n\}$.

Beweis: siehe **Anhang**

Anwendungen

- Zufällige Einbettungen (*Matrix-Sketching*) haben zahlreiche Anwendungen:
 - Matrixmultiplikation: $\mathbf{AB} \approx (\mathbf{AS}^\top)(\mathbf{SB})$
 - **KQ-Probleme**: $\min \|\mathbf{Ax} - \mathbf{b}\|_2 \approx \min \|\mathbf{SAx} - \mathbf{Sb}\|_2$
 - ...
- *Achtung*: Unser Theorem gilt je *festem* Vektor — hier hängt z.B. der Minimierer selbst von $\mathbf{S}$ ab! Dafür braucht man stärkere, *simultane* Garantien (*Subspace Embeddings*): gleiche Beweisidee, etwas größeres m .
- Wirklich schneller werden Algorithmen aber nur, wenn wir Produkte $\mathbf{Sx}$ sehr schnell berechnen können.
- *Beispiele*:
 - Wenn $\mathbf{S}$ m zufällige Koordinaten aus $\mathbf{x}$ zieht, braucht es dafür nur $O(m)$ Operationen.
 - Besonders leistungsfähig: *Subsampled Randomized Hadamard Transform* (SRHT) braucht $O(n \log m)$ Rechenschritte. (Vergleich der Sketching-Matrizen → **Anhang**)

Beispiel: Dimensionsreduktion mit Matrix-Sketching

Gegeben:

$n = 10000$ Dimensionen, $\delta = 0.05$, $\epsilon = 0.1$

Frage:

Wie klein kann m sein?

Aus dem Theorem:

Mit $\mathbf{s}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_n/m)$ gilt $K = 3$ und

$$\epsilon = \sqrt{\frac{K}{\delta m}} \implies m = \frac{K}{\delta \epsilon^2} = \frac{3}{0.05 \cdot 0.01} = 6000$$

Ergebnis:

Mit $m = 6000$ Dimensionen können wir $n = 10000$ Dimensionen so darstellen, dass jeder gegebene Abstand mit 95% Wahrscheinlichkeit um höchstens 10% verzerrt wird.

Praktische Konsequenz:

- Speicher & Rechenzeit pro Distanzberechnung: $O(n) \rightarrow O(m)$
- Die Schranke (via Tschebyscheff-Ungleichung) ist sehr *konservativ* — in der Praxis reichen oft viel kleinere m : die Demo oben kam mit $m = 50$ auf nur $\approx 3\%$ Abweichung!
- Anwendung: Schnelle Nächste-Nachbar-Suche, Clusteranalyse in hohen Dimensionen, ...

Self-Check & Zusammenfassung

Wrap-up

Heute gelernt

- Lösungsansätze für Eigenwertprobleme
- *Iterative Verfahren* nähern sich Lösung schrittweise durch *Fixpunktiteration*
- *Probabilistische Verfahren* ersetzen schweres durch zufälliges, leichteres (niedrigdimensionaleres) Problem mit approximativer Lösung
- Beide geben uns einen Trade-off zwischen *Laufzeit* und Genauigkeit

Nächste Vorlesung

- Tensoren und Tensorprodukte

Self-Check (1/2)

Frage 1: Sei $\rho \in (0, 1)$, $\epsilon > 0$, und

$$\|\mathbf{x}^{(k)} - \mathbf{x}\| \leq \rho^k \|\mathbf{x}^{(0)} - \mathbf{x}\|.$$

Wie groß muss k mindestens sein, so dass $\|\mathbf{x}^{(k)} - \mathbf{x}\| \leq \epsilon$?

1. $k = O(1)$
2. $k = O(\log(1/\epsilon))$
3. $k = O(1/\epsilon)$
4. $k = O(1/\epsilon^k)$

Lösung: 2. – mit $e_0 = \|\mathbf{x}^{(0)} - \mathbf{x}\|$: $\rho^k e_0 \leq \epsilon \implies \rho^k \leq \epsilon/e_0$; Logarithmieren ($0 < \rho < 1 \implies \log \rho < 0$!) gibt $k \geq \frac{\log(\epsilon) - \log(e_0)}{\log(\rho)} = \frac{\log(e_0) + \log(1/\epsilon)}{-\log(\rho)}$, also minimales $k = \left\lceil \frac{\log(e_0/\epsilon)}{-\log(\rho)} \right\rceil$ (falls $0 < \epsilon < e_0$, sonst reicht schon $k = 0$) – wächst wie $\log(1/\epsilon)$. ✓

Self-Check (2/2)

Frage 2: Worin unterscheidet sich *Subsampling* (m zufällige, skalierte Zeilen) von einer *Gauss-Sketching-Matrix* vor allem?

1. Subsampling kostet nur $O(m)$ statt $O(mn)$ (Zeit & Speicher), ist aber nur zuverlässig, wenn die Information über viele Koordinaten gestreut ist.
2. Gauss-Matrizen brauchen weniger Speicher als Subsampling.
3. Subsampling erhält alle Distanzen exakt.
4. Es gibt keinen Unterschied — beide sind optimal und gleich teuer.

Frage 3: Welcher Sketching-Typ kombiniert schnelle Anwendung ($O(n \log m)$ für $\mathbf{S}\mathbf{x}$) mit sehr guter Qualität (“State of the art”)?

1. Gauss
2. Rademacher
3. Subsampling
4. Subsampled Randomized Hadamard Transform (SRHT)

Anhang

Ähnlichkeit: Beispiel — Rechnung

$$\mathbf{A} = \begin{pmatrix} 5 & -2 \\ -2 & 8 \end{pmatrix}, \quad \mathbf{Q} = \frac{1}{\sqrt{5}} \begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} \text{ mit } \mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$$

Schritt 1: Berechne $\mathbf{A}\mathbf{Q}^\top$:

$$\mathbf{A}\mathbf{Q}^\top = \frac{1}{\sqrt{5}} \begin{pmatrix} 5 & -2 \\ -2 & 8 \end{pmatrix} \begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} = \frac{1}{\sqrt{5}} \begin{pmatrix} 8 & 9 \\ 4 & -18 \end{pmatrix}$$

Schritt 2: Berechne $\mathbf{B} = \mathbf{Q}(\mathbf{A}\mathbf{Q}^\top)$:

$$\mathbf{B} = \frac{1}{5} \begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} \begin{pmatrix} 8 & 9 \\ 4 & -18 \end{pmatrix} = \frac{1}{5} \begin{pmatrix} 20 & 0 \\ 0 & 45 \end{pmatrix} = \begin{pmatrix} 4 & 0 \\ 0 & 9 \end{pmatrix}$$

$\mathbf{B}$ diagonal $\implies$ Eigenwerte direkt ablesbar: 4 und 9.

↩ zurück: „Beispiel“

QR-Iteration: Beweis

Definiere $Q_k := Q^{(1)} Q^{(2)} \dots Q^{(k)}$, $R_k = R^{(k)} R^{(k-1)} \dots R^{(1)}$

Zeige: $A^k = Q_k R_k$

Beweis:

- Induktionsanfang ($k = 1$): $A = Q^{(1)} R^{(1)} = Q_1 R_1 \checkmark$
- Induktionsschritt ($k \rightarrow k + 1$): $A^{(i)} = R^{(i)} Q^{(i)}$,
 $A^{(i)} = Q^{(i+1)} R^{(i+1)} \implies R^{(i)} Q^{(i)} = Q^{(i+1)} R^{(i+1)} \quad (\star)$

$$\begin{aligned} A^{k+1} &= A A^k = (Q^{(1)} R^{(1)}) (Q_k R_k) \\ &= Q^{(1)} (R^{(1)} Q^{(1)}) Q^{(2)} \dots Q^{(k)} R^{(k)} \dots R^{(1)} \\ &\quad [\text{nutze } k\text{-mal } (\star) : R^{(1)} Q^{(1)} = Q^{(2)} R^{(2)}, \dots] \\ &= Q^{(1)} Q^{(2)} \dots Q^{(k+1)} R^{(k+1)} \dots R^{(1)} \\ &= Q_{k+1} R_{k+1} \quad \checkmark \end{aligned}$$

↔ zurück: „QR-Iteration” · ↔ zurück: „QR-Iteration: Interpretation”

Iterative LGS-Verfahren: Konvergenz — Beweis

Definiere $B = I - CA$, sodass $\|B\|_2 = \rho$ und

$$x^{(k)} = Bx^{(k-1)} + Cb, \quad k = 1, 2, \dots$$

Zunächst bemerken wir, dass x ein Fixpunkt dieser Iteration ist:

$$Bx + Cb = (I - CA)x + Cb = x - \underbrace{C(Ax - b)}_{=0} = x$$

also

$$\begin{aligned} \|x^{(k)} - x\|_2 &= \|Bx^{(k-1)} + Cb - x\|_2 \\ &= \|Bx^{(k-1)} + Cb - (Bx + Cb)\|_2 \\ &= \|B(x^{(k-1)} - x)\|_2 \\ &\leq \rho \|x^{(k-1)} - x\|_2. \end{aligned}$$

Iterieren wir die Ungleichung, erhalten wir

$$\|x^{(k)} - x\|_2 \leq \rho \|x^{(k-1)} - x\|_2 \leq \rho^2 \|x^{(k-2)} - x\|_2 \leq \dots \leq \rho^k \|x^{(0)} - x\|_2.$$



↩ zurück: „Iterative Lösungsverfahren für LGS“

Zufälliges Einbetten von Unterräumen: Beweis (1/2)

1. Für $\mathbf{x} = \mathbf{0}$ ist die Aussage trivial. Sei also $\mathbf{x} \neq \mathbf{0}$ und o.B.d.A. $\|\mathbf{x}\| = 1$ (andernfalls teile die Ungleichung durch $\|\mathbf{x}\|^2$). Zu zeigen also

$$\mathbb{P} \left(1 - \sqrt{K/\delta m} \leq \|\mathbf{S}\mathbf{x}\|^2 \leq 1 + \sqrt{K/\delta m} \right) = \mathbb{P} \left(|\|\mathbf{S}\mathbf{x}\|^2 - 1| \leq \sqrt{K/\delta m} \right) \geq 1 - \delta$$

2. Mit $\|\mathbf{S}\mathbf{x}\|^2 = \sum_{i=1}^m (\mathbf{s}_i^\top \mathbf{x})^2$ gilt

$$\mathbb{E} [\|\mathbf{S}\mathbf{x}\|^2] = \sum_{i=1}^m \mathbb{E} [(\mathbf{s}_i^\top \mathbf{x})^2] = m\mathbf{x}^\top \mathbb{E} [\mathbf{s}_1 \mathbf{s}_1^\top] \mathbf{x} = \|\mathbf{x}\|^2.$$

3. Weil die Summanden unabhängig sind, gilt außerdem

$$\text{Var} [\|\mathbf{S}\mathbf{x}\|^2] = \sum_{i=1}^m \text{Var} [(\mathbf{s}_i^\top \mathbf{x})^2] \leq m\mathbb{E} [(\mathbf{s}_1^\top \mathbf{x})^4] \leq \frac{K}{m}$$

nach der Momentannahme (mit $\|\mathbf{x}\| = 1$).

Zufälliges Einbetten von Unterräumen: Beweis (2/2)

(Fortsetzung)

4. Nun gibt die Tschebyscheff-Ungleichung (= Markov, angewandt auf die quadrierte Abweichung)

$$\mathbb{P} \left(\left| \|\mathbf{S}\mathbf{x}\|^2 - \mathbb{E}[\|\mathbf{S}\mathbf{x}\|^2] \right| > \epsilon \right) \leq \frac{\text{Var}[\|\mathbf{S}\mathbf{x}\|^2]}{\epsilon^2} \leq \frac{m \mathbb{E}[(\mathbf{s}_i^\top \mathbf{x})^4]}{\epsilon^2} \leq \frac{K}{\epsilon^2 m}.$$

Die Wahl $\epsilon = \sqrt{K/\delta m}$ liefert die Behauptung. ■

↪ zurück: „Zufälliges Einbetten von Unterräumen“

Approximative SVD: Setup & Fehlervergleich

Setup-Code des Benchmarks (im Foliensatz ausgeführt):

```
1 set.seed(1312)
2 library(irlba)
3 suppressMessages(library(magick))
4 img <- paste0(
5   "https://upload.wikimedia.org/wikipedia/commons/7/7e/",
6   "Bolzano_City_Image_-_Photo_by_Giovanni_Ussi_-_In_Black_and_White_19.jpg")
7 X <- (image_read(img) |>
8       image_data(channels = "gray") |>
9       as.numeric())[,,1]
```

Relative Frobenius-Fehler der niedrigrangigen Approximationen (der exakten Rang-50-SVD zu $\mathbf{X}$, und der iterativen Verfahren zur exakten Rang-50-SVD):

```
1 X_r <- with(svd(X, nu = r, nv = r), u %*% diag(d[1:r]) %*% t(v))
2 X_irlba <- with(irlba::irlba(X, nu = r, nv = r), u %*% diag(d) %*% t(v))
3 norm(X - X_r, "F") / norm(X, "F")
```

```
[1] 0.129
```

```
1 c(norm(X_r - X_irlba, "F"), norm(X_r - X_svdr, "F")) / norm(X_r, "F")
```

```
[1] 2.27e-06 1.68e-04
```

→ `irlba/svdr` weichen von der exakten Rang-50-SVD um Größenordnungen weniger ab, als diese vom Originalbild.

Approximative SVD: Bildvergleich

Original



SVD ($r = 50$)



IRLBA ($r = 50$)



SVDR ($r = 50$)



↔ zurück: „Anwendung: Approximative SVD“

Vergleich Sketching-Matrizen

Typ	Speicher	Zeit für Sx	Qualität
Gauss	$O(mn)$	$O(mn)$	Optimal
Rademacher ($\pm 1/\sqrt{m}$)	$O(mn)$	$O(mn)$	Optimal
Subsampling	$O(m)$	$O(m)$	Gut (bei über viele Koordinaten <i>gestreuter</i> Information)
SRHT (Hadamard)	$O(n)$	$O(n \log m)$	Sehr gut

Trade-offs:

- **Gauss:** Theoretisch optimal, aber teuer zu speichern und anzuwenden
- **Rademacher:** Genauso gut wie Gauss, einfacher zu generieren (nur ± 1)
- **Subsampling:** Wählt m zufällige Zeilen - extrem schnell, aber nur gut, wenn die Information über viele Koordinaten gestreut ist (schlecht, wenn sie in wenigen Koordinaten konzentriert ist)
- **SRHT:** kombiniert hohe Geschwindigkeit und gute Approximationsqualität; Speicher $O(n)$ wegen der n Vorzeichen der vorgeschalteten Diagonalmatrix (sofern man sie nicht aus einem Seed neu erzeugt)
 - *Subsampled Randomized Hadamard Transform*
 - S wird nicht explizit gespeichert, sondern durch schnelle(!) Transformationen angewendet

Tensoren und Tensorprodukte

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)

Material: Thomas Nagler

LMU München

Verwendete Vorkenntnisse

- **Lineare Algebra**

- Vektorräume und Unterräume
- Basis und Dimension
- Lineare Abbildungen und Multilinearität
- Äußeres Produkt: $\boldsymbol{v} \otimes \boldsymbol{w} = \boldsymbol{vw}^T$
- $\text{span}(\{\boldsymbol{v}_1, \dots, \boldsymbol{v}_n\}) = \{\boldsymbol{w} : \boldsymbol{w} = \sum_i c_i \boldsymbol{v}_i; c_i \in \mathbb{R}\}$

- **Analysis**

- Multivariate Funktionen $f(\boldsymbol{x}) = f(x_1, x_2, \dots, x_n)$

- **Gleichungssysteme** (Kapitel 5): LGS $\boldsymbol{M}\boldsymbol{z} = \boldsymbol{c}$ lösen (LU, Cholesky)

Multilineare Abbildungen

Multilineare Abbildungen

- Bisher wurde ein Vektor $\boldsymbol{v} \in V$ linear auf einen Vektor $\boldsymbol{w} \in W$ abgebildet.
- Jetzt werden Paare oder Tupel von Vektoren $(\boldsymbol{v}_1, \dots, \boldsymbol{v}_n) \in V_1 \times \dots \times V_n$ “linear” auf ein $\boldsymbol{w} \in W$ abgebildet,
 - $\rightarrow$ Wir sprechen von einer *multilinearen Abbildung*.
- Linearität bezieht sich hier auf die *einzelnen* Argumente der Abbildung.

Multilineare Abbildungen

Def.: Multilineare Abbildung

Seien $V_1, \dots, V_n, W$ Vektorräume. Eine **multilineare Abbildung** ist eine Abbildung $f: V_1 \times \dots \times V_n \rightarrow W$, sodass:

$$\begin{aligned} & f(v_1, \dots, cv_i + c'v'_i, \dots, v_n) \\ &= cf(v_1, \dots, v_i, \dots, v_n) + c'f(v_1, \dots, v'_i, \dots, v_n) \end{aligned}$$

für alle $i = 1, \dots, n, c, c' \in \mathbb{R}, v_i, v'_i \in V_i$ und $v_\ell \in V_\ell$ ($\ell \neq i$).

Anmerkungen:

- Im Spezialfall $n = 2$ nennen wir die Abbildung **bilinear**.
- **Achtung:** **Multilineare Abbildungen** sind in der Regel nicht linear!

Intuition: Linear vs. Multilinear

Fläche eines Rechtecks als Funktion seiner Seitenlängen:

$$f: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}, \quad (x, y) \mapsto xy$$

Warum ist f nicht linear?

- Für $c = 2$: $f(2x, 2y) = 2x \cdot 2y = 4xy = 4f(x, y) \neq 2f(x, y)$ (für $xy \neq 0$)
- Linear wäre: $f(cx, cy) = c \cdot f(x, y)$

Aber f ist bilinear:

$$y \text{ konstant: } f(x_1 + x_2, y) = (x_1 + x_2) \cdot y = x_1y + x_2y = f(x_1, y) + f(x_2, y)$$

$$f(cx, y) = (cx) \cdot y = c(xy) = cf(x, y)$$

$$x \text{ konstant: } f(x, y_1 + y_2) = x \cdot (y_1 + y_2) = xy_1 + xy_2 = f(x, y_1) + f(x, y_2)$$

$$f(x, cy) = x \cdot (cy) = c(xy) = cf(x, y)$$

$\implies$ Die Funktion ist **in jedem Argument einzeln linear!**

Multilineare Abbildungen

Beispiel:

- Multiplikation von zwei reellen Zahlen, $f: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}, (x_1, x_2) \mapsto x_1 x_2$, ist bilinear (s.a. Bsp. zur Fläche)

Beispiel:

- Jedes Skalarprodukt $f: V \times V \rightarrow \mathbb{R}, (\mathbf{v}_1, \mathbf{v}_2) \mapsto \langle \mathbf{v}_1, \mathbf{v}_2 \rangle$ ist bilinear.
- Das folgt aus Distributivgesetz, Homogenität und Symmetrie:
$$\langle c\mathbf{v} + c'\mathbf{v}', \mathbf{w} \rangle = \langle c\mathbf{v}, \mathbf{w} \rangle + \langle c'\mathbf{v}', \mathbf{w} \rangle = c\langle \mathbf{v}, \mathbf{w} \rangle + c'\langle \mathbf{v}', \mathbf{w} \rangle$$

Beispiel:

- Die Matrizenmultiplikation $\mathbb{R}^{m \times k} \times \mathbb{R}^{k \times n} \rightarrow \mathbb{R}^{m \times n}$ mit $(\mathbf{A}, \mathbf{B}) \mapsto \mathbf{AB}$ ist bilinear.

Quiz

TRUE/FALSE?

Für jedes multilineare f gilt $f(\mathbf{v}_1, \dots, \mathbf{v}_n) = \mathbf{0}$ sobald $\mathbf{v}_i = \mathbf{0}$ für ein i .

Darstellung von multilinearen Abbildungen

Theorem

Eine Abbildung $T: \mathbb{R}^{n_1} \times \cdots \times \mathbb{R}^{n_k} \rightarrow \mathbb{R}^m$ ist genau dann multilinear, wenn es reelle Koeffizienten

$$A = (a_{i_1, \dots, i_k, j} : 1 \leq i_1 \leq n_1, \dots, 1 \leq i_k \leq n_k, 1 \leq j \leq m)$$

gibt, sodass

$$T(\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(k)}) = \sum_{i_1=1}^{n_1} \cdots \sum_{i_k=1}^{n_k} \sum_{j=1}^m a_{i_1, \dots, i_k, j} x_{i_1}^{(1)} \cdots x_{i_k}^{(k)} \mathbf{e}_j,$$

wobei $\mathbf{e}_j$ der j -te Einheitsvektor des $\mathbb{R}^m$ ist. Die Koeffizienten $a_{i_1, \dots, i_k, j}$ sind eindeutig.

Tensoren

Tensoren

- Setzen wir im vorherigen Theorem $k = 1$, erhalten wir Koeffizienten $A = (a_{i,j}: 1 \leq i \leq n, 1 \leq j \leq m)$, sodass

$$T(\mathbf{x}) = \sum_{i=1}^n \sum_{j=1}^m a_{i,j} x_i \mathbf{e}_j = \sum_{i=1}^n x_i \mathbf{a}_i = \mathbf{A} \mathbf{x},$$

mit $\mathbf{a}_i = (a_{i,1}, \dots, a_{i,m})^\top \in \mathbb{R}^m$ und $\mathbf{A} = (\mathbf{a}_1, \dots, \mathbf{a}_n) \in \mathbb{R}^{m \times n}$, d.h. $(\mathbf{A})_{j,i} = a_{i,j}$.

- Die Ansammlung $\mathbf{A}$ aus nm Koeffizienten nennen wir **Matrix**.
- Sie hat zwei “Dimensionen” weil sie durch zwei Subskripte i und j indiziert wird.
- Allgemeiner beschreibt jede solche Ansammlung von Koeffizienten mit $k + 1$ Indizes (bei $m = 1$ also mit k) eine entsprechende multilineare Funktion

Bsp: Bilineare Abbildung ($k = 2, n_1 = n_2 = 2, m = 1$)

$T: \mathbb{R}^2 \times \mathbb{R}^2 \rightarrow \mathbb{R}$ mit $T(\mathbf{x}, \mathbf{y}) = 2x_1y_1 + 3x_1y_2 - x_2y_1 + 4x_2y_2$

Koeffizienten ($a_{i_1, i_2}: 1 \leq i_1, i_2 \leq 2$):

$$a_{1,1} = 2, \quad a_{1,2} = 3, \quad a_{2,1} = -1, \quad a_{2,2} = 4 \implies \mathbf{A} = \begin{pmatrix} 2 & 3 \\ -1 & 4 \end{pmatrix}$$

Verifikation:

$$\begin{aligned} T(\mathbf{x}, \mathbf{y}) &= \sum_{i_1=1}^2 \sum_{i_2=1}^2 a_{i_1, i_2} x_{i_1} y_{i_2} = a_{1,1}x_1y_1 + a_{1,2}x_1y_2 + a_{2,1}x_2y_1 + a_{2,2}x_2y_2 \\ &= 2x_1y_1 + 3x_1y_2 - x_2y_1 + 4x_2y_2 \quad \checkmark \end{aligned}$$

Numerisches Beispiel:

$$\text{Für } \mathbf{x} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \mathbf{y} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}: T(\mathbf{x}, \mathbf{y}) = 2 \cdot 1 \cdot 3 + 3 \cdot 1 \cdot 4 - 1 \cdot 2 \cdot 3 + 4 \cdot 2 \cdot 4 = 6 + 12 - 6 + 32 = 44$$

Alternativ in Tensorschreibweise:

$$T(\mathbf{x}, \mathbf{y}) = \mathbf{x}^\top \mathbf{A} \mathbf{y} = \begin{pmatrix} 1 & 2 \end{pmatrix} \begin{pmatrix} 2 & 3 \\ -1 & 4 \end{pmatrix} \begin{pmatrix} 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 0 & 11 \end{pmatrix} \begin{pmatrix} 3 \\ 4 \end{pmatrix} = 44$$

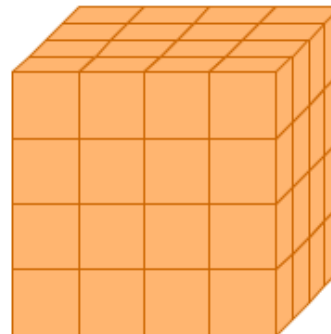
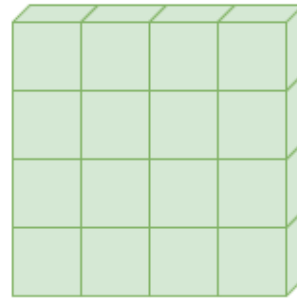
Definition: Tensor

Def.: Tensor

Ein **Tensor** ist eine durch k Indizes indizierte Ansammlung von reellen Zahlen

$$A = (a_{i_1, \dots, i_k} : 1 \leq i_1 \leq n_1, \dots, 1 \leq i_k \leq n_k) \in \mathbb{R}^{n_1 \times \dots \times n_k}.$$

Die Zahl k wird auch **Stufe** des Tensors genannt.



Eine (4×4) -Matrix besteht aus $4^2 = 16$ Zahlen (links); ein $(4 \times 4 \times 4)$ -Tensor besteht aus $4^3 = 64$ Zahlen (rechts). (Quelle: Deisenroth/Faisal/Ong, Fig 5.13)

- Die Menge $\mathbb{R}^{n_1 \times \cdots \times n_k}$ aller $(n_1 \times \cdots \times n_k)$ -Tensoren mit koeffizientenweiser Addition und Multiplikation mit Skalaren ist ein Vektorraum.

Anwendung: Tensoren in Deep Learning

- Eine Matrix stellt z.B. ein (Graustufen-)Bild dar: Dimension 1 indiziert die Pixel vertikal (Höhe), Dimension 2 horizontal (Breite).
- **RGB-Bild:** dritte Dimension indiziert die Farbkanäle $\implies$ Stufe-3-Tensor $\mathbf{I} \in \mathbb{R}^{224 \times 224 \times 3}$ ($\approx 150 \cdot 10^3$ Werte pro Bild)
- **Batch/Video:** vierte Dimension indiziert mehrere Bilder — z.B. $\stackrel{\sim}{=} 3$ Kanal-Matrizen verarbeitet ein CNN 32 Bilder gleichzeitig $\implies$ Stufe-4-Tensor $\mathbf{B} \in \mathbb{R}^{32 \times 224 \times 224 \times 3}$ ($\approx 4.8 \cdot 10^6$ Werte)
- **Feature Maps:** Convolutional Layers bilden Tensoren auf Tensoren ab: $c: \mathbb{R}^{32 \times 224 \times 224 \times 3} \rightarrow \mathbb{R}^{32 \times 56 \times 56 \times 64}$
- Rechnungen auf Tensoren können durch spezielle Hardware besonders schnell durchgeführt werden (Google TPUs, NVIDIA Tensor Cores, ...) — essentiell für KI.



(Details & Feature-Map-Visualisierung $\rightarrow$ Anhang)

Produkt von Tensoren

Produkt von Tensoren

Def.: Äußeres Produkt

Das **äußere Produkt** von zwei Vektoren $\mathbf{v} \in \mathbb{R}^m$ und $\mathbf{w} \in \mathbb{R}^n$ ist definiert als

$$\mathbf{v} \otimes \mathbf{w} := \mathbf{v}\mathbf{w}^\top = \begin{pmatrix} v_1 w_1 & \cdots & v_1 w_n \\ \vdots & & \vdots \\ v_m w_1 & \cdots & v_m w_n \end{pmatrix} \in \mathbb{R}^{m \times n}.$$

- **Äußeres Produkt**: $\otimes: \mathbb{R}^m \times \mathbb{R}^n \rightarrow \mathbb{R}^{m \times n}$
- vgl Skalarprodukt (“inneres Produkt”): $\langle \cdot, \cdot \rangle: \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R} \quad \langle \mathbf{v}, \mathbf{w} \rangle = \mathbf{v}^\top \mathbf{w}$

Äußeres Produkt: Beispiel & Eigenschaften

Beispiel:

$$\mathbf{v} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \quad \text{und} \quad \mathbf{w} = \begin{pmatrix} -2 \\ 3 \\ -11 \end{pmatrix}$$

$$\begin{aligned} \mathbf{v} \otimes \mathbf{w} &= \mathbf{v} \mathbf{w}^\top = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \begin{pmatrix} -2 & 3 & -11 \end{pmatrix} \\ &= \begin{pmatrix} -2 & 3 & -11 \\ -4 & 6 & -22 \end{pmatrix} \end{aligned}$$

- Alle Spalten sind Vielfache von $\mathbf{v}$, alle Zeilen sind Vielfache von $\mathbf{w}^\top$
 - Rang ist 1, sofern $\mathbf{v} \neq \mathbf{0}$ und $\mathbf{w} \neq \mathbf{0}$
- Was für eine lineare Abbildung ist $\mathbf{x} \rightarrow (\mathbf{v} \otimes \mathbf{w})\mathbf{x}$? (im Folgenden: $\mathbf{v} \neq \mathbf{0}, \mathbf{w} \neq \mathbf{0}$)
 - $(\mathbf{v} \otimes \mathbf{w})\mathbf{x} = \mathbf{v} \mathbf{w}^\top \mathbf{x} = \mathbf{v} \langle \mathbf{w}, \mathbf{x} \rangle$:
Vektor $\mathbf{v}$ skaliert mit Skalarprodukt von $\mathbf{w}$ und $\mathbf{x}$
 - $\text{col}(\mathbf{v} \otimes \mathbf{w}) = \text{span}(\mathbf{v})$: Bild ist eindimensional
 - $\ker(\mathbf{v} \otimes \mathbf{w}) = \{\mathbf{x} : \mathbf{w}^\top \mathbf{x} = 0\}$: Hyperebene orthogonal zu $\mathbf{w}$

Anwendungsbeispiele:

- SVD: $\mathbf{A} = \mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}_r^\top = \sum_{i=1}^r \sigma_i \mathbf{u}_i \otimes \mathbf{v}_i$ (Darstellung einer Matrix als gewichtete Summe von normierten Rang-1-Matrizen)
- Statistik: $\text{Cov}(\mathbf{x}) = \mathbb{E} \left[(\mathbf{x} - \mathbb{E}[\mathbf{x}]) (\mathbf{x} - \mathbb{E}[\mathbf{x}])^\top \right] = \mathbb{E} [\mathbf{x} \otimes \mathbf{x}] - \mathbb{E} [\mathbf{x}] \otimes \mathbb{E} [\mathbf{x}]$
- KI: $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{1}{\sqrt{d_k}} \mathbf{Q} \mathbf{K}^\top \right) \mathbf{V}$

Tensorprodukt

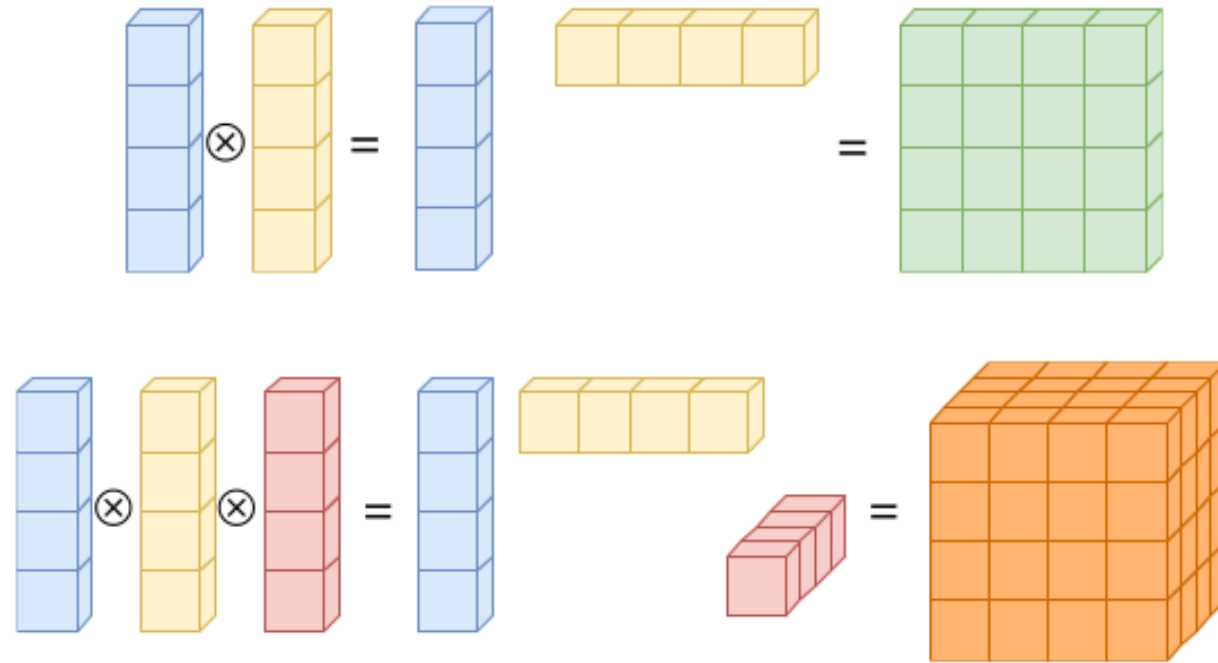
Def.: Tensorprodukt

Das **Tensorprodukt** $\otimes: \mathbb{R}^{m_1 \times \dots \times m_p} \times \mathbb{R}^{n_1 \times \dots \times n_q} \rightarrow \mathbb{R}^{m_1 \times \dots \times m_p \times n_1 \times \dots \times n_q}$ von zwei Tensoren $\mathbf{A} \in \mathbb{R}^{m_1 \times \dots \times m_p}$ und $\mathbf{B} \in \mathbb{R}^{n_1 \times \dots \times n_q}$ ist definiert als der Tensor $\mathbf{C} \in \mathbb{R}^{m_1 \times \dots \times m_p \times n_1 \times \dots \times n_q}$ mit

$$c_{i_1, \dots, i_p, j_1, \dots, j_q} = a_{i_1, \dots, i_p} b_{j_1, \dots, j_q}.$$

- **Äußeres Produkt** ist **Tensorprodukt** zweier Stufe-1-Tensoren ($p = q = 1$, also Vektoren)
- Das **Tensorprodukt** ist bilinear

Produkte von Tensoren



Oben: Das Produkt von zwei Stufe-1-Tensoren ergibt einen Stufe-2-Tensor. Unten: Das Produkt von drei Stufe-1-Tensoren ergibt einen Stufe-3-Tensor. Quelle: Deisenroth/Faisal/Ong, Fig 5.13

Bsp: Tensorprodukt

$$\mathbf{u} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \in \mathbb{R}^2, \quad \mathbf{v} = \begin{pmatrix} 3 \\ 5 \end{pmatrix} \in \mathbb{R}^2, \quad \mathbf{w} = \begin{pmatrix} 7 \\ 11 \end{pmatrix} \in \mathbb{R}^2$$

Tensorprodukt $\mathbf{T} = \mathbf{u} \otimes \mathbf{v} \otimes \mathbf{w} \in \mathbb{R}^{2 \times 2 \times 2}$ hat Einträge $T_{i,j,k} = u_i v_j w_k$:

$$\text{Slice 1 (} k = 1, \text{ also } w_k = w_1 = 7\text{): } T_{1,1,1} = 1 \cdot 3 \cdot 7 = 21, \quad T_{1,2,1} = 1 \cdot 5 \cdot 7 = 35$$

$$T_{2,1,1} = 2 \cdot 3 \cdot 7 = 42, \quad T_{2,2,1} = 2 \cdot 5 \cdot 7 = 70$$

$$\text{Slice 2 (} k = 2, \text{ also } w_k = w_2 = 11\text{): } T_{1,1,2} = 1 \cdot 3 \cdot 11 = 33, \quad T_{1,2,2} = 1 \cdot 5 \cdot 11 = 55$$

$$T_{2,1,2} = 2 \cdot 3 \cdot 11 = 66, \quad T_{2,2,2} = 2 \cdot 5 \cdot 11 = 110$$

$$\implies \mathbf{T}_{\cdot,\cdot,1} = \begin{pmatrix} 21 & 35 \\ 42 & 70 \end{pmatrix}; \quad \mathbf{T}_{\cdot,\cdot,2} = \begin{pmatrix} 33 & 55 \\ 66 & 110 \end{pmatrix}$$

Jede *slice* (Stufe-2-Tensor) dieses Stufe-3-Tensors ist eine Rang-1-Matrix:

- $\mathbf{T}_{\cdot,\cdot,1} = w_1 \cdot (\mathbf{u} \otimes \mathbf{v})$ und $\mathbf{T}_{\cdot,\cdot,2} = w_2 \cdot (\mathbf{u} \otimes \mathbf{v})$

Allgemein:

Ein Tensor der Form $\mathbf{u} \otimes \mathbf{v} \otimes \mathbf{w}$ hat *Rang 1* (für $\mathbf{u}, \mathbf{v}, \mathbf{w} \neq \mathbf{0}$).

Kroneckerprodukt

Def.: Kroneckerprodukt

Das **Kroneckerprodukt** von zwei Matrizen $\mathbf{A} \in \mathbb{R}^{m \times n}$ und $\mathbf{B} \in \mathbb{R}^{p \times q}$ ist definiert als

$$\begin{aligned} \mathbf{A} \otimes_{\mathbb{K}} \mathbf{B} &:= \begin{pmatrix} a_{11}\mathbf{B} & \cdots & a_{1n}\mathbf{B} \\ \vdots & \vdots & \vdots \\ a_{m1}\mathbf{B} & \cdots & a_{mn}\mathbf{B} \end{pmatrix} \\ &= \begin{pmatrix} a_{11}b_{11} & \cdots & a_{11}b_{1q} & \cdots & a_{1n}b_{11} & \cdots & a_{1n}b_{1q} \\ \vdots & & \vdots & & \vdots & & \vdots \\ a_{11}b_{p1} & \cdots & a_{11}b_{pq} & \cdots & a_{1n}b_{p1} & \cdots & a_{1n}b_{pq} \\ \vdots & & \vdots & & \vdots & & \vdots \\ a_{m1}b_{11} & \cdots & a_{m1}b_{1q} & \cdots & a_{mn}b_{11} & \cdots & a_{mn}b_{1q} \\ \vdots & & \vdots & & \vdots & & \vdots \\ a_{m1}b_{p1} & \cdots & a_{m1}b_{pq} & \cdots & a_{mn}b_{p1} & \cdots & a_{mn}b_{pq} \end{pmatrix} \in \mathbb{R}^{mp \times nq}. \end{aligned}$$

Obacht – Notation:

$\otimes$ ist oft doppelt belegt (Tensorprodukt und Kroneckerprodukt) – wir schreiben ab hier $\otimes_{\mathbb{K}}$ fürs **Kroneckerprodukt**.

Kroneckerprodukt vs. Tensorprodukt

Kroneckerprodukt ist Variante des Tensorprodukts für Matrizen – **dasselbe Zahlenmaterial, andere Anordnung:**

Für $A \in \mathbb{R}^{m \times n}$, $B \in \mathbb{R}^{p \times q}$

- **Tensorprodukt** ergibt hier **Stufe-4-Tensor**

$$C = A \otimes B \in \mathbb{R}^{m \times n \times p \times q} \quad \text{mit } c_{i_1, i_2, j_1, j_2} = a_{i_1, i_2} b_{j_1, j_2}$$

- **Kroneckerprodukt** ergibt **Matrix**

$$D = A \otimes_K B \in \mathbb{R}^{mp \times nq} \quad \text{mit } d_{(i_1-1)p+j_1, (i_2-1)q+j_2} = a_{i_1, i_2} b_{j_1, j_2}$$

- C und D unterscheiden sich (nur) darin wie ihre Einträge angeordnet sind – D ist “abgeflachte” Variante von C

Notation:

- In der Literatur steht meist $\otimes$ für *beides* (!); welche Bedeutung gemeint ist, verrät dort das **Format** des Ergebnisses.
- In FMM ist $\otimes_K$ immer das **Kroneckerprodukt**
- $\otimes$ bleibt **äußerem Produkt** bzw. **Tensorprodukt** vorbehalten.

Bsp: Kroneckerprodukt

Beispiel:

- Für

$$\mathbf{A} = \begin{pmatrix} 1 & 0 \\ 2 & 5 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 3 & 0 & 0 \\ 0 & 2 & 0 \\ -1 & 0 & -1 \end{pmatrix}$$

- Gilt

$$\mathbf{A} \otimes_{\mathbf{K}} \mathbf{B} = \begin{pmatrix} 1 \cdot \mathbf{B} & 0 \cdot \mathbf{B} \\ 2 \cdot \mathbf{B} & 5 \cdot \mathbf{B} \end{pmatrix} = \begin{pmatrix} 3 & 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 \\ -1 & 0 & -1 & 0 & 0 & 0 \\ 6 & 0 & 0 & 15 & 0 & 0 \\ 0 & 4 & 0 & 0 & 10 & 0 \\ -2 & 0 & -2 & -5 & 0 & -5 \end{pmatrix}$$

Vorsicht: $\mathbf{A} \otimes_{\mathbf{K}} \mathbf{B} \neq \mathbf{B} \otimes_{\mathbf{K}} \mathbf{A}$! Aber: $(\mathbf{A} \otimes_{\mathbf{K}} \mathbf{B})^{\top} = \mathbf{A}^{\top} \otimes_{\mathbf{K}} \mathbf{B}^{\top}$ – Transponieren dreht die Reihenfolge **nicht** um.

Quiz

Frage 1: Sei $\mathbf{A} \in \mathbb{R}^{3 \times 2}$ und $\mathbf{B} \in \mathbb{R}^{2 \times 4}$. In welchem Raum liegt das **Kroneckerprodukt** $\mathbf{A} \otimes_{\mathbb{K}} \mathbf{B}$?

1. $\mathbb{R}^{6 \times 8}$
2. $\mathbb{R}^{5 \times 6}$
3. $\mathbb{R}^{3 \times 2 \times 2 \times 4}$
4. $\mathbb{R}^{8 \times 6}$

Frage 2: Sei $\mathbf{S} \in \mathbb{R}^{p \times p}$ eine Matrix mit Einträgen s_{ij} . Wie sieht das Kroneckerprodukt $\mathbf{S} \otimes_{\mathbb{K}} \mathbf{I}_n$ aus?

Anwendung: Blockdiagonale Matrizen

Kroneckerprodukt um Blockdiagonalmatrizen zu erzeugen:

$$\mathbf{I}_n \otimes_{\mathbf{K}} \underbrace{\mathbf{S}}_{p \times p} = \underbrace{\begin{pmatrix} \mathbf{S} & & \\ & \mathbf{S} & \\ & & \ddots \\ & & & \mathbf{S} \end{pmatrix}}_{np \times np}$$

- Nützlich z.B. in Statistik für Kovarianzmatrizen von Vektoren mit unabhängigen Blöcken mit identischer Kovarianz
- **Achtung:**
 $\mathbf{I}_n \otimes_{\mathbf{K}} \mathbf{S}$ ist blockdiagonal, $\mathbf{S} \otimes_{\mathbf{K}} \mathbf{I}_n$ ist es i.A. *nicht* — auf die **Reihenfolge der Faktoren** kommt es an!

Anwendung: Separierbare Kovarianz

- z.B. Daten an m verschiedenen *Orten* zu n verschiedenen *Zeitpunkten* (Annahme hier: Daten geordnet Ort 1 Zeit 1, Ort 2 Zeit 1, Ort 1 Zeit 2, Ort 2 Zeit 2)
- **Modellannahme: Separierbare** Kovarianz mit $\Sigma = \Sigma_T \otimes_K \Sigma_S \in \mathbb{R}^{mn \times mn}$, wobei
 - $\Sigma_T \in \mathbb{R}^{n \times n}$: Kovarianzmatrix zwischen Zeitpunkten
 - $\Sigma_S \in \mathbb{R}^{m \times m}$: Kovarianzmatrix zwischen Orten
 - **Vorteile:** Parametersparsam: $mn(mn + 1)/2$ vs. $m(m + 1)/2 + n(n + 1)/2$; einfachere Interpretation

Beispiel $m = 2$ Orte, $n = 2$ Zeitpunkte, $\Sigma_T = \begin{pmatrix} 1.0 & 0.8 \\ 0.8 & 1.0 \end{pmatrix}$, $\Sigma_S = \begin{pmatrix} 2.0 & 0.5 \\ 0.5 & 2.0 \end{pmatrix}$:

$$\Sigma = \Sigma_T \otimes_K \Sigma_S = \begin{pmatrix} 1.0 \cdot \Sigma_S & 0.8 \cdot \Sigma_S \\ 0.8 \cdot \Sigma_S & 1.0 \cdot \Sigma_S \end{pmatrix} = \begin{pmatrix} 2.0 & 0.5 & 1.6 & 0.4 \\ 0.5 & 2.0 & 0.4 & 1.6 \\ 1.6 & 0.4 & 2.0 & 0.5 \\ 0.4 & 1.6 & 0.5 & 2.0 \end{pmatrix} \in \mathbb{R}^{4 \times 4}$$

Interpretation:

Eintrag (1, 1): Varianz in Ort 1 zu Zeit 1 = 2.0; Eintrag (1, 3): Kovarianz zwischen Ort 1, Zeit 1 und Ort 1, Zeit 2 = $1.6 = 0.8 \cdot 2.0$

Anwendung: vec-Trick — Matrixgleichungen als LGS

Für $\mathbf{X} \in \mathbb{R}^{m \times n}$ stapelt $\text{vec}(\mathbf{X}) = (x_{11}, \dots, x_{m1}, x_{12}, \dots, x_{mn})^\top \in \mathbb{R}^{mn}$ die Spalten von $\mathbf{X}$ zu einem Vektor.

vec-Trick

Für $\mathbf{A} \in \mathbb{R}^{k \times m}$, $\mathbf{X} \in \mathbb{R}^{m \times n}$, $\mathbf{B} \in \mathbb{R}^{n \times l}$ gilt

$$\text{vec}(\mathbf{AXB}) = (\mathbf{B}^\top \otimes_{\mathbb{K}} \mathbf{A}) \text{vec}(\mathbf{X}).$$

- Macht lineare **Matrix**gleichungen zu gewöhnlichen **LGS** $\mathbf{M}\mathbf{z} = \mathbf{c} \rightarrow$ lösbar mit den Methoden aus Kapitel *Gleichungssysteme* (LU, Cholesky, ...)!
- Wichtige Spezialfälle ($\mathbf{I}_n$: Einheitsmatrix):

$$\text{vec}(\mathbf{AX}) = (\mathbf{I}_n \otimes_{\mathbb{K}} \mathbf{A}) \text{vec}(\mathbf{X}), \quad \text{vec}(\mathbf{XB}) = (\mathbf{B}^\top \otimes_{\mathbb{K}} \mathbf{I}_m) \text{vec}(\mathbf{X})$$

- **Reihenfolge merken:**

$\mathbf{B}^\top \otimes_{\mathbb{K}} \mathbf{A}$ und $\mathbf{A} \otimes_{\mathbb{K}} \mathbf{B}^\top$ haben *immer* dieselbe Größe — ein Vertauschen fällt nicht durch unpassende Formate auf!

Bsp: vec-Trick löst die Sylvester-Gleichung

Gesucht:

$\mathbf{X} \in \mathbb{R}^{2 \times 2}$ mit $\mathbf{A}\mathbf{X} + \mathbf{X}\mathbf{B} = \mathbf{C}$ (z.B. stationäre Kovarianzen von Zeitreihenmodellen: Lyapunov-Gleichung)

vec-Trick auf beide Summanden ($m = n = 2$):

$$\text{vec}(\mathbf{A}\mathbf{X} + \mathbf{X}\mathbf{B}) = (\mathbf{I}_2 \otimes_K \mathbf{A} + \mathbf{B}^\top \otimes_K \mathbf{I}_2) \text{vec}(\mathbf{X}) = \text{vec}(\mathbf{C})$$

$\implies$ ein ganz normales 4×4 -LGS für $\text{vec}(\mathbf{X})$:

```
1 A <- matrix(c(3, 1, 0, 2), 2, 2)
2 B <- matrix(c(0, 1, 1, 1), 2, 2)
3 C <- matrix(c(1, 2, 3, 4), 2, 2)
4 # Systemmatrix des LGS via Kroneckerprodukte:
5 M <- kronecker(diag(2), A) + kronecker(t(B), diag(2))
6 X <- matrix(solve(M, as.vector(C)), 2, 2)
7 A %*% X + X %*% B          # = C ?
8 all.equal(A %*% X + X %*% B, C)
```

```
      [,1] [,2]
[1,]     1     3
[2,]     2     4
[1] TRUE
```

- Ohne vec-Trick keine LGS-Form für unsere Löser aus Kap. 5 — mit: `solve()` genügt. (Für große Matrizen gibt es effizientere spezialisierte Sylvester-Löser.)

Tensorprodukt von Vektorräumen

Tensorprodukt von Vektorräumen

Def.: Tensorprodukt von Vektorräumen

Seien V und W zwei Vektorräume. Dann ist das **Tensorprodukt** $V \otimes W$ definiert als der Vektorraum

$$V \otimes W = \text{span}\{\boldsymbol{v} \otimes \boldsymbol{w} : \boldsymbol{v} \in V, \boldsymbol{w} \in W\},$$

wobei $(\boldsymbol{v}, \boldsymbol{w}) \mapsto \boldsymbol{v} \otimes \boldsymbol{w}$ eine bilineare Abbildung ist.

- **Bilinearität** bedeutet: $(c\boldsymbol{v}) \otimes \boldsymbol{w} = c(\boldsymbol{v} \otimes \boldsymbol{w}) = \boldsymbol{v} \otimes (c\boldsymbol{w})$, $(\boldsymbol{v}_1 + \boldsymbol{v}_2) \otimes \boldsymbol{w} = \boldsymbol{v}_1 \otimes \boldsymbol{w} + \boldsymbol{v}_2 \otimes \boldsymbol{w}$ und $\boldsymbol{v} \otimes (\boldsymbol{w}_1 + \boldsymbol{w}_2) = \boldsymbol{v} \otimes \boldsymbol{w}_1 + \boldsymbol{v} \otimes \boldsymbol{w}_2$

Beispiel:

- Sei $V = \mathbb{R}^m$, $W = \mathbb{R}^n$ und $\otimes$ das äußere Produkt. Dann ist $V \otimes W = \mathbb{R}^m \otimes \mathbb{R}^n = \mathbb{R}^{m \times n}$.

Beispiel:

- Seien $V_1 = \mathbb{R}^m$, $V_2 = \mathbb{R}^n$, $V_3 = \mathbb{R}^q$ und $\otimes$ das Tensorprodukt. Dann ist

$$V_1 \otimes V_2 \otimes V_3 = \mathbb{R}^m \otimes \mathbb{R}^n \otimes \mathbb{R}^q = (\mathbb{R}^m \otimes \mathbb{R}^n) \otimes \mathbb{R}^q = \mathbb{R}^{m \times n} \otimes \mathbb{R}^q = \mathbb{R}^{m \times n \times q}.$$

Bsp: $V \otimes W$ vs. $v \otimes w$

Wichtige Unterscheidung:

- $v \otimes w$: Ein einzelnes Tensorprodukt (Rang-1-Matrix für $v, w \neq 0$)
- $V \otimes W$: Der **gesamte Vektorraum**, der von allen solchen Produkten erzeugt wird

Beispiel: Sei $V = W = \mathbb{R}^2$

- $A = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ **nicht** als einzelnes Tensorprodukt $v \otimes w$ von $v, w \in \mathbb{R}^2$ darstellbar (Rang 2 $\neq$ 1).
- **Aber:** $A \in V \otimes W = \mathbb{R}^{2 \times 2}$:

$$\begin{aligned} A &= e_1 \otimes e_1 + e_2 \otimes e_2 = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} \begin{pmatrix} 0 & 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad \checkmark \end{aligned}$$

Allgemein:

Jede Matrix $A \in \mathbb{R}^{m \times n}$ kann als Summe von höchstens $\min(m, n)$ Tensorprodukten geschrieben werden:
 $A = \sum_{i=1}^r \sigma_i u_i \otimes v_i$ mit $r = \text{rank}(A) \rightarrow$ **Singulärwertzerlegung!**

Tensorproduktbasis für Vektorräume

Theorem

Sind $\mathcal{B}_V = \{\mathbf{v}_1, \dots, \mathbf{v}_m\}$ und $\mathcal{B}_W = \{\mathbf{w}_1, \dots, \mathbf{w}_n\}$ Basen von V bzw. W , dann ist

$$\mathcal{B}_{V \otimes W} = \{\mathbf{v} \otimes \mathbf{w} : \mathbf{v} \in \mathcal{B}_V, \mathbf{w} \in \mathcal{B}_W\}$$

eine Basis von $V \otimes W$ und wird **Tensorproduktbasis** genannt. Insbesondere gilt

$$V \otimes W = \text{span}(\mathcal{B}_{V \otimes W}) = \left\{ \sum_{i=1}^m \sum_{j=1}^n c_{i,j} \mathbf{v}_i \otimes \mathbf{w}_j : c_{1,1}, \dots, c_{m,n} \in \mathbb{R} \right\}$$

und $\dim(V \otimes W) = \dim(V) \dim(W)$.

Bsp: Tensorproduktbasis für bivariate Polynome

Univariate Basis:

Raum der Polynome vom Grad ≤ 1 auf $[0, 1]$: $\mathcal{P}_1 = \{p(t) = a_0 + a_1 t : a_0, a_1 \in \mathbb{R}\}$ mit Basis $\mathcal{B} = \{b_1, b_2\}$, $b_1(t) = 1, b_2(t) = t$.

Tensorproduktbasis für bivariate Funktionen $f: [0, 1] \times [0, 1] \rightarrow \mathbb{R}$:

Basisfunktionen $\phi_{jk} = b_j \otimes b_k$ mit $(b_j \otimes b_k)(x, y) = b_j(x) \cdot b_k(y)$, also

$$\begin{aligned} \phi_{11}(x, y) &= b_1(x)b_1(y) = 1 & ; & & \phi_{12}(x, y) &= b_1(x)b_2(y) = y \\ \phi_{21}(x, y) &= b_2(x)b_1(y) = x & ; & & \phi_{22}(x, y) &= b_2(x)b_2(y) = xy \end{aligned}$$

Approximation einer bivariaten Funktion:

Z.B. $f(x, y) = 2 - x + 3y + 5xy = 2\phi_{11}(x, y) + 3\phi_{12}(x, y) - 1\phi_{21}(x, y) + 5\phi_{22}(x, y)$

Dimension: $\dim(\mathcal{P}_1 \otimes \mathcal{P}_1) = 2 \cdot 2 = 4 \checkmark$

Allgemein: Für k -variate Tensorprodukt-Polynome vom Grad $\leq d$ in jeder Variablen (univariat:

$$\mathcal{P}_d = \left\{ p(t) = \sum_{r=0}^d a_r t^r : a_r \in \mathbb{R} \right\} : \dim(\mathcal{P}_d^{\otimes k}) = (d+1)^k$$

Anwendung in der Praxis:

- **Statistik:** Tensorprodukt-Splines für multivariate Regression (GAMs) — ausführlich im Kapitel *Funktionsapproximation*
- **Finite-Elemente-Methoden (FEM):** Approximation von PDEs mit tensorbasierten Polynomen
- **Numerik:** Sparse Grids vermeiden die exponentielle Anzahl $(d+1)^k$ durch geschickte Auswahl

Bsp: Tensorproduktbasis für bivariate Polynome

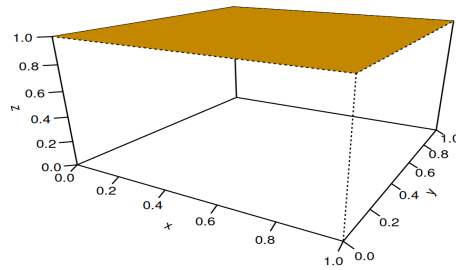
Tensorproduktbasis

$$P_1 \times P_1$$

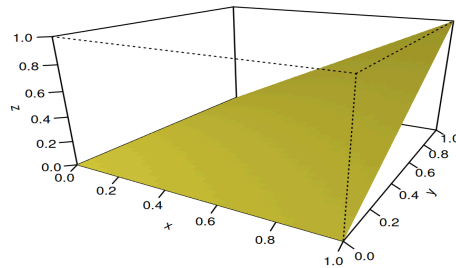
Univariate Basen:

$$B_x = \{1, x\}$$

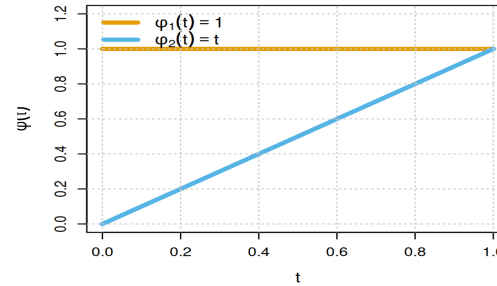
$$\varphi_1 = 1 \times 1$$



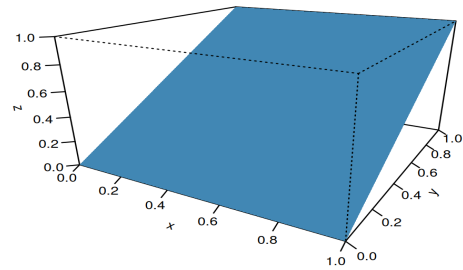
$$\varphi_4 = x \times y$$



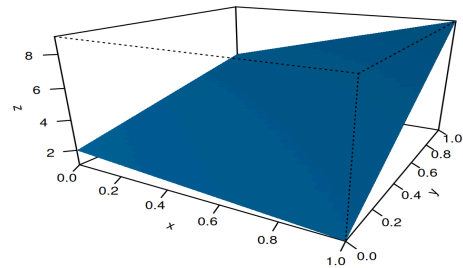
Univariate Basen



$$\varphi_2 = 1 \times y$$



Beispiel: $f(x, y) = 2 - x + 3y + 5xy$



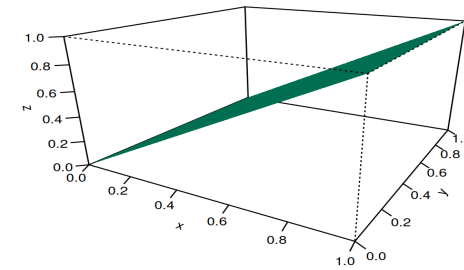
$$B_y = \{1, y\}$$

Tensorprodukt:

$$B_x \times B_y$$

erzeugt 4 bivariate

$$\varphi_3 = x \times 1$$



Anwendung: Tensorprodukt-Splines — Designmatrix

Vorgriff aufs Kapitel *Funktionsapproximation*: flexible univariate Funktionen via **B-Spline**-Basis $B_1, \dots, B_K$; bivariat via **Tensorproduktbasis** $\phi_{jk}(x, y) = B_j(x)B_k(y)$.

- Auswertung von $f(x, y) = \sum_{j,k} c_{jk} \phi_{jk}(x, y)$ auf einem **Gitter** aus x - und y -Werten:

$$\underbrace{\mathbf{F}}_{f(x_i, y_l)} = \mathbf{B}_x \mathbf{C} \mathbf{B}_y^\top$$

mit univariaten Basismatrizen $\mathbf{B}_x \in \mathbb{R}^{n_x \times K}$,
 $(\mathbf{B}_x)_{ij} = B_j(x_i)$ (analog $\mathbf{B}_y$)

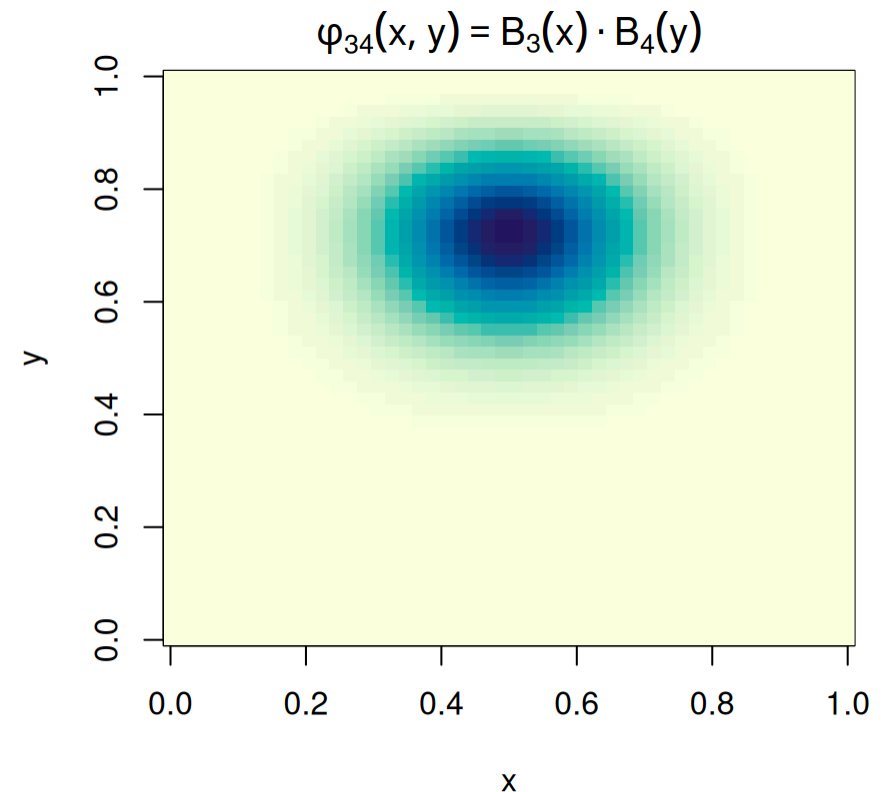
- **vec-Trick!**

$$\text{vec}(\mathbf{F}) = (\mathbf{B}_y \otimes_K \mathbf{B}_x) \text{vec}(\mathbf{C})$$

$\implies$ Designmatrix des Gitters ist das **Kroneckerprodukt**
 $\mathbf{B} = \mathbf{B}_y \otimes_K \mathbf{B}_x$

```
1 library(splines)
2 x <- seq(0, 1, length.out = 50)
3 Bx <- bs(x, df = 6) # B-Spline-Basis
4 By <- bs(x, df = 6)
5 B <- kronecker(By, Bx)
6 dim(B) # 2500 Gitterpunkte, 6*6 Koeffizienten
```

[1] 2500 36



Eine bivariate Basisfunktion = äußeres Produkt zweier univariater B-Splines. $\rightarrow$ Kap. *Funktionsapproximation I/II*: gute univariate Basen $\otimes$ -kombinieren!

Self-Check & Zusammenfassung

Kernkonzepte & Anwendungen

- **Tensoren**: höher-dimensionale Verallgemeinerung von Vektoren und Matrizen
 - Extrem wichtig für Deep Learning, Bild-/Videobearbeitung, moderne Statistik
- **Multilinearität** vs Linearität:
 - Linear: $f(a\mathbf{v} + b\mathbf{w}) = af(\mathbf{v}) + bf(\mathbf{w})$ vs. Multilinear:
 $f(\mathbf{v}_1, \dots, a\mathbf{v}_i + b\mathbf{w}_i, \dots, \mathbf{v}_n) = af(\mathbf{v}_1, \dots, \mathbf{v}_i, \dots, \mathbf{v}_n) + bf(\mathbf{v}_1, \dots, \mathbf{w}_i, \dots, \mathbf{v}_n)$
 - Bsp: Matrixmultiplikation bilinear aber nicht linear.
- **Äußeres Produkt**: Erzeugt (Rang-1-)Matrizen aus Vektoren ($\rightarrow$ **SVD**)
- **Tensorprodukt**: Verallgemeinerung des äußeren Produkts auf Tensoren höherer Stufe
- **Kroneckerprodukt** $\otimes_{\mathbb{K}}$: Spezielle Variante des Tensorprodukts von Matrizen ($\otimes$)
 - **vec-Trick** $\text{vec}(\mathbf{AXB}) = (\mathbf{B}^\top \otimes_{\mathbb{K}} \mathbf{A})\text{vec}(\mathbf{X})$: Matrixgleichungen (Sylvester/Lyapunov) $\rightarrow$ gewöhnliche LGS
 - Statistik: separierbare/Kronecker-strukturierte Kovarianzen, blockdiagonale Matrizen
- *Tensorprodukte bauen komplexe Räume auf:*
 - Konstruiere multivariate (Funktions-)Räume aus univariaten — Designmatrix auf Gittern = Kroneckerprodukt $\rightarrow$ **Funktionsapproximation**, (Höherdimensionale) **Splines**

Nächste Vorlesungen:

Fortgeschrittene Differentialrechnung

Self-Check

Frage 1: Daten an $m = 10$ Orten zu $n = 20$ Zeitpunkten. Eine unstrukturierte Kovarianzmatrix des 200-dimensionalen Datenvektors hat $\frac{200 \cdot 201}{2} = 20100$ freie Parameter. Wie viele Parameter haben die beiden Faktoren Σ_T, Σ_S des separierbaren Modells $\Sigma = \Sigma_T \otimes_K \Sigma_S$ zusammen?

- a) 265 b) 11550 c) 20100 d) 200

Frage 2: Sei $f: \mathbb{R}^2 \times \mathbb{R}^2 \rightarrow \mathbb{R}$ bilinear. Was ist $f(2v, 3w)$?

- a) $2f(v, w)$ b) $5f(v, w)$ c) $6f(v, w)$ d) $f(v, w)$

Frage 3: Der vec-Trick besagt $\text{vec}(\mathbf{A}\mathbf{X}\mathbf{B}) = ?$

- a) $(\mathbf{A} \otimes_K \mathbf{B}) \text{vec}(\mathbf{X})$ b) $(\mathbf{B}^\top \otimes_K \mathbf{A}) \text{vec}(\mathbf{X})$ c) $(\mathbf{B} \otimes_K \mathbf{A}^\top) \text{vec}(\mathbf{X})$ d) $(\mathbf{A} \otimes_K \mathbf{B}^\top) \text{vec}(\mathbf{X})$

Anhang

Anwendung: RGB-Bilder als Tensoren

Beispiel 1: RGB-Bild

- Farbbild der Größe 224×224 Pixel wird als Tensor $\mathbf{I} \in \mathbb{R}^{224 \times 224 \times 3}$ dargestellt:
 - $\mathbf{I}_{i,j,1}$: Rot-Intensität an Position (i, j) (Wert zwischen 0 und 255)
 - $\mathbf{I}_{i,j,2}$: Grün-Intensität an Position (i, j)
 - $\mathbf{I}_{i,j,3}$: Blau-Intensität an Position (i, j)
- Einträge: $224 \times 224 \times 3 \approx 150 \cdot 10^3$ Werte pro Bild



$\approx$



↔ zurück: „Tensoren in Deep Learning“

Anwendung: Batch-Tensoren in CNNs

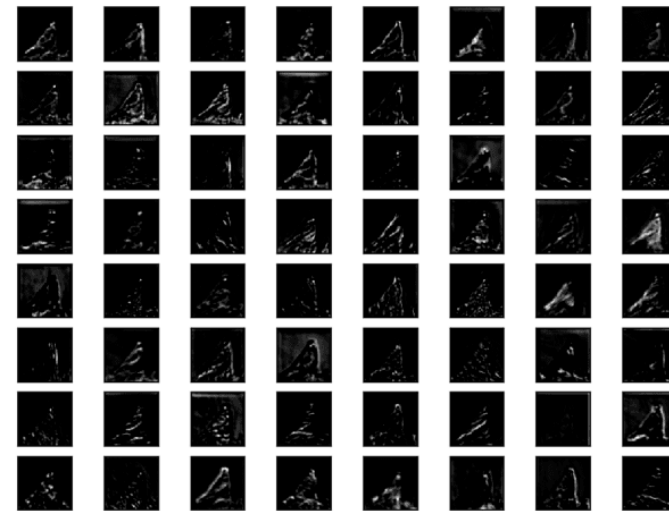
Beispiel 2: Batch von Bildern (Convolutional Neural Networks) (CNNs)

- CNN verarbeitet typischerweise mehrere Bilder gleichzeitig: $\mathbf{B} \in \mathbb{R}^{32 \times 224 \times 224 \times 3}$
 1. Dimension (Batch-Size): 32 Bilder gleichzeitig
 2. Dimension (Höhe): 224 Pixel
 3. Dimension (Breite): 224 Pixel
 4. Dimension (Kanäle): 3 Farbkanäle (RGB)
- Einträge: $32 \times 224 \times 224 \times 3 \approx 4.8 \cdot 10^6$ Werte

Anwendung: Feature Maps in CNNs

Beispiel 3: Feature Maps in CNNs

- Convolutional Layer $c: \mathbb{R}^{32 \times 224 \times 224 \times 3} \rightarrow \mathbb{R}^{32 \times 56 \times 56 \times 64}$
- Reduziert (üblicherweise) räumliche Auflösung ($224 \times 224 \rightarrow 56 \times 56$)
- Erhöht Anzahl der “Kanäle” (*features*) ($3 \rightarrow 64$)



Feature Map: Block 3 des VGG16-Modells; Quelle: machinelearningmastery.com

↔ zurück: „Tensoren in Deep Learning“

Universelle Eigenschaft des Tensorprodukts

- Das **Tensorprodukt** $V \otimes W$ hat eine *universelle Eigenschaft*: **Jede** bilineare Abbildung $f: V \times W \rightarrow Z$ kann eindeutig als lineare Abbildung $\tilde{f}: V \otimes W \rightarrow Z$ faktorisiert werden:

$$f(\boldsymbol{v}, \boldsymbol{w}) = \tilde{f}(\boldsymbol{v} \otimes \boldsymbol{w}) \quad \text{für alle } \boldsymbol{v} \in V, \boldsymbol{w} \in W$$

- Diese Eigenschaft macht das Tensorprodukt zur “natürlichen” Konstruktion für die Linearisierung bilinearer Abbildungen.

Differentialrechnung I

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)

Material: Thomas Nagler

LMU München

Verwendete Vorkenntnisse

- Matrix-*Spur* & *Matrixnormen*
- Landau-Notation: $g(h) = o(h) \iff \lim_{h \rightarrow 0} \frac{g(h)}{h} = 0$
- **Analysis** (essentiell)
 - Ableitungen: $f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$
 - Partielle Ableitungen: $\frac{\partial f(\mathbf{x})}{\partial x_i}$
 - Lineare Approximation: $f(x + h) \approx f(x) + f'(x)h$
 - Grundlagen Taylorreihen
- **Lineare Algebra**
 - Matrixinversion und Determinante
 - Normierte (Vektor-)Räume
 - Beschränkte lineare Operatoren

Opener-Quiz: Was wissen Sie noch?

Frage 1: Seien $g, h: \mathbb{R} \rightarrow \mathbb{R}$ differenzierbar. Welche Rechenregeln stimmen?

1. $(g + h)'(x) = g'(x) + h'(x)$
2. $(g \cdot h)'(x) = g'(x) \cdot h'(x)$
3. $(g \cdot h)'(x) = g'(x)h(x) + g(x)h'(x)$
4. $(g \circ h)'(x) = g'(h(x)) \cdot h'(x)$

Frage 2: Welche der folgenden Abbildungen sind *linear*?

1. $L(\mathbf{x}) = \mathbf{A}\mathbf{x}$ mit $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{x} \in \mathbb{R}^n$
2. $L(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}$ mit $\mathbf{b} \neq \mathbf{0}$
3. $L(x) = c \cdot x$ mit $c \in \mathbb{R}$, $x \in \mathbb{R}$
4. $L(x) = x^2$

Überblick

- Aus Analysis bekannt: Ableitungen, *Differenzierbarkeit*
- Neue wichtige Technik: **Allgemeinerer Ableitungsbegriff** (*Fréchet-Ableitung*)
 - Ableitungen mit **Matrix-/Vektor-Inputs** oder **-Outputs**
 - Ableitungen in *allgemeineren (normierten) Räumen*

Ableitung von univariaten Funktionen

Recap: Ableitung von univariaten Funktionen

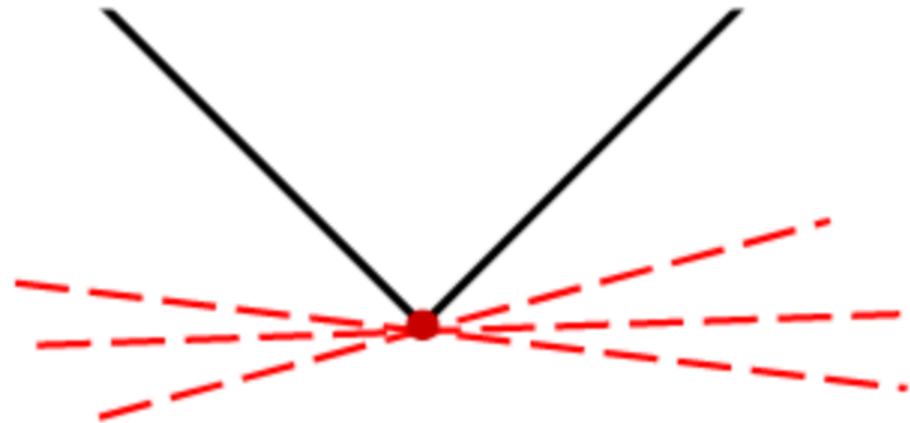
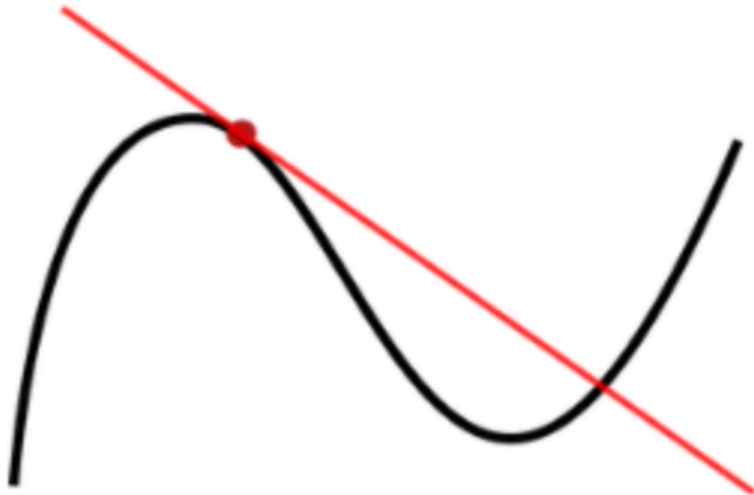
Def.: Differenzierbarkeit

Sei $S \subseteq \mathbb{R}$ offen und $f: S \rightarrow \mathbb{R}$ eine Funktion.

Wir nennen f an der Stelle $x \in S$ **differenzierbar** mit **Ableitung** $f'(x)$, falls

$$\lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} = f'(x).$$

- $f'(x)$ ist die **Steigung einer Tangente an f durch x** .
- Falls diese Steigung nicht eindeutig ist, ist f an der Stelle x nicht differenzierbar:



Quiz

Sei f differenzierbar in x mit Ableitung $f'(x)$.

Frage: Welche Aussagen sind wahr?

1. f' ist eine lineare Funktion.
2. $h \mapsto f'(x)h$ ist eine lineare Funktion (für festes x).
3. $\lim_{h \rightarrow 0} \frac{|f(x+h) - f(x) - f'(x)h|}{h} = 0$.
4. $f(x+h) = f(x) + f'(x)h + o(h)$.

Beweis (3) $\iff$ (4): $\rightarrow$ Anhang.

Allgemeiner Ableitungsbegriff

Allgemeiner Ableitungsbegriff

- Ableitungen können auf verschiedene Arten auf allgemeinere, **normierte** Räume erweitert werden.
- Die Idee ist immer: **Ableitung** = **lineare Approximation**.
- Die Definition nach Fréchet reicht aus für
 - alle endlich-dimensionalen Räume,
 - einige unendlich-dimensionale Räume.
- Allgemeinere Definitionen: *Gateaux*-, *Hadamard*-Ableitungen.

Allgemeiner Ableitungsbegriff

Def.: (Fréchet-)differenzierbar

Seien $\mathbb{D}$ und $\mathbb{E}$ zwei normierte Räume und $f: \mathbb{D} \rightarrow \mathbb{E}$.

Wir nennen f an der Stelle $x \in \mathbb{D}$ **(Fréchet-)differenzierbar**, falls eine beschränkte lineare Abbildung $D_x f: \mathbb{D} \rightarrow \mathbb{E}$ existiert, sodass

$$f(x + h) = f(x) + D_x f(h) + o(\|h\|) \quad \text{für } h \rightarrow 0.$$

- $D_x f(h)$ beschränkt: Es gibt $M < \infty$, sodass $\|D_x f(h)\| \leq M \|h\| \quad \forall h$.
- $h \rightarrow D_x f(h)$ ist linear in h , hängt aber in beliebiger Weise von x ab.
- Beispiel:
 - Für $f: \mathbb{R} \rightarrow \mathbb{R}$ ist $D_x f(h) = f'(x)h$.
 - $D_x f$ ist beschränkt, wenn $|f'(x)| < \infty$.
- Fehler $o(\|h\|)$: wird relativ zu $\|h\|$ beliebig klein, wenn $h \rightarrow 0$

Konkret: Fréchet-Ableitung von $f(x) = x^2$

Sei $f: \mathbb{R} \rightarrow \mathbb{R}$ mit $f(x) = x^2$. Zu Zeigen: f Fréchet-differenzierbar mit $D_x f(h) = 2xh$.

Prüfen der Definition:

$$\begin{aligned} f(x+h) &= (x+h)^2 = x^2 + 2xh + h^2 \\ &= f(x) + 2xh + h^2 \\ &= f(x) + D_x f(h) + h^2 \\ \implies D_x f(h) &= 2xh \text{ (linear in } h) \end{aligned}$$

- $\lim_{h \rightarrow 0} \frac{|h^2|}{|h|} = \lim_{h \rightarrow 0} |h| = 0 \implies h^2 = o(|h|) \quad \checkmark$
- $|D_x f(h)| = |2xh| = 2|x| \cdot |h| \leq M|h| \quad \forall h \text{ mit } M = 2|x| < \infty$
 $\implies D_x f$ beschränkt $\checkmark$

$\longrightarrow$ entspricht $f'(x) = 2x$, da $D_x f(h) = f'(x)h = 2x \cdot h$.

Wichtige Spezialfälle

Wichtige Spezialfälle

- Die bereits bekannten Ableitungen sind alle Spezialfälle der Fréchet-Ableitung.
- Besonders wichtige neue Spezialfälle: Ableitungen von Funktionen, die
 - *Skalare, Vektoren oder Matrizen*
 - als *Input oder Output* haben.
- Insbesondere betrachten wir die folgenden Fälle:

Input \ Output	Skalar	Vektor	Matrix
Skalar	$\frac{dy}{dx}$	$\frac{dy}{dx}$	$\frac{dY}{dx}$
Vektor	$\frac{\partial y}{\partial x}$	$\frac{\partial y}{\partial x}$	
Matrix	$\frac{\partial y}{\partial X}$		

Spezialfall: Vektor zu Skalar

Spezialfall: Vektor zu Skalar

Def.: Gradient

Sei $f: \mathbb{R}^n \rightarrow \mathbb{R}$ eine differenzierbare Funktion.

Wir nennen den **Gradienten** von f in $\mathbf{x} \in \mathbb{R}^n$ den *Zeilenvektor*

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \nabla f(\mathbf{x}) = \left(\frac{\partial f(\mathbf{x})}{\partial x_1}, \dots, \frac{\partial f(\mathbf{x})}{\partial x_n} \right) \in \mathbb{R}^{1 \times n}$$

$$\implies f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + D_{\mathbf{x}}f(\mathbf{h}) + o(\|\mathbf{h}\|)$$

mit $D_{\mathbf{x}}f(\mathbf{h}) = \nabla f(\mathbf{x})\mathbf{h}$.

Anmerkung:

Oft wird ∇f auch als Spaltenvektor definiert.

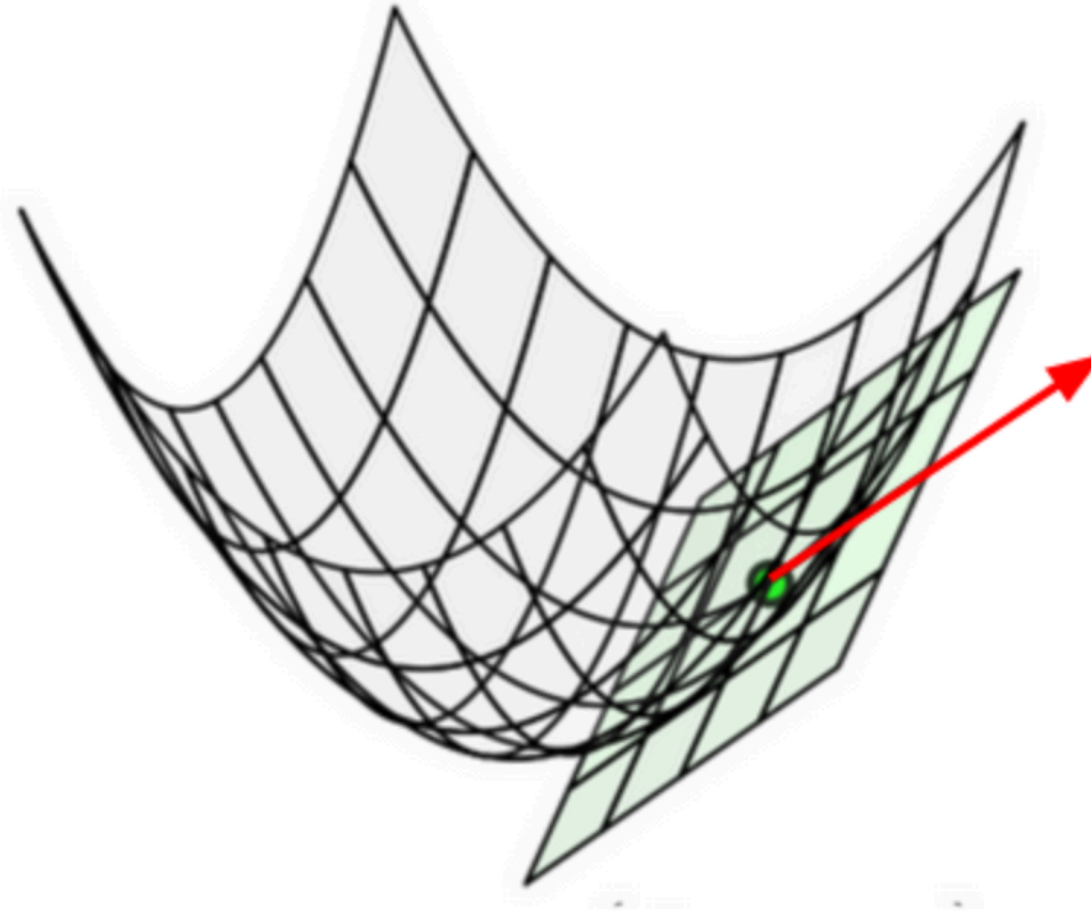
Für uns ist er als **Zeilenvektor** $\in \mathbb{R}^{1 \times n}$ konsistenter und praktischer:

- Der Gradient ist dann genau die $1 \times n$ -**Jacobimatrix** von f .
- Damit wird die Kettenregel zu einer simplen Matrixmultiplikation von Jacobimatrizen, ganz ohne Transponieren.

(→ Kapitel *Differentialrechnung II*)

Spezialfall: Vektor zu Skalar

$$f: \mathbb{R}^n \rightarrow \mathbb{R}$$



- Graph von f in grau, der grüne Punkt ist $(\mathbf{x}, f(\mathbf{x}))$.
- Der rote Pfeil zeigt (in die Tangentialebene angehoben) die Richtung des Gradienten $\nabla f(\mathbf{x})^\top$. $\implies$ Richtung, in die f an Stelle $\mathbf{x}$ am **stärksten** steigt
- Die grüne Ebene ist der Graph von $\mathbf{u} \mapsto f(\mathbf{x}) + \nabla f(\mathbf{x})(\mathbf{u} - \mathbf{x})$.

Beispiel: Gradient einer quadratischen Funktion

$$f: \mathbb{R}^2 \rightarrow \mathbb{R} \text{ mit } f(\mathbf{x}) = x_1^2 + 3x_1x_2 + 2x_2^2$$

Schritt 1: Berechne partielle Ableitungen

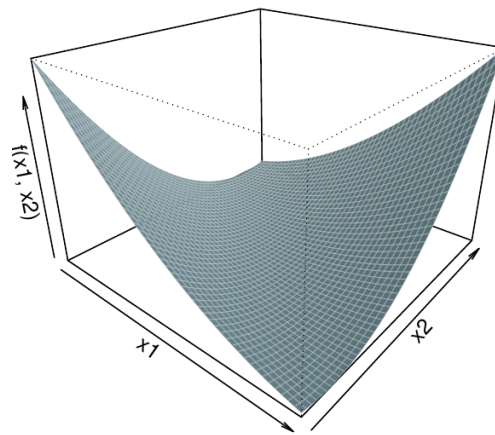
$$\frac{\partial f(\mathbf{x})}{\partial x_1} = 2x_1 + 3x_2;$$

$$\frac{\partial f(\mathbf{x})}{\partial x_2} = 3x_1 + 4x_2$$

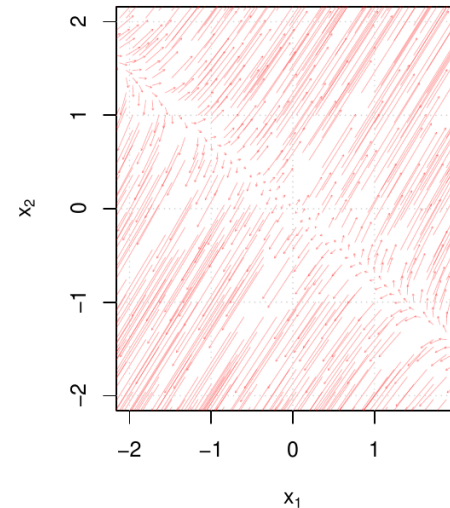
Schritt 2: Gradient als Zeilenvektor

$$\nabla f(\mathbf{x}) = (2x_1 + 3x_2 \quad 3x_1 + 4x_2) \quad (\implies \nabla f: \mathbb{R}^2 \rightarrow \mathbb{R}^{1 \times 2}!)$$

$$f(x_1, x_2) = x_1^2 + 3x_1x_2 + 2x_2^2$$

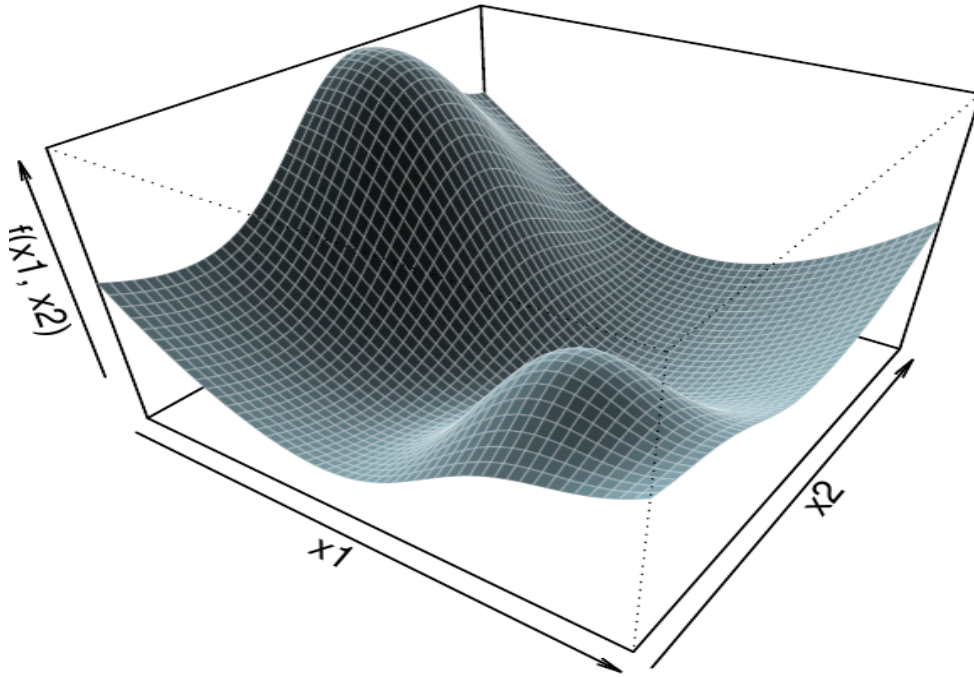


$$\nabla f(\mathbf{x}) = (2x_1 + 3x_2, 3x_1 + 4x_2)$$

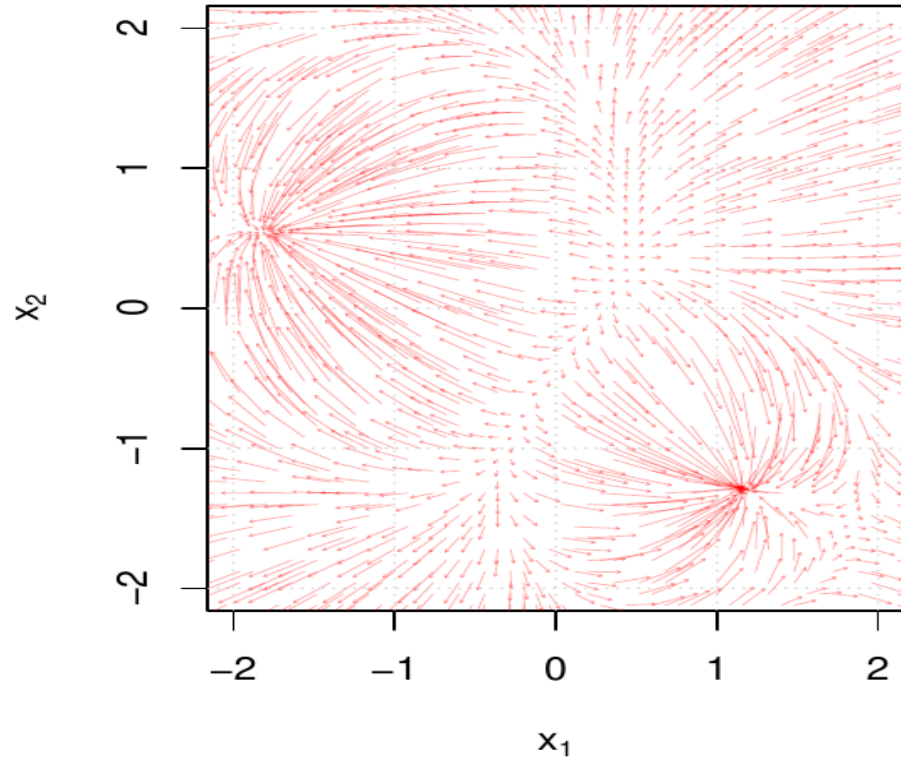


Beispiel: Gradient einer komplexeren Funktion

$$f(x_1, x_2)$$



$$\nabla f(x_1, x_2)$$



Übung: Vektor zu Skalar

Sei $\mathbf{a}, \mathbf{x} \in \mathbb{R}^n$ und

$$f(\mathbf{x}) = \mathbf{a}^\top \mathbf{x}.$$

Frage:

Was ist $\nabla f(\mathbf{x})$?

Antwort:

- Es gilt $f(\mathbf{x}) = \sum_{i=1}^n a_i x_i$.
- Deshalb $\frac{\partial f(\mathbf{x})}{\partial x_i} = a_i$.
- Also

$$\nabla f(\mathbf{x}) := \left(\frac{\partial f(\mathbf{x})}{\partial x_1}, \dots, \frac{\partial f(\mathbf{x})}{\partial x_n} \right) = (a_1, \dots, a_n) = \mathbf{a}^\top.$$

Übung: Gradient der quadratischen Form

Sei $\mathbf{A} \in \mathbb{R}^{n \times n}$ symmetrische Matrix und $f: \mathbb{R}^n \rightarrow \mathbb{R}$ mit

$$f(\mathbf{x}) = \mathbf{x}^\top \mathbf{A} \mathbf{x}.$$

1. $f(\mathbf{x}) = \sum_{i=1}^n \sum_{j=1}^n x_i a_{ij} x_j.$

2. Ableiten nach x_k :

$$\begin{aligned} \frac{\partial f(\mathbf{x})}{\partial x_k} &= \sum_{i=1}^n x_i a_{ik} + \sum_{j=1}^n a_{kj} x_j \\ &= \sum_{i=1}^n a_{ki} x_i + \sum_{j=1}^n a_{kj} x_j \quad (\text{da } \mathbf{A} \text{ symmetrisch}) \\ &= 2 \sum_{i=1}^n a_{ki} x_i = 2(\mathbf{A} \mathbf{x})_k \\ &\implies \nabla f(\mathbf{x}) = 2\mathbf{x}^\top \mathbf{A} \end{aligned}$$

Spezialfall: Ist $\mathbf{A} = \mathbf{I}$, dann $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{x} = \|\mathbf{x}\|_2^2$ und $\nabla f(\mathbf{x}) = 2\mathbf{x}^\top$.

Anwendung: Gradient Descent

Problemstellung:

Minimiere eine Verlustfunktion $L(\theta)$

(z.B. Mean Squared Error eines Modells mit Parametern θ).

Gradient Descent (Gradientenabstieg):

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla L(\theta^{(t)})^\top$$

- $\alpha > 0$: Lernrate (Schrittweite)
- $-\nabla L(\theta^{(t)})^\top$: Negativer Gradient zeigt in Richtung des steilsten *Abstiegs*
- Gradient sagt uns, in welche Richtung wir θ ändern müssen, um näher an ein Minimum (= “*optimales*” Modell) zu kommen
- fundamentaler Basisalgorithmus zur **Optimierung** von neuronalen Netze, komplexen Regressionsmodellen, ...
→ *Optimierung I & II*

Spezialfall: Vektor zu Vektor

Spezialfall: Vektor zu Vektor

Def.: Jacobimatrix

Sei $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ eine differenzierbare Funktion mit $f(\mathbf{x}) = (f_1(\mathbf{x}), \dots, f_m(\mathbf{x}))^\top$.

Die **Jacobimatrix** $\mathbf{J}_f(\mathbf{x}) \in \mathbb{R}^{m \times n}$ von f in $\mathbf{x} \in \mathbb{R}^n$ ist

$$\mathbf{J}_f(\mathbf{x}) := \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \begin{pmatrix} \partial f_1(\mathbf{x})/\partial x_1 & \cdots & \partial f_1(\mathbf{x})/\partial x_n \\ \vdots & \ddots & \vdots \\ \partial f_m(\mathbf{x})/\partial x_1 & \cdots & \partial f_m(\mathbf{x})/\partial x_n \end{pmatrix}.$$

$$f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + D_{\mathbf{x}} f(\mathbf{x}) \mathbf{h} + o(\|\mathbf{h}\|)$$

mit $D_{\mathbf{x}} f(\mathbf{x}) = \mathbf{J}_f(\mathbf{x})$.

- Die Zeilen der **Jacobimatrix** sind **Gradienten** der Komponenten von $f(\mathbf{x}) = (f_1(\mathbf{x}), \dots, f_m(\mathbf{x}))^\top$:

$$\mathbf{J}_f(\mathbf{x}) = \begin{pmatrix} \nabla f_1(\mathbf{x}) \\ \vdots \\ \nabla f_m(\mathbf{x}) \end{pmatrix}$$

Spezialfall: Vektor zu Vektor

Wichtige Identitäten:

- $f(\mathbf{x}) = \mathbf{c} \in \mathbb{R}^m \implies \mathbf{J}_f(\mathbf{x}) = \mathbf{0}_{m \times n}$
- $f(\mathbf{x}) = \mathbf{x}$ (d.h. $m = n$) $\implies \mathbf{J}_f(\mathbf{x}) = \mathbf{I}_n$
- $f(\mathbf{x}) = \mathbf{A}\mathbf{x}$ mit $\mathbf{A} \in \mathbb{R}^{m \times n} \implies \mathbf{J}_f(\mathbf{x}) = \mathbf{A}$
- $f(\mathbf{x}) = (\mathbf{x}^\top \mathbf{A})^\top = \mathbf{A}^\top \mathbf{x}$ mit $\mathbf{A} \in \mathbb{R}^{n \times m} \implies \mathbf{J}_f(\mathbf{x}) = \mathbf{A}^\top$

Übung

$$f(\mathbf{x}) = \mathbf{A}\mathbf{x} \text{ mit } \mathbf{A} \in \mathbb{R}^{m \times n} :$$

$$f_i(\mathbf{x}) = \sum_{j=1}^n a_{ij}x_j \quad (\text{komponentenweise})$$

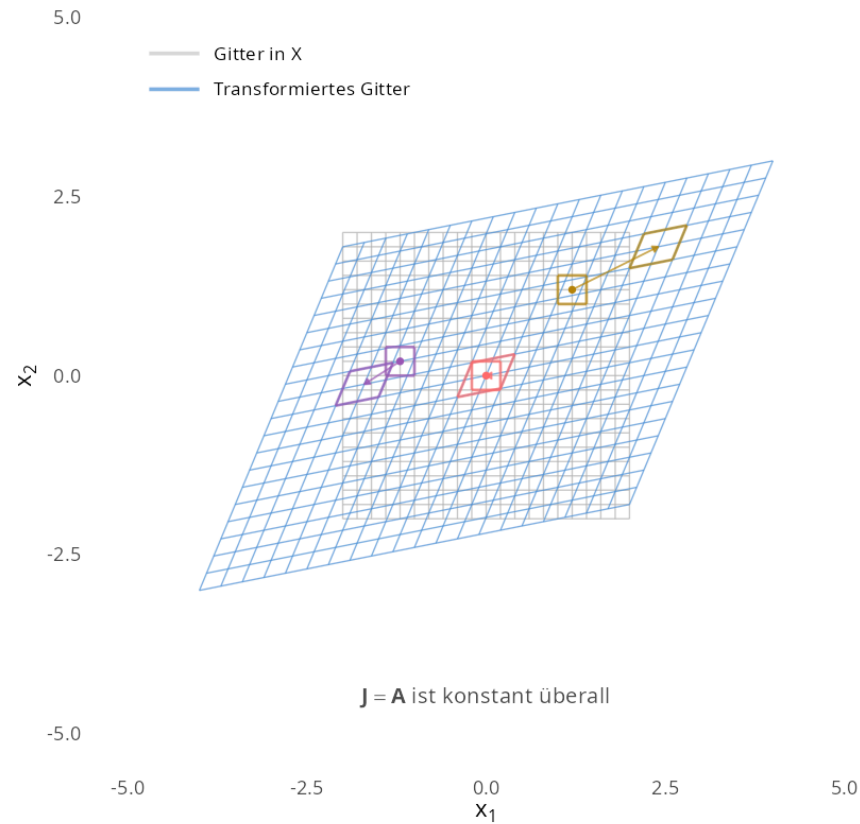
$$\frac{\partial f_i(\mathbf{x})}{\partial x_k} = \frac{\partial}{\partial x_k} \left(\sum_{j=1}^n a_{ij}x_j \right) = a_{ik}$$

$$\implies \mathbf{J}_f(\mathbf{x}) = \begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{pmatrix} = \mathbf{A} \quad \checkmark$$

- Lineare Funktionen haben konstante Jacobimatrizen
- Für lineare Funktionen gilt $f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + \mathbf{J}_f(\mathbf{x})\mathbf{h}$

Visualisierung: Lineare Transformation $\mathbb{R}^2 \rightarrow \mathbb{R}^2$

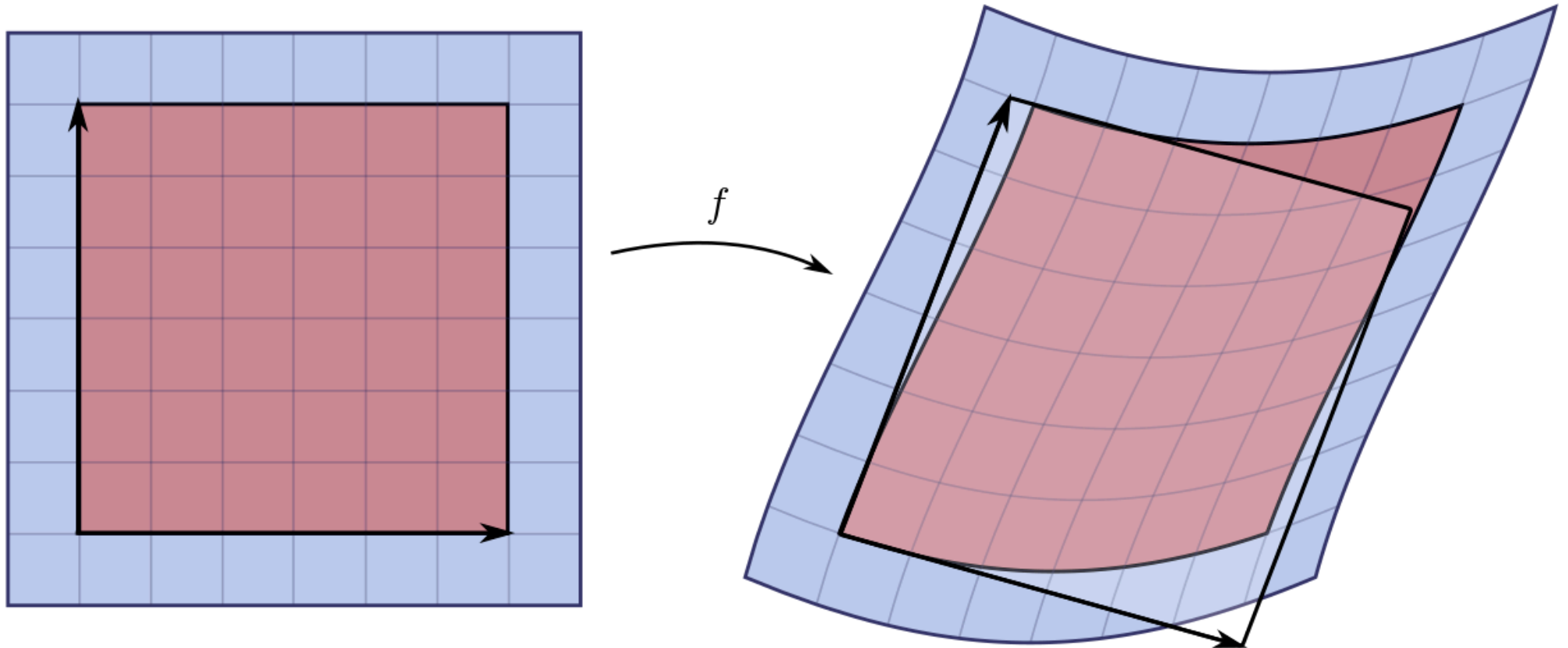
$$f(x_1, x_2) = \mathbf{A} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \text{ mit } \mathbf{A} = \begin{bmatrix} 1.5 & 0.5 \\ 0.3 & 1.2 \end{bmatrix}$$



Lineare Funktion: $f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + \mathbf{J}_f(\mathbf{x})\mathbf{h}$

Geometrische Intuition

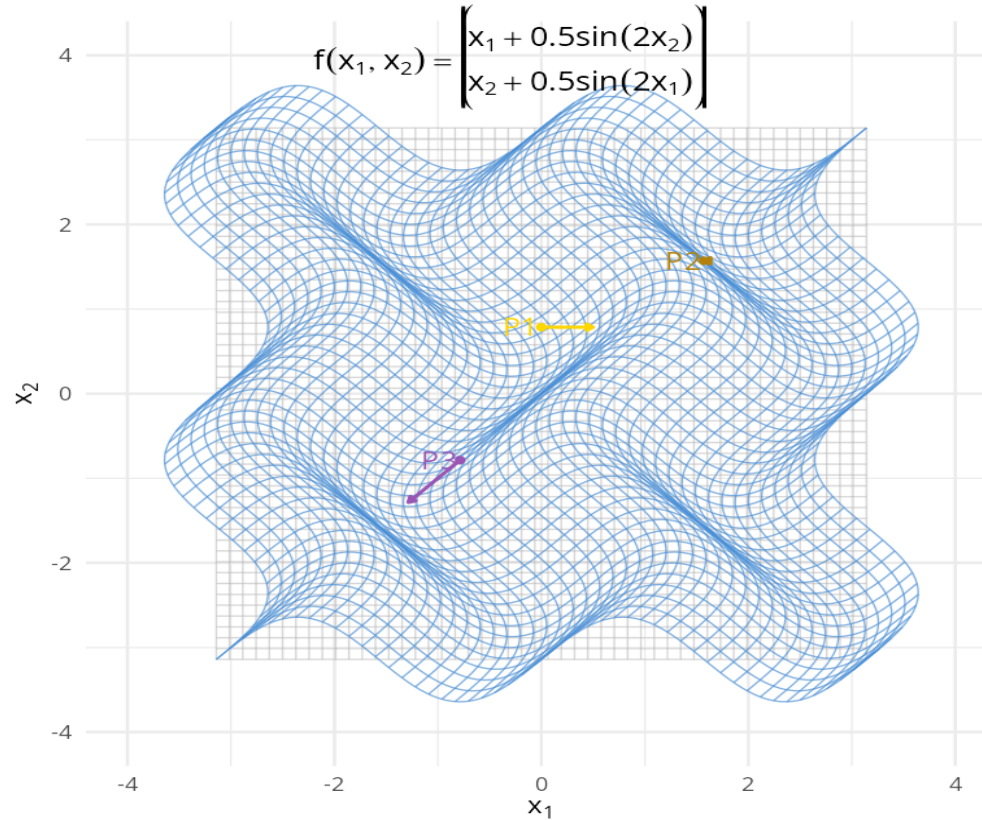
$$f: \mathbb{R}^n \rightarrow \mathbb{R}^m$$



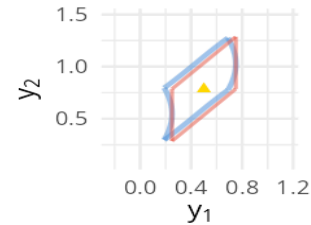
- $f: \mathbb{R}^2 \rightarrow \mathbb{R}^2$ transformiert das rote Quadrat links zur roten Fläche rechts.
- Die gekrümmte Darstellung soll verdeutlichen, dass f den Raum lokal verbiegt und streckt, obwohl Urbild und Bild dieselbe Dimension haben.
- Das schwarze Quadrat rechts zeigt das Bild der linearen Approximation durch $\mathbf{J}_f$.

Geometrische Intuition — Jacobimatrix $\mathbb{R}^2 \rightarrow \mathbb{R}^2$

Jacobi-Matrix: Lokale Lineare Approximation



P1:(0.00, 0.79) $\rightarrow$ (0.50, 0.79)



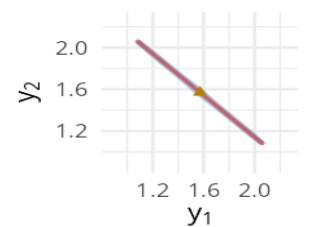
$$J_f(P_1) = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

— Wahres $f(\square)$

— Jacobi-Approx.

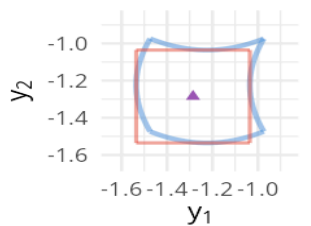
— Gitter (in X oder Y)

P2:(1.57, 1.57) $\rightarrow$ (1.57, 1.57)



$$J_f(P_2) = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

P3:(-0.79, -0.79) $\rightarrow$ (-1.29, -1.29)



$$J_f(P_3) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Jacobi-Matrix:

$$J_f(x_1, x_2) = \begin{bmatrix} 1 & \cos(2x_2) \\ \cos(2x_1) & 1 \end{bmatrix}$$

$$f(x + h) \approx f(x) + J_f(x)h$$

Spezialfall: Skalar zu Matrix

Spezialfall: Skalar zu Matrix

$$\mathbf{F}: \mathbb{R} \rightarrow \mathbb{R}^{m \times n}$$

- Sei $x \in \mathbb{R}$, seien alle f_{ij} in x differenzierbar, und

$$\mathbf{F}(x) = \begin{pmatrix} f_{11}(x) & \cdots & f_{1n}(x) \\ \vdots & \ddots & \vdots \\ f_{m1}(x) & \cdots & f_{mn}(x) \end{pmatrix}.$$

- Definiere

$$\frac{\partial \mathbf{F}(x)}{\partial x} = \begin{pmatrix} \frac{\partial f_{11}(x)}{\partial x} & \cdots & \frac{\partial f_{1n}(x)}{\partial x} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_{m1}(x)}{\partial x} & \cdots & \frac{\partial f_{mn}(x)}{\partial x} \end{pmatrix} \in \mathbb{R}^{m \times n}.$$

- Dann ist

$$\mathbf{F}(x + h) = \mathbf{F}(x) + D_x \mathbf{F}(h) + o(|h|) \quad \text{mit} \quad D_x \mathbf{F}(h) = \frac{\partial \mathbf{F}(x)}{\partial x} h.$$

Übung: Skalar zu Matrix

Sei, für $x > 0$,

$$\mathbf{F}(x) = \begin{pmatrix} x^2 & 2e^x & 0 \\ 0 & x & \ln(x) \end{pmatrix}.$$

Frage:

Was ist $\frac{\partial \mathbf{F}(x)}{\partial x}$?

Antwort:

$$\frac{\partial \mathbf{F}(x)}{\partial x} = \begin{pmatrix} 2x & 2e^x & 0 \\ 0 & 1 & 1/x \end{pmatrix}.$$

Skalar zu Matrix: Wichtige Identitäten

Für *quadratisches, differenzierbares*

$$\mathbf{F}: \mathbb{R} \rightarrow \mathbb{R}^{n \times n}$$

(erste beide Formeln: $\mathbf{F}(x)$ zusätzlich invertierbar):

- $\frac{\partial \mathbf{F}(x)^{-1}}{\partial x} = -\mathbf{F}(x)^{-1} \frac{\partial \mathbf{F}(x)}{\partial x} \mathbf{F}(x)^{-1}.$
- $\frac{\partial \det(\mathbf{F}(x))}{\partial x} = \det(\mathbf{F}(x)) \operatorname{tr} \left[\mathbf{F}(x)^{-1} \frac{\partial \mathbf{F}(x)}{\partial x} \right].$
- $\frac{\partial \operatorname{tr}(\mathbf{F}(x))}{\partial x} = \operatorname{tr} \left(\frac{\partial \mathbf{F}(x)}{\partial x} \right).$

(Rechenbeispiel zur Determinanten-Formel: → *Anhang* — wichtig z.B. für Ableitungen von Likelihood-Funktionen!)

Beweis: Ableitung der Spur

$$\frac{\partial \text{tr}(\mathbf{F}(x))}{\partial x} = \text{tr}\left(\frac{\partial \mathbf{F}(x)}{\partial x}\right)$$

Beweis:

$$\begin{aligned}\text{tr}(\mathbf{F}(x)) &= \sum_{i=1}^n f_{ii}(x) \\ \frac{\partial}{\partial x} \text{tr}(\mathbf{F}(x)) &= \frac{\partial}{\partial x} \left(\sum_{i=1}^n f_{ii}(x) \right) \\ &= \sum_{i=1}^n \frac{\partial f_{ii}(x)}{\partial x} \\ &= \text{tr}\left(\frac{\partial \mathbf{F}(x)}{\partial x}\right) \quad \checkmark\end{aligned}$$

Spezialfall: Matrix zu Skalar

Spezialfall: Matrix zu Skalar

$$f: \mathbb{R}^{m \times n} \rightarrow \mathbb{R}$$

- Sei f in $\mathbf{X} \in \mathbb{R}^{m \times n}$ differenzierbar. Definiere:

$$\frac{\partial f(\mathbf{X})}{\partial \mathbf{X}} = \begin{pmatrix} \partial f(\mathbf{X})/\partial x_{11} & \cdots & \partial f(\mathbf{X})/\partial x_{1n} \\ \vdots & \ddots & \vdots \\ \partial f(\mathbf{X})/\partial x_{m1} & \cdots & \partial f(\mathbf{X})/\partial x_{mn} \end{pmatrix} \in \mathbb{R}^{m \times n}.$$

- Dann ist

$$f(\mathbf{X} + \mathbf{H}) = f(\mathbf{X}) + D_{\mathbf{X}}f(\mathbf{H}) + o(\|\mathbf{H}\|) \quad \text{mit} \quad D_{\mathbf{X}}f(\mathbf{H}) = \text{tr} \left(\left(\frac{\partial f(\mathbf{X})}{\partial \mathbf{X}} \right)^{\top} \mathbf{H} \right).$$

Matrix zu Skalar: Beispiel

Beispiel:

- Sei $f(\mathbf{X}) = \mathbf{a}^\top \mathbf{X} \mathbf{b}$ mit $\mathbf{X} \in \mathbb{R}^{m \times n}$, $\mathbf{a} \in \mathbb{R}^m$, $\mathbf{b} \in \mathbb{R}^n$.
- Dann ist

$$f(\mathbf{X}) = \sum_{i=1}^m \sum_{j=1}^n a_i x_{ij} b_j \implies \frac{\partial f(\mathbf{X})}{\partial x_{ij}} = a_i b_j.$$

- Daraus folgt

$$\begin{aligned} \frac{\partial \mathbf{a}^\top \mathbf{X} \mathbf{b}}{\partial \mathbf{X}} &= \begin{pmatrix} a_1 b_1 & \cdots & a_1 b_n \\ \vdots & \ddots & \vdots \\ a_m b_1 & \cdots & a_m b_n \end{pmatrix} = \mathbf{a} \mathbf{b}^\top \\ \implies D_{\mathbf{X}} f(\mathbf{H}) &= \text{tr} \left((\mathbf{a} \mathbf{b}^\top)^\top \mathbf{H} \right) = \text{tr} (\mathbf{b} \mathbf{a}^\top \mathbf{H}) = \mathbf{a}^\top \mathbf{H} \mathbf{b} \end{aligned}$$

- also hier:

$$f(\mathbf{X} + \mathbf{H}) = \mathbf{a}^\top \mathbf{X} \mathbf{b} + \mathbf{a}^\top \mathbf{H} \mathbf{b} = f(\mathbf{X}) + D_{\mathbf{X}} f(\mathbf{H})$$

Wichtige Identitäten: Matrix zu Skalar

- **Quadratische Form:** $f(\mathbf{X}) = \text{tr}(\mathbf{X}\mathbf{A}\mathbf{X}^\top)$ mit symmetrischem $\mathbf{A} \in \mathbb{R}^{n \times n}$

$$\frac{\partial f(\mathbf{X})}{\partial \mathbf{X}} = \mathbf{X}(\mathbf{A} + \mathbf{A}^\top) = 2\mathbf{X}\mathbf{A}$$

- **Trace-Produkt:** $f(\mathbf{X}) = \text{tr}(\mathbf{A}^\top \mathbf{X})$ mit $\mathbf{A} \in \mathbb{R}^{m \times n}$

$$\frac{\partial f(\mathbf{X})}{\partial \mathbf{X}} = \mathbf{A}$$

- **Frobenius-Norm:** $f(\mathbf{X}) = \|\mathbf{X}\|_F^2 = \text{tr}(\mathbf{X}^\top \mathbf{X})$

$$\frac{\partial f(\mathbf{X})}{\partial \mathbf{X}} = 2\mathbf{X}$$

(Anwendungsbeispiel Matrixzerlegung / Matrix Completion: → *Anhang*)

Bemerkungen

- Wir haben bewusst ein paar Fälle ausgelassen:
 - Matrix zu Vektor
 - Vektor zu Matrix
 - Matrix zu Matrix
- Diese Fälle werden seltener gebraucht und können (z.B.) über *Tensoren* dargestellt werden.
- Zur *Störungsanalyse* von Matrixrechnungen kann man auch $h \mapsto f(\mathbf{X} + h\mathbf{E})$ nach Skalar h ableiten (= Ableitung in “Richtung” $\mathbf{E}$.)
- Formeln für die meisten in Anwendungen vorkommenden Ableitungen findet man in dem *Matrix Cookbook* von Petersen & Pedersen (2012).
- Abbildungen zwischen Funktionenräumen → Übung.

Self-Check & Zusammenfassung

Wrap-up

Heute gelernt

- Fréchet-Ableitung auf normierten Räumen: **Ableitung = lineare Approximation**
- Spezialfälle für Skalare, Vektoren, Matrizen: *Gradient* (Zeilenvektor!), *Jacobimatrix*, Matrixableitungen

Nächste Vorlesung

- Eigenschaften, Rechenregeln, höhere Ableitungen

Self-Check

Frage 1: Sei $f(\mathbf{x}) = \mathbf{a}^\top \mathbf{x}$ mit $\mathbf{a}, \mathbf{x} \in \mathbb{R}^n$. Was ist $\nabla f(\mathbf{x})$?

1. $\mathbf{a}^\top$
2. $\mathbf{a}\mathbf{a}^\top$
3. $\mathbf{x}^\top$
4. $\mathbf{a}^\top \mathbf{x}$

Frage 2: Sei $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{A} \mathbf{x}$ mit *symmetrischem* $\mathbf{A} \in \mathbb{R}^{n \times n}$. Was ist $\nabla f(\mathbf{x})$?

1. $\mathbf{A}\mathbf{x}$
2. $2\mathbf{x}^\top \mathbf{A}$
3. $2\mathbf{A}$
4. $\mathbf{x}^\top \mathbf{A}\mathbf{x}$

Frage 3: Sei $f: \mathbb{R}^3 \rightarrow \mathbb{R}^2$ differenzierbar. Welche Dimension hat die Jacobimatrix $\mathbf{J}_f(\mathbf{x})$?

1. 3×2
2. 2×3
3. 3×3
4. 2×2

Anhang

Quiz: Beweis Aussage (3) $\Longleftrightarrow$ (4)

“ $\implies$ ”: Angenommen (3) gilt, also $\lim_{h \rightarrow 0} \frac{|f(x+h) - f(x) - f'(x)h|}{h} = 0$.

- Definiere $r(h) := f(x+h) - f(x) - f'(x)h$.
- Also: $\lim_{h \rightarrow 0} \frac{|r(h)|}{h} = 0$, also $r(h) = o(h)$.
- Und damit: $f(x+h) = f(x) + f'(x)h + r(h) = f(x) + f'(x)h + o(h)$. ✓

“ $\Longleftarrow$ ”: Angenommen (4) gilt, also $f(x+h) = f(x) + f'(x)h + o(h)$.

- Dann ist $f(x+h) - f(x) - f'(x)h = o(h)$.
- Per Definition von $o(h)$: $\lim_{h \rightarrow 0} \frac{|f(x+h) - f(x) - f'(x)h|}{h} = \lim_{h \rightarrow 0} \frac{|o(h)|}{h} = 0$. ✓

↔ zurück: „Quiz“

Beispiel: Ableitung der Determinante

Vertiefung zu “Skalar zu Matrix: Wichtige Identitäten” — **wichtig** für Ableitungen von Likelihood-Funktionen in der Statistik!

$$\frac{\partial \det(\mathbf{F}(x))}{\partial x} = \det(\mathbf{F}(x)) \operatorname{tr} \left[\mathbf{F}(x)^{-1} \frac{\partial \mathbf{F}(x)}{\partial x} \right]$$

Beispiel: $\mathbf{F}(x) = \begin{pmatrix} x & 0 \\ 0 & 2x \end{pmatrix}$.

Direkte Berechnung:

$$\det(\mathbf{F}(x)) = x \cdot 2x = 2x^2 \quad \implies \quad \frac{\partial}{\partial x} \det(\mathbf{F}(x)) = 4x$$

Überprüfung (für $x \neq 0$, sonst existiert $\mathbf{F}(x)^{-1}$ nicht):

$$\begin{aligned} \frac{\partial \mathbf{F}(x)}{\partial x} &= \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}, \quad \mathbf{F}(x)^{-1} = \begin{pmatrix} 1/x & 0 \\ 0 & 1/(2x) \end{pmatrix} \\ \mathbf{F}(x)^{-1} \frac{\partial \mathbf{F}(x)}{\partial x} &= \begin{pmatrix} 1/x & 0 \\ 0 & 1/(2x) \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix} = \begin{pmatrix} 1/x & 0 \\ 0 & 1/x \end{pmatrix} \\ \operatorname{tr} \left[\mathbf{F}(x)^{-1} \frac{\partial \mathbf{F}(x)}{\partial x} \right] &= 1/x + 1/x = 2/x \\ \det(\mathbf{F}(x)) \cdot \operatorname{tr} \left[\mathbf{F}(x)^{-1} \frac{\partial \mathbf{F}(x)}{\partial x} \right] &= 2x^2 \cdot \frac{2}{x} = 4x \quad \checkmark \end{aligned}$$

↩ zurück: „Skalar zu Matrix: Wichtige Identitäten“

Anwendung: Matrixzerlegung & Matrix Completion

Vertiefung zu “Wichtige Identitäten: Matrix zu Skalar”.

Gegeben: Unvollständig nur an Indizes $(i, j) \in \Omega$ beobachtete Matrix $\mathbf{Y} \in \mathbb{R}^{m \times n}$ (z.B. User-Ratings von Filmen)

Ziel: Finde Low-Rank-Approximation $\mathbf{Y} \approx \mathbf{UV}^\top$ mit $\mathbf{U} \in \mathbb{R}^{m \times k}$, $\mathbf{V} \in \mathbb{R}^{n \times k}$.

Verlustfunktion:

$$L(\mathbf{U}, \mathbf{V}) = \frac{1}{2} \sum_{(i,j) \in \Omega} \left(y_{ij} - [\mathbf{UV}^\top]_{ij} \right)^2 = \frac{1}{2} \|\mathbf{P}_\Omega \odot (\mathbf{Y} - \mathbf{UV}^\top)\|_F^2; \quad \mathbf{P}_\Omega = [\mathbb{I}_{(i,j) \in \Omega}]$$

($\odot$: elementweises (Hadamard-)Produkt)

Gradienten für Optimierung:

$$\begin{aligned} \frac{\partial L}{\partial \mathbf{U}} &= -(\mathbf{P}_\Omega \odot (\mathbf{Y} - \mathbf{UV}^\top))\mathbf{V} \\ \frac{\partial L}{\partial \mathbf{V}} &= -(\mathbf{P}_\Omega \odot (\mathbf{Y} - \mathbf{UV}^\top))^\top \mathbf{U} \end{aligned}$$

Anwendungen: Missing Data Imputation, Recommender Systems / Collaborative Filtering, ...

↩ zurück: „Wichtige Identitäten: Matrix zu Skalar”

Differentialrechnung II

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)

Material: Thomas Nagler

LMU München

Verwendete Vorkenntnisse

- **Fréchet-Differenzierbarkeit:** $f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + D_{\mathbf{x}}f(\mathbf{h}) + o(\|\mathbf{h}\|)$
- **Gradient** $\nabla f(\mathbf{x})$ für $f : \mathbb{R}^n \rightarrow \mathbb{R}$
- **Jacobimatrix** $\mathbf{J}_f(\mathbf{x})$ für $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$
- Matrixableitungen: $\partial f / \partial \mathbf{X}$
- **Analysis:**
 - Produktregel: $(fg)' = f'g + fg'$ & Kettenregel: $(g \circ f)' = g'(f) \cdot f'$
 - Höhere Ableitungen: $f'', f''', \dots$
 - Taylorreihen: $f(x + h) = f(x) + f'(x)h + (1/2)f''(x)h^2 + \dots$
- **Lineare Algebra:**
 - **Multilineare** & **Bilineare** Abbildungen: $\langle \cdot, \cdot \rangle$
 - Quadratische Formen: $\mathbf{x}^\top \mathbf{A} \mathbf{x}$
 - Komposition von Funktionen

Opener-Quiz: Was wissen Sie noch?

Frage 1: Sei $f: \mathbb{R} \rightarrow \mathbb{R}$ differenzierbar in x mit Ableitung $f'(x)$. Welche Aussagen sind wahr?

1. f' ist eine lineare Funktion.
2. $h \mapsto f'(x)h$ ist eine lineare Funktion (für festes x).
3. $f(x+h) = f(x) + f'(x)h + o(h)$ für $h \rightarrow 0$.
4. “ $o(h)$ ” steht für eine Funktion $r(h)$ mit $\lim_{h \rightarrow 0} \frac{r(h)}{h} = 0$.

Frage 2: Sei $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ differenzierbar. Welche Dimension hat die Jacobimatrix $\mathbf{J}_f(\mathbf{x})$?

1. $n \times m$
2. $m \times n$
3. $n \times n$
4. $m \times m$

Eigenschaften und Rechenregeln

Wichtige Eigenschaften: Stetigkeit

Theorem: Stetigkeit aus Differenzierbarkeit

Seien $\mathbb{D}$ und $\mathbb{E}$ normierte Räume und $f: \mathbb{D} \rightarrow \mathbb{E}$ in $\boldsymbol{x} \in \mathbb{D}$ differenzierbar.
Dann ist f in $\boldsymbol{x}$ stetig.

Beweis.

- Per Definition ist $D_{\boldsymbol{x}}f$ beschränkt und linear. Insbesondere gilt

$$\|D_{\boldsymbol{x}}f(\boldsymbol{h})\| \leq M \|\boldsymbol{h}\|, \quad \text{d.h.} \quad D_{\boldsymbol{x}}f(\boldsymbol{h}) = O(\|\boldsymbol{h}\|).$$

- Also ist

$$f(\boldsymbol{x} + \boldsymbol{h}) = f(\boldsymbol{x}) + D_{\boldsymbol{x}}f(\boldsymbol{h}) + o(\|\boldsymbol{h}\|) = f(\boldsymbol{x}) + O(\|\boldsymbol{h}\|) \quad \text{für } \|\boldsymbol{h}\| \rightarrow 0.$$

- Für $\|\boldsymbol{h}\| \rightarrow 0$ erhalten wir $f(\boldsymbol{x} + \boldsymbol{h}) \rightarrow f(\boldsymbol{x})$. ■

Notation:

$r(\boldsymbol{h}) = O(\|\boldsymbol{h}\|)$ für $\|\boldsymbol{h}\| \rightarrow 0$ heißt:

$\|r(\boldsymbol{h})\| \leq C \|\boldsymbol{h}\|$ für alle hinreichend kleinen $\boldsymbol{h}$ –

im Unterschied zu $r(\boldsymbol{h}) = o(\|\boldsymbol{h}\|)$, wo $\|r(\boldsymbol{h})\| / \|\boldsymbol{h}\| \rightarrow 0$ gilt.

Bsp: Stetig aber nicht differenzierbar

Betrachte $f: \mathbb{R} \rightarrow \mathbb{R}$ mit $f(x) = |x|$.

- f stetig in $x = 0$: Für $h \rightarrow 0$ gilt $f(0 + h) = |h| \rightarrow 0 = f(0)$. ✓
- f nicht differenzierbar in $x = 0$:

$$\begin{aligned}\lim_{h \rightarrow 0^+} \frac{f(h) - f(0)}{h} &= \lim_{h \rightarrow 0^+} \frac{|h|}{h} = \lim_{h \rightarrow 0^+} \frac{h}{h} = 1, \\ \lim_{h \rightarrow 0^-} \frac{f(h) - f(0)}{h} &= \lim_{h \rightarrow 0^-} \frac{|h|}{h} = \lim_{h \rightarrow 0^-} \frac{-h}{h} = -1.\end{aligned}$$

Da die linksseitigen und rechtsseitigen Grenzwerte verschieden sind, existiert $f'(0)$ nicht. ✕

Fazit:

- Differenzierbarkeit $\implies$ Stetigkeit
- aber Stetigkeit $\not\implies$ Differenzierbarkeit!

Wichtige Eigenschaften: Linearität

Theorem: Linearität der Ableitungsoperation

Seien $\mathbb{D}$ und $\mathbb{E}$ normierte Räume und $f, g: \mathbb{D} \rightarrow \mathbb{E}$ in $\boldsymbol{x} \in \mathbb{D}$ differenzierbar. Dann ist für alle $c_1, c_2 \in \mathbb{R}$ auch $c_1 f + c_2 g$ in $\boldsymbol{x}$ differenzierbar, mit

$$D_{\boldsymbol{x}}(c_1 f + c_2 g) = c_1 D_{\boldsymbol{x}} f + c_2 D_{\boldsymbol{x}} g.$$

Beweis: $\rightarrow$ Übung

Wichtige Eigenschaften: Linearität

Beispiele

- Für $\mathbb{D} = \mathbb{R}, \mathbb{E} = \mathbb{R}$: $(c_1 f + c_2 g)' = c_1 f' + c_2 g'$
- Für $\mathbb{D} = \mathbb{R}^n, \mathbb{E} = \mathbb{R}$:

$$\nabla [c_1 f(\mathbf{x}) + c_2 g(\mathbf{x})] = c_1 \nabla f(\mathbf{x}) + c_2 \nabla g(\mathbf{x})$$

- Für $\mathbb{D} = \mathbb{R}^n, \mathbb{E} = \mathbb{R}^m$:

$$\mathbf{J}_{c_1 f + c_2 g}(\mathbf{x}) = c_1 \mathbf{J}_f(\mathbf{x}) + c_2 \mathbf{J}_g(\mathbf{x})$$

- Für $\mathbb{D} = \mathbb{R}, \mathbb{E} = \mathbb{R}^{m \times n}$:

$$\frac{\partial [c_1 \mathbf{F}(x) + c_2 \mathbf{G}(x)]}{\partial x} = c_1 \frac{\partial \mathbf{F}(x)}{\partial x} + c_2 \frac{\partial \mathbf{G}(x)}{\partial x}$$

- Für $\mathbb{D} = \mathbb{R}^{m \times n}, \mathbb{E} = \mathbb{R}$:

$$\frac{\partial [c_1 f(\mathbf{X}) + c_2 g(\mathbf{X})]}{\partial \mathbf{X}} = c_1 \frac{\partial f(\mathbf{X})}{\partial \mathbf{X}} + c_2 \frac{\partial g(\mathbf{X})}{\partial \mathbf{X}}$$

Bsp: Linearität der Ableitung

Gegeben: $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{x}$ und $g(\mathbf{x}) = \mathbf{a}^\top \mathbf{x}$ mit $\mathbf{a} = (1, 2)^\top$.

Berechne $\nabla h(\mathbf{x})$ für $h(\mathbf{x}) = 3f(\mathbf{x}) - 2g(\mathbf{x})$ an der Stelle $\mathbf{x} = (1, -1)^\top$.

(Achtung: h ist hier ausnahmsweise ein Funktionsname, nicht der Zuwachs h .)

Lösung:

1. Berechne Gradienten:

- Für $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{x} = x_1^2 + x_2^2$ gilt: $\nabla f(\mathbf{x}) = (2x_1, 2x_2)$
- Für $g(\mathbf{x}) = \mathbf{a}^\top \mathbf{x} = x_1 + 2x_2$ gilt: $\nabla g(\mathbf{x}) = \mathbf{a}^\top = (1, 2)$

2. Wende Linearität an:

$$\nabla h(\mathbf{x}) = 3\nabla f(\mathbf{x}) - 2\nabla g(\mathbf{x}) = 3(2x_1, 2x_2) - 2(1, 2) = (6x_1 - 2, 6x_2 - 4)$$

3. Setze $\mathbf{x} = (1, -1)^\top$ ein:

$$\nabla h \left(\begin{pmatrix} 1 \\ -1 \end{pmatrix} \right) = (6 \cdot 1 - 2, 6 \cdot (-1) - 4) = (4, -10)$$

Rechenregeln: Produktregel

Theorem: Produktregel

Seien $\mathbb{D}, \mathbb{E}, \mathbb{B}$ normierte Räume, $f, g: \mathbb{D} \rightarrow \mathbb{E}$ in $\mathbf{x} \in \mathbb{D}$ differenzierbar, $\langle \cdot, \cdot \rangle: \mathbb{E} \times \mathbb{E} \rightarrow \mathbb{B}$ eine beschränkte bilineare Abbildung und $b(\mathbf{x}) = \langle f(\mathbf{x}), g(\mathbf{x}) \rangle$.

Dann gilt

$$D_{\mathbf{x}}b(\mathbf{h}) = \langle D_{\mathbf{x}}f(\mathbf{h}), g(\mathbf{x}) \rangle + \langle f(\mathbf{x}), D_{\mathbf{x}}g(\mathbf{h}) \rangle.$$

Beweis (kompakt):

Differenzierbarkeit von f, g einsetzen, Bilinearität ausmultiplizieren:

$$\begin{aligned} b(\mathbf{x} + \mathbf{h}) &= \langle f(\mathbf{x}) + D_{\mathbf{x}}f(\mathbf{h}) + o(\|\mathbf{h}\|), g(\mathbf{x}) + D_{\mathbf{x}}g(\mathbf{h}) + o(\|\mathbf{h}\|) \rangle \\ &= b(\mathbf{x}) + \underbrace{\langle D_{\mathbf{x}}f(\mathbf{h}), g(\mathbf{x}) \rangle + \langle f(\mathbf{x}), D_{\mathbf{x}}g(\mathbf{h}) \rangle}_{= D_{\mathbf{x}}b(\mathbf{h}) \text{ (linear in } \mathbf{h})} + \underbrace{\langle D_{\mathbf{x}}f(\mathbf{h}), D_{\mathbf{x}}g(\mathbf{h}) \rangle + o(\|\mathbf{h}\|)}_{= O(\|\mathbf{h}\|^2) + o(\|\mathbf{h}\|) = o(\|\mathbf{h}\|)} \end{aligned} \quad \blacksquare$$

Kreuzterm: $D_{\mathbf{x}}f(\mathbf{h}), D_{\mathbf{x}}g(\mathbf{h})$ sind beide $O(\|\mathbf{h}\|)$ (beschränkte lineare Abbildungen), die bilineare Abbildung multipliziert die Größenordnungen $\rightarrow O(\|\mathbf{h}\|^2)$.

Rechenregeln: Produktregel

Beispiele:

- $\mathbb{D} = \mathbb{R}, \mathbb{E} = \mathbb{R}, \mathbb{B} = \mathbb{R}$:

$$b(x) = f(x)g(x) \implies b'(x) = f'(x)g(x) + g'(x)f(x).$$

- $\mathbb{D} = \mathbb{R}^n, \mathbb{E} = \mathbb{R}, \mathbb{B} = \mathbb{R}$:

$$b(\mathbf{x}) = f(\mathbf{x})g(\mathbf{x}) \implies \nabla b(\mathbf{x}) = g(\mathbf{x})\nabla f(\mathbf{x}) + f(\mathbf{x})\nabla g(\mathbf{x}).$$

- $\mathbb{D} = \mathbb{R}^n, \mathbb{E} = \mathbb{R}^m, \mathbb{B} = \mathbb{R}$:

$$b(\mathbf{x}) = f(\mathbf{x})^\top g(\mathbf{x}) \implies \nabla b(\mathbf{x}) = g(\mathbf{x})^\top \mathbf{J}_f(\mathbf{x}) + f(\mathbf{x})^\top \mathbf{J}_g(\mathbf{x}).$$

- $\mathbb{D} = \mathbb{R}, \mathbb{E} = \mathbb{R}^{n \times n}, \mathbb{B} = \mathbb{R}$:

$$b(x) = \text{tr} [\mathbf{F}(x)\mathbf{G}(x)] \implies b'(x) = \text{tr} \left[\frac{\partial \mathbf{F}(x)}{\partial x} \mathbf{G}(x) \right] + \text{tr} \left[\mathbf{F}(x) \frac{\partial \mathbf{G}(x)}{\partial x} \right].$$

Übung: Ableitung quadratischer Formen

Sei $b(\boldsymbol{x}) = f(\boldsymbol{x})^\top g(\boldsymbol{x}) = \boldsymbol{x}^\top \boldsymbol{A} \boldsymbol{x}$ mit $f(\boldsymbol{x}) = \boldsymbol{x}$ und $g(\boldsymbol{x}) = \boldsymbol{A} \boldsymbol{x}$.

Aufgabe:

Berechne $\nabla b(\boldsymbol{x})$.

Lösung:

- Es gilt $\boldsymbol{J}_f(\boldsymbol{x}) = \boldsymbol{I}_n$ und $\boldsymbol{J}_g(\boldsymbol{x}) = \boldsymbol{A}$.
- Dann ist nach der Produktregel

$$\begin{aligned}\nabla b(\boldsymbol{x}) &= g(\boldsymbol{x})^\top \boldsymbol{J}_f(\boldsymbol{x}) + f(\boldsymbol{x})^\top \boldsymbol{J}_g(\boldsymbol{x}) \\ &= (\boldsymbol{A} \boldsymbol{x})^\top \boldsymbol{I}_n + \boldsymbol{x}^\top \boldsymbol{A} = \boldsymbol{x}^\top (\boldsymbol{A} + \boldsymbol{A}^\top)\end{aligned}$$

- falls $\boldsymbol{A}$ symmetrisch also: $\nabla b(\boldsymbol{x}) = 2\boldsymbol{x}^\top \boldsymbol{A}$

Rechenregeln: Kettenregel

Theorem: Kettenregel

Seien $\mathbb{D}_1, \mathbb{D}_2, \mathbb{D}_3$ normierte Räume und $f: \mathbb{D}_1 \rightarrow \mathbb{D}_2, g: \mathbb{D}_2 \rightarrow \mathbb{D}_3$, sodass $g \circ f: \mathbb{D}_1 \rightarrow \mathbb{D}_3$.
Ist f in $\boldsymbol{x} \in \mathbb{D}_1$ und g in $\boldsymbol{y} = f(\boldsymbol{x}) \in \mathbb{D}_2$ differenzierbar, dann ist

$$D_{\boldsymbol{x}}(g \circ f) = D_{\boldsymbol{y}}g \circ D_{\boldsymbol{x}}f.$$

Kettenregel: Beweis

- Da f differenzierbar ist, gilt

$$f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + D_{\mathbf{x}}f(\mathbf{h}) + o(\|\mathbf{h}\|).$$

- Daraus folgt

$$\begin{aligned}(g \circ f)(\mathbf{x} + \mathbf{h}) &= g(f(\mathbf{x}) + D_{\mathbf{x}}f(\mathbf{h}) + o(\|\mathbf{h}\|)) \\[g \text{ diff'bar:}] &= g(f(\mathbf{x})) + D_{\mathbf{y}}g(D_{\mathbf{x}}f(\mathbf{h}) + o(\|\mathbf{h}\|)) + o(\|\mathbf{h}\|) \\[D_{\mathbf{y}}g \text{ linear:}] &= g(f(\mathbf{x})) + D_{\mathbf{y}}g(D_{\mathbf{x}}f(\mathbf{h})) + D_{\mathbf{y}}g(o(\|\mathbf{h}\|)) + o(\|\mathbf{h}\|) \\[D_{\mathbf{y}}g \text{ beschr.:}] &= g(f(\mathbf{x})) + D_{\mathbf{y}}g(D_{\mathbf{x}}f(\mathbf{h})) + o(\|\mathbf{h}\|) \\ &= (g \circ f)(\mathbf{x}) + (D_{\mathbf{y}}g \circ D_{\mathbf{x}}f)(\mathbf{h}) + o(\|\mathbf{h}\|)\end{aligned}$$

Zeile 2:

Der Restterm von g ist zunächst nur klein im weitergereichten Zuwachs $\mathbf{k} = D_{\mathbf{x}}f(\mathbf{h}) + o(\|\mathbf{h}\|)$, also $o(\|\mathbf{k}\|)$.

Da $D_{\mathbf{x}}f$ beschränkt ist ($\|D_{\mathbf{x}}f(\mathbf{h})\| \leq M_f \|\mathbf{h}\|$), gilt für kleine $\mathbf{h}$ aber $\|\mathbf{k}\| \leq (M_f + 1) \|\mathbf{h}\|$ –

damit ist $o(\|\mathbf{k}\|)$ auch $o(\|\mathbf{h}\|)$.

Rechenregeln: Kettenregel

Beispiele:

Anwendung: Kleinste Quadrate via Matrixkalkül

Problem (Kapitel 7 — Lineare Regression):

Response $\mathbf{y} \in \mathbb{R}^n$, Designmatrix $\mathbf{X} \in \mathbb{R}^{n \times p}$:

$$\ell(\boldsymbol{\beta}) = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 \rightarrow \min_{\boldsymbol{\beta}}$$

Schritt 1 — als Skalarprodukt schreiben:

$$\ell(\boldsymbol{\beta}) = (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) = \langle f(\boldsymbol{\beta}), f(\boldsymbol{\beta}) \rangle \quad \text{mit } f(\boldsymbol{\beta}) = \mathbf{y} - \mathbf{X}\boldsymbol{\beta}$$

Schritt 2 — Jacobimatrix des inneren Terms (Linearität):

$$\mathbf{J}_f(\boldsymbol{\beta}) = -\mathbf{X}$$

Schritt 3 — Produktregel (mit $g = f$, beide Summanden gleich):

$$\begin{aligned} \nabla \ell(\boldsymbol{\beta}) &= f(\boldsymbol{\beta})^\top \mathbf{J}_f(\boldsymbol{\beta}) + f(\boldsymbol{\beta})^\top \mathbf{J}_f(\boldsymbol{\beta}) \\ &= 2 (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top (-\mathbf{X}) = -2 (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top \mathbf{X} \end{aligned}$$

Anwendung: Kleinste Quadrate — Normalengleichungen

Schritt 4 — Gradient nullsetzen:

Am Minimum muss der Gradient verschwinden. (*Optimalitätsbedingung* → *Optimierung I*):

$$\nabla \ell(\hat{\beta}) = -2 (\mathbf{y} - \mathbf{X}\hat{\beta})^\top \mathbf{X} \stackrel{!}{=} \mathbf{0}^\top$$

Transponieren und umstellen:

$$\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\hat{\beta}) = \mathbf{0} \quad \Longrightarrow \quad \boxed{\mathbf{X}^\top \mathbf{X} \hat{\beta} = \mathbf{X}^\top \mathbf{y}}$$

→ die *Normalengleichungen*: **die KQ-Lösung aus Kapitel 7 — jetzt hergeleitet!**

Payoff:

- Produkt- & Kettenregel im Matrixkalkül liefern die KQ-Lösung in 4 Zeilen — ohne eine einzige partielle Ableitung auszuschreiben.
- “ $\nabla \ell = \mathbf{0}^\top$ ” als Optimalitätsbedingung wird systematisch ausgebaut: → *Optimierung I & II*.
- Gleiche Rechnung mit Strafterm $\lambda \|\beta\|_2^2$ ergibt die Ridge-Regression (→ Übung).

Anwendung: Backprop in Neuronalen Netzen

- NN ist eine Funktion $f(\mathbf{x}) = (f_K \circ \dots \circ f_2 \circ f_1)(\mathbf{x}) = \hat{\mathbf{y}}$ aufgebaut aus verketteten Funktionen:

$$\text{Input } \mathbf{x} \xrightarrow{f_1} \mathbf{z}_1 \xrightarrow{f_2} \mathbf{z}_2 \xrightarrow{\dots} \dots \xrightarrow{f_K} \text{Output } \hat{\mathbf{y}}$$

- Jedes layer f_k
 - hat eine Gewichtsmatrix $\mathbf{W}_k \in \mathbb{R}^{d_k \times d_{k-1}}$ und
 - berechnet $f_k(\mathbf{z}_{k-1}) = \sigma_k(\mathbf{W}_k \mathbf{z}_{k-1})$,
 - wobei $\sigma_k : \mathbb{R}^{d_k} \rightarrow \mathbb{R}^{d_k}$ komponentenweise wirkt.
- Verlustfunktion $L(\hat{\mathbf{y}})$ misst Güte der Outputs.
- Wir brauchen $\frac{\partial L}{\partial \mathbf{W}_k}$ zur Optimierung.

Kettenregel mit Jacobimatrizen:

$$\frac{\partial L}{\partial \mathbf{W}_k} = \frac{\partial L}{\partial \hat{\mathbf{y}}} \cdot \mathbf{J}_{f_K}(\mathbf{z}_{K-1}) \cdot \mathbf{J}_{f_{K-1}}(\mathbf{z}_{K-2}) \cdots \mathbf{J}_{f_{k+1}}(\mathbf{z}_k) \cdot \frac{\partial f_k(\mathbf{z}_{k-1})}{\partial \mathbf{W}_k}$$

Backpropagation = effiziente Berechnung dieser Jacobimatrizen-Kette!

Beispiel:

Mit ReLU-Aktivierung $\sigma(x) = \max(0, x)$ ist $f_k(\mathbf{z}_{k-1}) = \max(\mathbf{0}, \mathbf{W}_k \mathbf{z}_{k-1})$, also:

$$\mathbf{J}_{f_k}(\mathbf{z}_{k-1}) = \text{diag}(\mathbb{I}_{\mathbf{W}_k \mathbf{z}_{k-1} > 0}) \cdot \mathbf{W}_k$$

(nur für $(\mathbf{W}_k \mathbf{z}_{k-1})_i \neq 0$: an den Knickstellen ist $\max(0, \cdot)$ nicht differenzierbar)

Übung

Sei $h(\mathbf{x}) = \|\mathbf{x}\|_2 = \sqrt{\mathbf{x}^\top \mathbf{x}}$.

Frage:

Was ist $\nabla h(\mathbf{x})$ für $\mathbf{x} \neq \mathbf{0}$?

Antwort:

- Für $g(x) = \sqrt{x} = x^{1/2}$ gilt $g'(x) = \frac{1}{2x^{1/2}}$.
- Für $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{x}$ gilt $\nabla f(\mathbf{x}) = 2\mathbf{x}^\top$.
- Nach der Kettenregel ist

$$\nabla h(\mathbf{x}) = g'(f(\mathbf{x})) \nabla f(\mathbf{x}) = \frac{2\mathbf{x}^\top}{2f(\mathbf{x})^{1/2}} = \frac{\mathbf{x}^\top}{\|\mathbf{x}\|}.$$

- (In $\mathbf{x} = \mathbf{0}$ ist $\|\cdot\|_2$ nicht differenzierbar.)

Ableitungen höheren Grades

Ableitungen höheren Grades

Def.: k -mal (Fréchet-)differenzierbar

Seien $\mathbb{D}$ und $\mathbb{E}$ zwei normierte Räume und $f: \mathbb{D} \rightarrow \mathbb{E}$. Definiere $D_x^0 f := f(x)$.

Wir nennen f an der Stelle $x \in \mathbb{D}$ **k -mal (Fréchet-)differenzierbar**, falls für $j = 1, \dots, k$ beschränkte, multilineare Abbildungen $D_x^j f: \mathbb{D}^j \rightarrow \mathbb{E}$ existieren, sodass für alle $h_1, \dots, h_{j-1}$ und für $h \rightarrow 0$ gilt:

$$D_x^j f(h_1, \dots, h_{j-1}, h) = D_{x+h}^{j-1} f(h_1, \dots, h_{j-1}) - D_x^{j-1} f(h_1, \dots, h_{j-1}) + o(\|h\|) \prod_{i=1}^{j-1} \|h_i\|$$

- Höhere Ableitungen sind also wieder (multi-)lineare Annäherungen an die (lokale) Veränderung der nächstniedrigeren Ableitung von $f \implies$ Ableitungen berechnen wir wie gewohnt iterativ
- Höhere Ableitungen involvieren i.a. **Tensoren**, deren Stufe wächst mit j .

Konkret: Hesse-Matrix ($k = 2$)

Sei $f: \mathbb{R}^n \rightarrow \mathbb{R}$ (z.B. eine Log-Likelihood) und $f \in \mathcal{C}^2$.

- $j = 1$:
 $D_x f(\mathbf{h}_1) = \nabla f(\mathbf{x}) \mathbf{h}_1$ – lineare Abbildung, dargestellt durch den **Gradienten** (Zeilenvektor).

- $j = 2$:
Die Definition fragt: wie verändert sich Df selbst, wenn wir $\mathbf{x}$ um $\mathbf{h}$ verschieben?

$$D_{\mathbf{x}+\mathbf{h}} f(\mathbf{h}_1) - D_{\mathbf{x}} f(\mathbf{h}_1) = [\nabla f(\mathbf{x} + \mathbf{h}) - \nabla f(\mathbf{x})] \mathbf{h}_1$$

- Lineare Approximation des Gradienten (jede Komponente von ∇f ist selbst differenzierbar):

$$\nabla f(\mathbf{x} + \mathbf{h}) - \nabla f(\mathbf{x}) = \mathbf{h}^\top \mathbf{H}_f(\mathbf{x}) + o(\|\mathbf{h}\|)$$

- Einsetzen ergibt die *bilineare* Abbildung

$$D_{\mathbf{x}}^2 f(\mathbf{h}_1, \mathbf{h}) = \mathbf{h}^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h}_1$$

dargestellt durch die Matrix $\mathbf{H}_f(\mathbf{x})$ der zweiten partiellen Ableitungen: die **Hesse-Matrix**

(Symmetrie von $\mathbf{H}_f$ für $f \in \mathcal{C}^2$: $\rightarrow$ Satz von Schwarz, gleich.)

Höhergradige Ableitung für Vektor-zu-Skalar

Def.: Hesse-Matrix

Die **Hesse-Matrix** $\mathbf{H}_f(\mathbf{x})$ einer zweimal differenzierbaren Funktion $f: S \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$ (S offen) ist die Matrix der zweiten partiellen Ableitungen, also

$$\mathbf{H}_f(\mathbf{x}) := \left(\frac{\partial^2 f(\mathbf{x})}{\partial x_i \partial x_j} \right)_{i,j=1,\dots,n} \in \mathbb{R}^{n \times n}.$$

Hier ist:

- $D_{\mathbf{x}} f(\mathbf{h}) = \nabla f(\mathbf{x}) \mathbf{h}$
- $D_{\mathbf{x}}^2 f(\mathbf{h}_1, \mathbf{h}_2) = \mathbf{h}_1^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h}_2$ mit Hesse-Matrix $\mathbf{H}_f(\mathbf{x}) \in \mathbb{R}^{n \times n}$.
- $(D_{\mathbf{x}}^3 f \in \mathbb{R}^{n \times n \times n})$ wäre ein Tensor mit $(D_{\mathbf{x}}^3 f)_{i,j,k} = \frac{\partial^3 f(\mathbf{x})}{\partial x_k \partial x_j \partial x_i}$ und $D_{\mathbf{x}}^3 f(\mathbf{h}_1, \mathbf{h}_2, \mathbf{h}_3) = \sum_{i,j,k} (D_{\mathbf{x}}^3 f)_{i,j,k} h_{1i} h_{2j} h_{3k}$

Symmetrische Hesse-Matrizen

Theorem: Satz von Schwarz

Sei $f: \mathbb{R}^n \rightarrow \mathbb{R}$ zweimal stetig differenzierbar ($f \in \mathcal{C}^2$). Dann gilt:

$$\frac{\partial^2 f(\mathbf{x})}{\partial x_i \partial x_j} = \frac{\partial^2 f(\mathbf{x})}{\partial x_j \partial x_i} \quad \text{für alle } i, j.$$

$\implies$ Hesse-Matrix für $f \in \mathcal{C}^2$ ist symmetrisch: $\mathbf{H}_f(\mathbf{x}) = \mathbf{H}_f(\mathbf{x})^\top$.

- Reihenfolge der partiellen Ableitungen spielt unter Stetigkeitsbedingungen keine Rolle
- Reduziert Anzahl der zu berechnenden Ableitungen von n^2 auf $\frac{n(n+1)}{2}$
- Wichtig für numerische Verfahren und Optimierung: nutze Symmetrie der Hesse-Matrix

Geometrische Interpretation und Konvexität

Hesse-Matrix charakterisiert die *lokale Krümmung* einer Funktion:

1. Definitheit der Hesse-Matrix:

Für $f \in \mathcal{C}^2$, $f: \mathbb{R}^n \rightarrow \mathbb{R}$, an einem **kritischen Punkt** $\mathbf{x}^*$ (d.h. $\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$):

- $\mathbf{H}_f(\mathbf{x}^*)$ **positiv definit** ($\mathbf{h}^\top \mathbf{H}_f(\mathbf{x}^*) \mathbf{h} > 0$ für alle $\mathbf{h} \neq \mathbf{0}$) $\implies \mathbf{x}^*$ ist ein **lokales Minimum**
- $\mathbf{H}_f(\mathbf{x}^*)$ **negativ definit** ($\mathbf{h}^\top \mathbf{H}_f(\mathbf{x}^*) \mathbf{h} < 0$ für alle $\mathbf{h} \neq \mathbf{0}$) $\implies \mathbf{x}^*$ ist ein **lokales Maximum**
- $\mathbf{H}_f(\mathbf{x}^*)$ **indefinit** (positive und negative Eigenwerte) $\implies \mathbf{x}^*$ ist ein **Sattelpunkt**

2. Konvexität:

$f \in \mathcal{C}^2$ ist **konvex** auf einer *offenen, konvexen* Menge $S \subseteq \mathbb{R}^n \iff \mathbf{H}_f(\mathbf{x}) \succeq 0$ (*positiv semidefinit*) für alle $\mathbf{x} \in S$.

3. Quadratische Approximation:

$$f(\mathbf{x} + \mathbf{h}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x}) \mathbf{h} + \frac{1}{2} \mathbf{h}^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h}$$

- beste quadratische Approximation von f in der Umgebung von $\boldsymbol{x}$ ($\rightarrow$ Taylorentwicklung)
- $\boldsymbol{H}_f(\boldsymbol{x})$ beschreibt die lokale Krümmung von f in Richtung $\boldsymbol{h}$
- Eigenvektoren der Hesse-Matrix geben die Hauptkrümmungsrichtungen an, Eigenwerte die Stärke der Krümmung in diesen Richtungen

Praxisrelevanz Hesse-Matrix

- Heuristik: in typischen hochdimensionalen (nicht-konvexen) Problemen sind echte lokale Minima / Maxima **selten**, die meisten kritischen Punkte sind **Sattelpunkte!** $\implies$ SGD & co funktionieren
 $\implies$ *Singularity incoming* (maybe....)
- **Statistik:**
 - Wichtiger Spezialfall für $f: \mathbb{R}^p \rightarrow \mathbb{R}$: *(Log-)Likelihood-Funktion*
 - Hesse-Matrix der negativen Log-Likelihood ($\rightarrow$ beobachtete *Fisher-Information*) bestimmt Unsicherheit von Schätzern
- **ML:**
Hesse-Matrix der Verlustfunktion bestimmt Konditionierung von Optimierungsproblemen
- **Optimierung:**
Hesse-Matrix bestimmt Konvergenzverhalten von Optimierungsalgorithmen

Anwendung: Fisher-Information

Def.: Fisher-Informationsmatrix

Für ein statistisches Modell mit Log-Likelihood $\ell(\boldsymbol{\theta})$ und Parameter $\boldsymbol{\theta} \in \mathbb{R}^p$ ist die **Fisher-Informationsmatrix**, definiert als

$$\mathbf{I}(\boldsymbol{\theta}) := \mathbb{E}[-\mathbf{H}_{\ell}(\boldsymbol{\theta})] = -\mathbb{E}\left[\frac{\partial^2 \ell(\boldsymbol{\theta})}{\partial \theta_i \partial \theta_j}\right]_{i,j} \in \mathbb{R}^{p \times p}.$$

Intuition:

- Stark gekrümmte Log-Likelihood $\implies$ Daten sind “informativ” über optimales $\boldsymbol{\theta}$
- Flache Log-Likelihood $\implies$ wenig Information, Parameter schwer zu schätzen

Wichtige Konsequenz:

Asymptotische Effizienz des ML-Schätzers:

Für ML-Schätzer $\hat{\boldsymbol{\theta}}_{ML}$ gilt (unter Regularitätsbedingungen) für großes n approximativ

$$\text{Var}(\hat{\boldsymbol{\theta}}_{ML}) \approx \mathbf{I}(\boldsymbol{\theta})^{-1},$$

der MLE erreicht damit asymptotisch die *Cramér-Rao-Schranke*, die untere Schranke für die Varianz unverzerrter Schätzer.

$\implies$ mehr Information $\rightarrow$ kleinere Varianz $\rightarrow$ präzisere Schätzung

Höhergradige Ableitung für Vektor-zu-Vektor

Wichtiger Fall:

$$f: \mathbb{R}^n \rightarrow \mathbb{R}^m$$

- **Beispiel:** Score-Funktion $\nabla \ell(\boldsymbol{\theta})^\top$ (in der Zeilenkonvention ist $\nabla \ell$ eine Zeile, erst die Transponierte bildet $\mathbb{R}^p \rightarrow \mathbb{R}^p$ ab), Momenten-Gleichungen
- j -te Ableitung ist multilineare Abbildung $D_{\mathbf{x}}^j f: (\mathbb{R}^n)^j \rightarrow \mathbb{R}^m$: $\rightarrow$ Stufe $j + 1$ -Tensor mit Dimensionen $m \times \underbrace{n \times n \times \cdots \times n}_{j\text{-mal}}$

$$D_{\mathbf{x}}^j f(\mathbf{h}_1, \dots, \mathbf{h}_j) = \sum_{i_1, \dots, i_j=1}^n \underbrace{\frac{\partial^j f(\mathbf{x})}{\partial x_{i_j} \cdots \partial x_{i_1}}}_{\in \mathbb{R}^m} h_{1,i_1} \cdots h_{j,i_j}$$

- Insbesondere:

$$D_{\mathbf{x}} f(\mathbf{h}) = \mathbf{J}_f(\mathbf{x}) \mathbf{h}, \quad D_{\mathbf{x}}^2 f(\mathbf{h}_1, \mathbf{h}_2) = \sum_{i=1}^n \frac{\partial \mathbf{J}_f(\mathbf{x})}{\partial x_i} \mathbf{h}_1 \cdot h_{2,i}$$

(ausgeschriebene Summenformeln für $j = 1, 2, 3$: $\rightarrow$ *Anhang*)

Taylorapproximation

Taylorentwicklung: univariate Funktionen

Theorem: Taylorentwicklung I

Sei $f: \mathbb{R} \rightarrow \mathbb{R}$ $(k + 1)$ -mal stetig differenzierbar.

Dann gilt für $h \rightarrow 0$:

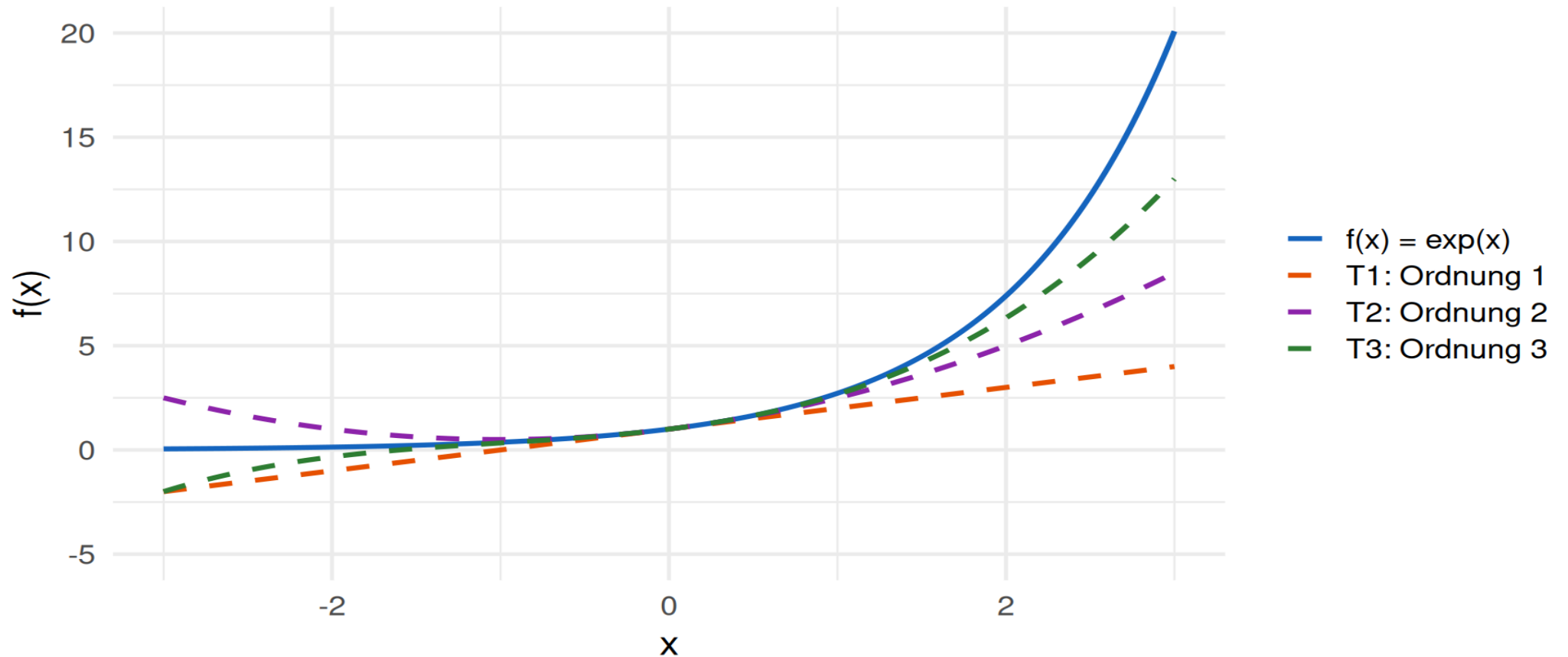
$$f(x + h) = f(x) + \sum_{j=1}^k \frac{1}{j!} f^{(j)}(x) h^j + o(|h|^k).$$

- **Taylorpolynom** $T_k(x + h) := \sum_{j=0}^k \frac{1}{j!} f^{(j)}(x) h^j$ (Entwicklungspunkt x)
- Dies ist nützlich um Funktionen lokal durch einfaches Polynom zu approximieren.
- Je höher der Grad k , desto besser die *lokale* Approximation für $h \rightarrow 0$ – *global* muss sie dadurch nicht besser werden!
- Essenzielles Tool in Konvergenzanalysen (z.B. asymptotische Normalität des Maximum-Likelihood-Schätzers).

Beweisskizze: $\rightarrow$ *Anhang*.

Taylorentwicklung: univariate Funktionen

$$f(x) \approx T_k(x) = f(0) + f'(0)x + \frac{1}{2}f''(0)x^2 + \frac{1}{6}f'''(0)x^3 + \dots$$



Taylor-Approximationen für $\exp(x)$ in $x = 0$

Taylorentwicklung: Allgemein

Theorem: Taylorentwicklung II

Seien V, W normierte Räume und $f: V \rightarrow W$ k -mal stetig Fréchet-differenzierbar ($f \in \mathcal{C}^k$).
Dann gilt für $\mathbf{h} \rightarrow \mathbf{0}$:

$$f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + \sum_{j=1}^k \frac{1}{j!} D_{\mathbf{x}}^j f(\underbrace{\mathbf{h}, \dots, \mathbf{h}}_{j\text{-mal}}) + o(\|\mathbf{h}\|^k).$$

- Gleiche Struktur wie univariat: Polynom in $\mathbf{h}$ mit multilinearen Koeffizienten $D_{\mathbf{x}}^j f$.

Beweisskizze (Reduktion auf den 1D-Fall): $\rightarrow$ Anhang.

Taylorapproximation für Vektor-zu-Skalar

- Es gilt:

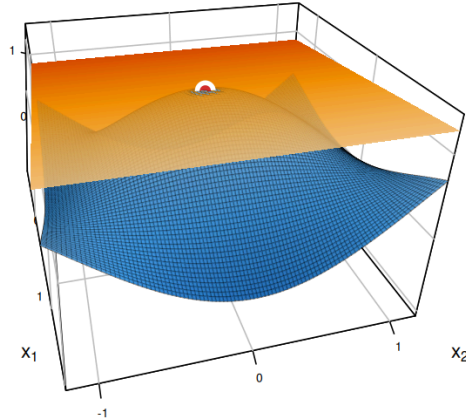
$$\begin{aligned}f(\mathbf{x} + \mathbf{h}) &= f(\mathbf{x}) + \nabla f(\mathbf{x})\mathbf{h} + o(\|\mathbf{h}\|) \\&= f(\mathbf{x}) + \nabla f(\mathbf{x})\mathbf{h} + \frac{1}{2}\mathbf{h}^\top \mathbf{H}_f(\mathbf{x})\mathbf{h} + o(\|\mathbf{h}\|^2) \\&= f(\mathbf{x}) + \nabla f(\mathbf{x})\mathbf{h} + \frac{1}{2}\mathbf{h}^\top \mathbf{H}_f(\mathbf{x})\mathbf{h} + \frac{1}{6} \sum_{i,j,l=1}^n \frac{\partial^3 f(\mathbf{x})}{\partial x_i \partial x_j \partial x_l} h_i h_j h_l + o(\|\mathbf{h}\|^3) \\&= \dots\end{aligned}$$

- Alternative Formulierung mit Entwicklungspunkt $\mathbf{x}_0$:

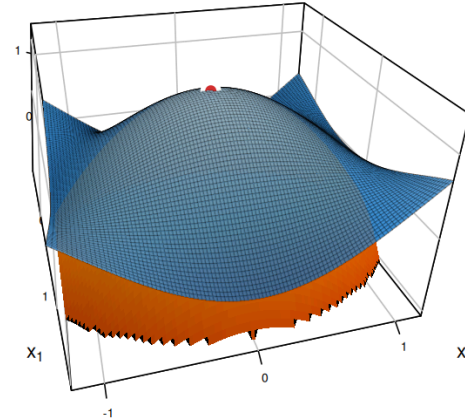
$$\begin{aligned}f(\mathbf{x}) &= f(\mathbf{x}_0) + \nabla f(\mathbf{x}_0)(\mathbf{x} - \mathbf{x}_0) + o(\|\mathbf{x} - \mathbf{x}_0\|) \\&= f(\mathbf{x}_0) + \nabla f(\mathbf{x}_0)(\mathbf{x} - \mathbf{x}_0) + \frac{1}{2}(\mathbf{x} - \mathbf{x}_0)^\top \mathbf{H}_f(\mathbf{x}_0)(\mathbf{x} - \mathbf{x}_0) + o(\|\mathbf{x} - \mathbf{x}_0\|^2) \\&= \dots\end{aligned}$$

Taylorapproximation für Vektor-zu-Skalar

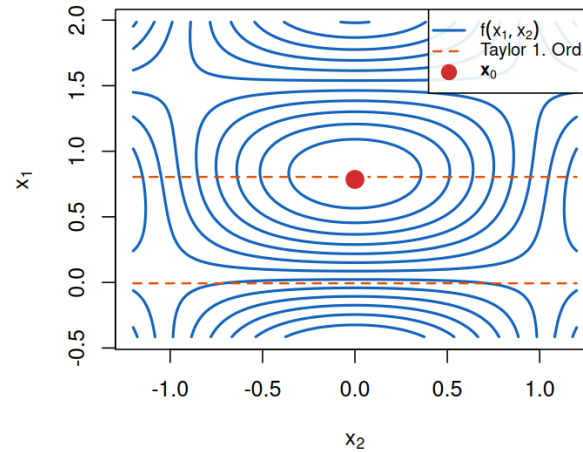
1. Ordnung: $f(\mathbf{x}_0) + \nabla f(\mathbf{x}_0)^T(\mathbf{x} - \mathbf{x}_0)$



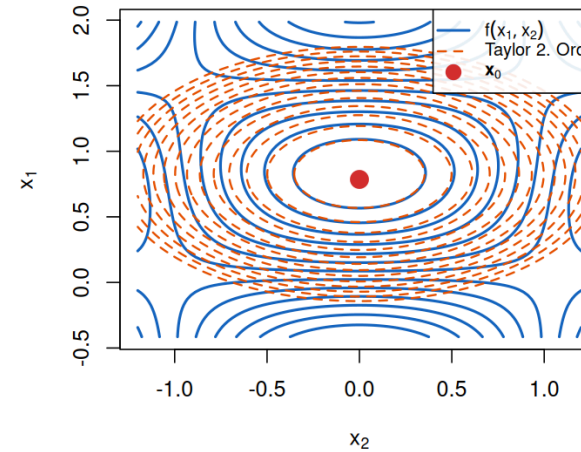
2. Ordnung: $\dots + \frac{1}{2}(\mathbf{x} - \mathbf{x}_0)^T \mathbf{H}_f(\mathbf{x} - \mathbf{x}_0)$



1. Ordnung: Höhenlinien



2. Ordnung: Höhenlinien



Selbst ausprobieren: [Shiny-App](#)

Anwendung: Newton-Raphson-Verfahren

Problemstellung:

Finde $\mathbf{x}^* = \arg \min_{\mathbf{x}} f(\mathbf{x})$ für zweimal stetig diff.bare $f: \mathbb{R}^n \rightarrow \mathbb{R}$

Idee:

Approximiere $f(\mathbf{x})$ um aktuellen Punkt $\mathbf{x}^{(k)}$ durch das Taylor-Polynom 2. Ordnung (Entwicklungspunkt $\mathbf{x}^{(k)}$):

$$f(\mathbf{x}) \approx T_2(\mathbf{x}) = f(\mathbf{x}^{(k)}) + \nabla f(\mathbf{x}^{(k)}) (\mathbf{x} - \mathbf{x}^{(k)}) + \frac{1}{2} (\mathbf{x} - \mathbf{x}^{(k)})^\top \mathbf{H}_f(\mathbf{x}^{(k)}) (\mathbf{x} - \mathbf{x}^{(k)})$$

Newton-Schritt:

Minimiere $T_2(\mathbf{x})$ bzgl. $\mathbf{x}$. (für $\mathbf{H}_f(\mathbf{x}^{(k)})$ positiv definit, sonst hat T_2 kein Minimum):

$$\begin{aligned} \nabla T_2(\mathbf{x}) &= \nabla f(\mathbf{x}^{(k)}) + (\mathbf{x} - \mathbf{x}^{(k)})^\top \mathbf{H}_f(\mathbf{x}^{(k)}) \stackrel{!}{=} \mathbf{0}^\top \\ \implies (\mathbf{x} - \mathbf{x}^{(k)})^\top &= -\nabla f(\mathbf{x}^{(k)}) \mathbf{H}_f(\mathbf{x}^{(k)})^{-1} \end{aligned}$$

Newton-Iteration:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \left(\nabla f(\mathbf{x}^{(k)}) \mathbf{H}_f(\mathbf{x}^{(k)})^{-1} \right)^\top$$

Enorme Relevanz in ML/Statistics:

Maximum-Likelihood-Schätzung; IRLS/IWLS für GLMs; Trust-Region-Methoden ($\rightarrow$ Optimierung II)

Self-Check & Zusammenfassung

Wrap-up

Heute gelernt

- Eigenschaften: Stetigkeit und Linearität der Ableitung
- Rechenregeln: *Produktregel* und *Kettenregel* → Matrixkalkül liefert z.B. die *Normalengleichungen* der KQ-Schätzung in 4 Zeilen
- Höhergradige Ableitungen: multilineare Abbildungen, *Hesse-Matrix* & Krümmung
- *Taylorapproximation* und Newton-Raphson-Verfahren

Self-Check

Frage 1: Sei $b(\mathbf{x}) = \mathbf{x}^\top \mathbf{A} \mathbf{x}$ mit symmetrischem $\mathbf{A} \in \mathbb{R}^{n \times n}$. Was ist $\nabla b(\mathbf{x})$?

1. $2\mathbf{x}^\top \mathbf{A}$
2. $\mathbf{x}^\top \mathbf{A}$
3. $2\mathbf{A}$
4. $2\mathbf{A}\mathbf{x}\mathbf{x}^\top$

Frage 2: Sei f $(k + 1)$ -mal stetig differenzierbar und T_k das Taylorpolynom vom Grad k um x . Welche Ordnung hat der Approximationsfehler $f(x + h) - T_k(x + h)$ für $h \rightarrow 0$?

1. $o(|h|^k)$
2. $o(|h|^{k+1})$
3. $O(1)$
4. $o(|h|)$, unabhängig von k

Frage 3: Ein Newton-Schritt zur Minimierung von $f: \mathbb{R}^n \rightarrow \mathbb{R}$ basiert auf ...

1. dem Taylorpolynom 1. Ordnung plus einer Schrittweite.
2. dem Taylorpolynom 2. Ordnung: er braucht Gradient **und** Hesse-Matrix.
3. nur der Hesse-Matrix.
4. dem Satz von Schwarz.

Nächste Vorlesung

- Konvexität

Anhang

Alternatives Beispiel (Kettenregel): Logistische Regression

Problemstellung:

Binäre Klassifikation mit $Y \in \{0, 1\}$ und Features $\mathbf{x} \in \mathbb{R}^p$: $P(Y = 1 \mid \mathbf{x}) = \sigma(\boldsymbol{\beta}^\top \mathbf{x})$ mit $\sigma(t) = \frac{1}{1+e^{-t}}$

Verlustfunktion für eine Beobachtung $(\mathbf{x}, y)$ ist:

$$\ell(\boldsymbol{\beta}) = - [y \log(\sigma(\boldsymbol{\beta}^\top \mathbf{x})) + (1 - y) \log(1 - \sigma(\boldsymbol{\beta}^\top \mathbf{x}))] = - \begin{cases} \log(P(Y = 1 \mid \mathbf{x})) & \text{für } y = 1 \\ \log(P(Y = 0 \mid \mathbf{x})) & \text{für } y = 0 \end{cases}$$

Ableitung für $y = 1$: $\ell(\boldsymbol{\beta})|_{y=1} = -\log \sigma(\boldsymbol{\beta}^\top \mathbf{x}) = g(f(\boldsymbol{\beta}))$

- $f(\boldsymbol{\beta}) = \boldsymbol{\beta}^\top \mathbf{x}$ mit $\nabla f(\boldsymbol{\beta}) = \mathbf{x}^\top$
- $g(t) = -\log \sigma(t)$ mit $g'(t) = -\frac{\sigma'(t)}{\sigma(t)}$
- $\sigma'(t) = \frac{e^{-t}}{(1+e^{-t})^2} = \sigma(t)(1 - \sigma(t))$, also $g'(t) = -\frac{\sigma(t)(1-\sigma(t))}{\sigma(t)} = -(1 - \sigma(t)) = \sigma(t) - 1$

Mit Kettenregel und $\hat{y} = \sigma(\boldsymbol{\beta}^\top \mathbf{x})$:

$$\nabla \ell(\boldsymbol{\beta}) = g'(f(\boldsymbol{\beta})) \nabla f(\boldsymbol{\beta}) = (\sigma(\boldsymbol{\beta}^\top \mathbf{x}) - 1) \mathbf{x}^\top = (\hat{y} - 1) \mathbf{x}^\top$$

- analog für $y = 0$:

$$\nabla \ell(\boldsymbol{\beta})|_{y=0} = \hat{y} \mathbf{x}^\top$$

Taylorentwicklung I: Beweisskizze (univariat)

Vertiefung: Beweisskizze.

Sei $h > 0$ (Fall $h < 0$ analog). Definiere auf $[x, x + h]$:

$$F(t) := \sum_{j=0}^k \frac{f^{(j)}(t)}{j!} (x + h - t)^j, \quad \text{also } F(x + h) = f(x + h), \quad F(x) = T_k(x + h),$$

$$G(t) := (x + h - t)^{k+1}, \quad \text{also } G(x + h) = 0, \quad G(x) = h^{k+1},$$

mit $F'(t) = \frac{f^{(k+1)}(t)}{k!} (x + h - t)^k$ (Teleskopsumme!) und $G'(t) = -(k + 1)(x + h - t)^k \neq 0$ auf $(x, x + h)$.

Cauchy-Mittelwertsatz: es gibt $\xi \in (x, x + h)$ mit

$$\frac{F(x + h) - F(x)}{G(x + h) - G(x)} = \frac{F'(\xi)}{G'(\xi)} \implies f(x + h) - T_k(x + h) = \frac{f^{(k+1)}(\xi)}{(k + 1)!} h^{k+1} = o(|h|^k). \blacksquare$$

Wieso $o(|h|^k)$? $f^{(k+1)}$ stetig $\implies$ lokal beschränkt durch ein M , also $|f^{(k+1)}(\xi) h^{k+1} / (k + 1)!| / |h|^k \leq \frac{M}{(k+1)!} |h| \xrightarrow{h \rightarrow 0} 0$.

↩ zurück: „Taylorentwicklung: univariate Funktionen“

Taylorentwicklung II: Beweisskizze (allgemein)

Vertiefung: Beweisskizze.

Fixiere $\boldsymbol{x} \in V$. Für $\boldsymbol{h} = \mathbf{0}$ ist die Aussage trivial; sei also $\boldsymbol{h} \neq \mathbf{0}$.

- Setze $s := \|\mathbf{h}\|$ und $\mathbf{u} := \mathbf{h}/s$ (also $\|\mathbf{u}\| = 1, \mathbf{h} = s\mathbf{u}$).
- Definiere $\psi: \mathbb{R}_{\geq 0} \rightarrow W$ durch $\psi(w) := f(\mathbf{x} + w\mathbf{u})$.
- Wiederholte Kettenregel liefert: $\psi^{(j)}(w) = D_{\mathbf{x}+w\mathbf{u}}^j \underbrace{f(\mathbf{u}, \dots, \mathbf{u})}_{j\text{-mal}}$.
- 1D-Taylor für ψ um $w = 0$ – in der *Peano-Form*, die nur $\psi \in \mathcal{C}^k$ braucht (Satz I setzt stärker $\mathcal{C}^{k+1}$ voraus):

$$\psi(s) = \sum_{j=0}^k \frac{s^j}{j!} \psi^{(j)}(0) + o(s^k) \quad \text{für } s \rightarrow 0.$$
- Mit $\psi(s) = f(\mathbf{x} + \mathbf{h})$, $\psi(0) = f(\mathbf{x})$ und $\psi^{(j)}(0) = D_{\mathbf{x}}^j f(\mathbf{u}, \dots, \mathbf{u})$:

$$f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + \sum_{j=1}^k \frac{s^j}{j!} D_{\mathbf{x}}^j f(\mathbf{u}, \dots, \mathbf{u}) + o(s^k).$$

- Multilinearität von $D_{\mathbf{x}}^j f$: $D_{\mathbf{x}}^j f(\mathbf{u}, \dots, \mathbf{u}) = D_{\mathbf{x}}^j f\left(\frac{\mathbf{h}}{s}, \dots, \frac{\mathbf{h}}{s}\right) = \frac{1}{s^j} D_{\mathbf{x}}^j f(\mathbf{h}, \dots, \mathbf{h})$.
- Mit $s = \|\mathbf{h}\|$:

$$f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + \sum_{j=1}^k \frac{1}{j!} D_{\mathbf{x}}^j f(\mathbf{h}, \dots, \mathbf{h}) + o(\|\mathbf{h}\|^k). \blacksquare$$

(Drei technische Feinheiten des Arguments: $\rightarrow$ *Anhang*.)

$\leftarrow$ zurück: „Taylorentwicklung: Allgemein“

Taylorentwicklung II: Beweisskizze — Feinheiten

Drei Punkte machen die Skizze zum vollständigen Beweis — hier für den im Kurs relevanten Fall $V = \mathbb{R}^n$, $W = \mathbb{R}^m$ (allgemeine normierte Räume brauchen andere Argumente):

- 1) Definitionsbereich:** f muss auf einer Umgebung von $\mathbf{x}$ definiert sein, die das ganze Segment $\mathbf{x} + w\mathbf{u}$, $0 \leq w \leq s$, enthält — für hinreichend kleines $\|\mathbf{h}\|$ ist das erfüllt.
- 2) W -wertiges 1D-Taylor:** Der 1D-Satz mit Peano-Restglied gilt zunächst für reellwertige Funktionen; wegen $\dim W < \infty$ wendet man ihn **koordinatenweise** auf $\psi = (\psi_1, \dots, \psi_m)$ an und sammelt die Restglieder ein.
- 3) Gleichmäßigkeit in der Richtung:** Das $o(s^k)$ aus dem 1D-Satz gilt zunächst *pro fester Richtung* $\mathbf{u}$. Damit daraus $o(\|\mathbf{h}\|^k)$ für **beliebige** $\mathbf{h} \rightarrow \mathbf{0}$ wird, muss der Fehlerterm gleichmäßig über alle Richtungen klein werden: Betrachte das Restglied für $j = k$ in Lagrange-Form — die Differenz $D_{\mathbf{x}+\xi\mathbf{u}}^k f - D_{\mathbf{x}}^k f$ wird wegen der **Stetigkeit von** $D^k f$ und der **Kompaktheit der Einheitssphäre** $\{\|\mathbf{u}\| = 1\}$ gleichmäßig klein.

↔ zurück: „Taylorentwicklung II: Beweisskizze (allgemein)“

Höhere Ableitungen für Vektor-zu-Vektor: explizite Formeln

Für $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ ist insbesondere:

$$D_{\mathbf{x}} f(\mathbf{h}) = \sum_{i=1}^n \frac{\partial f(\mathbf{x})}{\partial x_i} h_i = \mathbf{J}_f(\mathbf{x}) \mathbf{h},$$

$$D_{\mathbf{x}}^2 f(\mathbf{h}_1, \mathbf{h}_2) = \sum_{i=1}^n \sum_{j=1}^n \frac{\partial^2 f(\mathbf{x})}{\partial x_j \partial x_i} h_{1,i} h_{2,j} = \sum_{i=1}^n \frac{\partial \mathbf{J}_f(\mathbf{x})}{\partial x_i} \mathbf{h}_1 \cdot h_{2,i},$$

$$D_{\mathbf{x}}^3 f(\mathbf{h}_1, \mathbf{h}_2, \mathbf{h}_3) = \sum_{i=1}^n \sum_{j=1}^n \sum_{k=1}^n \frac{\partial^3 f(\mathbf{x})}{\partial x_k \partial x_j \partial x_i} h_{1,i} h_{2,j} h_{3,k} = \sum_{i=1}^n \sum_{j=1}^n \frac{\partial^2 \mathbf{J}_f(\mathbf{x})}{\partial x_j \partial x_i} \mathbf{h}_1 \cdot h_{2,i} \cdot h_{3,j}$$

↔ zurück: „Höhergradige Ableitung für Vektor-zu-Vektor“

Konvexität

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)

Material: Thomas Nagler

LMU München

Verwendete Vorkenntnisse

- Normen, Skalarprodukträume
- Gradient ∇f & Hesse-Matrix $\mathbf{H}_f$ (Kap. 10/11)
- **Lineare Algebra**
 - Vektorräume
 - Linearkombinationen: $\sum w_i \mathbf{v}_i$
 - Positiv definite Matrizen
- **Analysis**
 - Grenzwerte; Supremum & Infimum
 - Abgeschlossene Mengen
- **Wahrscheinlichkeitstheorie**
 - Erwartungswert $\mathbb{E}[X]$ als gewichteter Durchschnitt
 - Jensen-Ungleichung

Einführung

Einführung

- *Konvexität* ist ein wichtiges Konzept in der Mathematik, besonders in
 - Optimierung
 - Wahrscheinlichkeitstheorie
- Der Begriff bezieht sich meist auf die (verwandten) Konzepte
 - *konvexe Mengen*
 - *konvexe Funktionen*
- Beides ist gewissermaßen eine Verallgemeinerung von Linearitätskonzepten

Diese Folien: → wichtigste Definitionen und Eigenschaften

Warum Konvexität wichtig ist

In Optimierung / ML / Statistik:

- Konvexe Optimierungsprobleme sind *gutartig*: jedes lokale Minimum ist *global* (→ *Optimierung I*)
- Viele Algorithmen basieren auf konvexer Optimierung, z.B.:
 - Maximum Likelihood (*meist* konkave Log-Likelihoods = konvexe *negative* Log-Likelihoods): GLMs etc.
 - Support Vector Machines
 - Ridge Regression

In der Wahrscheinlichkeitstheorie:

- *Jensen'sche Ungleichung* nutzt Konvexität für Erwartungswerte
- KL-Divergenz ist konvex (wichtig für *Variational Inference*)
- Varianz als Konvexitätseigenschaft: $\mathbb{E}[X^2] \geq (\mathbb{E}[X])^2$

Konvexkombinationen

Konvexkombinationen

Def.: Konvexkombination

Sei V ein Vektorraum und $\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_k\} \subset V$.

Dann nennen wir $\mathbf{x}$ eine **Konvexkombination** von Vektoren in $\mathcal{X}$, falls es Koeffizienten $w_1, \dots, w_k \geq 0$ gibt mit

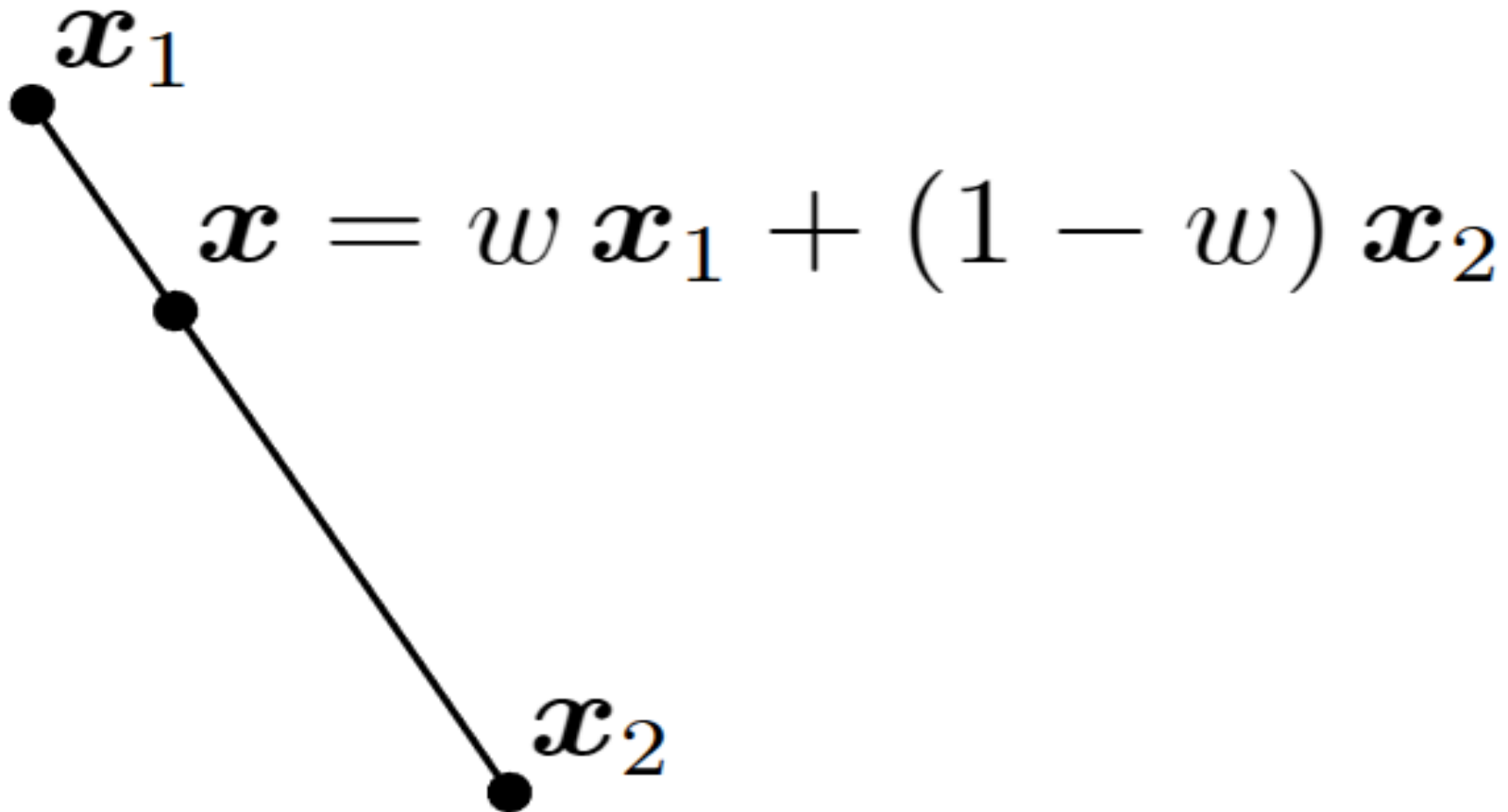
$$\mathbf{x} = \sum_{i=1}^k w_i \mathbf{x}_i \quad \text{und} \quad \sum_{i=1}^k w_i = 1.$$

- **Konvexkombinationen** sind spezielle Linearkombinationen.
- Sie können als gewichteter Durchschnitt interpretiert werden.
- **Bsp.:** Erwartungswert $\mathbb{E}(X)$ für diskrete Zufallsvariable X mit endlichem Träger ist Konvexkombination der Werte des Trägers $\mathcal{X} = T_X$:

$$\mathbb{E}(X) = \sum_{x_j \in T_X} \mathbb{P}(X = x_j) x_j$$

Konvexkombinationen

- Die Menge aller Konvexkombinationen zweier Vektoren x_1, x_2 ist das *Liniensegment*, das x_1 und x_2 verbindet.



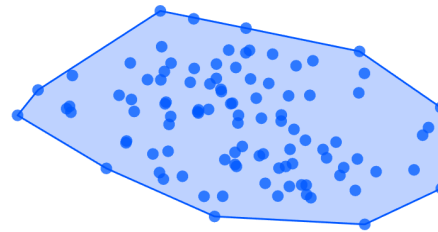
Konvexe Hülle

Def.: Konvexe Hülle

Sei $\mathcal{X} \subset V$ eine beliebige Menge von Vektoren.

Die Menge aller Konvexkombinationen von endlich vielen Vektoren aus $\mathcal{X}$ wird als **konvexe Hülle** $\text{conv}(\mathcal{X})$ von $\mathcal{X}$ bezeichnet:

$$\text{conv}(\mathcal{X}) = \left\{ \mathbf{x} = \sum_{i=1}^N w_i \mathbf{x}_i : N \in \mathbb{N}, \mathbf{x}_i \in \mathcal{X}, w_i \geq 0, \sum_{i=1}^N w_i = 1 \right\}$$



- **Extrempunkte** sind die Punkte $\mathbf{x}_i \in \mathcal{X}$, die sich nur mit $w_i = 1$ (und allen übrigen Gewichten 0) darstellen lassen.
- Die **konvexe Hülle** ist die kleinste konvexe Menge, die $\mathcal{X}$ enthält.

Konkretes Beispiel im $\mathbb{R}^2$: → *Anhang*.

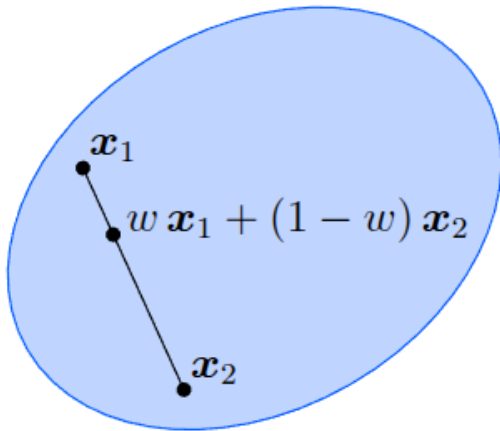
Konvexe Mengen

Konvexe Mengen: Definition

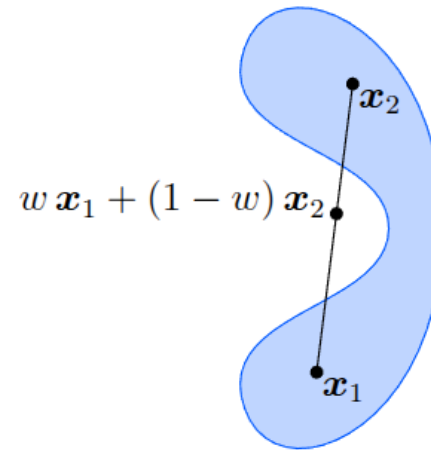
Def.: Konvexe Menge

Eine Menge $\mathcal{X} \subset V$ in Vektorraum V wird als **konvex** bezeichnet, falls für alle $\mathbf{x}, \mathbf{y} \in \mathcal{X}$ und alle $\lambda \in [0, 1]$ gilt:

$$\lambda \mathbf{x} + (1 - \lambda) \mathbf{y} \in \mathcal{X}.$$



konvex



nicht konvex

Konvexe Mengen enthalten für alle Paare von Punkten auch das Liniensegment zwischen diesen Punkten $\implies$ sie haben keine “Einbuchtungen” oder “Löcher”.

Quiz: Konvexe Mengen

Welche der folgenden Mengen ist konvex?

1. Die leere Menge $\emptyset$.
2. Ein Vektorraum V .
3. Die Einheitssphäre $\{\mathbf{x} \in V: \|\mathbf{x}\| = 1\}$.
4. Der geschlossene Ball $\{\mathbf{x} \in V: \|\mathbf{x}\| \leq r\}$.
5. Die Menge $\{(x, y) \in \mathbb{R}^2: y \leq x^2\}$.

Konvexe Mengen: Beispiele

Weitere Beispiele

- Ein offener Ball $\{\mathbf{x} \in V: \|\mathbf{x}\| < r\}$.
- Der *Simplex*

$$\Delta^N = \left\{ \mathbf{x} \in \mathbb{R}^N: x_i \geq 0, \sum_{i=1}^N x_i = 1 \right\}.$$

- Die Menge aller *positiv semi-definiten* Matrizen in $\mathbb{R}^{n \times n}$.

Simplex - Beispiel und Anwendung

Beispiel in $\mathbb{R}^3$:

Simplex Δ^3 besteht aus allen Punkten $\mathbf{x} = (x_1, x_2, x_3)$ mit $x_1, x_2, x_3 \geq 0$ und $x_1 + x_2 + x_3 = 1 = \|\mathbf{x}\|_1$

Extrempunkte:

$$\mathbf{e}_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \mathbf{e}_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \mathbf{e}_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

Anwendung in der Statistik:

- Δ^N definiert (die Menge aller) Wahrscheinlichkeitsverteilungen über N Ereignisse. Beispiel:
 $\mathbf{p} = (0.2, 0.5, 0.3)$ beschreibt eine kategoriale Verteilung mit drei Kategorien
- Wichtig für: Softmax-Regression/Multinomial-Modelle, Dirichlet-Verteilungen, ...

Geometrisch:

Δ^2 ist die Strecke von $(1, 0)$ nach $(0, 1)$,
 Δ^3 das Dreieck zwischen $\mathbf{e}_1, \mathbf{e}_2$ und $\mathbf{e}_3$,
 Δ^4 ein Tetraeder, ...

PSD-Matrizen bilden eine konvexe Menge

Def.: Positiv Semi-Definit

Eine *symmetrische* Matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ ist **positiv semi-definit (PSD)**

$$\iff \mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0 \quad \forall \mathbf{x} \in \mathbb{R}^n.$$

Wir schreiben: $\mathbf{A} \succeq 0$

Seien $\mathbf{A}_1, \mathbf{A}_2 \succeq 0$ und $\lambda \in [0, 1]$.

Zu zeigen:

$$\mathbf{A}_\lambda = \lambda \mathbf{A}_1 + (1 - \lambda) \mathbf{A}_2 \succeq 0.$$

Für beliebiges $\mathbf{x} \in \mathbb{R}^n$:

$$\begin{aligned} \mathbf{x}^\top \mathbf{A}_\lambda \mathbf{x} &= \mathbf{x}^\top [\lambda \mathbf{A}_1 + (1 - \lambda) \mathbf{A}_2] \mathbf{x} \\ &= \lambda \mathbf{x}^\top \mathbf{A}_1 \mathbf{x} + (1 - \lambda) \mathbf{x}^\top \mathbf{A}_2 \mathbf{x} \\ &\geq \lambda \cdot 0 + (1 - \lambda) \cdot 0 = 0 \quad (\text{da } \mathbf{A}_1, \mathbf{A}_2 \succeq 0) \quad \blacksquare \end{aligned}$$

Anwendung:

Kovarianzmatrizen sind stets **PSD** (**SPD** nur, falls keine Komponente linear von den übrigen abhängt) und bilden eine **konvexe Menge**.

Konvexe Mengen: Eigenschaften

- Viele wichtige Operationen erhalten Konvexität.

Theorem: Konvexitätserhaltung

Seien $\mathcal{X}, \mathcal{X}_1, \mathcal{X}_2, \dots$ konvexe Mengen. Dann sind auch die folgenden Mengen konvex:

- *Schnittmenge:*
 $\mathcal{S} = \bigcap_{i \in I} \mathcal{X}_i$ für eine beliebige Indexmenge I (alle $\mathcal{X}_i$ im selben Vektorraum)
- *Kartesisches Produkt:*
 $\mathcal{X}_1 \times \dots \times \mathcal{X}_k$
- *Affine Abbildungen:*
 $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ mit $f(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}$ und $\mathcal{X} \subseteq \mathbb{R}^n$: Bildmenge
 $f(\mathcal{X}) = \{\mathbf{y} = f(\mathbf{x}) : \mathbf{x} \in \mathcal{X}\}$ ist konvex.
- *Summen:*
 $\mathcal{X}_1 \oplus \mathcal{X}_2 := \{\mathbf{x}_1 + \mathbf{x}_2 : \mathbf{x}_1 \in \mathcal{X}_1, \mathbf{x}_2 \in \mathcal{X}_2\}$ ($\mathcal{X}_1, \mathcal{X}_2$ im selben Vektorraum)

- **Beispiel** für die Nützlichkeit der Schnitt-Eigenschaft:
Polyeder (= Schnitte von **Halbräumen**) sind konvex – wichtig für *lineare Programmierung* (→ *Optimierung unter linearen Nebenbedingungen*)!

Konvexe Hülle: Eigenschaften

- Eine **konvexe Hülle** ist selbst eine **konvexe Menge** (Konvexkombinationen von Konvexkombinationen aus $\mathcal{X}$ sind wieder Konvexkombinationen aus $\mathcal{X}$).
- Sie ist die **kleinste konvexe Menge, die ganz $\mathcal{X}$ enthält**.

Proposition

Es gilt $\text{conv}(\mathcal{X}) = \bigcap_{\mathcal{X} \subseteq \mathcal{Y}, \mathcal{Y} \text{ konvex}} \mathcal{Y}$.

Beweis:

Sei $\mathcal{S} = \bigcap_{\mathcal{X} \subseteq \mathcal{Y}, \mathcal{Y} \text{ konvex}} \mathcal{Y}$.

- Da $\mathcal{X} \subseteq \mathcal{Y}$ und $\mathcal{Y}$ konvex ist, ist jede Konvexkombination von Punkten aus $\mathcal{X}$ auch in jedem $\mathcal{Y}$ (konvexe Mengen enthalten – per Induktion über die Anzahl der Punkte – alle Konvexkombinationen endlich vieler ihrer Punkte). Es folgt $\text{conv}(\mathcal{X}) \subseteq \mathcal{S}$.
- Da der Schnitt auch über die Menge $\mathcal{Y} = \text{conv}(\mathcal{X})$ geht, ist automatisch auch $\mathcal{S} \subseteq \text{conv}(\mathcal{X})$.

Konvexe Mengen: Anwendung

- Konvexe Mengen/Hüllen sind wichtig in der Optimierung.

Beispiel:

- Sei $\mathcal{X} \subseteq \mathbb{N}^n$. Jedes $\mathbf{x} \in \mathcal{X}$ beschreibt eine Zusammensetzung von n verschiedenen Gegenständen: $x_i = k$ bedeutet, dass k Einheiten des i -ten Gegenstands vorhanden sind.
- Jeder Gegenstand hat einen Wert v_i und wir wollen das $\mathbf{x} \in \mathcal{X}$ mit maximalem Wert finden:

$$\mathbf{x}^* \in \arg \max_{\mathbf{x} \in \mathcal{X}} \sum_{i=1}^n v_i x_i.$$

- Falls $\mathcal{X}$ sehr groß ist, dauert es zu lang, den Wert jeden Vektors auszurechnen.
- Man kann zeigen, dass das Maximum (auch) an einem Extrempunkt von $\text{conv}(\mathcal{X})$ angenommen wird $\rightarrow$ oft viel leichter zu finden.

Konvexe Mengen: Projektionstheorem

- Eine weitere wichtige Eigenschaft ist Existenz & Eindeutigkeit von **Projektionen** auf konvexe Mengen:

Theorem: Projektionstheorem

Sei $\mathcal{X} \neq \emptyset$ eine abgeschlossene, konvexe Teilmenge eines *vollständigen* Skalarproduktraums V (z.B. $V = \mathbb{R}^n$).

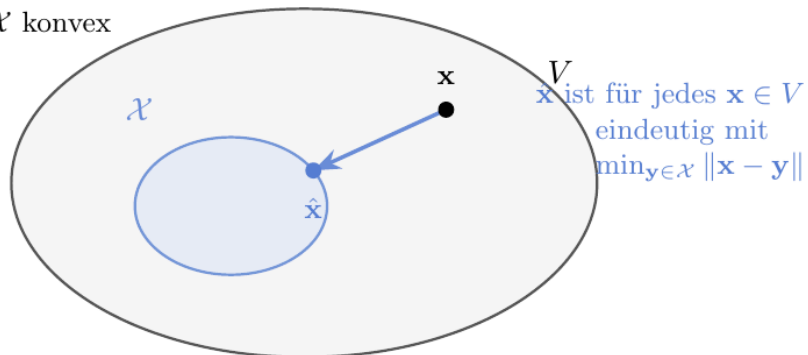
Dann existiert für jeden Punkt $\mathbf{x} \in V$ genau ein Punkt $\hat{\mathbf{x}} \in \mathcal{X}$, der den minimalen Abstand zu $\mathbf{x}$ hat:

$$\|\mathbf{x} - \hat{\mathbf{x}}\| = \min_{\mathbf{y} \in \mathcal{X}} \|\mathbf{x} - \mathbf{y}\|.$$

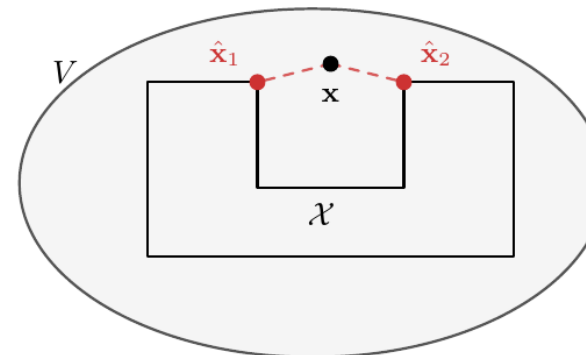
Dieser Punkt ist die **Projektion** von $\mathbf{x}$ auf $\mathcal{X}$.

Projektionstheorem:

$\mathcal{X}$ konvex



Wenn $\mathcal{X}$ nicht konvex ist, ist $\hat{\mathbf{x}}$ nicht immer eindeutig.



$$\begin{aligned} \|\mathbf{x} - \hat{\mathbf{x}}_1\| &= \|\mathbf{x} - \hat{\mathbf{x}}_2\| \\ &= \min_{\mathbf{y} \in \mathcal{X}} \|\mathbf{x} - \mathbf{y}\| \end{aligned}$$

Konvexe Funktionen

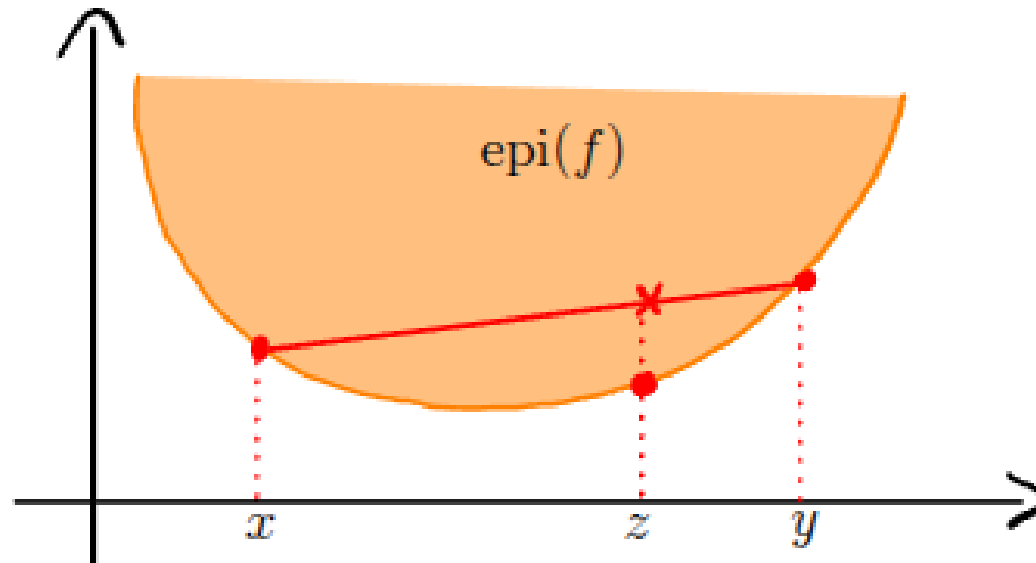
Konvexe Funktionen

Def.: Epigraph

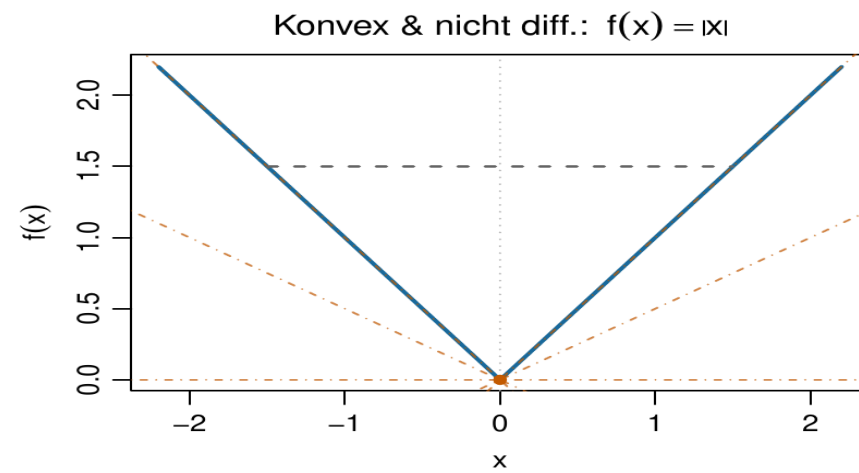
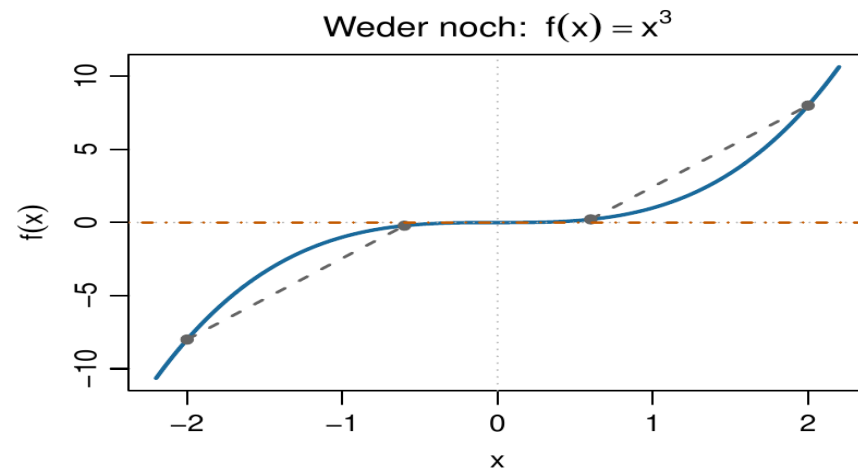
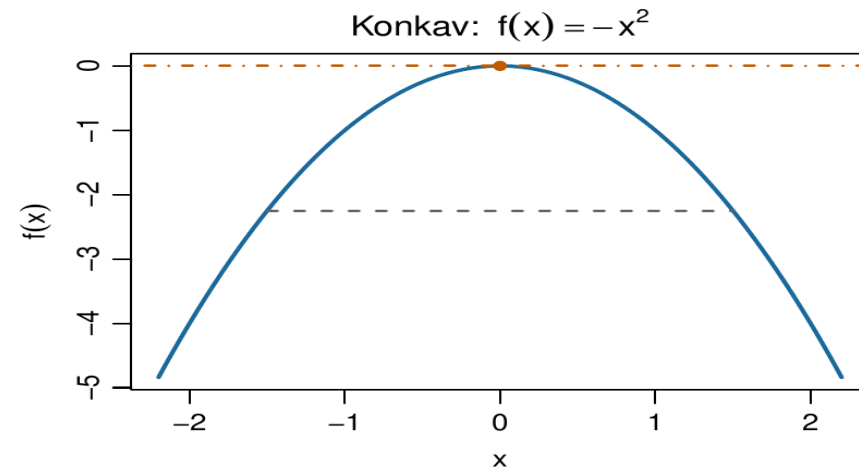
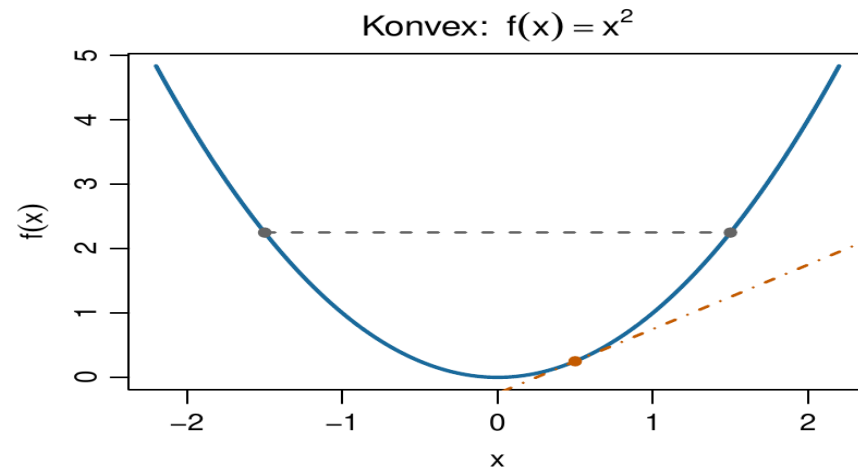
Der **Epigraph** einer Funktion $f: \mathcal{X} \rightarrow \mathbb{R}$ ist die Menge

$$\text{epi}(f) = \{(\mathbf{x}, t) \in \mathcal{X} \times \mathbb{R} : t \geq f(\mathbf{x})\}$$

- Der **Epigraph** enthält alle Punkte in $\mathcal{X} \times \mathbb{R}$, die auf oder über dem Graphen von f liegen.
- **Konvexe Funktionen** $f: \mathcal{X} \rightarrow \mathbb{R}$ sind solche, deren **Epigraph** eine konvexe Menge ist. ($\mathcal{X}$ ist dann automatisch konvex.)



Konvex vs. Konkav



Konvexe Funktionen

- Eine gängigere Definition erfolgt direkt über Ungleichungen.

Def.: Konvexe Funktion

Sei $\mathcal{X}$ konvex und $f: \mathcal{X} \rightarrow \mathbb{R}$ eine Funktion.

Dann heißt f **konvex**, falls für alle $\mathbf{x}, \mathbf{y} \in \mathcal{X}$ und alle $\lambda \in [0, 1]$ gilt

$$f(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) \leq \lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y}).$$

- Intuitiv sagt die Definition (für $f: \mathbb{R}^n \rightarrow \mathbb{R}$):
Die Strecke zwischen den Graphenpunkten $(\mathbf{x}, f(\mathbf{x}))$ und $(\mathbf{y}, f(\mathbf{y}))$ liegt *nie unter* dem Funktionsgraphen.
Konvexe Funktionen sind also “nach oben geöffnet”.
- Wir nennen eine Funktion f **konkav**, wenn $-f$ **konvex** ist.

Konvexe Funktion - Verifikation

Zu zeigen:

$f(x) = |x|$ ist konvex auf $\mathbb{R}$.

Seien $x, y \in \mathbb{R}$ beliebig und $\lambda \in [0, 1]$. Zu prüfen:

$$f(\lambda x + (1 - \lambda)y) \stackrel{?}{\leq} \lambda f(x) + (1 - \lambda)f(y)$$

Beweis (Dreiecksungleichung):

$$\begin{aligned} f(\lambda x + (1 - \lambda)y) &= |\lambda x + (1 - \lambda)y| \\ &\leq |\lambda x| + |(1 - \lambda)y| \\ &= \lambda|x| + (1 - \lambda)|y| \\ &= \lambda f(x) + (1 - \lambda)f(y) \quad \blacksquare \end{aligned}$$

Konvexe Funktionen: Beispiele

Die folgenden multivariaten Funktionen sind konvex:

- **Affine Funktionen:**

$$f(\mathbf{x}) = \mathbf{b}^\top \mathbf{x} + a \text{ mit } \mathbf{b} \in \mathbb{R}^n, a \in \mathbb{R}.$$

Sie sind gleichzeitig auch **konkav**!

- **Manche quadratischen Funktionen:**

$$f(\mathbf{x}) = \mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{b}^\top \mathbf{x} + a \text{ mit } \mathbf{Q} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n, a \in \mathbb{R}, \text{ falls der symmetrische Anteil } \frac{1}{2}(\mathbf{Q} + \mathbf{Q}^\top) \text{ positiv semidefinit ist (für symmetrisches } \mathbf{Q} \text{ also: falls } \mathbf{Q} \succeq 0).$$

- **Jede Norm:** $f(\mathbf{x}) = \|\mathbf{x}\|$, d.h. jede Funktion mit den Eigenschaften

- $f(c\mathbf{x}) = |c|f(\mathbf{x})$
- $f(\mathbf{x} + \mathbf{y}) \leq f(\mathbf{x}) + f(\mathbf{y})$
- $f(\mathbf{x}) \geq 0$ und $f(\mathbf{x}) = 0 \implies \mathbf{x} = \mathbf{0}$

Anwendung in ML / Statistik:

- **Kleinste-Quadrate-Verlust:**

$$f(\mathbf{w}) = \|\mathbf{A}\mathbf{w} - \mathbf{y}\|_2^2 = \mathbf{w}^\top (\mathbf{A}^\top \mathbf{A}) \mathbf{w} - 2\mathbf{y}^\top \mathbf{A} \mathbf{w} + \|\mathbf{y}\|_2^2 \text{ mit } \mathbf{Q} = \mathbf{A}^\top \mathbf{A} \succeq 0 \text{ (immer PSD!) } \implies \text{konvexes Optimierungsproblem}$$

- **Ridge-Regression:**

$$f(\mathbf{w}) = \|\mathbf{A}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}\|_2^2 \text{ mit } \lambda > 0 \text{ ist strikt konvex,}$$

d.h. die Konvexitäts-Ungleichung gilt strikt für $\mathbf{w} \neq \mathbf{w}'$

($\rightarrow$ eindeutige Lösung; formale Definition und Konsequenzen im Kapitel *Optimierung I*)

Konvexe Funktionen: Eigenschaften

Proposition

Seien $f_1, f_2, \dots: \mathcal{X} \rightarrow \mathbb{R}$ konvexe Funktionen. Dann sind auch die folgenden Funktionen konvex:

- i. $f(\mathbf{x}) = f_1(\mathbf{x}) + f_2(\mathbf{x})$.
- ii. $f(\mathbf{x}) = cf_1(\mathbf{x}), c \geq 0$.
- iii. $f(\mathbf{x}) = \max_{i=1, \dots, k} f_i(\mathbf{x})$.
- iv. $f(\mathbf{x}) = \lim_{n \rightarrow \infty} f_n(\mathbf{x})$, vorausgesetzt der Grenzwert existiert für alle $\mathbf{x} \in \mathcal{X}$.

Beweis für (ii) + (iv): → Anhang.

Jensen-Ungleichung

Satz: Jensen-Ungleichung

Sei $\mathcal{X} \subseteq \mathbb{R}^n$ eine *konvexe Menge* und $f: \mathcal{X} \rightarrow \mathbb{R}$ eine *konvexe Funktion*.

Dann gilt für alle $N \in \mathbb{N}$, alle $\mathbf{x}_1, \dots, \mathbf{x}_N \in \mathcal{X}$ und alle $w_1, \dots, w_N \geq 0$ mit $\sum_{i=1}^N w_i = 1$:

$$f\left(\sum_{i=1}^N w_i \mathbf{x}_i\right) \leq \sum_{i=1}^N w_i f(\mathbf{x}_i).$$

- Die *Jensen-Ungleichung* verallgemeinert die Definition von Konvexität auf beliebig viele Punkte.
- Der Beweis erfolgt per Induktion über N .

Anwendung: Nichtnegative Varianz

Für eine Zufallsvariable X mit endlichem Träger T_X ist $\mathbb{E}[X] = \sum_{x \in T_X} P(X = x)x$ eine **Konvexkombination** der Werte in T_X (Gewichte = Wahrscheinlichkeiten).

Mit der **Jensen-Ungleichung** und da $x \mapsto x^2$ **konvex** ist gilt also:

$$\mathbb{E}[X]^2 = f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)] = \mathbb{E}[X^2]$$

Umstellen ergibt:

$$\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 \geq 0$$

Für allgemeine integrierbare X gilt die Ungleichung ebenfalls – der Beweis läuft dann über Stützgeraden statt über endliche Konvexkombinationen.

Konvexe Vektor-zu-Skalar-Funktionen

Theorem

Sei $\mathcal{X} \subseteq \mathbb{R}^n$ eine offene, *konvexe Menge* und $f: \mathcal{X} \rightarrow \mathbb{R}$ zweimal stetig differenzierbar. Dann sind die folgenden Aussagen äquivalent:

- i. f ist konvex.
 - ii. $f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})(\mathbf{y} - \mathbf{x})$ für alle $\mathbf{x}, \mathbf{y} \in \mathcal{X}$.
 - iii. $\mathbf{H}_f(\mathbf{x})$ ist positiv semidefinit für alle $\mathbf{x} \in \mathcal{X}$.
- (i): f ist konvex genau dann wenn die Sehne zwischen zwei Punkten des Graphen nie *unter* dem Funktionsgraphen verläuft.
 - (ii): f ist konvex genau dann wenn die Tangente an f nie *über* dem Funktionsgraphen verläuft.
 - (iii): f ist konvex genau dann wenn die **Hesse-Matrix** positiv semidefinit ist (d.h.: nicht-negative Krümmung in alle Richtungen an jeder Stelle).

Beweis: → *Anhang*.

Konvexität & Spektrum der Hesse-Matrix

Für $f \in \mathcal{C}^2$ ist $\mathbf{H}_f(\mathbf{x})$ symmetrisch

$\implies \mathbf{H}_f(\mathbf{x})$ hat reelle Eigenwerte λ_i und orthonormale Eigenvektoren $\mathbf{v}_i$:

- Lokale Taylor-Approximation um $\mathbf{x}$:

$$f(\mathbf{x} + \mathbf{h}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x}) \mathbf{h} + \frac{1}{2} \mathbf{h}^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h} = f(\mathbf{x}) + \nabla f(\mathbf{x}) \mathbf{h} + \frac{1}{2} \sum_{i=1}^n \lambda_i (\mathbf{v}_i^\top \mathbf{h})^2$$

- $\lambda_i > 0$: in $\mathbf{v}_i$ -Richtung um $\mathbf{x}$ herum **konvex** (nach oben gekrümmt)
- $\lambda_i < 0$: in $\mathbf{v}_i$ -Richtung um $\mathbf{x}$ herum **konkav** (nach unten gekrümmt)
- $|\lambda_i|$ misst Stärke der Krümmung in $\mathbf{v}_i$ -Richtung

Wrap-up

Heute gelernt

- **Konvexkombinationen** = gewichtete Durchschnitte; **konvexe Hülle** = kleinste konvexe Obermenge
- **Konvexe Mengen** enthalten alle Verbindungsstrecken; Konvexität bleibt unter Schnitten, Produkten, affinen Abbildungen und Summen erhalten
- **Projektionen** auf nichtleere abgeschlossene konvexe Mengen (in vollständigen Skalarprodukträumen) existieren und sind eindeutig
- **Konvexe Funktionen**: **Epigraph** konvex $\iff$ Sehnen-Ungleichung; **Jensen-Ungleichung**
- $\mathcal{C}^2$ -Charakterisierung: f konvex $\iff$ **Hesse-Matrix** überall PSD

Nächste Vorlesung

- Optimierung I – u.a.: warum konvexe Optimierungsprobleme “gutartig” sind

Self-Check: Konvexe Funktionen

Welche der folgenden Funktionen ist konvex?

1. $f: \mathbb{R} \rightarrow \mathbb{R}, f(x) = ax + b, a, b \in \mathbb{R}.$

2. $f: \mathbb{R} \rightarrow \mathbb{R}, f(x) = x^4.$

3. $f: \mathbb{R}_+ \rightarrow \mathbb{R}, f(x) = \sqrt{x}.$

4. $f: \mathbb{R} \rightarrow \mathbb{R}, f(x) = e^x.$

5. $f: \mathbb{R}_+ \rightarrow \mathbb{R}, f(x) = \ln x.$

Self-Check: Klassifikation

Klassifizieren Sie: **konvex**, **strikt konvex** (d.h., die Ungleichung gilt strikt für $x \neq y$, $\lambda \in (0, 1)$) oder **keins von beiden**?

1. $f: \mathbb{R} \rightarrow \mathbb{R}, f(x) = |x|$
2. $f: \mathbb{R} \rightarrow \mathbb{R}, f(x) = e^x$
3. $f: \mathbb{R} \rightarrow \mathbb{R}, f(x) = ax + b, a, b \in \mathbb{R}$
4. $f: \mathbb{R}_+ \rightarrow \mathbb{R}, f(x) = \sqrt{x}$

Anhang

Konvexe Hülle - Beispiel im $\mathbb{R}^2$

(Zusatzbeispiel)

Sei $\mathcal{X} = \{(0, 0), (2, 0), (1, 2)\} \subset \mathbb{R}^2$ (die Ecken eines Dreiecks).

Extrempunkte:

Alle drei Punkte in $\mathcal{X}$ sind *Extrempunkte*, da sie nur durch $w_i = 1, w_j = 0 \ \forall j \neq i$ erreicht werden können.

Konvexe Hülle:

$\text{conv}(\mathcal{X})$ ist die Menge aller Punkte der Form

$$\mathbf{x} = w_1 \begin{pmatrix} 0 \\ 0 \end{pmatrix} + w_2 \begin{pmatrix} 2 \\ 0 \end{pmatrix} + w_3 \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

mit $w_1, w_2, w_3 \geq 0$ und $w_1 + w_2 + w_3 = 1$.

Konkreter Punkt: Für $w_1 = w_2 = w_3 = \frac{1}{3}$:

$$\mathbf{x} = \frac{1}{3} \begin{pmatrix} 0 \\ 0 \end{pmatrix} + \frac{1}{3} \begin{pmatrix} 2 \\ 0 \end{pmatrix} + \frac{1}{3} \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} 1 \\ \frac{2}{3} \end{pmatrix}$$

Dieser Punkt (Schwerpunkt!) liegt im Inneren des Dreiecks.

↩ zurück: „Konvexe Hülle“

Beispiel: Schnitt konvexer Mengen

(Zusatzbeispiel)

Konkrete Mengen im $\mathbb{R}^2$:

- $\mathcal{X}_1 = \{(x, y) \in \mathbb{R}^2 : x + y \leq 1\}$ (Halbraum, konvex)
- $\mathcal{X}_2 = \{(x, y) \in \mathbb{R}^2 : x \geq 0\}$ (Halbraum, konvex)
- $\mathcal{X}_3 = \{(x, y) \in \mathbb{R}^2 : y \geq 0\}$ (Halbraum, konvex)

Schnitt:

$$\mathcal{S} = \mathcal{X}_1 \cap \mathcal{X}_2 \cap \mathcal{X}_3 = \{(x, y) : x \geq 0, y \geq 0, x + y \leq 1\}$$

Dies ist ein Dreieck mit Ecken $(0, 0)$, $(1, 0)$, $(0, 1)$ - offensichtlich konvex!

Generell:

Polyeder (als Schnitte von Halbräumen) sind konvex – wichtig für *lineare Programmierung* (Optimierung unter linearen Nebenbedingungen)!

↔ zurück: „Konvexe Mengen: Eigenschaften“

Konvexitätserhaltung: Beweis

- (i) Schnitt:

Sei $\mathbf{x}, \mathbf{y} \in \mathcal{S}$, $\lambda \in [0, 1]$ und $\mathbf{z} = \lambda \mathbf{x} + (1 - \lambda) \mathbf{y}$.

Dann gilt $\mathbf{x}, \mathbf{y} \in \mathcal{S} \implies \mathbf{x}, \mathbf{y} \in \mathcal{X}_i \forall i \in I$.

$\mathcal{X}_i$ konvex $\forall i \in I \implies \mathbf{z} \in \mathcal{X}_i \forall i \in I \implies \mathbf{z} \in \mathcal{S}$.

- (ii) Kartesisches Produkt:

Seien $(\mathbf{x}_1, \dots, \mathbf{x}_k), (\mathbf{y}_1, \dots, \mathbf{y}_k) \in \mathcal{X}_1 \times \dots \times \mathcal{X}_k; \lambda \in [0, 1]$.

Weil $\mathcal{X}_i$ konvex, ist auch

$$\begin{aligned} & \lambda(\mathbf{x}_1, \dots, \mathbf{x}_k) + (1 - \lambda)(\mathbf{y}_1, \dots, \mathbf{y}_k) \\ &= (\lambda \mathbf{x}_1 + (1 - \lambda) \mathbf{y}_1, \dots, \lambda \mathbf{x}_k + (1 - \lambda) \mathbf{y}_k) \in (\mathcal{X}_1 \times \dots \times \mathcal{X}_k) \end{aligned}$$

- (iii) Affine Abb.:

Sei $\mathbf{y}_1 = f(\mathbf{x}_1), \mathbf{y}_2 = f(\mathbf{x}_2) \in f(\mathcal{X})$ für $\mathbf{x}_1, \mathbf{x}_2 \in \mathcal{X}; \lambda \in [0, 1]$. Dann gilt:

$$\begin{aligned} \lambda \mathbf{y}_1 + (1 - \lambda) \mathbf{y}_2 &= \lambda(\mathbf{A} \mathbf{x}_1 + \mathbf{b}) + (1 - \lambda)(\mathbf{A} \mathbf{x}_2 + \mathbf{b}) \\ &= \mathbf{A}(\lambda \mathbf{x}_1 + (1 - \lambda) \mathbf{x}_2) + \mathbf{b} \\ &= f(\lambda \mathbf{x}_1 + (1 - \lambda) \mathbf{x}_2) \in f(\mathcal{X}). \end{aligned}$$

- (iv) Summe:

Folgt aus (ii) und (iii):

$\mathcal{X}_1 \times \mathcal{X}_2$ ist konvex, $f(\mathbf{x}_1, \mathbf{x}_2) = \mathbf{x}_1 + \mathbf{x}_2$ ist linear, also ist auch $f(\mathcal{X}_1 \times \mathcal{X}_2) = \mathcal{X}_1 \oplus \mathcal{X}_2$ konvex.

Projektionstheorem: Beweisskizze

- **Existenz:**

Eine Minimierungsfolge ist beschränkt und besitzt daher einen Grenzwert (in $\mathbb{R}^n$: Bolzano-Weierstraß;
allgemein: Parallelogrammgleichung $\rightarrow$ Cauchy-Folge + Vollständigkeit von V);
 $\mathcal{X}$ abgeschlossen $\implies$ dieser Grenzwert liegt in $\mathcal{X}$

- **Eindeutigkeit:**

Angenommen, es gäbe $\hat{\mathbf{x}}_1 \neq \hat{\mathbf{x}}_2$ mit demselben minimalen Abstand d zu $\mathbf{x}$.

Für jedes $\lambda \in [0, 1]$ gilt wegen Konvexität

$\mathbf{z} = \lambda \hat{\mathbf{x}}_1 + (1 - \lambda) \hat{\mathbf{x}}_2 \in \mathcal{X}$ und nach der Dreiecksungleichung:

$$\|\mathbf{x} - \mathbf{z}\| \leq \lambda \|\mathbf{x} - \hat{\mathbf{x}}_1\| + (1 - \lambda) \|\mathbf{x} - \hat{\mathbf{x}}_2\| = d$$

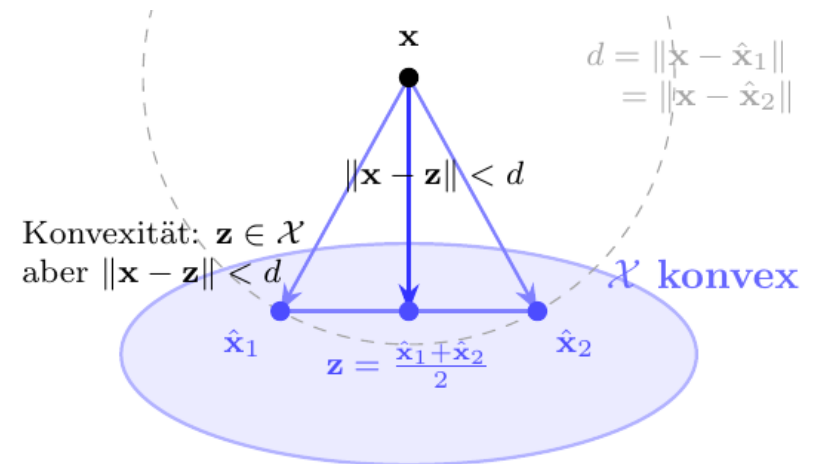
Wegen der Minimalität von d gilt sogar $\|\mathbf{x} - \mathbf{z}\| = d$.

Wähle Mittelpunkt $\mathbf{z} = (\hat{\mathbf{x}}_1 + \hat{\mathbf{x}}_2)/2$ und setze $\mathbf{u} := \mathbf{x} - \hat{\mathbf{x}}_1$,

$\mathbf{v} := \mathbf{x} - \hat{\mathbf{x}}_2$.

Aus der Definition von $\mathbf{z}$ folgt $\mathbf{x} - \mathbf{z} = \frac{\mathbf{u} + \mathbf{v}}{2}$, $\mathbf{z} - \hat{\mathbf{x}}_1 = \frac{\mathbf{u} - \mathbf{v}}{2}$,

und diese sind orthogonal: $\langle \mathbf{x} - \mathbf{z}, \mathbf{z} - \hat{\mathbf{x}}_1 \rangle = \frac{1}{4} \langle \mathbf{u} + \mathbf{v}, \mathbf{u} - \mathbf{v} \rangle = \frac{1}{4} (\|\mathbf{u}\|^2 - \|\mathbf{v}\|^2) = \frac{1}{4} (d^2 - d^2) = 0$



Damit also auch

$$\begin{aligned}\|\mathbf{x} - \hat{\mathbf{x}}_1\|^2 &= \|(\mathbf{x} - \mathbf{z}) + (\mathbf{z} - \hat{\mathbf{x}}_1)\|^2 \\ &= \|\mathbf{x} - \mathbf{z}\|^2 + \|\mathbf{z} - \hat{\mathbf{x}}_1\|^2 \\ &> \|\mathbf{x} - \mathbf{z}\|^2,\end{aligned}$$

$$[(\mathbf{x} - \mathbf{z}) \perp (\mathbf{z} - \hat{\mathbf{x}}_1)]$$

also: *Widersprüchlich* $d^2 > d^2 \implies$ Projektion eindeutig. ■

↔ zurück: „Konvexe Mengen: Projektionstheorem“

Konvexe Funktionen: Beweise I

(Beweis der $\mathcal{C}^2$ -Charakterisierung; (i) $\iff$ (ii))

- (i) $\implies$ (ii): Setze $t = (1 - \lambda)$. Dann gilt

$$\begin{aligned} & f(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) \leq \lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y}) \\ \iff & f(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) - f(\mathbf{x}) \leq (1 - \lambda) [f(\mathbf{y}) - f(\mathbf{x})] \\ \iff & f(\mathbf{y}) - f(\mathbf{x}) \geq \frac{f(\mathbf{x} + t(\mathbf{y} - \mathbf{x})) - f(\mathbf{x})}{t} \quad \text{für } t > 0. \quad \blacksquare \end{aligned}$$

(Für $t \rightarrow 0$ erhalten wir (ii).)

- (ii) $\implies$ (i): Sei $\mathbf{x}, \mathbf{y} \in \mathcal{X}$, $\lambda \in [0, 1]$ und definiere $\mathbf{z} = \lambda \mathbf{x} + (1 - \lambda) \mathbf{y}$. (ii) impliziert

$$\begin{aligned} \lambda f(\mathbf{x}) &\geq \lambda f(\mathbf{z}) + \lambda \nabla f(\mathbf{z})(\mathbf{x} - \mathbf{z}) \\ \text{und } (1 - \lambda) f(\mathbf{y}) &\geq (1 - \lambda) f(\mathbf{z}) + (1 - \lambda) \nabla f(\mathbf{z})(\mathbf{y} - \mathbf{z}). \end{aligned}$$

Summieren wir die beiden Ungleichungen, erhalten wir

$$\lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y}) \geq f(\mathbf{z}) + 0 = f(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}). \quad \blacksquare$$

Konvexe Funktionen: Beweise II

(Beweis der $\mathcal{C}^2$ -Charakterisierung, fortgesetzt; (ii) $\iff$ (iii))

- (ii) $\implies$ (iii): Sei $\mathbf{h} \in \mathbb{R}^n$ beliebig. Nach Definition der zweiten Ableitung gilt für $t \rightarrow 0$,

$$\begin{aligned} f(\mathbf{x} + t\mathbf{h}) - f(\mathbf{x}) - t\nabla f(\mathbf{x})\mathbf{h} - \frac{t^2}{2}\mathbf{h}^\top \mathbf{H}_f(\mathbf{x})\mathbf{h} &= o(t^2) \quad [\text{Taylorpolynom}] \\ \iff \mathbf{h}^\top \mathbf{H}_f(\mathbf{x})\mathbf{h} &= 2 \frac{f(\mathbf{x} + t\mathbf{h}) - f(\mathbf{x}) - t\nabla f(\mathbf{x})\mathbf{h}}{t^2} + o(1) \\ &\geq 0 + o(1) \rightarrow 0. \end{aligned}$$

(Da die linke Seite nicht von t abhängt, folgt (iii).)

- (iii) $\implies$ (ii): *Taylor's Theorem*: für $f \in \mathcal{C}^2$ und jedes $\mathbf{x}, \mathbf{y} \in \mathcal{X} \exists t \in (0, 1)$, so dass:

$$\begin{aligned} f(\mathbf{y}) &= f(\mathbf{x}) + \nabla f(\mathbf{x})(\mathbf{y} - \mathbf{x}) + \frac{1}{2}(\mathbf{y} - \mathbf{x})^\top \mathbf{H}_f(\mathbf{x} + t(\mathbf{y} - \mathbf{x}))(\mathbf{y} - \mathbf{x}) \\ &\geq f(\mathbf{x}) + \nabla f(\mathbf{x})(\mathbf{y} - \mathbf{x}) \end{aligned}$$

Konvexe Funktionen: Beweis der Eigenschaften

(Proposition Summe/Skalierung/Maximum/Grenzwert)

Seien $\mathbf{x}, \mathbf{y} \in \mathcal{X}$, $\lambda \in [0, 1]$ und $\mathbf{z} := \lambda \mathbf{x} + (1 - \lambda) \mathbf{y}$.

(ii) Skalierung: Für $f = cf_1$ mit $c \geq 0$ gilt

$$\begin{aligned} f(\mathbf{z}) &= c f_1(\mathbf{z}) \\ &\leq c [\lambda f_1(\mathbf{x}) + (1 - \lambda) f_1(\mathbf{y})] \\ &= \lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y}). \end{aligned}$$

Die Ungleichung folgt aus der Konvexität von f_1 ; Multiplikation mit $c \geq 0$ erhält ihre Richtung.

(iv) Grenzwert: Wegen der Konvexität von f_n gilt für jedes n

$$f_n(\mathbf{z}) \leq \lambda f_n(\mathbf{x}) + (1 - \lambda) f_n(\mathbf{y}).$$

Da alle beteiligten Folgen konvergieren, können wir den Grenzwert bilden:

$$\begin{aligned} f(\mathbf{z}) &= \lim_{n \rightarrow \infty} f_n(\mathbf{z}) \\ &\leq \lambda \lim_{n \rightarrow \infty} f_n(\mathbf{x}) + (1 - \lambda) \lim_{n \rightarrow \infty} f_n(\mathbf{y}) \\ &= \lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y}). \end{aligned}$$

Optimierung I: Grundlagen & Verfahren erster Ordnung

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)

Material: Thomas Nagler

LMU München

Opener-Quiz

Opener-Quiz

Beim Reinkommen: QR-Code scannen und die 2 Einstiegsfragen beantworten.

Frage 1: Sei $[a, b] \subset \mathbb{R}$ und $f: [a, b] \rightarrow \mathbb{R}$. Wir wollen $f(x) = 0$ lösen. Welche der folgenden Aussagen ist wahr?

1. Falls f stetig ist, hat $f(x) = 0$ genau eine Lösung.
2. Falls $f(a) < 0$ und $f(b) > 0$, hat $f(x) = 0$ mindestens eine Lösung.
3. Falls f streng monoton ist, hat $f(x) = 0$ höchstens eine Lösung.

Frage 2: Sei $f: \mathbb{R}^n \rightarrow \mathbb{R}$ konvex und differenzierbar mit $\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$. Was folgt?

1. $\mathbf{x}^*$ ist ein lokales, aber nicht unbedingt globales Minimum.
2. $\mathbf{x}^*$ ist ein globales Minimum.
3. $\mathbf{x}^*$ kann auch ein Sattelpunkt sein.

Verwendete Vorkenntnisse

- **Gradient** $\nabla f(\mathbf{x})$ für $f : \mathbb{R}^n \rightarrow \mathbb{R}$; $\nabla f(\mathbf{x})^\top$ ist die Richtung des steilsten *Anstiegs*
- **Hesse-Matrix** $\mathbf{H}_f(\mathbf{x})$ als Matrix zweiter Ableitungen: *Krümmung*
- **Konvexität von Mengen und Funktionen**
- **Analysis:**
 - Notwendige Bedingung für Extrema: $f'(\mathbf{x}^\star) = 0$
 - Hinreichende Bedingung für Minima: $f'(\mathbf{x}^\star) = 0$ **und** $f''(\mathbf{x}^\star) > 0$
 - Zwischenwertsatz
- **Lineare Algebra:**
 - **positiv definite** Matrizen
 - **Konditionszahl** einer Matrix

Motivation & Einführung

Einführung

- Ein **nichtlineares Gleichungssystem** für $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ hat die Form

$$f(\mathbf{x}) = \mathbf{0}$$

→ Wie finden wir Lösungen $\mathbf{x}^*$?

- Ein **unbeschränktes Optimierungsproblem** für $f: \mathbb{R}^n \rightarrow \mathbb{R}$ hat die Form

$$\mathbf{x}^* = \arg \min_{\mathbf{x}} f(\mathbf{x}) \quad \text{bzw.} \quad \mathbf{x}^* = \arg \max_{\mathbf{x}} f(\mathbf{x}),$$

⇒ Falls f **differenzierbar** ist, muss $\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$ gelten.

- Ein **beschränktes Optimierungsproblem** stellt zusätzliche Bedingungen an Lösungen $\mathbf{x}^*$:

$$\mathbf{x}^* = \arg \min_{\mathbf{x} \in S} f(\mathbf{x})$$

mit **Nebenbedingungen** $S \subset \mathbb{R}^n$, z.B:

- $S = \{\mathbf{x} \in \mathbb{R}^n: g_i(\mathbf{x}) = 0, i = 1, \dots, m\}$ (Gleichungsnebenbedingungen) und/oder
 - $S = \{\mathbf{x} \in \mathbb{R}^n: h_j(\mathbf{x}) \leq 0, j = 1, \dots, p\}$ (Ungleichungsnebenbedingungen) → Kapitel *Optimierung II*
- Alle diese Probleme werden meist **iterativ** gelöst.

Warum Optimierung?

Statistik und ML bestehen grōßteils aus **Optimierungsproblemen**:

- **Maximum-Likelihood-Schätzung** für i.i.d. $y_1, \dots, y_n$ mit Dichte $f_Y(y \mid \theta)$:

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \left(\sum_{i=1}^n \log f_Y(y_i \mid \theta) \right)$$

- **Supervised Learning** mit Verlustfunktion $L(y, \hat{y})$ und Vorhersagen $\hat{y}_i := p_{\theta}(\mathbf{x}_i)$ aus Features $\mathbf{x}_i$:

$$\hat{\theta} = \arg \min_{\theta} \left(\sum_{i=1}^n L(y_i, p_{\theta}(\mathbf{x}_i)) \right)$$

z.B. **Lineare Regression**: $\hat{\beta} = \arg \min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|_2^2$ oder **Neuronale Netze**.

- **Regularisierte Regression (Ridge: $p = 2$, Lasso: $p = 1$):**

$$\hat{\beta} = \arg \min_{\beta} \left(\|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_p^p \right)$$

kann als **beschränktes Optimierungsproblem** mit $S = \{\beta : \|\beta\|_p^p \leq c(\lambda)\}$ formuliert werden:

$$\arg \min_{\beta: \|\beta\|_p^p \leq c(\lambda)} \|\mathbf{y} - \mathbf{X}\beta\|_2^2$$

Nichtlineare Gleichungssysteme

Lösbarkeit

Theorem

Sei $f: [a, b] \rightarrow \mathbb{R}$ streng monoton, stetig und $f(a) \cdot f(b) \leq 0$.

Dann hat die Gleichung $f(x) = 0$ eine eindeutige Lösung.

Beweis.

- Weil f streng monoton ist, kann es höchstens eine Lösung geben.
- Falls $f(a) \cdot f(b) = 0$, ist $f(a) = 0$ oder $f(b) = 0$.
- $f(a) \cdot f(b) < 0$ bedeutet, dass $f(a)$ und $f(b)$ verschiedene Vorzeichen haben.
- Nehmen wir o.B.d.A. an, dass $f(a) < 0 < f(b)$.

Weil f stetig ist, gibt es nach dem Zwischenwertsatz ein $x \in (a, b)$, sodass $f(x) = 0$. ■

Anmerkung:

Diese Bedingungen sind *hinreichend*, aber nicht *notwendig* für Lösbarkeit.

Bisektionsverfahren

Ziel:

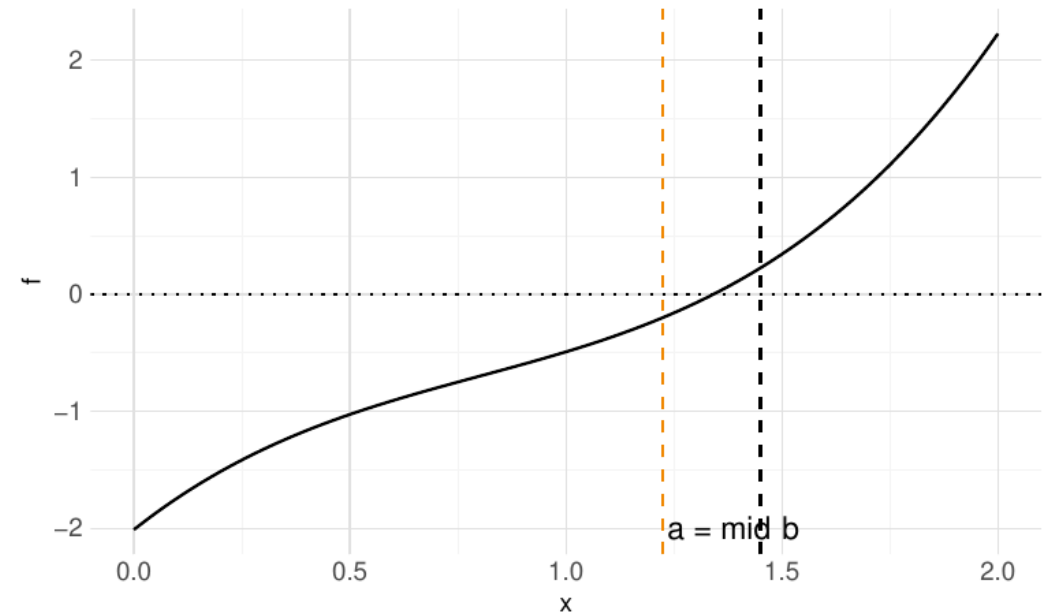
Löse $f(x) = 0$, $f: \mathbb{R} \rightarrow \mathbb{R}$.

Bisektionsverfahren

- Inputs:

- **Stetige** Funktion f
- Zwei Punkte a, b mit $f(a) \cdot f(b) < 0$ (also: unterschiedliche Vorzeichen)
- Ziel-Genauigkeit ϵ : $|x^* - x^{(k)}| < \epsilon$

```
1 bisect <- function(f, a, b, eps) {  
2   while (b - a > eps) {  
3     mid <- (a + b) / 2  
4     if (f(a) * f(mid) <= 0) b <- mid else a <- mid  
5   }  
6   return((a + b) / 2)  
7 }
```



Kommentare:

- Kann auch verwendet werden, um die **Umkehrfunktion** $f^{-1}(y)$ auszuwerten: Löse $f(x) - y = 0$.
- Der Algorithmus benötigt $\lceil \log_2 \left(\frac{b-a}{\epsilon} \right) \rceil$ Schritte.
- Eignet sich nur für **univariate** Funktionen.

Newton-Raphson-Verfahren

Ziel:

Löse $f(x) = 0$ für $f: \mathbb{R} \rightarrow \mathbb{R}$ differenzierbar.

Idee:

Approximiere f lokal durch eine Tangente und finde deren Nullstelle.

Taylor-Entwicklung um $x^{(k)}$:

$$f(x) \approx f(x^{(k)}) + f'(x^{(k)})(x - x^{(k)}) \stackrel{!}{=} 0$$

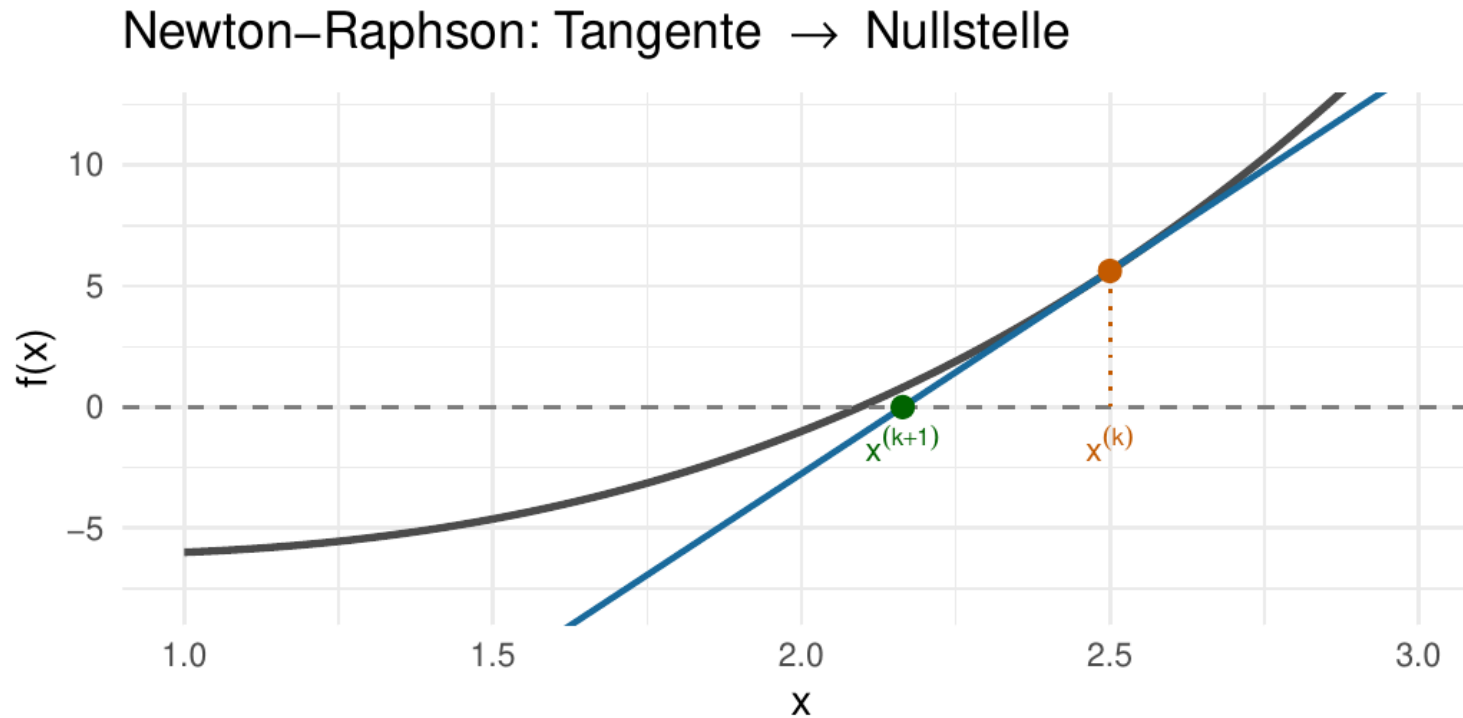
Newton-Raphson-Iteration

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}$$

Geometrisch:

Schnittpunkt der Tangente an f im Punkt $x^{(k)}$ mit der x -Achse.

Newton-Raphson: Visualisierung



- **Konvergenz:** Quadratisch nahe einer *einfachen* Nullstelle ($f'(x^*) \neq 0$): $e_{k+1} \approx C \cdot e_k^2$ für $e_k := |x^{(k)} - x^*|$ mit $C = \frac{|f''(x^*)|}{2|f'(x^*)|}$
- **Verbindung zur Optimierung:**
Newton für $\min f(x)$ ist Newton-Raphson für $\nabla f(x) = \mathbf{0}^\top$!
(→ Kapitel *Optimierung II*)
- Für multivariate Probleme heute: **Gradientenverfahren**.

Das Optimierungsproblem

Das Optimierungsproblem

Ziel:

Finde das Minimum von $f(\boldsymbol{x})$, $f: \mathbb{R}^n \rightarrow \mathbb{R}$.

$$\boldsymbol{x}^* = \arg \min_{\boldsymbol{x} \in \mathbb{R}^n} f(\boldsymbol{x})$$

Wichtige Beobachtungen:

- Maximierung ist äquivalent: $\max f(\boldsymbol{x}) = -\min(-f(\boldsymbol{x}))$
- Ist f differenzierbar, muss am Minimum gelten:

$$\nabla f(\boldsymbol{x}^*) = \mathbf{0}^\top$$

$\implies$ Optimierung $\simeq$ Lösen von Gleichungssystemen!

- **Problem:**

$\nabla f(\boldsymbol{x}) = \mathbf{0}^\top$ meist nicht analytisch lösbar

$\implies$ *iterative Verfahren* nötig

Konvexe Funktionen in der Optimierung

- Konvexe Funktionen ergeben *einfachere* Optimierungsprobleme:

Proposition

Für jede differenzierbare, konvexe Funktion $f: \mathbb{R}^n \rightarrow \mathbb{R}$ sind die folgenden Aussagen äquivalent:

- i. $\mathbf{x}^*$ ist ein globales Minimum von f .
- ii. $\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$.

- Das bedeutet:
 - Jeder kritische Punkt ist ein lokales Minimum.
 - Jedes lokale Minimum ist ein globales Minimum.

Wenn wir ein konvexes Optimierungsproblem lösen, finden wir garantiert ein *globales* Optimum!

(Beweis → *Anhang*)

Strikt konvexe Funktionen

- Um die **Eindeutigkeit** von Minima zu garantieren, benötigen wir eine stärkere Form von Konvexität.

Def.: Strikte Konvexität

Sei $\mathcal{X}$ konvex. Eine Funktion $f: \mathcal{X} \rightarrow \mathbb{R}$ heißt **strikt konvex**, falls für alle $\mathbf{x}, \mathbf{y} \in \mathcal{X}$ mit $\mathbf{x} \neq \mathbf{y}$ und alle $\lambda \in (0, 1)$ gilt

$$f(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) < \lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y}).$$

Proposition

Sei $f: \mathbb{R}^n \rightarrow \mathbb{R}$ *strikt konvex*.

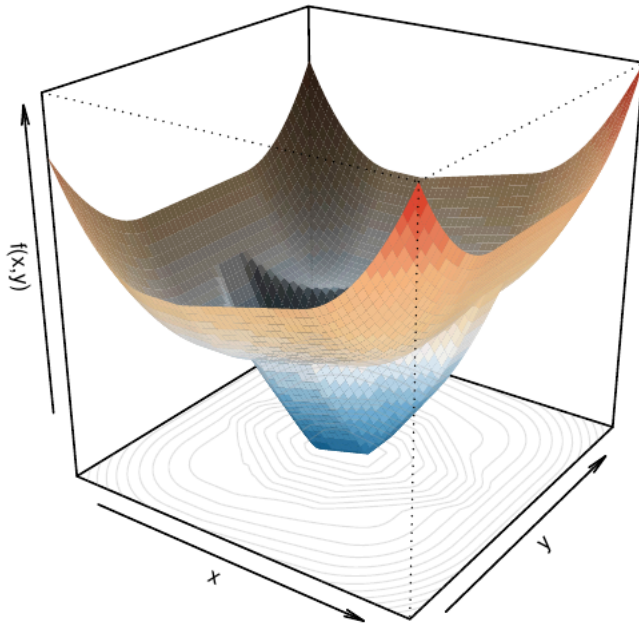
Dann hat f höchstens ein Minimum.

Wenn wir ein *strikt* konvexes Optimierungsproblem lösen, finden wir garantiert das *eindeutige globale Optimum*!

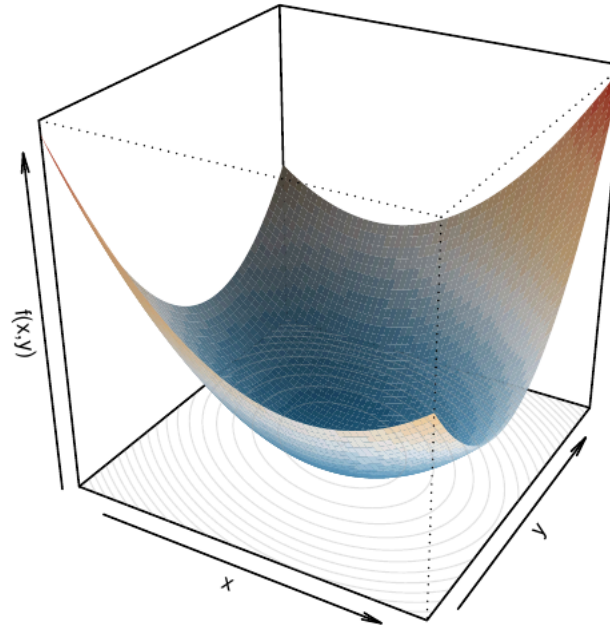
- Gilt auch mit konvexer zulässiger Menge;
Spezialfall **Projektionstheorem**: quadrierte Distanz $\|\mathbf{x} - \mathbf{y}\|^2$ ist strikt konvex in $\mathbf{y}$.

Optimierungslandschaften

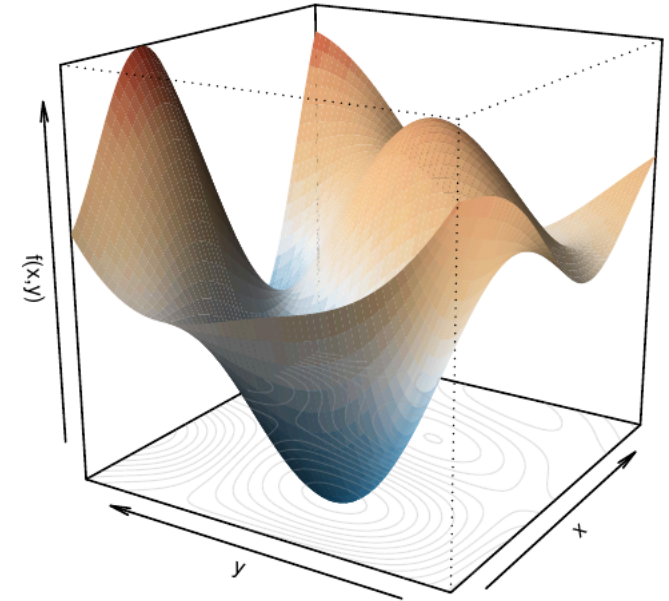
Konvex mit Plateaus



Strikt konvex



Nicht konvex



- Links: konvex mit Plateaus – *Minimumsregion*, flache Teilstücke
- Mitte: strikt konvex – “Schüssel” mit eindeutigen Minimum.
- Rechts: nicht konvex – mehrere Täler und lokale Minima ohne Garantie für Globalität.

(Non-)Konvexe Optimierung in ML & Statistik

1. **Strikt konvexe Probleme** haben eine einzige *eindeutige* Lösung, z.B.

- KQ-Probleme $\min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|_2^2$ für $\mathbf{X}$ mit vollem Spaltenrang ($\mathbf{H} = 2\mathbf{X}^\top \mathbf{X} \succ 0$)
- Ridge Regression $\min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_2^2$, strikt konvex für $\lambda > 0$ ($\rightarrow$ Übung) & Elastic Net
- genereller: ML-Schätzung für GLMs mit kanonischem Link (konkave Log-Likelihood $\implies$ neg. LL konvex)
- Support Vector Machines: konvexer Hinge-Loss $L(y, \hat{y}) = \max(0, 1 - y\hat{y})$ + strikt konvexe L_2 -Regularisierung (strikt konvex nur in $\mathbf{w}$ – der Achsenabschnitt b ist i.A. *nicht* eindeutig)
- Graph LASSO (regularisierte Kovarianzschätzung)

2. **Konvexe Probleme** (nicht *strikt*!) haben manchmal nur eine *Lösungsmenge*, z.B. LASSO, Quantilregression, KQ-Probleme ohne vollen Spaltenrang

3. **Nicht-konvexe Probleme** mit (lokalen Minima und) **Sattelpunkten** $\implies$ verschiedene Initialisierungen/Algorithmen $\rightarrow$ (evtl.) verschiedene Lösungen, zum Beispiel:

- **Neuronale Netze**
- Variablenselektion (L_0 , *Best Subset*, ...)
- **Clusteranalysen** (z.B. *k-means*, Mischverteilungsmodelle) - **Bäume** (CART, Random Forest, XGBoost, ...)
- Embeddings (tSNE, UMAP, [word2vec](#), ...)
- **Matrix-Completion**
- **Latente Variablen** (GLMM, SEM, Faktoranalysen, ...)
- **Hyperparameter-Tuning**: meist nichtmal stetig $\rightarrow$ Gitter-, Zufalls- oder Bayes-Optimierung

Sattelpunkte

Bei **nicht-konvexen** Funktionen: $\nabla f(\mathbf{x}) = \mathbf{0}^\top$ bedeutet nicht unbedingt Minimum!

Definition: Sattelpunkt

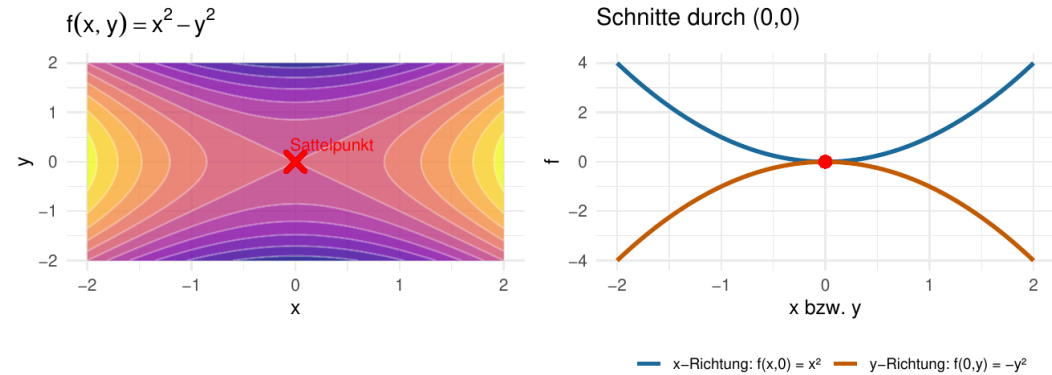
Ein *kritischer Punkt* $\mathbf{x}^*$ mit $\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$ heißt **Sattelpunkt**, wenn $\mathbf{H}_f(\mathbf{x}^*)$ *indefinit* ist (sowohl positive als auch negative Eigenwerte).

Beispiel $f(x, y) = x^2 - y^2$:

- $\nabla f = (2x, -2y) = \mathbf{0}^\top$ bei $(0, 0)^\top$ ✓
- $\mathbf{H}_f = \begin{pmatrix} 2 & 0 \\ 0 & -2 \end{pmatrix}$: Eigenwerte $2 > 0$, $-2 < 0$
 $\implies$ **indefinit**
- $(0, 0)^\top$ ist Minimum in x -Richtung, Maximum in y -Richtung

Konsequenzen:

- In vielen hochdimensionalen, nicht-konvexen Modellen treten Sattelpunkte auf.
- In hohen Dimensionen sind sie i.A. häufiger als lokale Minima.



Optimalitätsbedingungen: Zusammenfassung

Für $f : \mathbb{R}^n \rightarrow \mathbb{R}$ zweimal stetig differenzierbar:

Notwendige Bedingung (1. Ordnung)

$\mathbf{x}^*$ lokales Minimum $\implies \nabla f(\mathbf{x}^*) = \mathbf{0}^\top$ ($\mathbf{x}^*$ ist *stationärer Punkt*)

Hinreichende Bedingung (2. Ordnung)

$\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$ und $\mathbf{H}_f(\mathbf{x}^*) \succ 0 \implies \mathbf{x}^*$ ist lokales Minimum

Klassifikation stationärer Punkte ($\nabla f = \mathbf{0}^\top$):

$\mathbf{H}_f(\mathbf{x}^*)$	Punkt ist...
$\succ 0$ (positiv definit)	Striktes lokales Minimum
$\prec 0$ (negativ definit)	Striktes lokales Maximum
indefinit	Sattelpunkt
semidefinit, aber nicht definit	Unklar, weitere Analyse nötig

⚠ So wichtig:

Für **konvexe Funktionen** ist jedes *lokale* Minimum auch **global** (keine *Sattelpunkte*)!

Iterative Optimierungsverfahren

Taxonomie: Optimierungsverfahren

Klassifikation nach verwendeter **Ableitungsinformation**:

Ordnung	Verwendet	Beispiele
0-te Ordnung	nur $f(x)$	<i>Nelder-Mead</i> , Gittersuche
1-te Ordnung	$f, \nabla f$	<i>Gradient Descent</i> , <i>Momentum GD</i> , <i>SGD</i>
2-te Ordnung	$f, \nabla f, \mathbf{H}_f$	<i>Newton</i> (→ Kapitel <i>Optimierung II</i>)

Trade-off:

- 0-te Ordnung: robust, keine Ableitungen nötig, aber langsam
- Höhere Ordnung → schnellere Konvergenz, aber teurer pro Schritt
- In der Praxis oft: Superlineare Verfahren (z.B. *Quasi-Newton (BFGS)*, → Kapitel *Optimierung II*) als Kompromiss zwischen 1. und 2. Ordnung

Nelder-Mead: Ableitungsfreie Optimierung

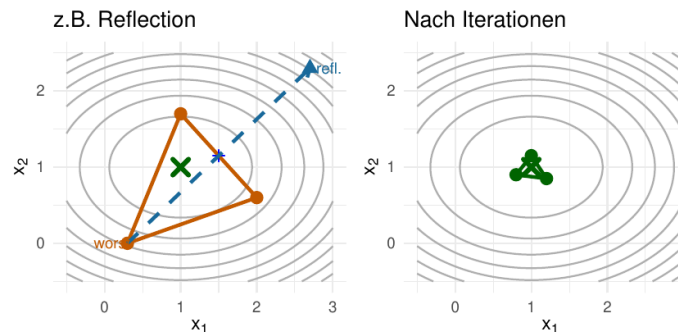
0-te Ordnung: Verwendet nur Funktionswerte $f(\mathbf{x})$, keine Ableitungen.

Simplex-Methode

- Arbeitet mit einem *Simplex*: $n + 1$ Punkte in $\mathbb{R}^n$ (Dreieck in 2D, Tetraeder in 3D)
- Iterativ: ersetze schlechtesten Punkt durch besseren

Simplex-Operationen: (*Interaktive Demo*)

1. **Reflection:** Spiegle schlechtesten Punkt am Schwerpunkt der anderen
2. **Expansion:** Gehe weiter in vielversprechende Richtung
3. **Contraction:** Ziehe zusammen wenn Reflection nicht hilft
4. **Shrink:** Verkleinere gesamten Simplex zum besten Punkt hin



Nelder-Mead: Eigenschaften

Wann Nelder-Mead verwenden?

Situation	Geeignet?
Ableitungen nicht verfügbar	✓
Niedrige Dimension ($n \leq 10$)	✓
Nicht-glatte Zielfunktion	✓
Hohe Dimension ($n > 50$)	✗ (zu langsam)
Präzise Konvergenz nötig	✗ (i.A. keine Garantien)

Komplexität:

$O(n)$ Funktionsauswertungen pro Iteration, meist *viele* Iterationen nötig.

Fazit:

Robuste Methode für “schwierige” Probleme ohne Ableitungen, aber langsam. Oft als **Startpunktsuche** vor gradientenbasierten Methoden verwendet.

Gradientenabstieg

Gradientenabstieg: Die Kernidee

Der Gradient zeigt bergauf:

⇒ Der negative Gradient zeigt bergab!

Gradientenabstieg (Gradient Descent / GD)

Die Iterationsvorschrift ist gegeben durch:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \gamma \nabla f(\mathbf{x}^{(k)})^\top$$

- $\gamma > 0$: **Schrittweite** (*learning rate*)
- $-\nabla f(\mathbf{x}^{(k)})^\top$: Richtung des steilsten Abstiegs an Stelle $\mathbf{x}^{(k)}$

Intuition: Gehe immer in die Richtung, in der es am steilsten bergab geht.

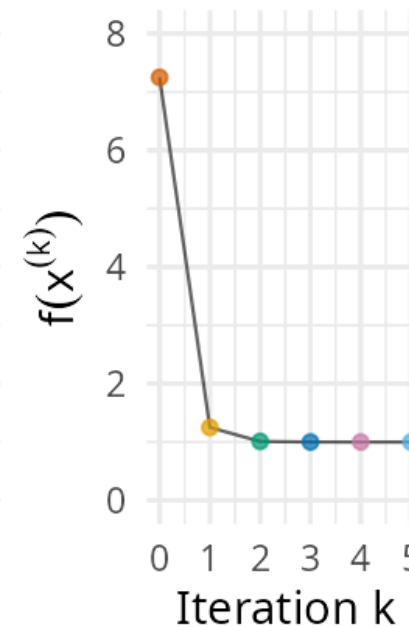
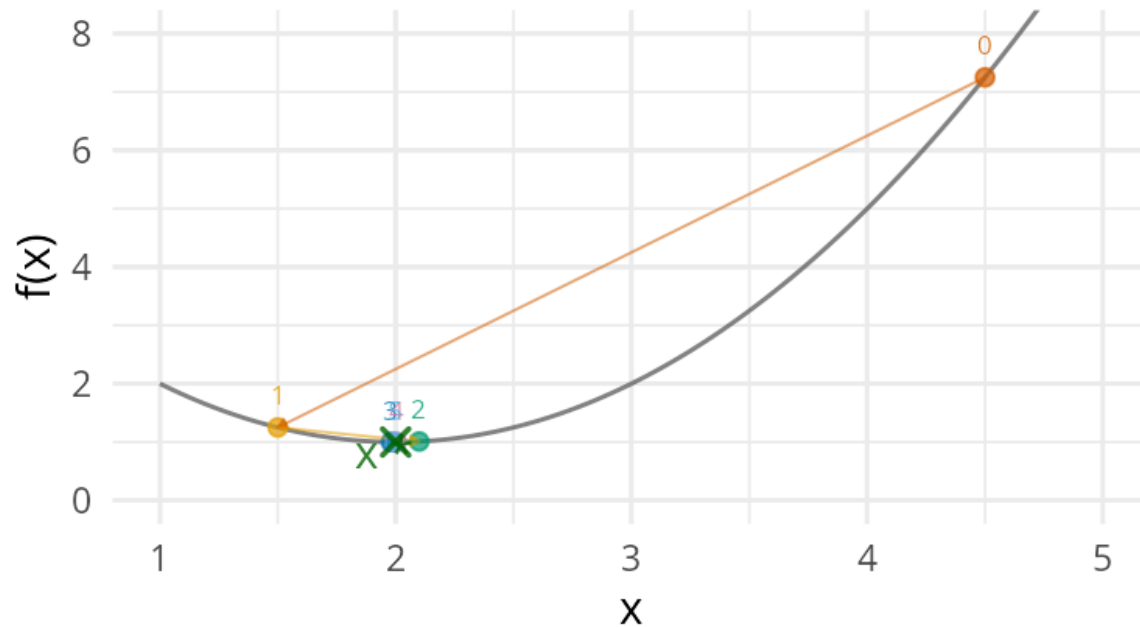
$-\nabla f(\mathbf{x})^\top$ ist für *jedes* differenzierbare f eine Abstiegsrichtung, solange $\nabla f(\mathbf{x}) \neq \mathbf{0}^\top$ – sie zeigt aber i.A. *nicht direkt* auf das Minimum!

Falls **Konvexität** gilt, ist jeder stationäre Punkt dann bereits *global optimal*.

Bemerkung:

Gradientenabstieg: $f : \mathbb{R} \rightarrow \mathbb{R}$

Gradient Descent für $f(x) = (x - 2)^2 + 1$, $\gamma = 0.6$



k	$x^{(k)}$	$f'(x^{(k)})$
0	4.500	5.000
1	1.500	-1.000
2	2.100	0.200
3	1.980	-0.040
4	2.004	0.008
5	1.999	-0.002

Idee:

Starte bei $x^{(0)}$ und verbessere schrittweise:

$$x^{(k+1)} = x^{(k)} - \gamma f'(x^{(k)})$$

Beispiel Gradientenabstieg: Schritt für Schritt

Problem:

Minimiere $f(x) = (x - 2)^2 + 1$ mit Startwert $x^{(0)} = 4.5$, Schrittweite $\gamma = 0.6$

Ableitung:

$$f'(x) = 2x - 4$$

Iterationen:

$$x^{(0)} = 4.5$$

$$x^{(1)} = x^{(0)} - 0.6 \cdot f'(x^{(0)}) = 4.5 - 0.6 \cdot (2 \cdot 4.5 - 4) = 4.5 - 3 = 1.5$$

$$x^{(2)} = x^{(1)} - 0.6 \cdot f'(x^{(1)}) = 1.5 - 0.6 \cdot (2 \cdot 1.5 - 4) = 1.5 + 0.6 = 2.1$$

$$x^{(3)} = 2.1 - 0.6 \cdot (2 \cdot 2.1 - 4) = 1.98$$

$\vdots$

Konvergenz:

$$x^{(k)} \rightarrow 2 \text{ (analytisches Minimum: } f'(x^*) = 0 \implies x^* = 2)$$

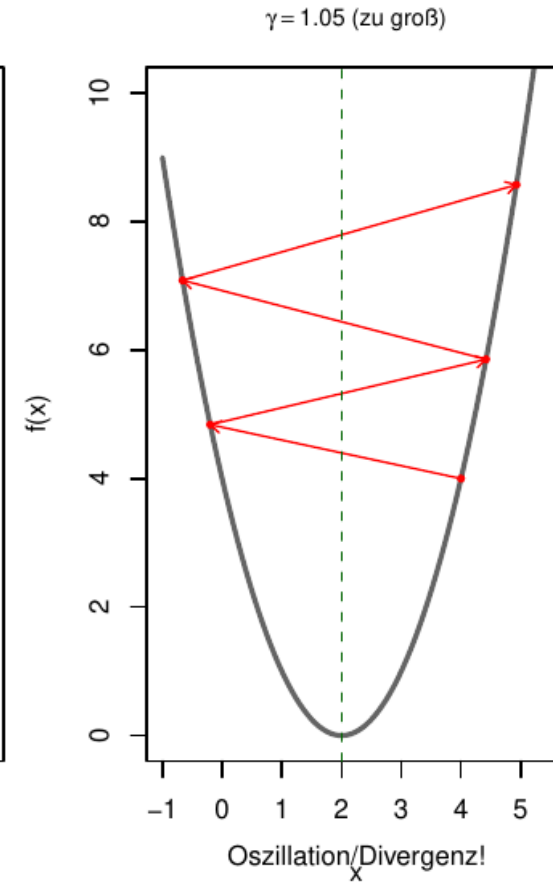
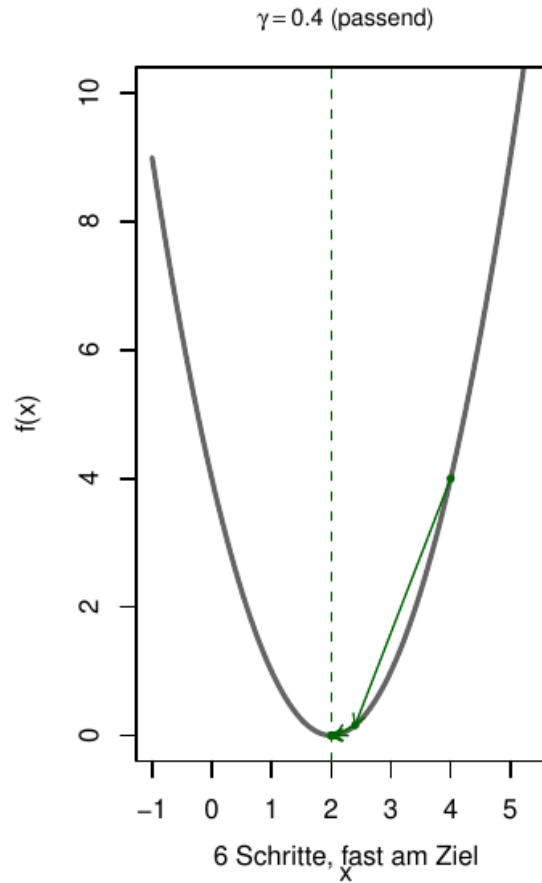
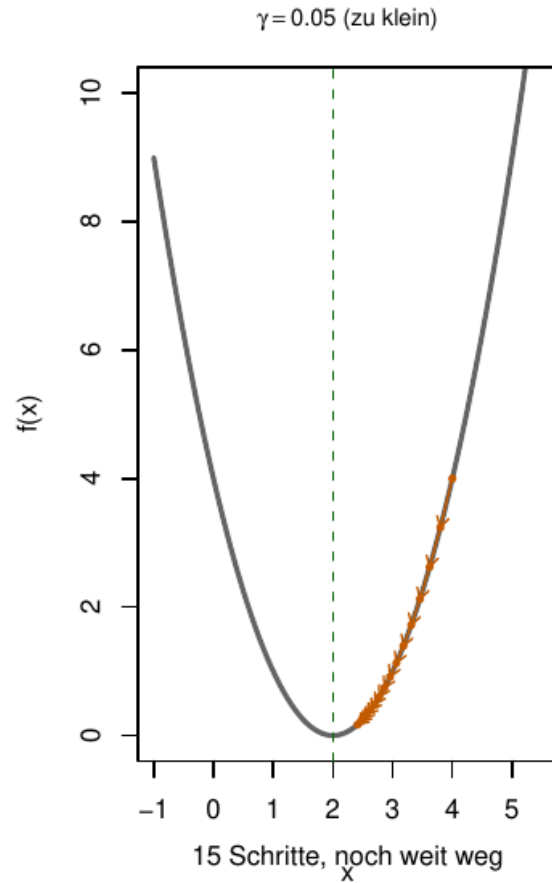
Quiz: Gradient Descent

Sei $f(\mathbf{x}) = x_1^2 + x_2^2$ und $\mathbf{x}^{(0)} = (4, 3)^\top$ mit Schrittweite $\gamma = 0.25$.

Frage: Wo liegt $\mathbf{x}^{(1)}$ nach einem GD-Schritt?

1. $(2, 1.5)^\top$
2. $(0, 0)^\top$
3. $(3, 2)^\top$
4. $(8, 6)^\top$

Die Rolle der Schrittweite γ

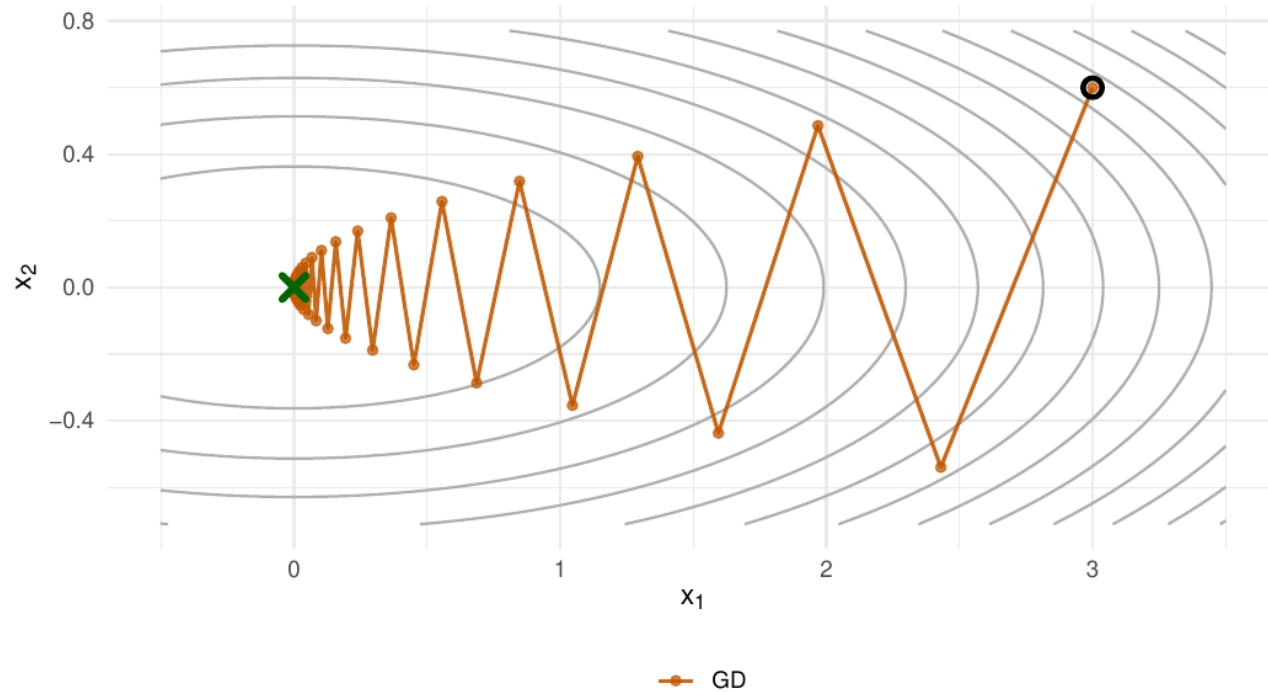


Dilemma:

γ muss

- klein genug sein für *Konvergenz*,
- aber groß genug für *Effizienz*...

Problem: Zick-Zack bei schlechter Kondition



Bei **hoher Konditionszahl** $\kappa(\mathbf{H}_f)$ oszilliert GD oft stark.

- $\kappa(\mathbf{H}_f)$ ist groß, wenn die Krümmung von f in verschiedenen Richtungen stark unterschiedlich ist (z.B. “Canyons”)
- Wann und wie schnell konvergiert GD also überhaupt? → jetzt
- **Interaktiv ausprobieren:** Shiny-App: Gradient Descent

↔ zurück: „Konvergenzrate & Kondition“

Konvergenz: Lipschitz-stetiger Gradient

Def.: Lipschitz-Stetigkeit

Eine Funktion $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ heißt **Lipschitz-stetig** mit Konstante $L > 0$, falls:

$$\|f(\mathbf{x}) - f(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\| \quad \forall \mathbf{x}, \mathbf{y}$$

Intuition:

Funktionswerte ändern sich *höchstens* L -mal so schnell wie Funktionsargumente. Je größer L , desto “gezackter” kann f sein $\implies$ kleinere Schrittweiten γ nötig.

- **Lipschitz-stetiger Gradient** ∇f :
 $\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \leq L\|\mathbf{x} - \mathbf{y}\|$
 $\rightarrow$ Steigung von f ändert sich nicht zu abrupt.
- Äquivalent (für zweimal differenzierbares f):
 $\|\mathbf{H}_f(\mathbf{x})\|_2 \leq L \quad \forall \mathbf{x}$ mit $L = \sup_{\mathbf{x}} \max_i |\lambda_i(\mathbf{H}_f(\mathbf{x}))|$
 $\rightarrow$ Krümmung von f ist beschränkt
- L heißt auch **Lipschitz-Konstante** von f (oder ∇f)

Konvergenzrate bei Konvexität

Konvergenzrate: Gradientenabstieg

Für *konvexes* f mit *Lipschitz-stetigem* Gradient ∇f und *Lipschitz-Konstante* L , das sein Minimum in $\mathbf{x}^*$ annimmt,
gilt für *Schrittweite* $\gamma \leq \frac{1}{L}$ und alle Iterationen $k \geq 1$:

$$f(\mathbf{x}^{(k)}) - f(\mathbf{x}^*) \leq \frac{\|\mathbf{x}^{(0)} - \mathbf{x}^*\|^2}{2\gamma k}$$

Interpretation:

Der *Funktionswertabstand* $f(\mathbf{x}^{(k)}) - f(\mathbf{x}^*)$ sinkt mit Rate $O(1/k)$
 $\implies$ es braucht $k \sim 1/\varepsilon$ Schritte, bis er $< \varepsilon$ ist (... relativ langsam!)

Achtung:

Über den Abstand $\|\mathbf{x}^{(k)} - \mathbf{x}^*\|$ sagt diese Schranke *nichts*.

Starke Konvexität: lineare Konvergenzrate

Def.: Starke Konvexität

Eine *zweimal differenzierbare* Funktion f heißt **stark konvex** (μ -strongly convex) mit Parameter $\mu > 0$, falls:

$$\mathbf{H}_f(\mathbf{x}) - \mu \mathbf{I} \succeq 0 \quad \forall \mathbf{x}$$

(ableitungsfrei äquivalent: $f(\mathbf{x}) - \frac{\mu}{2} \|\mathbf{x}\|^2$ ist konvex)

- $\mu = \inf_{\mathbf{x}} \lambda_{\min}(\mathbf{H}_f(\mathbf{x}))$: minimale (positive!) Krümmung von f

Konvergenzrate: Stark konvexer Gradientenabstieg

Für *stark konvexes* f mit Parameter $\mu > 0$ und *Lipschitz-stetigem* Gradienten mit *Lipschitz-Konstante* L gilt für $\gamma = \frac{1}{L}$:

$$f(\mathbf{x}^{(k)}) - f(\mathbf{x}^*) \leq \rho^k \cdot \left(f(\mathbf{x}^{(0)}) - f(\mathbf{x}^*) \right), \quad \rho = 1 - \frac{\mu}{L} < 1$$

$\implies$ schnellere Konvergenz bei starker Konvexität

Konvergenzrate & Kondition

Verallgemeinerte **Kondition** für Funktionen:

- $\rho = \frac{\kappa_f - 1}{\kappa_f}$ mit Konditionszahl $\kappa_f = \frac{\sup_{\mathbf{x}} \lambda_{\max}(\mathbf{H}_f(\mathbf{x}))}{\inf_{\mathbf{x}} \lambda_{\min}(\mathbf{H}_f(\mathbf{x}))} = \frac{L}{\mu}$
- Großes $\kappa_f \implies \rho \rightarrow 1 \implies$ langsame Konvergenz:
Zick-Zack-Trajektorien durch “Canyons” mit anisotropischer Krümmung
($\rightarrow$ genau das vorhin gesehene **Zick-Zack-Problem!**)
- Für quadratische Funktionen $f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top \mathbf{A} \mathbf{x}$ mit $\mathbf{A} \succ 0$ ist $\mathbf{H}_f = \mathbf{A}$,
also $\kappa_f = \kappa(\mathbf{A})$
 $\rightarrow$ selbe **Konditionszahl** wie in der numerischen linearen Algebra!

Abbruchkriterien (Stopping Criteria)

Wann beenden wir die Iteration $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \gamma \nabla f(\mathbf{x}^{(k)})^\top$?

Typische Abbruchkriterien

1. *Gradient klein:* $\|\nabla f(\mathbf{x}^{(k)})\| < \varepsilon_{\text{grad}}$
2. *Funktionswert stagniert:* $\frac{|f(\mathbf{x}^{(k+1)}) - f(\mathbf{x}^{(k)})|}{|f(\mathbf{x}^{(k)})|} < \varepsilon_f$
3. *Iterationen ausgeschöpft:* $k > k_{\text{max}}$

- Üblicherweise kombiniert man mehrere Kriterien
- Typische Werte: $\varepsilon_{\text{grad}} \approx 10^{-6}$ bis 10^{-8} , $k_{\text{max}} \approx 1000$ bis 10000

Achtung:

Kriterien können bei nicht stark konvexen oder schlecht konditionierten Problemen versagen — $\|\nabla f\|$ oder $|f^{(k+1)} - f^{(k)}|$ kann beliebig klein werden, obwohl $\mathbf{x}^{(k)}$ noch weit vom Optimum entfernt ist!

Line Search: Adaptive Schrittweite

Problem:

Eine feste Schrittweite γ ist oft suboptimal.

Idee:

Wähle $\gamma^{(k)}$ in jedem Schritt so, dass f “genug” sinkt.

Backtracking Line Search (Armijo)

Gegeben: Richtung $\mathbf{d} = -\nabla f(\mathbf{x}^{(k)})^\top$, Parameter $c \in (0, 1)$, $\rho \in (0, 1)$

1. Starte mit $\gamma = 1$

2. **while** $f(\mathbf{x}^{(k)} + \gamma \mathbf{d}) > f(\mathbf{x}^{(k)}) + c\gamma \nabla f(\mathbf{x}^{(k)}) \mathbf{d}$:

- $\gamma \leftarrow \rho\gamma$ (verkleinere Schrittweite)

3. **return** γ

Armijo-Bedingung:

Fordert “genügend Abstieg” — Funktionswert muss proportional zum erwarteten Abstieg sinken.

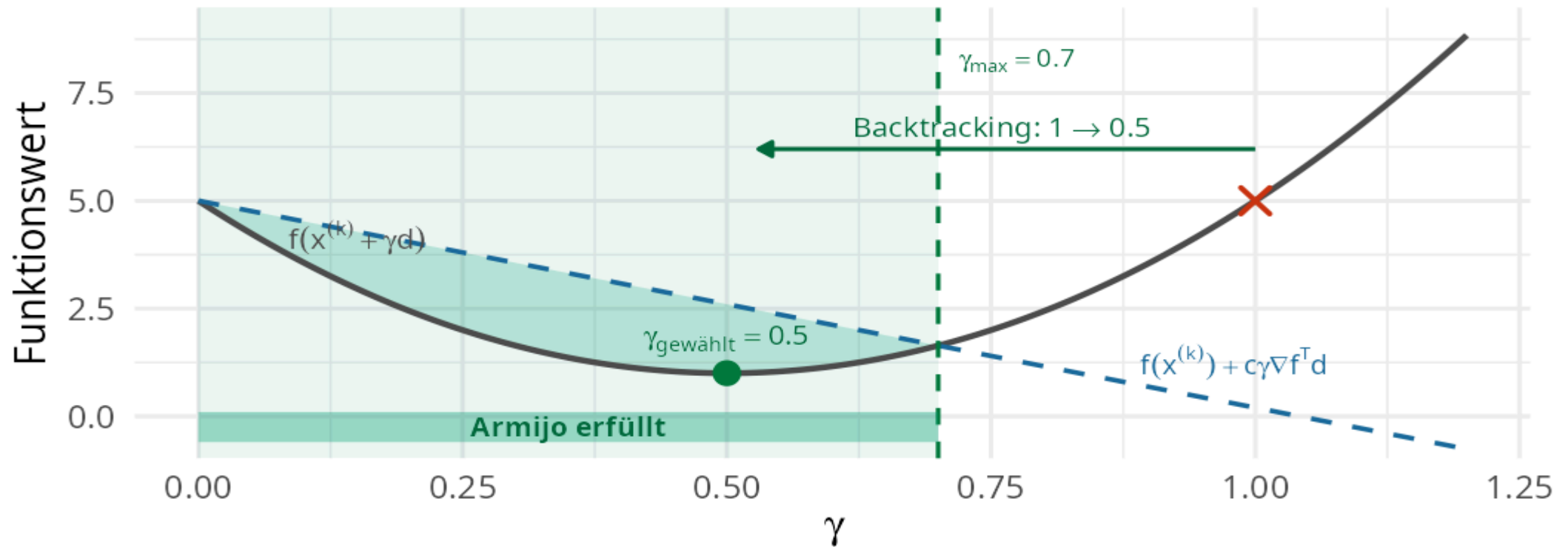
Achtung:

ρ ist hier der *Verkleinerungsfaktor* der Schrittweite — nicht die Konvergenzrate $\rho = 1 - \mu/L$ von vorhin.

Line Search: Visualisierung

Armijo Line Search: zulässige Schrittweiten

Illustration mit $c = 0.3$, $\rho = 0.5$



Typische Parameter: $c = 10^{-4}$, $\rho = 0.5$

Quasi-Newton: *BFGS* verwendet automatisch *Line Search* ($\rightarrow$ Kapitel *Optimierung II*)

Zusammenfassung

Self-Check: Gradientenabstieg

1. Sei $f(\mathbf{x}) = x_1^2 + 4x_2^2$ und $\mathbf{x}^{(0)} = (2, 1)^\top$ mit Schrittweite $\gamma = 0.25$. Wo liegt $\mathbf{x}^{(1)}$?

1. $(1, -1)^\top$

2. $(1.5, 0)^\top$

3. $(1, 0)^\top$

4. $(0, 0)^\top$

2. Welche Aussagen sind wahr?

1. GD konvergiert für jede Schrittweite $\gamma > 0$ gegen ein lokales Minimum.

2. Für konvexes f mit L -Lipschitz-Gradient und $\gamma \leq 1/L$ gilt: Fehler $= O(1/k)$.

3. Für stark konvexes f konvergiert GD linear, d.h. Fehler $= O(\rho^k)$ mit $\rho < 1$.

4. Ist $\|\nabla f(\mathbf{x}^{(k)})\|$ klein, dann ist $\mathbf{x}^{(k)}$ nahe am Minimum.

Zusammenfassung

Methode	Wann verwenden?
Bisektion	univariate Nullstellen; braucht nur Stetigkeit + Vorzeichenwechsel
Newton-Raphson	Nullstellen glatter Funktionen; quadratische Konvergenz
Nelder-Mead	Optimierung ohne Ableitungen, niedrige Dimension ($n \leq 10$)
Gradient Descent	Optimierung großer differenzierbarer Probleme; einfache Implementierung

- **Konvexität** $\implies$ jedes lokale Minimum ist global;
strikte Konvexität $\implies$ *höchstens ein* Minimum (also eindeutig, falls existent)
 - **Konvergenz von GD:**
 - Rate $O(1/k)$ (konvex) bzw. $O(\rho^k)$ (stark konvex)
 - $\rho = 1 - \frac{\mu}{L}$ bei $\gamma = \frac{1}{L}$
 - alternativ Schrittweite adaptiv mit *Line Search*

Im Kapitel Optimierung II:

Newton-Verfahren (“Newton-Raphson für $\nabla f = \mathbf{0}^\top$ ”), Quasi-Newton (BFGS), Momentum & SGD, Optimierung unter Nebenbedingungen (Lagrange/KKT)

Anhang

Fixpunktiteration erster Ordnung

(in der Vorlesung nur als Bemerkung: GD ist die Fixpunktiteration für $\nabla f(\mathbf{x})^\top = \mathbf{0}$)

Ziel: Löse $f(\mathbf{x}) = \mathbf{0}$, $f: \mathbb{R}^n \rightarrow \mathbb{R}^n$ monoton steigend (d.h. $(f(\mathbf{x}) - f(\mathbf{y}))^\top (\mathbf{x} - \mathbf{y}) \geq 0 \forall \mathbf{x}, \mathbf{y}$).

- Wir definieren die Iteration

$$\mathbf{x}^{(k)} = \mathbf{x}^{(k-1)} - \gamma f(\mathbf{x}^{(k-1)}).$$

- Ist f differenzierbar, erhalten wir aus einer Taylorapproximation 1. Ordnung um $\mathbf{x}^*$:

$$\begin{aligned} \|\mathbf{x}^{(k)} - \mathbf{x}^*\| &\approx \left\| \mathbf{x}^{(k-1)} - \mathbf{x}^* - \underbrace{\gamma f(\mathbf{x}^*)}_{=\mathbf{0}} - \gamma \mathbf{J}_f(\mathbf{x}^*) (\mathbf{x}^{(k-1)} - \mathbf{x}^*) \right\| \\ &= \left\| (\mathbf{I}_n - \gamma \mathbf{J}_f(\mathbf{x}^*)) (\mathbf{x}^{(k-1)} - \mathbf{x}^*) \right\| \\ &\leq \|\mathbf{I}_n - \gamma \mathbf{J}_f(\mathbf{x}^*)\| \|\mathbf{x}^{(k-1)} - \mathbf{x}^*\|. \end{aligned}$$

- Falls $\rho := \|\mathbf{I}_n - \gamma \mathbf{J}_f(\mathbf{x}^*)\| < 1$, gilt *lokal* (für Startwerte nahe genug an $\mathbf{x}^*$)

$$\|\mathbf{x}^{(k)} - \mathbf{x}^*\| = O(\rho^k)$$

- Exakt mit diesem ρ und *global* gilt das nur für affines f – dort verschwindet der oben weggelassene Restterm.

Fixpunktiteration erster Ordnung

Vorteile:

- Simple Verfahren auch für multivariate Funktionen
- Schnelle Konvergenz für gutartige Funktionen

Nachteile:

- Schrittweite γ muss gut gewählt sein:
 - zu klein: langsame Konvergenz
 - zu groß: gar keine Konvergenz

In der Praxis:

- Folge $\gamma_k \rightarrow 0$.
- Falls f fallend, betrachte $-f$.

Beweis: Globale Optimalität bei Konvexität

Proposition

Für jede differenzierbare, konvexe Funktion $f: \mathbb{R}^n \rightarrow \mathbb{R}$ sind äquivalent: (i) $\mathbf{x}^*$ ist ein globales Minimum von f . (ii) $\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$.

Beweis.

- $(i) \implies (ii)$: Jedes globale Minimum ist ein lokales Minimum. Dort muss $\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$ gelten. ■
- $(ii) \implies (i)$: Sei $\mathbf{x}^*$ ein Punkt mit $\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$. Dann gilt für alle $\mathbf{y} \in \mathbb{R}^n$

$$f(\mathbf{y}) \geq f(\mathbf{x}^*) + \nabla f(\mathbf{x}^*)(\mathbf{y} - \mathbf{x}^*) = f(\mathbf{x}^*).$$

Also ist $\mathbf{x}^*$ ein globales Minimum. ■

↩ zurück: „Konvexe Funktionen in der Optimierung“

Optimierung II: Verfahren 2. Ordnung, Stochastischer GD, Nebenbedingungen

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)

Material: Thomas Nagler

LMU München

Verwendete Vorkenntnisse

- **Aus Optimierung I:**

- *Gradient Descent* (Gradientenabstieg, GD): Iterationsvorschrift, Schrittweite, Konvergenzraten
- *Newton-Raphson-Verfahren* für nichtlineare Gleichungen $f(x) = 0$
- Optimalitätsbedingungen: $\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$, Definitheit von $\mathbf{H}_f(\mathbf{x}^*)$
- *Lipschitz-Konstante* und *Konditionszahl* $\kappa_f = L/\mu$

- **Aus Konvexität:**

- *Konvexe Mengen und Funktionen*; lokale = globale Minima

- **Lineare Algebra:** *positiv definite* Matrizen, Eigenwerte

- **Analysis:** Taylor-Approximation 1. und 2. Ordnung

Opener-Quiz: Was wissen Sie noch?

Frage 1: Wie lautet die Iterationsvorschrift des *Gradientenabstiegs*?

1. $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \gamma \nabla f(\mathbf{x}^{(k)})^\top$
2. $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \gamma \nabla f(\mathbf{x}^{(k)})^\top$
3. $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \gamma f(\mathbf{x}^{(k)})$
4. $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{H}_f(\mathbf{x}^{(k)})^{-1} \nabla f(\mathbf{x}^{(k)})^\top$

Frage 2: Sei $\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$ und $\mathbf{H}_f(\mathbf{x}^*)$ *indefinit*. Dann ist $\mathbf{x}^*$...

1. ein lokales Minimum
2. ein lokales Maximum
3. ein Sattelpunkt
4. ein globales Minimum

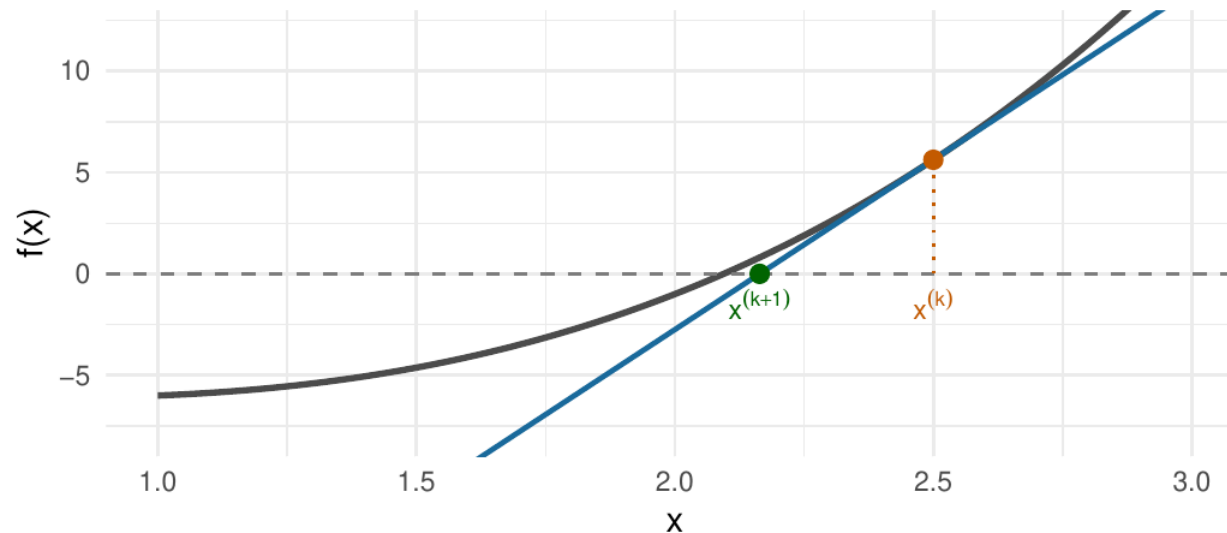
Brücke: Von Newton-Raphson zur Newton-Optimierung

Letzte Woche:

Newton-Raphson löst $f(x) = 0$ über die Nullstelle der Tangente:

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}$$

Newton-Raphson: Tangente → Nullstelle



Heute:

Am Minimum gilt $\nabla f(x^*) = \mathbf{0}^\top$.

⇒ **Newton-Optimierung** = Newton-Raphson, angewandt auf die Gleichung $\nabla f(x) = \mathbf{0}^\top$!

Newton-Verfahren

Newton-Verfahren: Idee & Schritt

Problem bei GD:

Berücksichtigt nur die *Steigung*, nicht die *Krümmung*

→ approximiert f lokal durch Tangente (Taylor 1. Ordnung)

Newton-Idee:

Approximiere f lokal durch eine *Parabel* (Taylor 2. Ordnung):

$$f(\mathbf{x} + \mathbf{h}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})\mathbf{h} + \frac{1}{2}\mathbf{h}^\top \mathbf{H}_f(\mathbf{x})\mathbf{h}$$

→ Minimiere diese Approximation bzgl. $\mathbf{h}$:

$$\nabla_{\mathbf{h}} [\dots] = \nabla f(\mathbf{x}) + \mathbf{h}^\top \mathbf{H}_f(\mathbf{x}) \stackrel{!}{=} \mathbf{0}^\top \quad \Longleftrightarrow \quad \mathbf{h} = -\mathbf{H}_f(\mathbf{x})^{-1} \nabla f(\mathbf{x})^\top$$

Newton-Schritt

In Iteration k wird der nächste Punkt $\mathbf{x}^{(k+1)}$ berechnet als:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{H}_f\left(\mathbf{x}^{(k)}\right)^{-1} \nabla f\left(\mathbf{x}^{(k)}\right)^\top$$

- **Voraussetzung:** $\mathbf{H}_f(\mathbf{x}^{(k)})$ muss invertierbar sein (Konvexität liefert nur $\mathbf{H}_f \succeq 0$!)
- Abbruchkriterien wie bei *Gradient Descent*
- In der Praxis löst man das LGS, statt $\mathbf{H}_f^{-1}$ explizit zu berechnen
— **Invertiere niemals eine Matrix!**

Verbindung: Newton-Raphson $\leftrightarrow$ Newton-Optimierung

Nullstellensuche	Optimierung
Löse $f(x) = 0$	Löse $\nabla f(\mathbf{x}) = \mathbf{0}^\top$
$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}$	$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{H}_f^{-1} \nabla f(\mathbf{x}^{(k)})^\top$

$\implies$

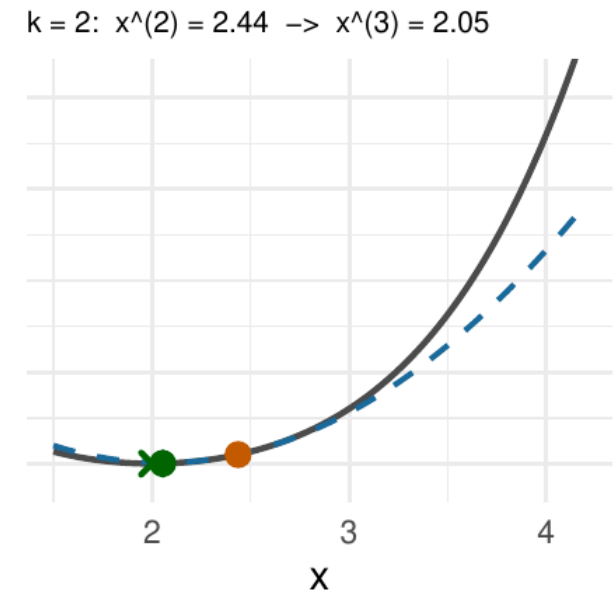
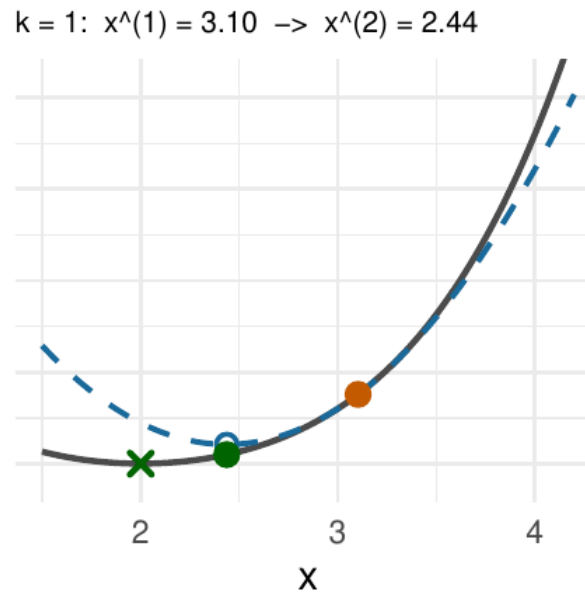
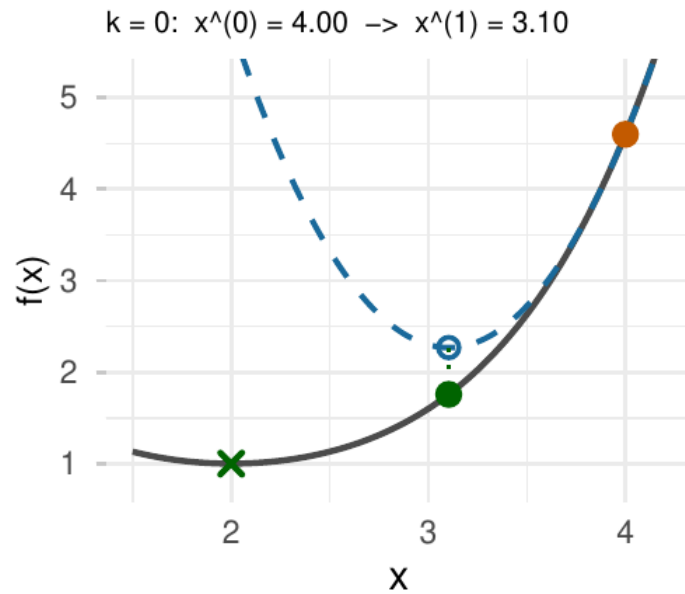
Newton für Optimierung $\simeq$ Newton-Raphson für $g(\mathbf{x}) := \nabla f(\mathbf{x})^\top = \mathbf{0}$

- Jacobi-Matrix von g : $\mathbf{J}_g = \mathbf{H}_f$
- Newton-Raphson:

$$\begin{aligned}\mathbf{x}^{(k+1)} &= \mathbf{x}^{(k)} - \mathbf{J}_g^{-1}(\mathbf{x}^{(k)}) g(\mathbf{x}^{(k)}) \\ &= \mathbf{x}^{(k)} - \mathbf{H}_f^{-1}(\mathbf{x}^{(k)}) \nabla f(\mathbf{x}^{(k)})^\top \quad \checkmark\end{aligned}$$

Newton-Verfahren: Geometrische Intuition

Newton-Iterationen für die konvexe Funktion $f(x) = 0.5(x-2)^2 + 0.1(x-2)^4 + 1$



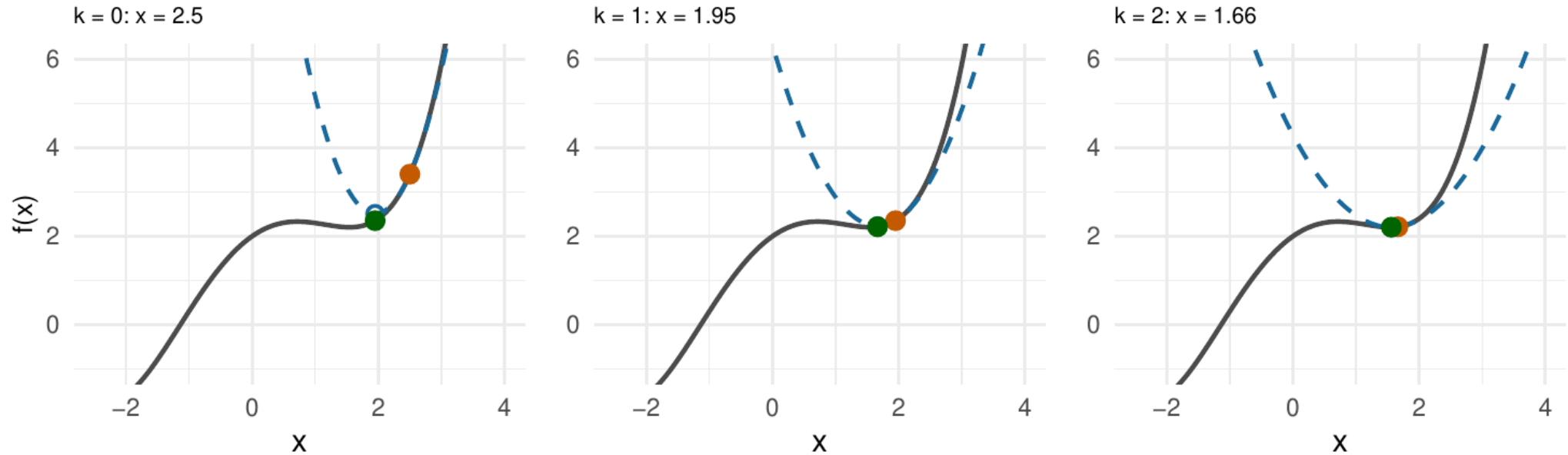
orange: aktueller Punkt

blau gestrichelt: quadratische Taylor-Approximation T_2

grün: nächster Iterationspunkt.

Newton für nicht-konvexe Funktionen

Newton für $f(x) = 0.1x^4 - 0.8x^2 + x + 2$ (nicht-konvex)



Bei nicht-konvexen Funktionen kann Newton auch zu beliebigen **lokalen** oder globalen kritischen Punkten — also auch Sattelpunkten oder *Maxima* — konvergieren.

Quasi-Newton-Verfahren

Quasi-Newton-Verfahren

Problem:

Hesse-Matrix $\mathbf{H}_f(\mathbf{x})$ ist teuer zu berechnen, zu speichern ($O(n^2)!$) und zu invertieren ($O(n^3)!$).

Idee:

Approximiere $\mathbf{H}_f^{-1}$ iterativ aus Gradienten-Informationen!

Quasi-Newton-Update

Der nächste Punkt $\mathbf{x}^{(k+1)}$ wird berechnet als:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \gamma_k \mathbf{B}_k \nabla f(\mathbf{x}^{(k)})^\top,$$

wobei $\mathbf{B}_k \approx \mathbf{H}_f^{-1}(\mathbf{x}^{(k)})$ aus vorherigen Schritten geschätzt wird und γ_k aus einer *Line Search* stammt.

- Vermeidet Invertieren von $\mathbf{H}_f$
- Oft *Low-Rank-Updates* von $\mathbf{B}_k$ (BFGS: Rang 2) oder Diagonal-Approximationen
- Schrittweite γ_k kommt automatisch aus einer *Line Search* (ohne sie kann schon der erste Schritt aus $\mathbf{B}_0 = \mathbf{I}$ den Funktionswert erhöhen)

BFGS-Verfahren

BFGS = Broyden–Fletcher–Goldfarb–Shanno

Aktualisiert $\mathbf{B}_k$ so, dass die **Sekantenbedingung** $f''(x) \approx \frac{f'(x_{k+1}) - f'(x_k)}{x_{k+1} - x_k}$ erfüllt ist:

$$\mathbf{B}_{k+1} \underbrace{\left(\nabla f(\mathbf{x}^{(k+1)}) - \nabla f(\mathbf{x}^{(k)}) \right)}_{\mathbf{y}_k}^\top = \underbrace{\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}}_{\mathbf{s}_k}$$

- **Achtung:** wegen $\mathbf{B}_k \approx \mathbf{H}_f^{-1}$ ist die *Matrixgleichung* der **Kehrwert** des Differenzenquotienten (in 1D: $b_{k+1} = s_k/y_k$ statt y_k/s_k)
- Setze $\mathbf{B}_0 = \mathbf{I}$
- BFGS-Update mit $\rho_k = \frac{1}{\mathbf{y}_k^\top \mathbf{s}_k} \left(\frac{\mathbf{y}_k^\top \mathbf{s}_k}{\mathbf{s}_k^\top \mathbf{s}_k} \approx \text{Krümmung in Richtung } \mathbf{s}_k \right)$:

$$\mathbf{B}_{k+1} = (\mathbf{I} - \rho_k \mathbf{s}_k \mathbf{y}_k^\top) \mathbf{B}_k (\mathbf{I} - \rho_k \mathbf{y}_k \mathbf{s}_k^\top) + \rho_k \mathbf{s}_k \mathbf{s}_k^\top$$

- Komplexität: $O(n^2)$ pro Schritt statt $O(n^3)$ bei Newton
- Konvergenz schneller als GD, fast so gut wie Newton (superlinear)
- **L-BFGS:** *Limited-memory* Variante für großes n — speichert nur letzte m Vektoren für Speicherkomplexität $O(mn)$ statt $O(n^2)$

Methodenvergleich: Zusammenfassung

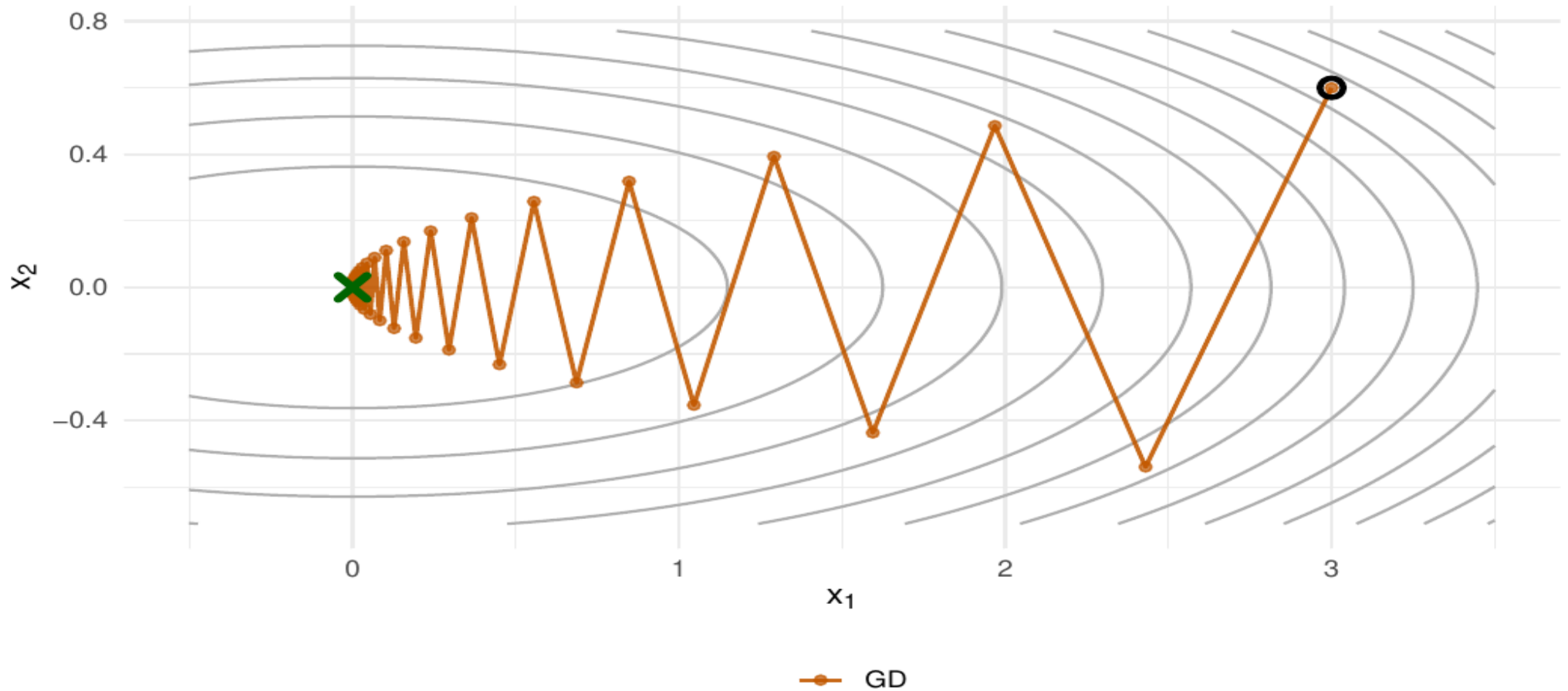
Eigenschaft	Nelder-Mead	Gradientenabstieg	Quasi-Newton	Newton
Verfahren	0. Ordnung	1. Ordnung	“1.5te” Ordnung	2. Ordnung
Benötigt	nur f	$f, \nabla f$	$f, \nabla f$	$f, \nabla f, \mathbf{H}_f$
Komplexität/Schritt	$O(n)$	$O(n)$	$O(n^2)$	$O(n^3)$
Konvergenz	Keine Garantie	Linear [×]	Superlinear [×]	Quadratisch [×]
Schrittweite	Autom.	Kritisch (γ)	<i>line search</i>	Autom.
Skalierbarkeit	Schlecht ($n \leq 10$)	Sehr gut	Gut	Schlecht

×: Konvergenzaussagen gelten lokal bzw. für glatte, stark konvexe Zielfunktionen und geeignete Schrittweiten/Line Search.

- GD/Newton/Quasi-Newton sind **Fixpunktiterationen** für $\nabla f(\mathbf{x}) = \mathbf{0}$
- **Nelder-Mead**: Default in `R` `optim()`; nützlich, wenn keine Ableitungen verfügbar
- **GD**: Grundlage moderner NN-Optimierer
- **Quasi-Newton (BFGS)**: in `R`: `optim(..., method = "BFGS")`, guter Kompromiss
- **Newton**: MaxLik-Inferenz (IWLS für GLMs), XGBoost, ...

Momentum & Stochastic Gradient Descent

Gradientenabstieg mit Momentum: Motivation



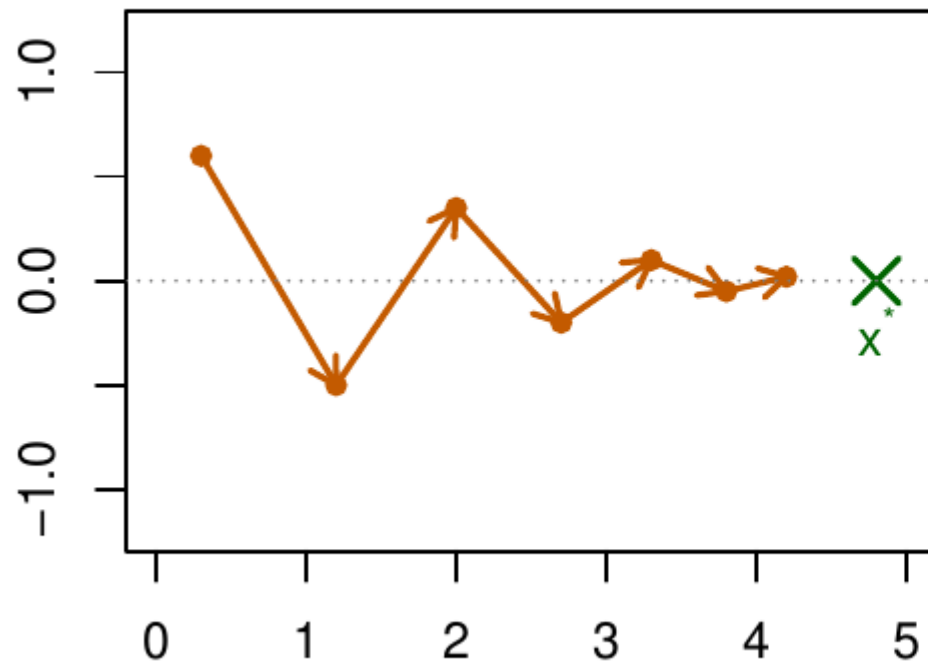
Bei **hoher Konditionszahl** $\kappa(\mathbf{H}_f)$ oszilliert GD oft stark.

Idee: Momentum ("Schwung")

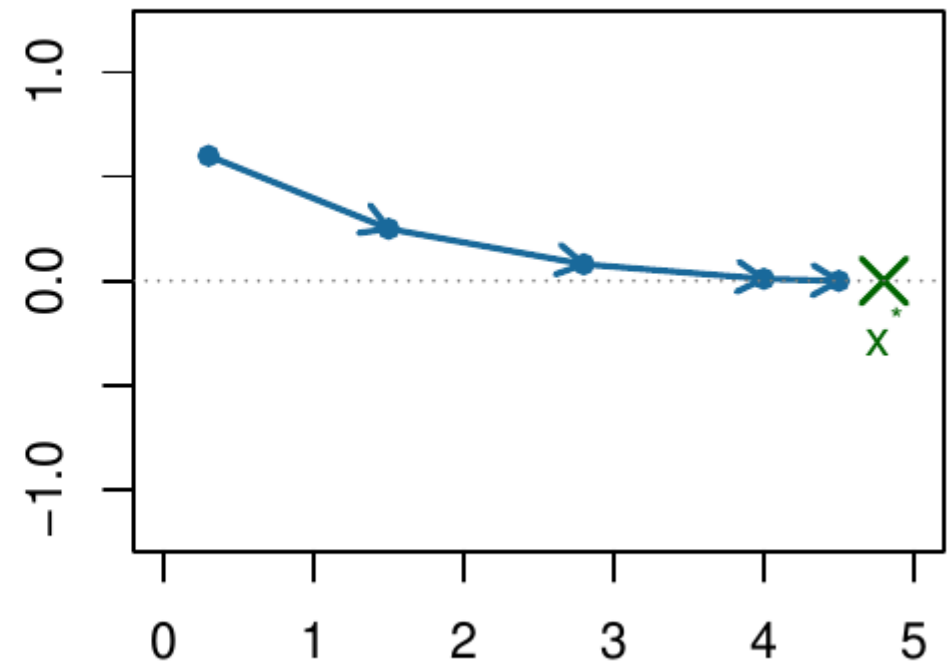
Analogie:

Eine Kugel, die einen Hügel herunterrollt, sammelt Schwung (*momentum*). Ihre Bewegung wird dann nicht durch jede kleine Unebenheit beeinflusst, sondern durch die *mittlere Steigung*.

Ohne Momentum



Mit Momentum



Momentum dämpft Oszillationen und beschleunigt konsistente Richtungen.

Heavy-Ball Momentum

Momentum-Update

Benutzt wird nicht mehr nur die aktuelle Steigung $\nabla f(\mathbf{x}^{(k)})^\top$, sondern ein speziell “gemittelter” Gradient $\mathbf{v}^{(k)}$:

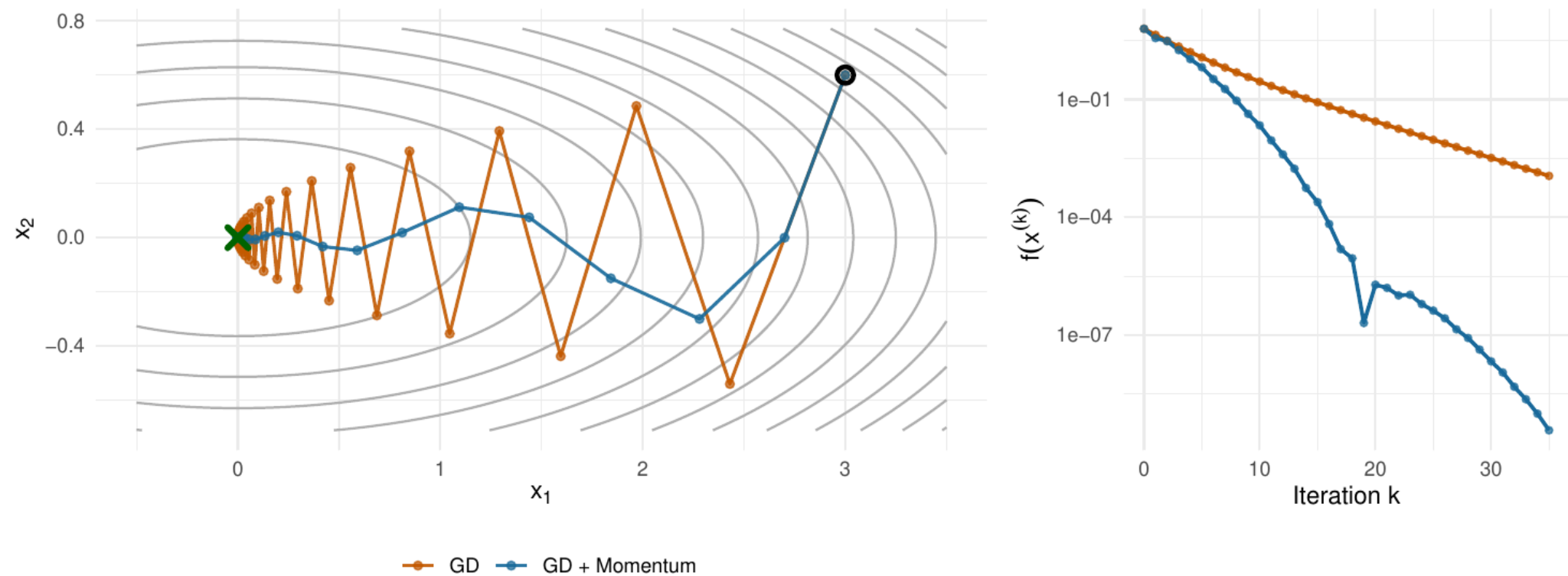
$$\begin{aligned}\mathbf{v}^{(k+1)} &= \alpha \mathbf{v}^{(k)} - \gamma \nabla f(\mathbf{x}^{(k)})^\top \\ \mathbf{x}^{(k+1)} &= \mathbf{x}^{(k)} + \mathbf{v}^{(k+1)}\end{aligned}$$

- $\mathbf{v}^{(k)}$: “Geschwindigkeit” (akkumulierte Bewegungsrichtung)
- $\alpha \in [0, 1)$: **Momentum**-Parameter (typisch: 0.9); gedämpft wird mit $1 - \alpha$
- γ : Schrittweite (wie bei GD)
- (Bessere) Alternative: **Nesterov-Momentum** benutzt Gradient an *voraussichtlichem* Punkt.

Wirkung:

- Glättet Oszillationen durch Mittelung früherer Gradienten
- Beschleunigt so Iterationsfortschritt in Richtungen mit konsistentem Gradienten

Momentum: Visualisierung



s.a. Shiny-App

Stochastic Gradient Descent: Motivation

Typisches ML/Statistik-Problem:

Minimiere das *empirische Risiko*

$$R(\boldsymbol{\theta}) := \frac{1}{N} \sum_{i=1}^N L(y_i, p_{\boldsymbol{\theta}}(\mathbf{x}_i))$$

- Modell $p_{\boldsymbol{\theta}}$ mit Features/Inputs $\mathbf{x}_i$, Labels/Outputs y_i und Parametern $\boldsymbol{\theta}$
- N = Anzahl Datenpunkte (kann sehr groß sein!)
- Verlustfunktion $L(y_i, p_{\boldsymbol{\theta}}(\mathbf{x}_i)) \simeq$ Fehler für Datenpunkt i bei Modellvorhersage $\hat{y}_i = p_{\boldsymbol{\theta}}(\mathbf{x}_i)$

Problem:

$$\nabla R(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N \nabla L(y_i, p_{\boldsymbol{\theta}}(\mathbf{x}_i))$$

Jeder Schritt erfordert Berechnung **aller** N Gradienten $\rightarrow$ **sehr teuer!**

Stochastic Gradient Descent (SGD)

Idee:

Gradient eines *zufälligen* Datenpunkts $\nabla L(y_i, p_{\theta}(\mathbf{x}_i))$ ist ein *unverzerrter Schätzer*:

$$\mathbb{E}_{i \sim \text{Unif}\{1, \dots, N\}} [\nabla L(y_i, p_{\theta}(\mathbf{x}_i))] = \nabla R(\theta)$$

(Erwartungswert *nur* über die Ziehung des Index i — bei festem Datensatz und festem θ)

SGD-Update

Wähle zufälligen Index i_k :

$$\theta^{(k+1)} = \theta^{(k)} - \gamma^{(k)} \nabla L(y_{i_k}, p_{\theta^{(k)}}(\mathbf{x}_{i_k}))^\top$$

- Kosten pro Schritt: $O(1)$ statt $O(N)$
- Richtung ist “im Mittel” korrekt, aber **verrauscht**

In der Praxis:

Verwende **Mini-Batches**: Gradient bestimmt aus Stichproben von $n \ll N$ Datenpunkten (oft: 32–256)

SGD: Grundlage moderner AI

Mini-Batch-Gradientenverfahren sind die Grundlage fast des gesamten aktuellen NN-Trainings.

- Zwei langlebige Standardfamilien:
 - **SGD mit Momentum** (ggf. Nesterov), besonders in Computer Vision
 - **AdamW**: adaptive erste und zweite Momente + entkoppelter *Weight Decay*; Standard bei Transformern
- Adaptive, veränderliche Schrittweiten: **Learning-Rate-Schedule** oft ebenso wichtig wie der Optimierer (*Warmup* → lineare oder cos-Abnahme.)
- **Gradient Clipping** begrenzt große Gradienten und stabilisiert das Training.
- Mini-Batch-Rauschen kann helfen, Sattelpunkte und Plateaus zu verlassen.

Weiterführend:

MSc-Kurs **Optimization for ML** behandelt Adam, Muon & Co, *learning rate schedules*, etc.

Beschränkte Optimierung (Constrained Optimization)

Optimierung mit Nebenbedingungen: Motivation

Bisher: **Unbeschränkte** Optimierung $\min_{\mathbf{x}} f(\mathbf{x})$ für $f : \mathbb{R}^n \rightarrow \mathbb{R}$.

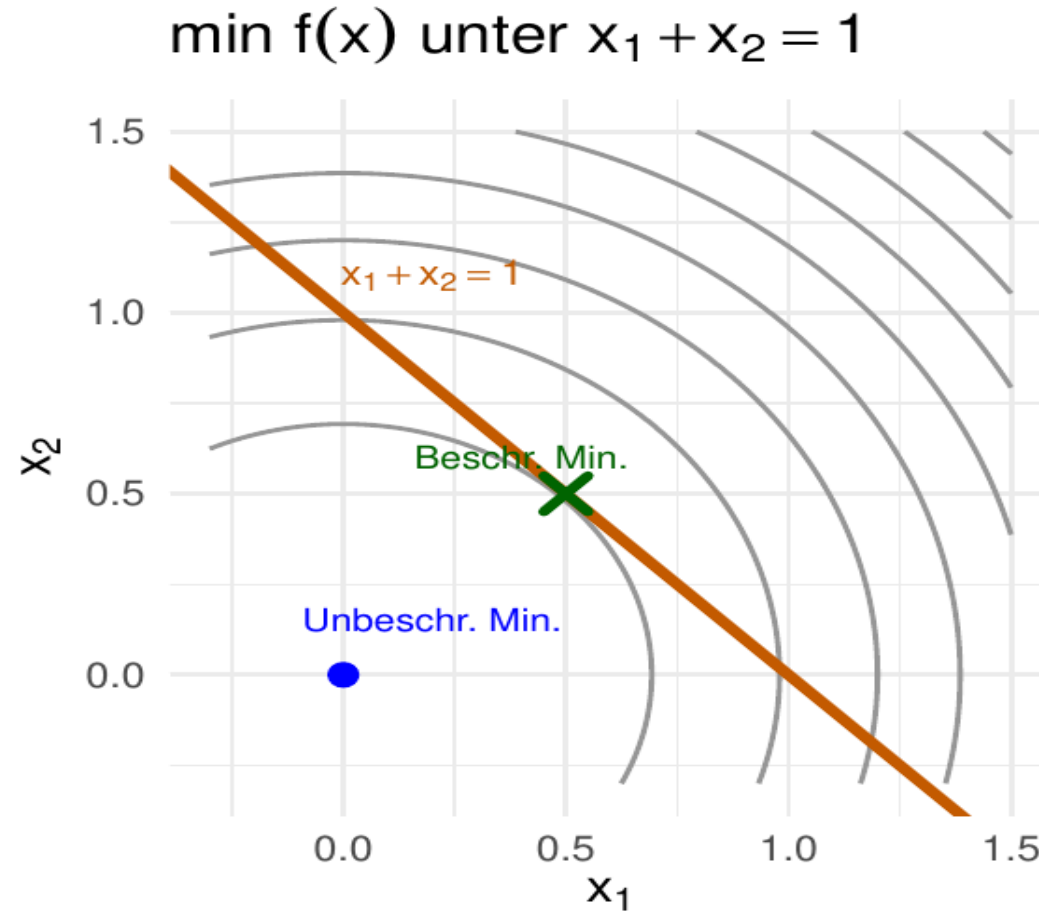
Oft haben wir aber **Einschränkungen**, z.B.:

- Parameter sind Wahrscheinlichkeiten: $p_i \geq 0$ und $\sum_i p_i = 1$
- Budget: Kosten $\leq$ Maximum
- Physik: Erhaltungsgrößen

Allgemeines Problem mit Nebenbedingungen

$\min_{\mathbf{x}}$	$f(\mathbf{x})$	(Zielfunktion)
u.d.N.	$g_i(\mathbf{x}) = 0, \ i = 1, \dots, m$	(Gleichungen)
	$h_j(\mathbf{x}) \leq 0, \ j = 1, \dots, p$	(Ungleichungen)

Geometrische Intuition



Hier also: $g(x) = x_1 + x_2 - 1$. Das beschränkte Optimum liegt **auf** der Nebenbedingung!

Lagrange-Multiplikatoren: Idee

Intuition:

1. Wenn wir die Nebenbedingung $g(\boldsymbol{x}) = 0$ als Kurve betrachten, dürfen wir uns nur entlang dieser Kurve bewegen.
2. Im Optimum gibt es entlang der Nebenbedingung g keine Steigung mehr in f
 - $\implies \nabla f$ senkrecht zur (Tangente der) Nebenbedingung
 - $\implies \nabla f$ parallel zu ∇g

Idee: Am Optimum $\boldsymbol{x}^*$ auf $g(\boldsymbol{x}) = 0$ muss also gelten:

$$\nabla f(\boldsymbol{x}^*) \text{ ist parallel zu } \nabla g(\boldsymbol{x}^*)$$

$$\implies \exists \lambda \in \mathbb{R} \text{ mit } \nabla f(\boldsymbol{x}^*) + \lambda \nabla g(\boldsymbol{x}^*) = \mathbf{0}^\top$$

Lagrange-Multiplikatoren: Theorie

Lagrange-Funktion

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \boldsymbol{\lambda}^\top \mathbf{g}(\mathbf{x})$$

$$\text{mit } \mathbf{g}(\mathbf{x}) = \begin{pmatrix} g_1(\mathbf{x}) \\ \vdots \\ g_m(\mathbf{x}) \end{pmatrix} \text{ und } \boldsymbol{\lambda} = \begin{pmatrix} \lambda_1 \\ \vdots \\ \lambda_m \end{pmatrix}$$

Notwendige Bedingung (Lagrange)

Sind die $\nabla g_i(\mathbf{x}^*)$ linear unabhängig, existiert am Optimum $\mathbf{x}^*$ ein $\boldsymbol{\lambda}^*$ mit:

$$\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}^*, \boldsymbol{\lambda}^*) = \mathbf{0}^\top \quad \text{und} \quad \mathbf{g}(\mathbf{x}^*) = \mathbf{0}$$

- Hinreichende Bedingungen sind komplexer (siehe z.B. Heath, Ch. 6.2.3)

Beispiel: Lagrange-Multiplikatoren

Problem:

Minimiere $f(x, y) = x^2 + y^2$ unter $g(x, y) = x + y - 1 = 0$.

Lagrange-Funktion:

$$\mathcal{L}(x, y, \lambda) = x^2 + y^2 + \lambda(x + y - 1)$$

Notwendige Bedingungen:

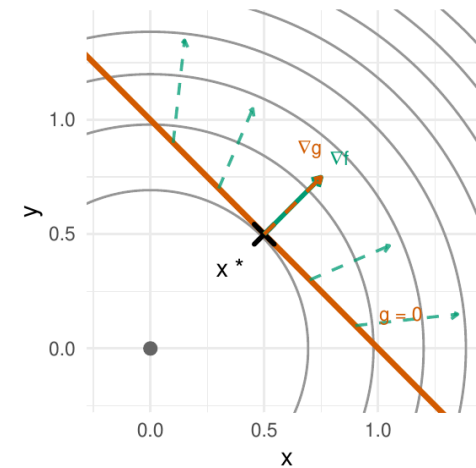
$$\frac{\partial \mathcal{L}}{\partial x} = 2x + \lambda = 0 \quad \Rightarrow \quad x = -\lambda/2$$

$$\frac{\partial \mathcal{L}}{\partial y} = 2y + \lambda = 0 \quad \Rightarrow \quad y = -\lambda/2$$

$$\frac{\partial \mathcal{L}}{\partial \lambda} = g(x, y) = x + y - 1 = 0 \quad \Rightarrow \quad -\lambda = 1$$

$$\text{Lösung: } x^* = y^* = \frac{1}{2}, \lambda^* = -1, f(x^*, y^*) = \frac{1}{2}$$

Am Optimum: $\nabla f = -\lambda \nabla g$



$$\nabla f(x, y) = (2x, 2y); \quad \nabla g(x, y) = (1, 1)$$

KKT-Bedingungen für Ungleichungen

Bei **Ungleichungs-Nebenbedingungen** $h_j(\mathbf{x}) \leq 0$ wird es komplizierter.

Karush-Kuhn-Tucker (KKT) Bedingungen

Für das Problem

$$\min f(\mathbf{x}) \text{ u.d.N. } g_i(\mathbf{x}) = 0, h_j(\mathbf{x}) \leq 0$$

gelten am Optimum $\mathbf{x}^*$:

1. Stationarität: $\nabla f(\mathbf{x}^*) + \sum_i \lambda_i \nabla g_i(\mathbf{x}^*) + \sum_j \mu_j \nabla h_j(\mathbf{x}^*) = \mathbf{0}^\top$
2. Primale Zulässigkeit: $g_i(\mathbf{x}^*) = 0, h_j(\mathbf{x}^*) \leq 0$
3. Duale Zulässigkeit: $\mu_j \geq 0$
4. Komplementarität: $\mu_j h_j(\mathbf{x}^*) = 0$ für alle j

Komplementarität bedeutet:

Für jede Ungleichung gilt

- $h_j(\mathbf{x}^*) < 0$ (inaktiv) $\implies \mu_j = 0$
- $\mu_j > 0 \implies h_j(\mathbf{x}^*) = 0$ (aktiv, "bindet")

Beispiel: Ungleichungs-Nebenbedingungen

Problem: $\min_x f(x) = (x - 3)^2$ unter $0 \leq x \leq 2$, umformuliert: $h_1(x) = -x \leq 0$, $h_2(x) = x - 2 \leq 0$

KKT-Bedingungen:

1. $2(x^* - 3) - \mu_1 + \mu_2 = 0$
2. $-x^* \leq 0$, $x^* - 2 \leq 0$
3. $\mu_1, \mu_2 \geq 0$
4. $\mu_1 x^* = 0$, $\mu_2(x^* - 2) = 0$

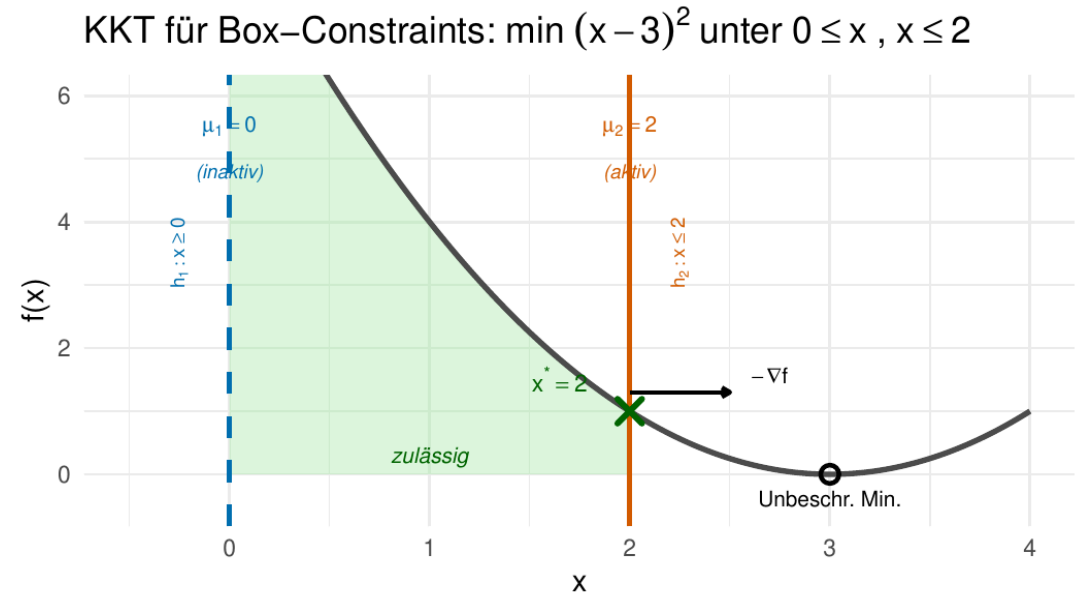
Lösung:

$x^* = 2$ (am rechten Rand), mit $\mu_1 = 0$, $\mu_2 = 2$

$\implies$ Obere Schranke ist **aktiv** (bindet), untere ist **inaktiv**.

Am Optimum $x^* = 2$:

neg. Gradient $-\nabla f(x^*) = -2(x^* - 3) = 2$ zeigt nach rechts (Richtung des unbeschränkten Minimums), aber die **aktive** Schranke h_2 blockiert weitere Bewegung.



Statistische Anwendung: Regularisierte Regression

Regularisierte Regression lässt sich als *beschränkte Optimierung* formulieren:

Ridge / Lasso (äquivalente Formulierungen)

$$\text{Penalisiert: } \min_{\boldsymbol{\beta}} (\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda \|\boldsymbol{\beta}\|_p^p)$$

$$\text{Beschränkt: } \min_{\boldsymbol{\beta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 \quad \text{u.d.N.} \quad \|\boldsymbol{\beta}\|_p^p \leq c$$

mit $p = 2$ (Ridge) bzw. $p = 1$ (Lasso).

Lagrange-Funktion:

$$\mathcal{L}(\boldsymbol{\beta}, \mu) = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \mu (\|\boldsymbol{\beta}\|_p^p - c)$$

KKT-Stationarität für Ridge ($p = 2$):

$$\nabla_{\boldsymbol{\beta}} \mathcal{L}^\top = -2\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) + 2\mu\boldsymbol{\beta} = \mathbf{0} \implies \hat{\boldsymbol{\beta}}_{\text{Ridge}} = (\mathbf{X}^\top \mathbf{X} + \mu \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$$

Für jedes $\lambda > 0$ existiert ein $c > 0$, sodass beide Formulierungen dieselbe Lösung haben –
umgekehrt nur, solange die Nebenbedingung auch *bindet* (ist der KQ-Schätzer selbst zulässig, erzwingt die Komplementarität $\mu = 0$).
Die Zuordnung $\lambda \leftrightarrow c$ hängt von den Daten ab.

Exkurs: Subgradienten

Proposition

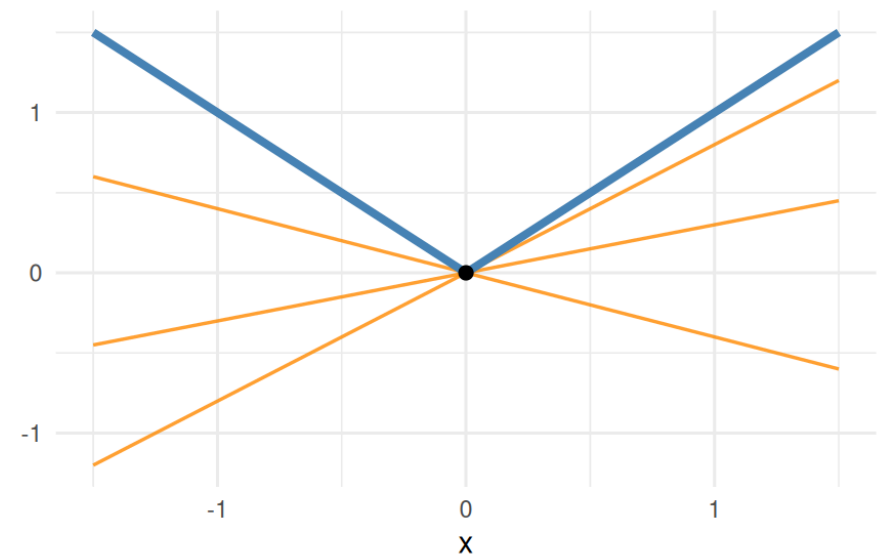
Sei $f: \mathbb{R}^n \rightarrow \mathbb{R}$ konvex.

Dann gibt es für jedes $\mathbf{x} \in \mathbb{R}^n$ einen Vektor $\mathbf{v} \in \mathbb{R}^n$, sodass

$$f(\mathbf{y}) \geq f(\mathbf{x}) + \mathbf{v}^\top (\mathbf{y} - \mathbf{x}) \quad \forall \mathbf{y} \in \mathbb{R}^n.$$

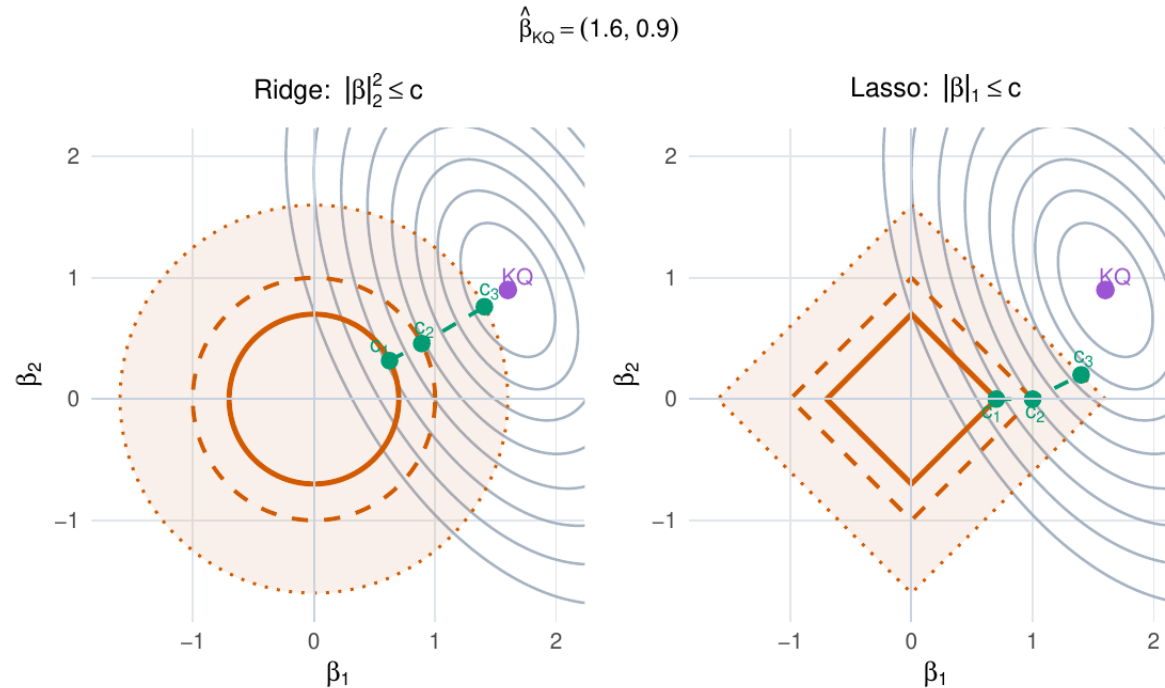
- Einfach gesagt: Wir können an jedem Punkt eine *affine* Funktion *unter* den Funktionsgraphen legen.
- Der Vektor $\mathbf{v}$ muss nicht eindeutig sein. *Jeder* Vektor, der diese Ungleichung erfüllt, wird **Subgradient** genannt. (Ist f in $\mathbf{x}$ differenzierbar, ist $\nabla f(\mathbf{x})^\top$ der *einzig*e Subgradient.)
- Die Funktion muss dafür **nicht differenzierbar** sein $\rightarrow$ **Subgradienten** wichtig für allgemeinere Optimierungsprobleme: z.B. LASSO, Quantilregression, u.v.m.

$f(x) = |x|$: Subgradienten am Knick $x = 0$



↩ zurück: „Geometrie: Ridge vs. Lasso“

Geometrie: Ridge vs. Lasso



Grau: Höhenlinien $f(\beta) = \|\mathbf{y} - \mathbf{X}\beta\|_2^2$; Lila: $\arg \min f(\beta)$; Grün: Lösung unter Nebenbedingungen.

- **Ridge** ($p = 2$): Kreisförmige Nebenbedingung $\rightarrow$ Lösungen auf dem Kreisbogen (sofern die Nebenbedingung *bindet*)
- **Lasso** ($p = 1$): Rautenförmige Nebenbedingung $\rightarrow$ Lösungen oft an *Ecken*
 $\implies$ **sparse** ($\exists j : \beta_j = 0$)

Achtung:

$\|\beta\|_1$ ist an den Ecken **nicht differenzierbar** — die KKT-Stationarität gilt fürs Lasso nur mit **Subgradienten**.
In der Praxis verwendet man Subgradienten- bzw. *Proximal-Verfahren*.

Optimierung in R

Optimierung in R: `optimize`

Univariat: `optimize()` für $f : \mathbb{R} \rightarrow \mathbb{R}$

```
1 f <- function(x) (x - 3)^2
2 optimize(f, interval = c(0, 10))
```

\$minimum

[1] 3

\$objective

[1] 0

- Benutzt **Brent-Verfahren**: **Golden Section Search** (Intervall-Schrumpfung ähnlich **Bisektion**) + parabolische Interpolation
- Findet für **unimodales** f das Minimum im Intervall — sonst evtl. nur ein *lokales* Minimum

Optimierung in R: `optim`

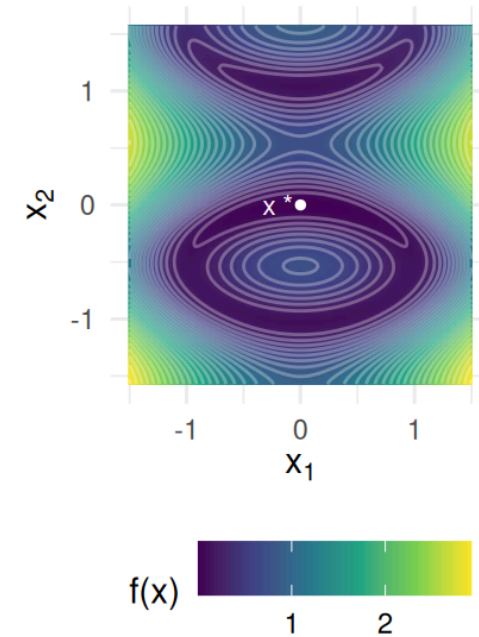
Multivariat: `optim()` für $f : \mathbb{R}^n \rightarrow \mathbb{R}$

```
1 f <- function(x) log1p((x[1]^2 + sin(3*x[2]))^2) +  
2   .1 * x[1]^2 + .1 * x[2]^2  
3 optim(c(-1, -0.5), f, method = "Nelder-Mead") |> #default  
4   str()
```

```
List of 5  
 $ par      : num [1:2] 5.54e-05 7.49e-06  
 $ value    : num 8.18e-10  
 $ counts   : Named int [1:2] 123 NA  
 ..- attr(*, "names")= chr [1:2] "function" "gradient"  
 $ convergence: int 0  
 $ message  : NULL
```

```
1 optim(c(-1, -0.5), f, method = "BFGS") |> str()
```

```
List of 5  
 $ par      : num [1:2] 6.88e-10 2.11e-10  
 $ value    : num 4.52e-19  
 $ counts   : Named int [1:2] 47 29  
 ..- attr(*, "names")= chr [1:2] "function" "gradient"  
 $ convergence: int 0  
 $ message  : NULL
```



optim: (Falsche) Konvergenz

Startwerte weit weg vom globalen Minimum $\mathbf{x}^* = (0, 0)$ bzw. zu kleines Iterationslimit:

```
1 # DANGER: Konvergenz zu lokalen Minima
2 optim(c(-1, 1), f)[1:2] |> str()
3 optim(c(-.5, -1), f, method = "BFGS")[1:2] |> str()
```

```
List of 2
 $ par   : num [1:2] 8.81e-05 1.04
 $ value : num 0.108
List of 2
 $ par   : num [1:2] -1.54e-05 -1.04
 $ value : num 0.108
```

```
1 # DANGER: Keine Konvergenz aber keine warnings():
2 optim(c(-1, -0.5), f, control = list(maxit = 50)) |> str()
```

```
List of 5
 $ par       : num [1:2] -0.584 -0.151
 $ value     : num 0.0454
 $ counts    : Named int [1:2] 51 NA
 ..- attr(*, "names")= chr [1:2] "function" "gradient"
 $ convergence: int 1
 $ message   : NULL
```

⇒ `convergence = 0` heißt nur: *Abbruchkriterium erfüllt* — **keine Garantie für ein (globales) Minimum!**

optim: Mit Gradienten oder Constraints

```
1 grad_f <- function(x) {  
2   log1p_dx <- 1 / (1 + (x[1]^2 + sin(3*x[2]))^2)  
3   c((4 * x[1] * (x[1]^2 + sin(3*x[2]))) * log1p_dx + .2 * x[1],  
4     (6 * cos(3*x[2]) * (x[1]^2 + sin(3*x[2]))) * log1p_dx + .2 * x[2])  
5 }  
6 str(optim(c(-1, -0.5), f, gr = grad_f, method = "BFGS")[-5])
```

```
List of 4  
$ par      : num [1:2] 6.88e-10 2.11e-10  
$ value    : num 4.52e-19  
$ counts   : Named int [1:2] 47 29  
..- attr(*, "names")= chr [1:2] "function" "gradient"  
$ convergence: int 0
```

```
1 str(optim(c(-1, 0.5), f, gr = grad_f, method = "L-BFGS-B",  
2         lower = c(-1, -1), upper = c(1, 1))[-5])
```

```
List of 4  
$ par      : num [1:2] 8.71e-07 2.65e-08  
$ value    : num 8.22e-14  
$ counts   : Named int [1:2] 14 14  
..- attr(*, "names")= chr [1:2] "function" "gradient"  
$ convergence: int 0
```

→ Analytischer Gradient schneller und genauer als numerische Approximation über finite Differenzen. → **L-BFGS-B** erlaubt Box-Nebenbedingungen (**KKT!**).

Self-Check & Zusammenfassung

Self-Check

Frage 1: Newton-Schritt für $f(x) = x^2 + 2$ mit $x^{(0)} = 4$: Was ist $x^{(1)}$?

1. $x^{(1)} = 0$
2. $x^{(1)} = 2$
3. $x^{(1)} = -4$
4. $x^{(1)} = 3.5$

Frage 2: Welche Aussage über die KKT-Bedingungen ist **falsch**?

1. Die Multiplikatoren für Ungleichungen erfüllen $\mu_j \geq 0$.
2. Komplementarität: $\mu_j h_j(\mathbf{x}^*) = 0$ für alle j .
3. Ist eine Ungleichung inaktiv ($h_j(\mathbf{x}^*) < 0$), gilt $\mu_j = 0$.
4. Ist eine Ungleichung inaktiv ($h_j(\mathbf{x}^*) < 0$), muss $\mu_j > 0$ gelten.

Frage 3: `optim()` liefert `convergence = 0`. Was wissen wir?

1. Das globale Minimum wurde gefunden.
2. Ein Abbruchkriterium wurde erfüllt — keine Garantie für ein (globales) Minimum.
3. Der Gradient ist exakt **0**.
4. Die Funktion ist konvex.

Wrap-up: Optimierung I & II

Methode	Wann verwenden?
Bisektion / Newton-Raphson	Nullstellen $f(x) = 0$, univariat
Nelder-Mead	Keine Ableitungen, niedrige Dimension ($n \leq 10$)
Gradient Descent	Große Probleme, einfache Implementierung
Momentum	GD + schlecht konditionierte Probleme
SGD	Sehr große Datensätze & große Dimension
Quasi-Newton (BFGS)	Mittlere Probleme, guter Kompromiss
Newton	Kleine Probleme, schnelle Konvergenz nötig
Lagrange/KKT	Nebenbedingungen

Kernideen der beiden Optimierungs-Sitzungen:

- Unbeschränkte differenzierbare Optimierung $\simeq$ Lösen von $\nabla f(x) = \mathbf{0}^\top$
 - meist **iterativ** mit - unter Nebenbedingungen: mit **Lagrange**, **KKT** prüfen.
- Mehr Ableitungsinformation $\rightarrow$ schnellere Konvergenz, teurere Schritte
- **Konvexität** garantiert globale Optima; **Konditionszahl** der **Hessematrix** steuert das Tempo
- Nebenbedingungen: **Lagrange** / **KKT** — Gradienten von Ziel und Nebenbedingung im Gleichgewicht

Nächste Vorlesung: Funktionsapproximation

Anhang

Momentum: Zusammenfassung

Wann Momentum verwenden?

- Hilfreich vor allem bei **schlecht konditionierten** Problemen; bei kleiner Konditionszahl kann $\alpha = 0.9$ sogar *langsamer* sein als reines GD
- Standard in Deep Learning ($\alpha = 0.9$ typisch)
- Kombination mit adaptiven Schrittweiten (Adagrad, RMSprop): **ADAM**, ...

Verbindung zur Konvexität:

- Bei **quadratischen** Zielfunktionen (und geeignet gewählten α, γ) beschleunigt Momentum die Konvergenz beweisbar auf die Rate $\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$; für allgemeine stark konvexe f garantiert das erst die *Nesterov*-Variante
- Reduziert effektiv die “Konditionszahl” des Problems

Funktionsapproximation: **Interpolation & Splines***

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)

Material: Thomas Nagler

LMU München

Opener-Quiz

Opener-Quiz

Beim Reinkommen: QR-Code scannen und die 2 Einstiegsfragen beantworten.

Frage 1:

Sei $\mathbf{v}_1, \dots, \mathbf{v}_d$ eine *Basis* eines Vektorraums V . Welche Aussagen sind wahr?

1. Jedes $\mathbf{v} \in V$ hat eine **eindeutige** Darstellung $\mathbf{v} = \sum_{j=1}^d \alpha_j \mathbf{v}_j$.
2. Basisvektoren dürfen linear abhängig sein, solange sie V aufspannen.
3. Die Anzahl d der Basisvektoren ist die *Dimension* von V .

Frage 2:

Die Menge *aller* Funktionen $f: [a, b] \rightarrow \mathbb{R}$ bildet mit punktweiser Addition und Skalarmultiplikation einen Vektorraum. Welche Dimension hat er?

1. 2 (Intervallgrenzen a und b)
2. n (Anzahl der Datenpunkte)
3. endlich, aber sehr groß
4. Er ist unendlich-dimensional.

Verwendete Vorkenntnisse

- **Lineare Algebra:**

- *Lineare Gleichungssysteme* $\mathbf{B}\mathbf{a} = \mathbf{y}$ und deren Lösbarkeit
- *Konditionszahl* einer Matrix als Maß für numerische Stabilität
- *Basen* und *lineare Unabhängigkeit* (Basisdarstellung von Funktionen)

- **Analysis:**

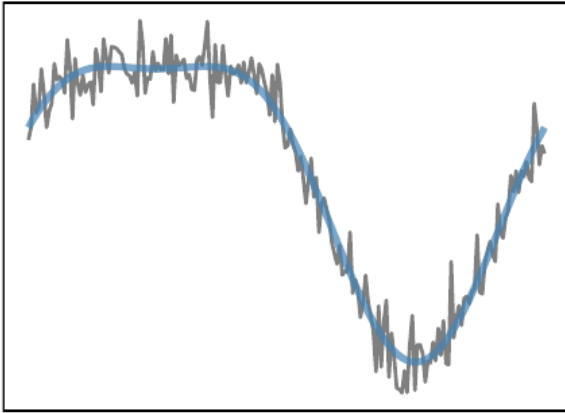
- Polynome und ihre Ableitungen (Polynomgrad, Monomialbasis)
- Stetigkeit und Differenzierbarkeit
- Stückweise definierte Funktionen
- *Normen* (z.B. $\|\cdot\|_\infty$ für Konvergenzaussagen)

Einführung

Einführung

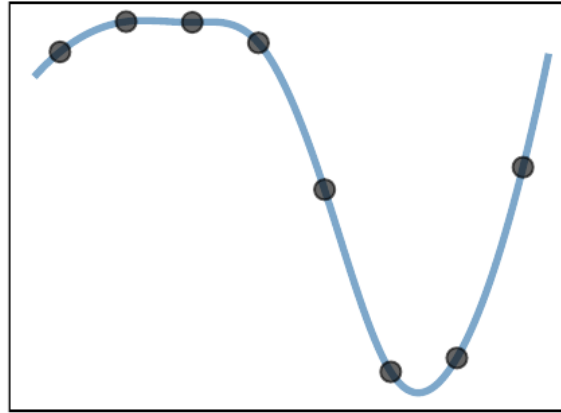
Drei Varianten: Finde $\hat{f}$, so dass ...

Approximation



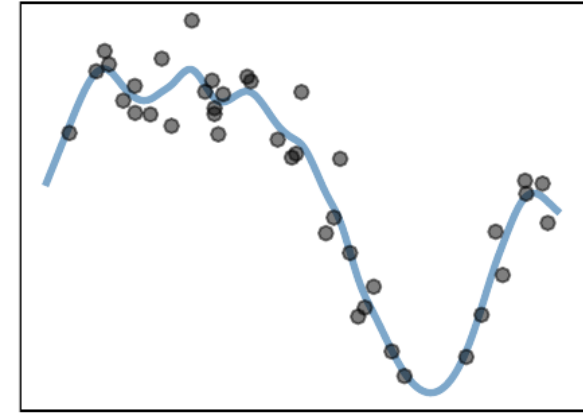
$$\|f - \hat{f}\| \approx 0$$

Interpolation



$$f(x_i) = \hat{f}(x_i) \quad \forall i$$

Glättung



$$y_i = \hat{f}(x_i) + \varepsilon_i \quad \forall i$$

Bei der Interpolation sind die Daten *rauschfrei*, d.h. $y_i = f(x_i)$ – deshalb steht dort später auch $y_i = \hat{f}(x_i)$.

Frage:

Wie sind die drei Varianten miteinander verwandt?

Kommentare

- Das *Approximationsproblem* lösen wir meist, indem wir es in ein *Interpolations*- oder *Glättungsproblem* überführen.
- Glättung ist die “schwierigste” Variante, weil die Fehler ε_i meist stochastisch sind.
- Wir kümmern uns zunächst um **Interpolation**.
- Wichtige Anwendungen:
 - Glatte Kurven zeichnen.
 - f approximativ, aber schneller auswerten.
 - **Beispiel:** $f(x) = \int_0^x g(y)dy$ oder $f(x) = g^{-1}(x)$ müssen numerisch bestimmt werden.
 - Ableitungen oder Integrale von f approximieren.

Anwendungen in ML und Data Science

- **Computergrafik:**
Interpolation von Pixelwerten bei Reformatierung/Rotation von Bildern.
- **Zeitreihen:**
Auffüllen fehlender Werte in Sensor- oder Finanzdaten.
- **Generative Modelle:**
Latent Space-Interpolation in VAEs/GANs
- **NLP:**
Positional Encodings in Transformern (“RoPE”) basieren auf Sinus-/Cosinus-Funktionen; *Long-Context*-Erweiterungen von RoPE interpolieren bzw. skalieren Positionsindizes.

(teils Interpolation im engeren Sinn, teils verwandte Konzepte/Analogien)

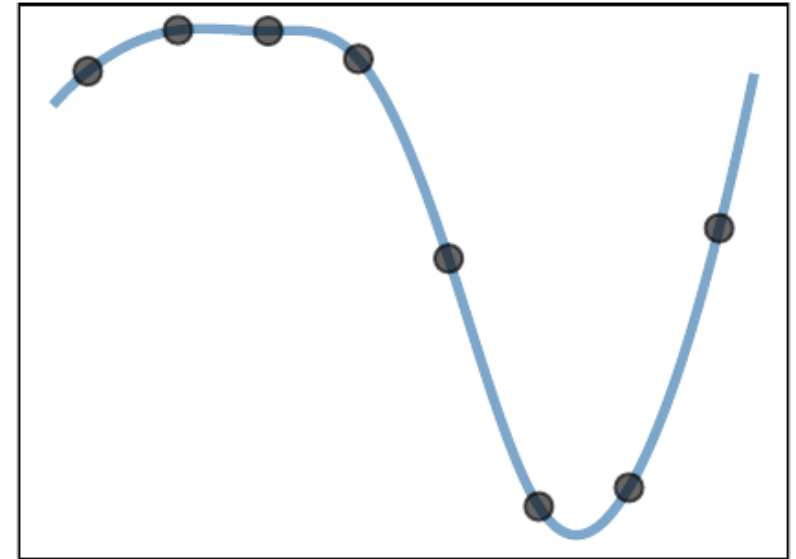
Quiz: Interpolationsproblem

Problem:

Finde $\hat{f}$, so dass

$$y_i = \hat{f}(x_i), \quad i = 1, \dots, n.$$

Interpolation



Welche der folgenden Aussagen ist wahr?

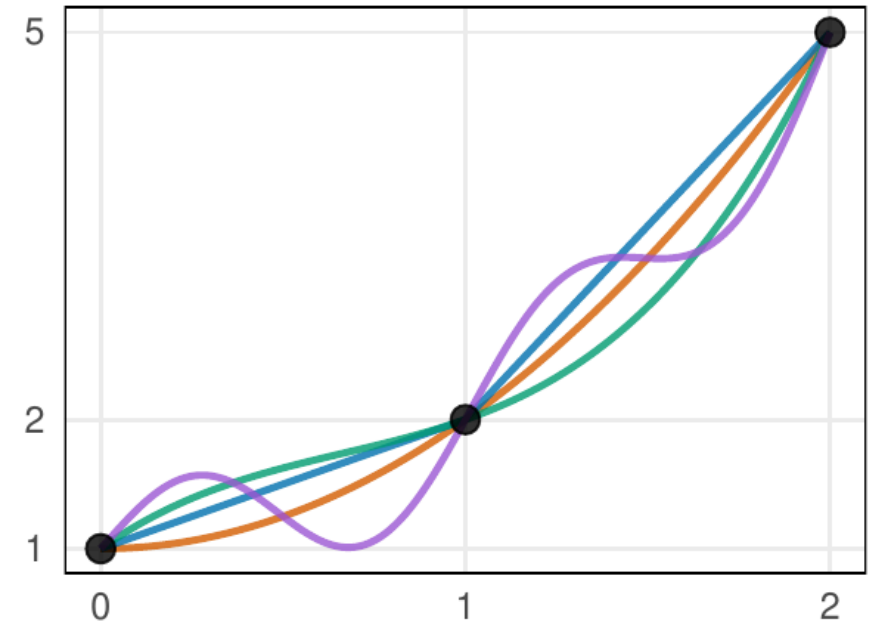
1. Es gibt unendlich viele Funktionen $\hat{f}$, die das Problem lösen.
2. Aus $y_i = \hat{f}(x_i), i = 1, \dots, n$, folgt $\|f - \hat{f}\| \approx 0$.
3. Das Problem hat eine eindeutige Lösung, wenn alle x_i verschieden sind.

Unendlich viele Lösungen: Beispiel

Betrachte $n = 3$ Punkte: $(0, 1)$, $(1, 2)$, $(2, 5)$.

Einige Interpolanten:

- **Parabel:** $\hat{f}_1(x) = 1 + x^2$
- **Stückweise linear:** $\hat{f}_2(x) = \begin{cases} x + 1 & x < 1 \\ 3x - 1 & x \geq 1 \end{cases}$
- **Kubisch:** $\hat{f}_3(x) = x^3 - 2x^2 + 2x + 1$
- **Vogelwild:** $\hat{f}_4(x) = 1 + x^2 + 0.5 \sin(2\pi x)$



Alle erfüllen $\hat{f}(x_i) = y_i$, aber sehen völlig unterschiedlich aus!

Allgemein:

Jedes $\hat{f} = \hat{f}_1 + g$ mit $g(x_i) = 0$ für alle i ist ebenfalls ein Interpolant.

⇒ Wir brauchen **Basissysteme** (Polynome, Splines), die die Lösungsmenge auf bestimmte Funktionenräume einschränken.

Funktionenräume und ihre Basen

Funktionenräume

- Menge aller Funktionen $f: [a, b] \rightarrow \mathbb{R}$ ist ein **Vektorraum**:
 - Addition: $(f + g)(x) = f(x) + g(x)$
 - Skalarmultiplikation: $(\alpha \cdot f)(x) = \alpha \cdot f(x)$
- Dieser Raum ist **unendlich-dimensional** $\implies \exists$ unendlich viele Interpolanten für n gegebene Punkte!

Lösung:

Einschränkung auf endlichdimensionale Unterräume

Basen: von $\mathbb{R}^n$ zu Funktionenräumen

$\{\phi_1, \dots, \phi_K\}$ ist eine **Basis** von $\mathcal{F}_K$ – *genauso* wie $\{v_1, \dots, v_n\}$ eine Basis des $\mathbb{R}^n$ ist:

	Vektorraum $\mathbb{R}^n$	Funktionsraum $\mathcal{F}_K$
“Vektor”	$v \in \mathbb{R}^n$	Funktion $f: [a, b] \rightarrow \mathbb{R}$
Basis	$v_1, \dots, v_n$	$\phi_1, \dots, \phi_K$
eindeutige Darstellung	$v = \sum_{i=1}^n \alpha_i v_i$	$f = \sum_{k=1}^K a_k \phi_k$
Koordinaten	$(\alpha_1, \dots, \alpha_n)$	Koeffizienten $(a_1, \dots, a_K)$
Dimension	n	K

- Alle Konzepte der linearen Algebra (lineare Unabhängigkeit, **Span**, Basiswechsel, ...) übertragen sich direkt auf Funktionen – eine Funktion *ist* ein Vektor (in $\mathbb{R}^\infty$...)
- Statt eine Funktion (in $\mathbb{R}^\infty$) finden zu müssen, brauchen wir jetzt nur noch ihren **Koordinatenvektor** $a = (a_1, \dots, a_K) \in \mathbb{R}^K$
→ ein Problem der (numerischen) linearen Algebra!

Interpolation durch Basisdarstellung

Interpolation durch Basisdarstellung

- **Problem:**

Finde $\hat{f}$, so dass

$$y_i = \hat{f}(x_i) \quad \forall i.$$

- **Lösung:**

1. Nehme an, dass $\hat{f}(x) = \sum_{k=1}^K a_k \phi_k(x)$ für

- **Basisfunktionen** $\phi_1, \dots, \phi_K$ (bekannt)
- **Basiskoeffizienten** $a_1, \dots, a_K$ (unbekannt)

2. Löse das Gleichungssystem

$$y_i = \sum_{k=1}^K a_k \phi_k(x_i), \quad i = 1, \dots, n.$$

→ **Interpolationsproblem** zurückgeführt auf **Lineares Gleichungssystem**!

Quiz: Vom Interpolationsproblem zum LGS

Das System

$$y_i = \sum_{k=1}^K a_k \phi_k(x_i), \quad i = 1, \dots, n$$

lässt sich als LGS $\mathbf{B}\mathbf{a} = \mathbf{y}$ darstellen. Was sind $\mathbf{B}$, $\mathbf{a}$, $\mathbf{y}$?

Beispiel: Basisdarstellung konkret

Gegeben:

$n = 3$ Datenpunkte $(0, 1), (1, 2), (2, 5)$, Monomialbasis $\phi_k(x) = x^{k-1}$, $K = 3$

$$\implies \mathbf{B} = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 1 \\ 2 \\ 5 \end{pmatrix}$$

Lösung durch Einsetzen:

$$\text{Zeile 1:} \quad a_1 = 1$$

$$\text{Zeile 2:} \quad a_2 + a_3 = 1 \quad \rightarrow a_2 = 1 - a_3$$

$$\text{Zeile 3:} \quad 2a_2 + 4a_3 = 4 \quad \rightarrow 2 + 2a_3 = 4$$

$$\implies a_3 = 1, a_2 = 0 \implies \hat{f}(x) = 1 + x^2$$

Probe:

$$\hat{f}(0) = 1, \hat{f}(1) = 2, \hat{f}(2) = 5 \quad \checkmark$$

Polynominterpolation

Fundamentalsatz der Algebra

Satz: Fundamentalsatz der Algebra

Jedes Polynom

$$p(x) = a_n x^n + \dots + a_1 x + a_0$$

mit $a_n \neq 0$ und $n \geq 1$ hat mindestens eine Nullstelle in $\mathbb{C}$.

Äquivalent:

Jedes Polynom vom Grad n hat genau n Nullstellen in $\mathbb{C}$ (mit Vielfachheit gezählt).

Folgerung:

Ein Polynom vom Grad $\leq n - 1$ mit n verschiedenen Nullstellen ist das Nullpolynom.

Beispiel:

Ein quadratisches Polynom (Grad 2) kann höchstens 2 (verschiedene) Nullstellen haben. Finden wir eines mit 3 Nullstellen, muss es $p(x) = 0$ für alle x sein.

Eindeutigkeit der Polynominterpolation

Satz: Polynominterpolation

Für n Paare $(x_i, y_i)_{i=1}^n$ mit paarweise verschiedenen x_i gibt es *genau* ein Polynom p vom Grad *höchstens* $(n - 1)$ mit $p(x_i) = y_i$ für alle i .

Beweisskizze (Eindeutigkeit):

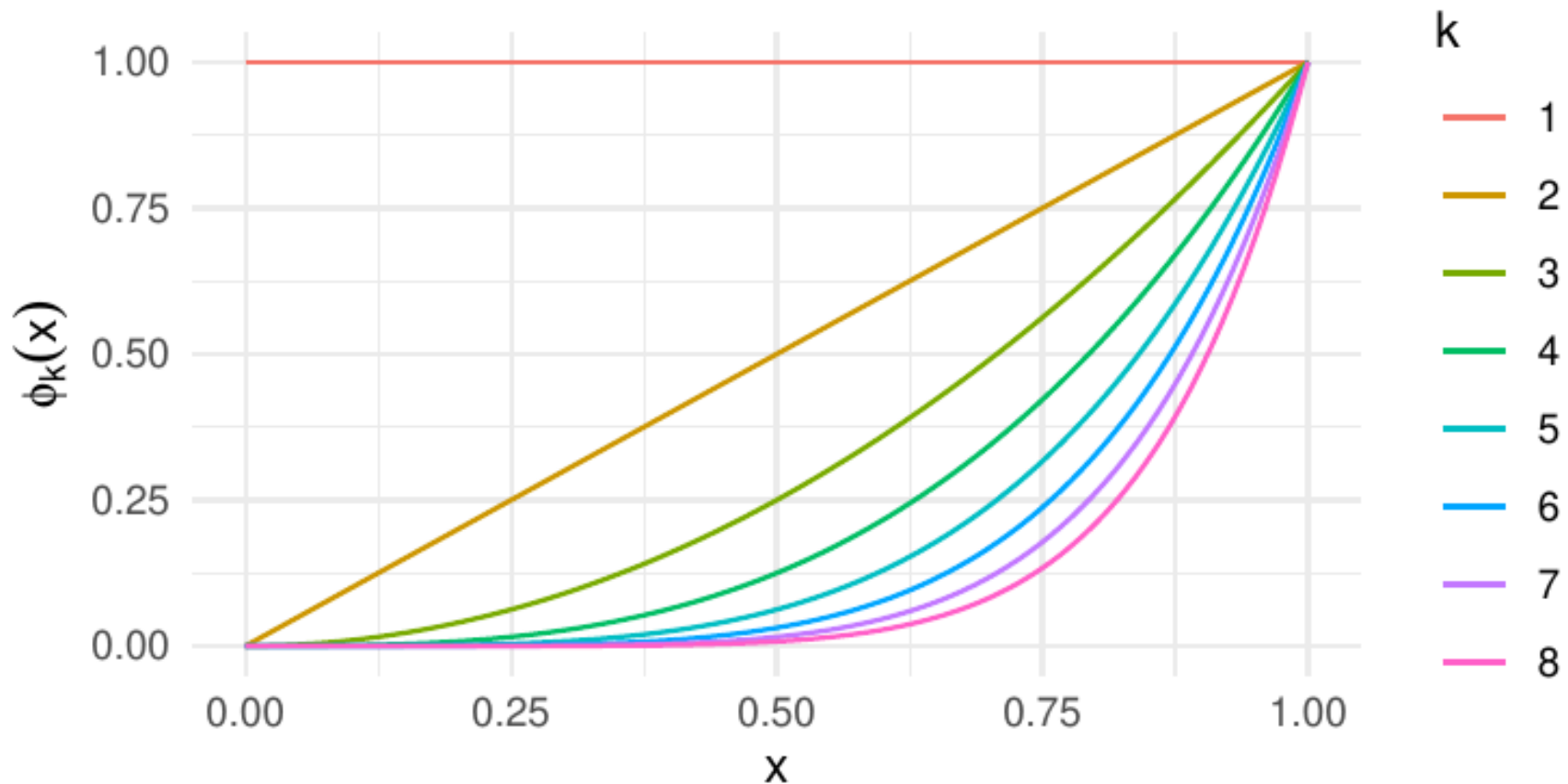
- Angenommen, p und q sind beides interpolierende Polynome vom Grad $\leq n - 1$.
- Dann gilt: $(p - q)(x_i) = p(x_i) - q(x_i) = y_i - y_i = 0$ für alle $i = 1, \dots, n$.
- Also hat $p - q$ mindestens n Nullstellen, aber Grad $\leq n - 1$.
- Nach dem **Fundamentalsatz**: $p - q$ muss das Nullpolynom sein.
- Daher $p = q$. ■

Polynominterpolation

- Alle Polynome vom Grad höchstens $(n - 1)$ können durch die *Monomialbasis*

$$\phi_1(x) = 1, \quad \phi_2(x) = x, \quad \dots, \quad \phi_n(x) = x^{n-1}$$

erzeugt werden.



Achtung: Häufiges Missverständnis

Es gibt **genau ein** Polynom von Grad **höchstens** $n - 1$ durch n Punkte.

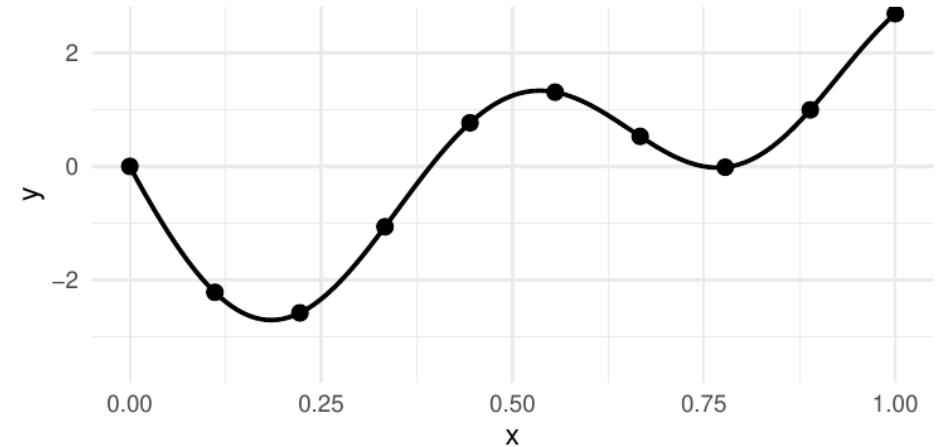
NICHT: Grad **genau** $n - 1$!

Beispiel: $n = 3$ Punkte $(0, 0)$, $(1, 1)$, $(2, 2)$ liegen auf einer Geraden.

- Interpolierendes Polynom: $p(x) = x$ (Grad 1, nicht 2!)
- Das eindeutige Polynom *höchstens* 2. Grades ist: $p(x) = 0 \cdot x^2 + 1 \cdot x + 0$

Polynominterpolation in R

```
1 x <- seq(0, 1, length = 10)
2 y <- f(x)
3
4 # Vandermonde-Matrix: cbind(1, x, x^2, ...)
5 B <- outer(x, 0:9, "^")
6 a <- solve(B, y)
7
8 xnew <- seq(0, 1, length = 100)
9 fhat <- outer(xnew, 0:9, "^") %*% a
10
11 ggplot() +
12   geom_point(aes(x, y)) +
13   geom_line(aes(xnew, fhat))
```



Problem 1: Kondition

- **Monomialbasis** führt zu einer schlecht konditionierten Basismatrix $\mathbf{B}$:

$$\mathbf{x} = (x_1, \dots, x_n)^\top \rightarrow \mathbf{B} = (1 \quad \mathbf{x} \quad \mathbf{x}^2 \quad \dots \quad \mathbf{x}^{n-1})$$

- Die Spalten sind die Vektoren $\mathbf{b}_k = (x_1^{k-1}, \dots, x_n^{k-1})^\top$.
- Für k groß sind diese Vektoren nahezu linear abhängig (vgl. Abbildung der Monomialbasis!)
 $\implies$ Matrix nahezu singulär / schlecht **konditioniert**.

Numerisches Beispiel:

n äquidistante Punkte in $[0, 1]$, Polynom vom Grad $n - 1$:

n	Polynomgrad	$\kappa(\mathbf{B})$
5	4	$\approx 10^3$
10	9	$\approx 10^7$
15	14	$\approx 10^{12}$
20	19	$\approx 10^{16}$

(Größenordnungen für κ_2 ; andere Normen ergeben sehr ähnliche Werte.

Die Kondition hängt zudem von der Knotenlage ab: dieselben Punkte auf $[-1, 1]$ ergeben für $n = 20$ nur $\kappa_2 \approx 10^8$.)

Bei doppelter Präzision (≈ 16 Dezimalstellen) ist für $n \geq 20$ fast alle Genauigkeit verloren!

$\rightarrow$ verwende **orthogonalisierte** Basis (Legendre-, Chebyshev-, Hermite-Polynome) —
in R: `cbind(1, poly(x, 3))` statt `cbind(1, x, x^2, x^3)`

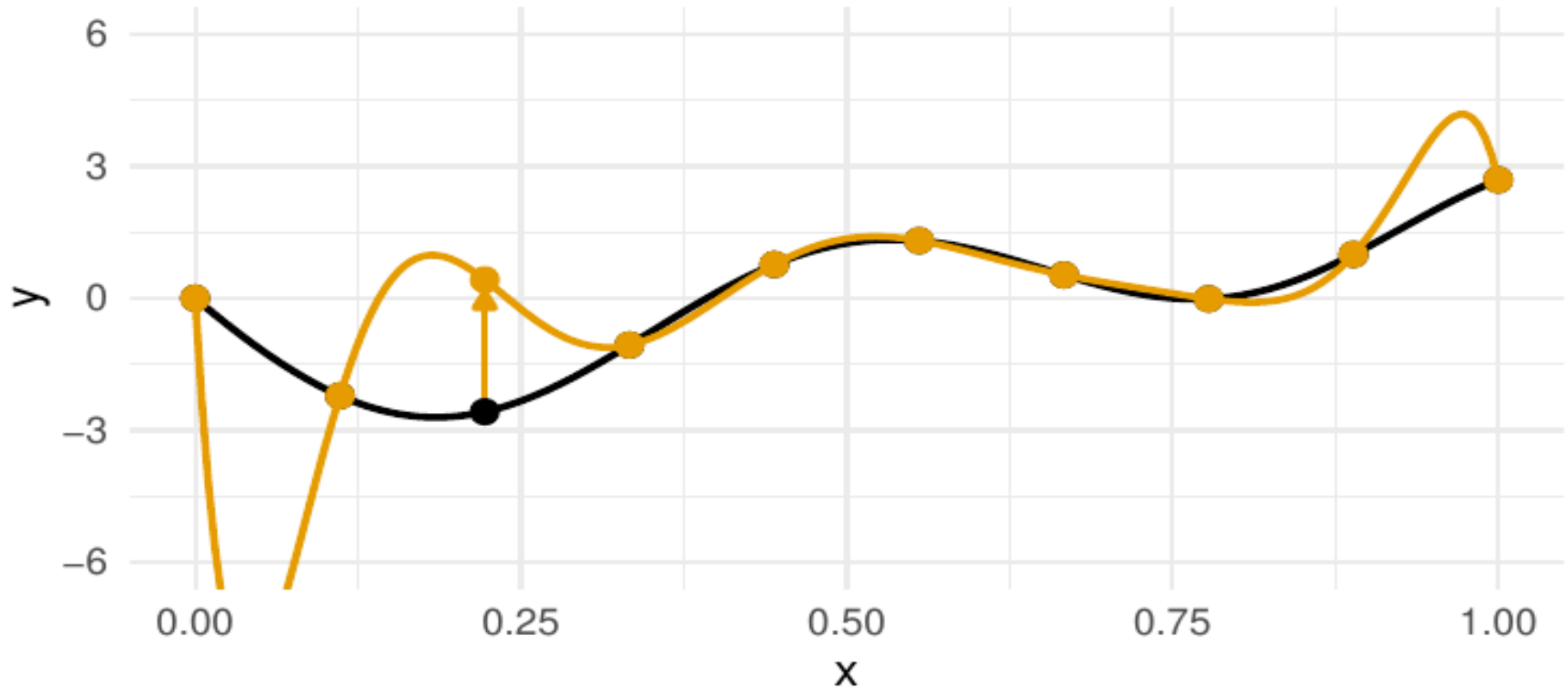
Orthogonale Polynome: Gram-Schmidt im Funktionenraum

- Auf $\mathcal{F}_K$ gibt es auch ein **Skalarprodukt**: $\langle f, g \rangle = \int f(x) g(x) dx$ (analog zu $\mathbf{x}^\top \mathbf{y}$ in $\mathbb{R}^n$)
- Damit funktioniert **Gram-Schmidt-Orthogonalisierung** ($\rightarrow$ **QR-Zerlegung**!) angewendet auf die Monome $\{1, x, x^2, \dots\}$ über $[-1, 1]$:

$$p_2(x) = x^2 - \frac{\langle x^2, 1 \rangle}{\langle 1, 1 \rangle} \cdot 1 - \underbrace{\frac{\langle x^2, x \rangle}{\langle x, x \rangle}}_{=0} \cdot x = x^2 - \frac{1}{3} \propto \underbrace{\frac{1}{2}(3x^2 - 1)}_{\text{Legendre-Polynom } P_2}$$

- **Legendre-, Chebyshev-, Hermite-Polynome** = Gram-Schmidt bzgl. unterschiedlicher Skalarprodukte (Gewichtsfunktionen/Intervalle)
- orthogonale Basisfunktionen $\implies \mathbf{B}$ deutlich besser konditioniert.
- `poly(x, k)` in `R` orthogonalisiert bzgl. des *diskreten* (vektoriellen) Skalarprodukts $\sum_i f(x_i)g(x_i)$.

Problem 2: Stabilität und Lokalität



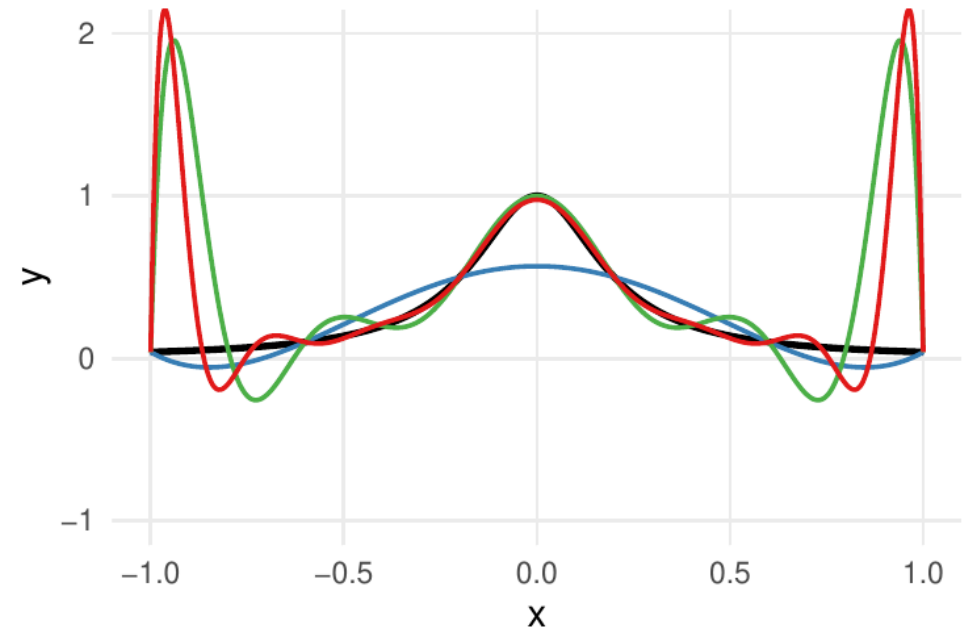
- Ändern wir nur ein y_i , verändert sich (potentiell) die gesamte Funktion. $\implies$ Die Methode ist instabil.
- Dies ist ein grundsätzliches Problem von Polynombasen.

Das Runge-Phänomen

Runge-Phänomen (1901):

Interpoliere $f(x) = \frac{1}{1+25x^2}$ auf $[-1, 1]$ mit n äquidistanten Knoten durch p_{n-1} (Grad $n - 1$).

- f ist glatt ($\in \mathcal{C}^\infty$!)
- Aber: $\|f - p_{n-1}\|_\infty \rightarrow \infty$ für $n \rightarrow \infty$!
- Das Wachstum ist **nicht monoton** (von $n = 5$ auf $n = 10$ fällt der Fehler zunächst); die Fehlermaxima wandern an den Rand.



Anzahl Knoten — $n = 6$ — $n = 11$ — $n = 16$

Lösung:

Stückweise Polynome statt eines globalen Polynoms $\rightarrow$ **Splines**

Splines

Splines

Def.: Polynom-Spline

Sei $[a, b] \subset \mathbb{R}$ ein Intervall.

Ein **Polynom-Spline q -ten Grades** ist eine Funktion $s: [a, b] \rightarrow \mathbb{R}$,

- definiert auf einem *Gitter* von *Knoten*

$$a = \xi_0 < \xi_1 < \dots < \xi_m = b,$$

- so dass

$$s(x) = p_k(x) \quad \text{für } x \in [\xi_{k-1}, \xi_k) \quad (\text{bzw. } x \in [\xi_{m-1}, \xi_m] \text{ für } k = m)$$

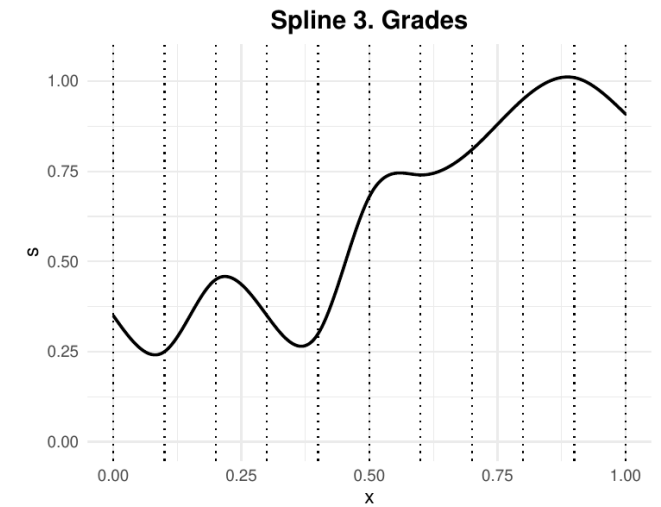
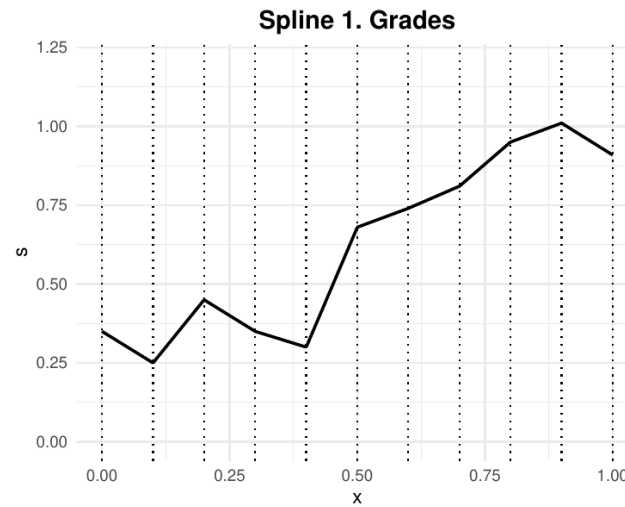
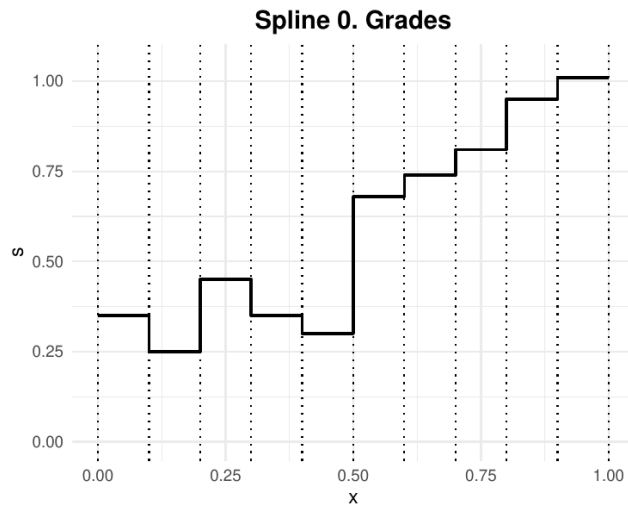
mit *Polynomen* p_k vom Grad höchstens q , $k = 1, \dots, m$,

- mit $(q - 1)$ stetigen Ableitungen: $s \in \mathcal{C}^{q-1}$.

Splines

Weniger formal:

Ein Spline ist eine Funktion, die Polynome an den Punkten $\xi_1, \dots, \xi_{m-1}$ zusammensetzt.



Warum $m + q$ Parameter?

Annahme: einfache innere Knoten, maximale Glattheit C^{q-1}

Parameterzählung:

- Ohne Stetigkeitsbedingungen: $m(q + 1)$ Parameter (m Intervalle, je $q + 1$ Koeffizienten)
- An jedem der $(m - 1)$ inneren Knoten: q Bedingungen (Stetigkeit von $s, s', \dots, s^{(q-1)}$)
- $\implies m(q + 1) - (m - 1) \cdot q = m + q$

Interpolation an $n = m + 1$ **Knoten** mit kubischem Spline ($q = 3$):

$m + 3$ Freiheitsgrade aber nur $n = m + 1$ Interpolationsbedingungen

$\implies$ Es fehlen **2 Bedingungen**!

Gängige **Randbedingungen**:

Typ	Bedingung
Natürlich	$s''(a) = s''(b) = 0$ (lineare Enden)
Eingespannt	$s'(a), s'(b)$ vorgegeben
Periodisch	$s'(a) = s'(b), s''(a) = s''(b)$ (für $y_0 = y_m$)

Natürliche Splines sind der Standard, wenn keine Randinformation vorliegt.

Kubischer Spline: Konstruktion Schritt für Schritt

Gegeben:

4 Punkte $(0, 0), (1, 1), (2, 0), (3, -1) \implies m = 3$ Intervalle

Ansatz:

Je Intervall ein kubisches Polynom $p_k(x)$ mit 4 Koeffizienten.

Bedingungen:

Typ	Anzahl	Beispiel
Interpolation	4	$p_1(\xi_0) = y_0, p_k(\xi_k) = y_k$
Stetigkeit	2	$p_1(1) = p_2(1)$
s' stetig	2	$p'_1(1) = p'_2(1)$
s'' stetig	2	$p''_1(1) = p''_2(1)$
Randbedingung (natürlich)	2	$s''(0) = s''(3) = 0$

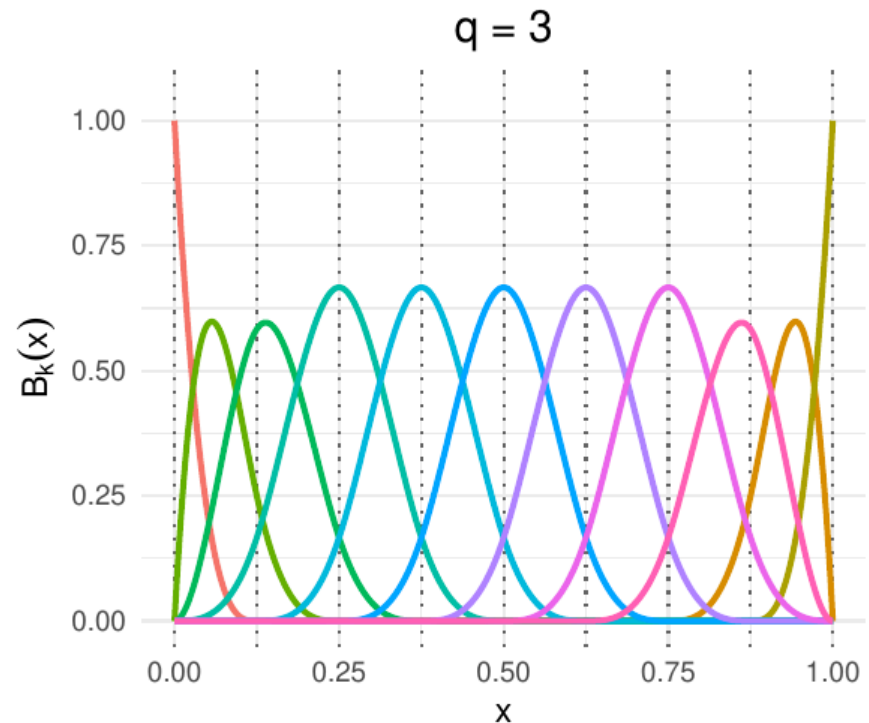
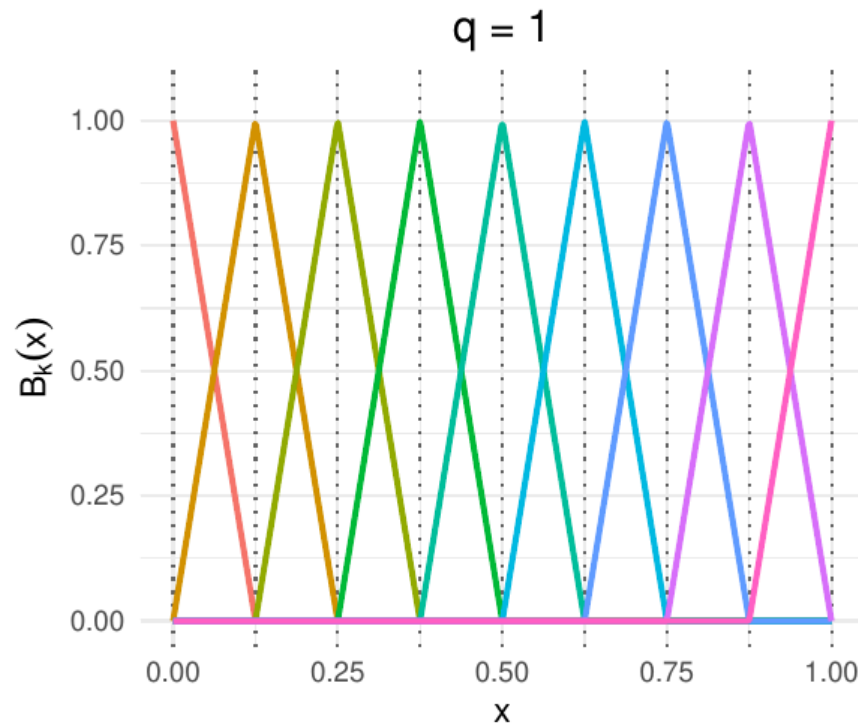
Summe: $4 + 2 + 2 + 2 + 2 = 12$ Bedingungen für $3 \times 4 = 12$ Unbekannte

$\implies$ LGS ist eindeutig lösbar!

B-Splines

- Jeder Spline von Grad q kann auch durch eine Linearkombination von $m + q$ **B-Spline**-Basisfunktionen B_k zum Grad q repräsentiert werden:

$$s(x) = \sum_{k=1}^{m+q} a_k B_k(x)$$



B-Splines: Rekursive Definition

- B-Splines können auf verschiedene Arten definiert werden.
- Rekursive Definition (*Cox-de-Boor-Rekursion*; nur zur Referenz/Transparenz):

- Erweitere die Knotenfolge zu $\tau_1, \dots, \tau_{m+2q+1}$:

$$\tau_1 = \dots = \tau_{q+1} = \xi_0, \quad \tau_{q+1+i} = \xi_i \quad (i = 1, \dots, m-1), \quad \tau_{m+q+1} = \dots = \tau_{m+2q+1} = \xi_m$$

- $q = 0$:

$$B_k^{(0)}(x) = \mathbb{1}(\tau_k \leq x < \tau_{k+1})$$

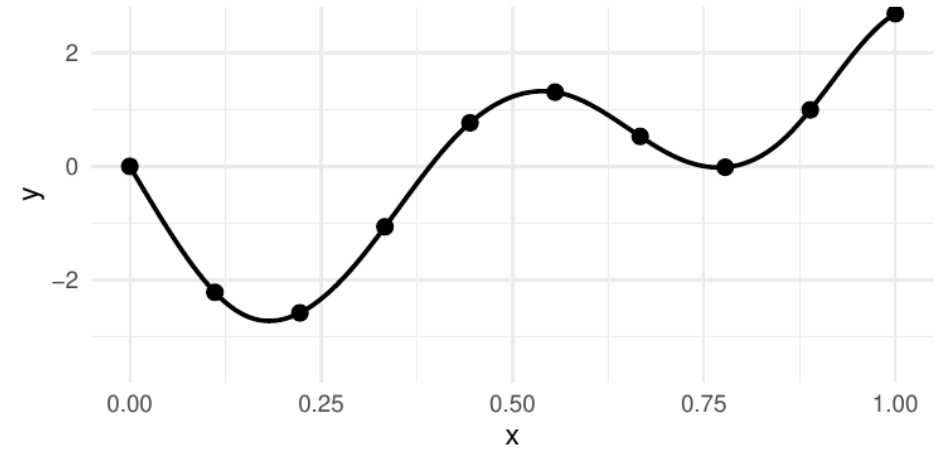
- $q > 0$:

$$B_k^{(q)}(x) = \frac{x - \tau_k}{\tau_{k+q} - \tau_k} B_k^{(q-1)}(x) + \frac{\tau_{k+q+1} - x}{\tau_{k+q+1} - \tau_{k+1}} B_{k+1}^{(q-1)}(x)$$

- in R: `splines::bs()`

B-Splines in R

```
1 y <- f(x)
2 B <- splines::bs(x, df = 10, degree = 3,
3               intercept = TRUE)
4 a <- solve(B, y)
5
6 xnew <- seq(0, 1, length = 100)
7 Bnew <- splines::bs(xnew,
8               knots = attr(B, "knots"),
9               intercept = TRUE)
10 fhat <- Bnew %*% a
```



B-Splines: Numerische Stabilität & Effizienz

Vergleich der Konditionszahlen für $n = 20$ Punkte in $[0, 1]$:

Basis	Polynomgrad	Konditionszahl
Monomialbasis	19	$\kappa(\mathbf{B}_{\text{mono}}) \approx 10^{16}$
B-Splines (kubisch, $K = 20$ Basisfunktionen)	3	$\kappa(\mathbf{B}_{\text{spline}}) \approx 10^2$

B-Splines sind **rund 14 Größenordnungen** besser konditioniert!

(Größenordnungen, hier κ_2 mit den Standardknoten von `splines::bs()`: $1,2 \cdot 10^{16}$ vs. 35; die genauen Werte hängen von Norm und Knotenwahl ab.)

Außerdem: Dünn besetzte **Bandstruktur** (nur $q + 1$ Einträge pro Zeile) der B-Spline-Matrix ermöglicht

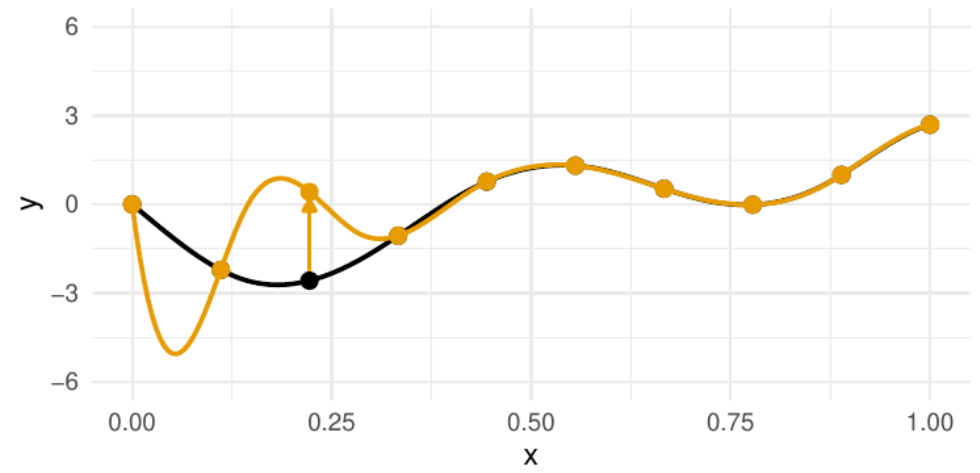
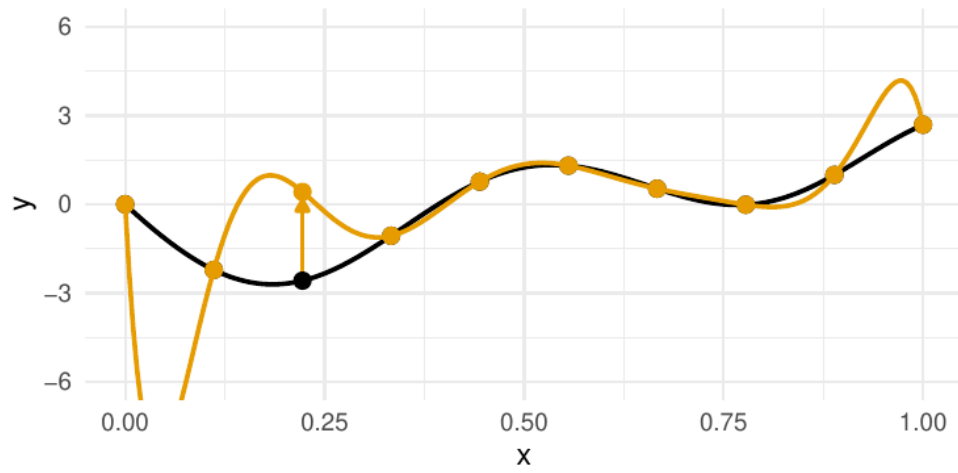
- LGS-Lösung in $O(nq^2)$ statt $O(n^3)$
- Für $n = 1000, q = 3$: Beschleunigungsfaktor $\approx 100\,000$

⇒ B-Splines sind der Standard in der Praxis, trotz komplizierter Definition.

↔ zurück: „(B-)Splines: Eigenschaften“

(B-)Splines: Eigenschaften

- Splines sind das gängigste Basissystem zur Funktionsapproximation, meist **kubisch** ($q = 3$):
 1. Wenn $\max_k |\xi_k - \xi_{k-1}|$ hinreichend klein ist, kann jede stetige Funktion f beliebig genau approximiert werden.
 2. Die Ableitung eines Grad- q -Splines ist ein Grad- $(q - 1)$ -Spline, das Integral ein Grad- $(q + 1)$ -Spline.
 3. Ableitungen/Integrale eines Splines approximieren auch die Ableitungen/Integrale der Zielfunktion — für die Ableitungen allerdings nur, wenn f selbst entsprechend oft differenzierbar ist, und mit je einer Ordnung langsamerer Konvergenz.
 4. B-Splines sind numerisch stabil (vgl. **B-Splines: Numerische Stabilität & Effizienz**);
haben **lokale Träger**: $B_k(x) = 0$ für $x \notin [\tau_k, \tau_{k+q+1}] \implies$ Approximation ist **lokal**, also stabiler;
 $\mathbf{B}$ hat *Band-Struktur* $\implies$ effizient lösbar.



Polynominterpolation (links) vs. Spline-Interpolation (rechts): Änderung *eines* Datenpunkts.

Self-Check & Zusammenfassung

Wrap-up

Heute gelernt

- *Approximation, Interpolation, Glättung*
- *Funktionsräume*: Funktionen als Vektoren, *Basisfunktionen* wie Basisvektoren
- Lösungsansatz durch *Basisdarstellung*: Interpolation $\rightarrow$ LGS $\mathbf{B}\mathbf{a} = \mathbf{y}$
- Polynombasen und ihre Probleme (Kondition, *Runge-Phänomen*); Ausweg: orthogonale Polynome = Gram-Schmidt im Funktionenraum
- *Splines, B-Splines* und ihre Vorteile (Lokalität, Stabilität, Effizienz)

Nächste Vorlesung: Funktionsapproximation II — Glättungsproblem, Bias-Varianz-Trade-off

Self-Check

Frage 1: Zur Charakterisierung eines Splines q -ten Grades auf $m + 1$ Knoten benötigen wir...

1. $q + 1$ Parameter.
2. $m(q + 1)$ Parameter.
3. $m + q$ Parameter.

Frage 2: Ein kubischer Spline auf dem Knotengitter $\xi_0 < \xi_1 < \dots < \xi_5$ (6 Knoten). Wie viele Basiskoeffizienten hat er?

1. 6
2. 8
3. 9
4. 24

Frage 3: Warum verwendet man in der Praxis B-Splines statt der Monomialbasis? (*mehrere Antworten möglich*)

1. Die B-Spline-Basismatrix ist deutlich besser konditioniert.
2. Nur B-Splines können jede stetige Funktion beliebig genau approximieren.
3. Lokale Träger $\rightarrow$ Bandstruktur $\rightarrow$ effiziente Lösung des LGS.
4. Mit der Monomialbasis hat das Interpolationsproblem keine eindeutige Lösung.

Ausblick: Weiterführende Themen

Weiterführende Themen & Vorlesungen:

- **Mehrdimensionale Interpolation:** Tensor-Produkt-Splines, radiale Basisfunktionen
- **Glättung mit Regularisierung:** P-Splines, Generalisierte Additive Modelle (→ *Statistik V: Konzepte Statistischer Modellierung*)
- **Moderne ML-Anwendungen:** Spline-basierte Aktivierungsfunktionen (KANs), *Normalizing Flows*

Funktionsapproximation: Krümmung, Approximation, Glättung

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)

Material: Thomas Nagler

LMU München

Verwendete Vorkenntnisse

- **Lineare Algebra:**

- *Kleinste Quadrate* und *Pseudoinverse*

- **Analysis:**

- $\mathcal{C}^k$ -Funktionen (insb. $\mathcal{C}^2$ und $\mathcal{C}^4$): k -mal stetig differenzierbar
- Integration $(\int |f''(x)|^2 dx)$

- **Funktionsapproximation I:**

- *Basen von Funktionenräumen*: $\hat{f}(x) = \sum_k a_k \phi_k(x)$ mit *Basisfunktionen* ϕ_k und *Basiskoeffizienten* a_k
- *(B-)Splines*
- *Interpolation* vs. *Glättung*

- **Statistik:**

- Designmatrix eines Linearen Modells
- Bias-Varianz-Zerlegung und MSE
- Kreuzvalidierung, Informationskriterien (AIC, BIC)

Opener-Quiz: Was wissen Sie noch?

Frage 1:

Was bedeutet $f \in \mathcal{C}^4[a, b]$?

1. f ist auf $[a, b]$ viermal differenzierbar.
2. f ist auf $[a, b]$ viermal *stetig* differenzierbar.
3. f ist stetig und durch 4 beschränkt.
4. f ist ein Polynom vom Grad höchstens 4.

Frage 2:

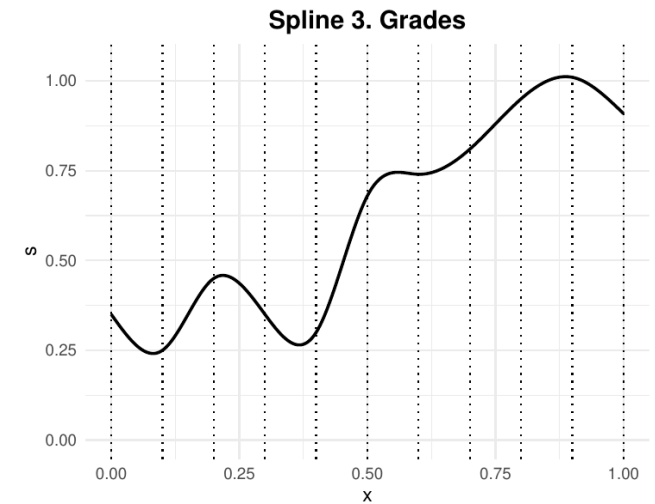
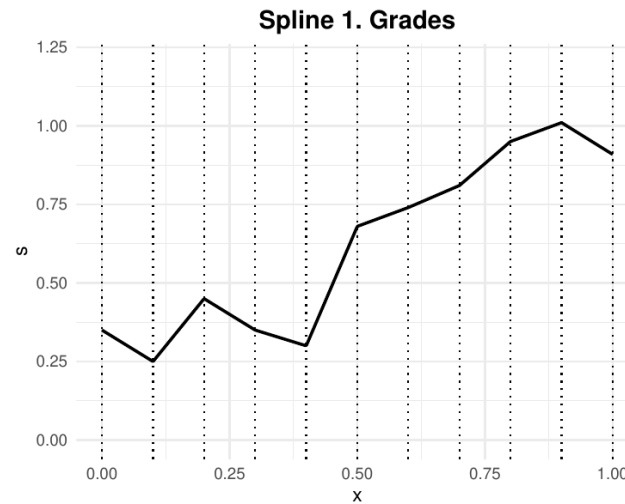
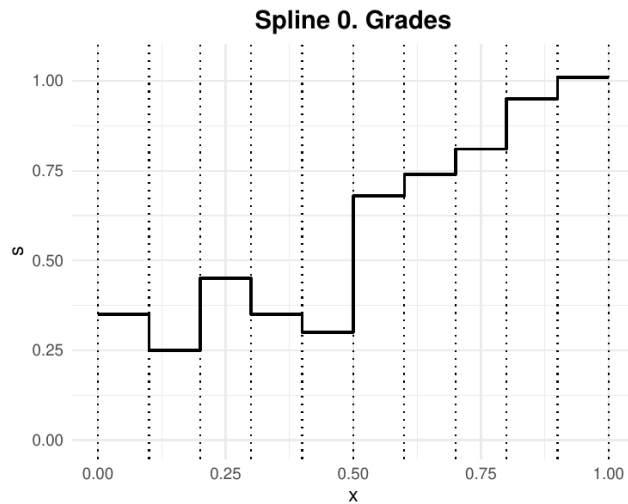
Sei $\mathbf{B} \in \mathbb{R}^{n \times K}$ mit vollem Spaltenrang ($K < n$). Die KQ-Lösung von $\min_{\mathbf{a}} \|\mathbf{y} - \mathbf{B}\mathbf{a}\|_2^2$ ist ...

1. $\hat{\mathbf{a}} = \mathbf{B}^{-1}\mathbf{y}$
2. $\hat{\mathbf{a}} = \mathbf{B}^\top \mathbf{y}$
3. $\hat{\mathbf{a}} = \mathbf{B}^+ \mathbf{y} = (\mathbf{B}^\top \mathbf{B})^{-1} \mathbf{B}^\top \mathbf{y}$
4. $\hat{\mathbf{a}} = (\mathbf{B}\mathbf{B}^\top)^{-1} \mathbf{B}\mathbf{y}$

Nützliche Theorie zu Splines

Splines

Erinnerung: Ein **Spline** vom Grad q ist eine Funktion, die Polynome an den Punkten $\xi_1, \dots, \xi_{m-1}$ **glatt** zusammensetzt: dort stimmen alle Ableitungen bis zur Ordnung $q - 1$ überein, d.h. $s \in \mathcal{C}^{q-1}$.



- Splines von Grad ℓ zu festen Knoten bilden einen **Funktionenraum**
- **B-Splines** sind eine besonders gutartige **Basis** dieses Raums.

Minimale Krümmung

Kubische Splines erzeugen besonders einfache “glatte” Kurven.

Das ist kein Zufall:

Man kann sie herleiten, indem man Interpolanden mit *minimaler Krümmung* sucht (diese Eigenschaft hat auf englisch auch den schönen Namen *minimal wiggleness*).

Theorem: Kubische Splines haben minimale Krümmung

Seien $(x_1, y_1), \dots, (x_n, y_n) \in [a, b] \times \mathbb{R}$ mit $a = x_1 < x_2 < \dots < x_n = b$ und s ein *natürlicher kubischer Spline*, der diese Punkte interpoliert (d.h. mit $s''(a) = s''(b) = 0$).

Sei $g \in \mathcal{C}^2[a, b]$ eine weitere Funktion mit $g(x_i) = y_i \quad \forall i$.

Dann gilt:

$$\int_a^b |s''(x)|^2 dx \leq \int_a^b |g''(x)|^2 dx.$$

“Krümmung” meint hier $\int_a^b |f''(x)|^2 dx$, nicht das Integral der geometrischen Krümmung $\kappa = |f''| / (1 + (f')^2)^{3/2}$ – beide fallen nur für flache Kurven mit $|f'| \ll 1$ zusammen. Der Name ist trotzdem eingebürgert.

Minimale Krümmung: Beweis

Sei s der natürliche kubische Spline-Interpolant zu (x_i, y_i) .

Für einen anderen Interpolanten $g \in \mathcal{C}^2$ setze $g = s + h$ mit $h(x_i) = 0 \forall i$.

Sei $J(g) := \int_a^b (g''(x))^2 dx$. Dann gilt

$$J(g) = J(s) + 2 \int_a^b s''(x)h''(x)dx + \int_a^b (h''(x))^2 dx.$$

Auf jedem Intervall $[x_{i-1}, x_i]$, $i = 2, \dots, n$, gilt durch partielle Integration

$$\int s''h''dx = \left[s''h' - s^{(3)}h \right]_{x_{i-1}}^{x_i} + \int s^{(4)}h dx.$$

- für kubische Splines ist $s^{(4)} = 0$
- $\sum_{i=2}^n \left[s''h' - s^{(3)}h \right]_{x_{i-1}}^{x_i} = \left[s''h' \right]_a^b = 0$, weil
 - $\left[s^{(3)}h \right]_{x_{i-1}}^{x_i} = 0$ wegen $h(x_i) = 0$,
 - sich die inneren $s''h'$ -Randterme paarweise aufheben (s'' und h' sind an den inneren Knoten stetig,
 - und $s''(a) = s''(b) = 0$ (natürlicher Spline).

$$\implies J(g) = J(s) + \int_a^b (h''(x))^2 dx \geq J(s) \implies \text{Gleichheit nur für } h \equiv 0. \quad \blacksquare$$

Letzter Schritt:

Gleichheit heißt $\int_a^b (h'')^2 dx = 0$, also $h'' \equiv 0$ (h'' ist stetig), also h affin;

da h an den $n \geq 2$ Stellen x_i verschwindet, folgt $h \equiv 0 \implies$ Der Spline s ist der **einzige** Minimierer.

Minimale Krümmung: Beispiel

Gegeben:

Punkte $(0, 0)$, $(1, 1)$, $(2, 0)$

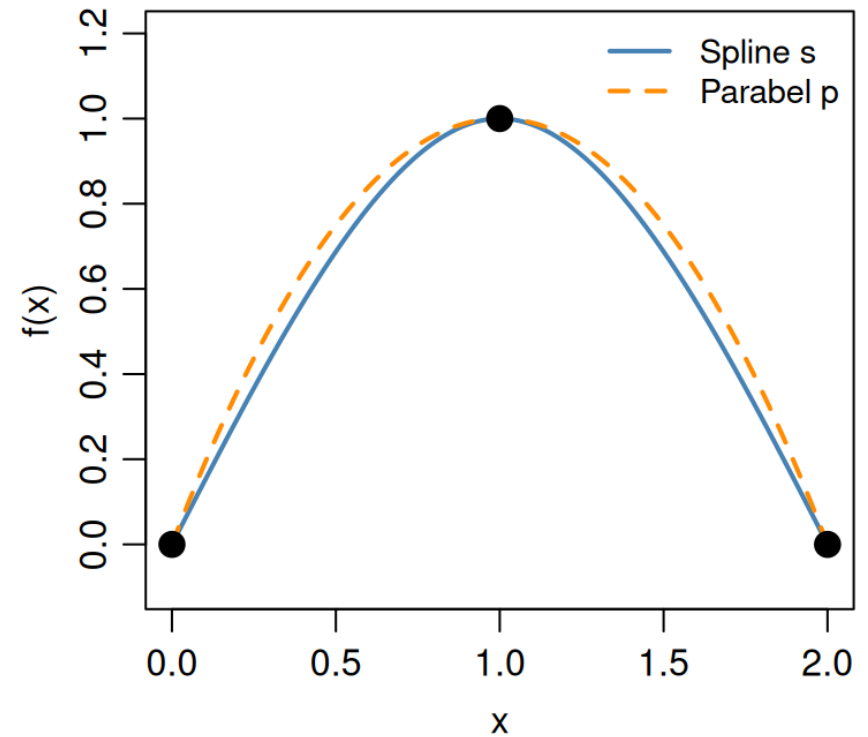
Natürlicher kubischer Spline:

$$s(x) = \begin{cases} \frac{3}{2}x - \frac{1}{2}x^3 & x < 1 \\ \frac{3}{2}(2-x) - \frac{1}{2}(2-x)^3 & x \geq 1 \end{cases}$$

$$s''(x) = -3x \text{ bzw. } s''(x) = -3(2-x) \\ \implies s''(0) = s''(2) = 0 \checkmark$$

Vergleich mit Parabel:

$$p(x) = -x^2 + 2x$$



Interpolant	$\int_0^2 f''(x) ^2 dx$
-------------	--------------------------

Spline s	6
------------	---

Parabel p	8
-------------	---

$\implies$ Spline hat kleinere Krümmung!

Approximationseigenschaft

- Bereits erwähnt: Wenn die *Gitterweite* (der größte Abstand zweier benachbarter Knoten) hinreichend klein ist, kann jede stetige Funktion f beliebig genau approximiert werden. Genauer:

Theorem: Approximationsfehler kubischer Splines

Sei $a = x_0 < x_1 < \dots < x_n = b$ eine Partition mit **Gitterweite** $h := \max_i |x_i - x_{i-1}|$. Für jedes $f \in \mathcal{C}^4[a, b]$ sei s der eindeutig bestimmte *eingespannte kubische Spline-Interpolant* mit

$$f(x_i) = s(x_i) \quad \forall i, \quad s'(a) = f'(a), \quad s'(b) = f'(b).$$

Dann gilt

$$\max_{x \in [a, b]} |f(x) - s(x)| \leq \frac{5}{384} h^4 \max_{x \in [a, b]} |f^{(4)}(x)|.$$

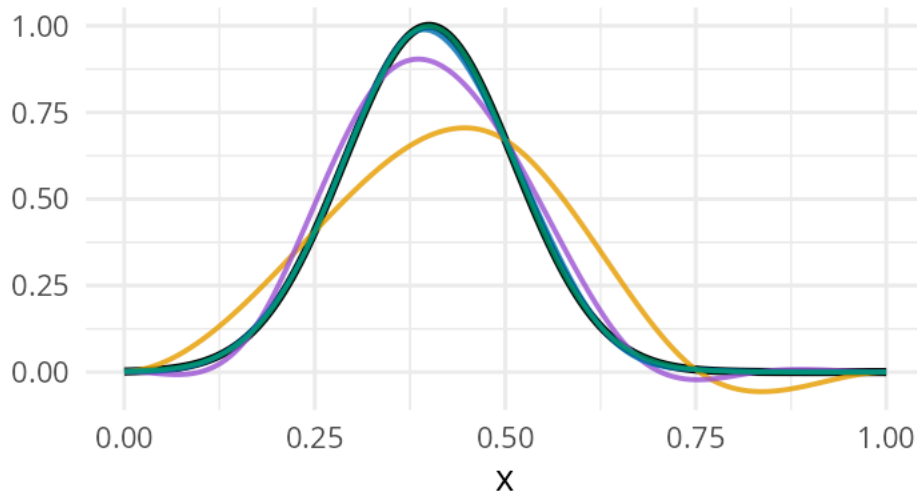
Erkenntnisse:

Approximation: Lokaler Fehler

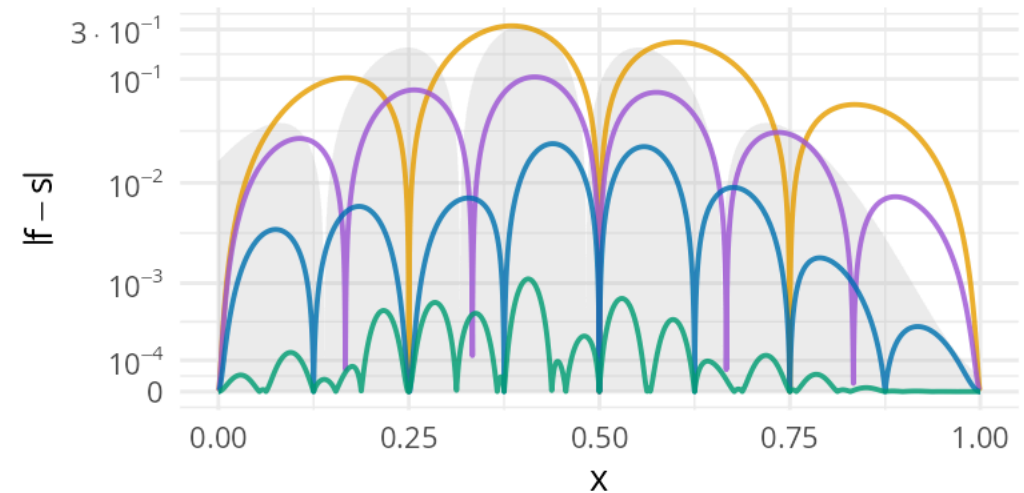
Gegeben:

$f(x) = \exp\left(-40\left(x - \frac{2}{5}\right)^2\right)$ auf $[0, 1]$ mit $\max |f^{(4)}(x)| = 19200$ (am schmalen Peak $x = \frac{2}{5}$, der kein Knoten ist)

Spline-Interpolanten



Lokale Fehler (log(1+)-Skala)



— $f(x)$ — $n+1 = 5$ — $n+1 = 7$ — $n+1 = 9$ — $n+1 = 17$ $|f^{(4)}|$ (skaliert)

Beobachtung:

Mit nur fünf Knoten ist die Abweichung deutlich sichtbar. Sie konzentriert sich um den schmalen Peak, wo $|f^{(4)}|$ groß ist; an den Knoten selbst ist der Interpolationsfehler natürlich null.

Approximation: h^4 -Rate

Knoten $n + 1$	Gitterweite h	Fehlerschranke $\frac{5}{384} h^4 \max_x f^{(4)}(x) $	Tatsächlicher Fehler $\max_x f(x) - s(x) $
5	0.25	≈ 0.977	≈ 0.324
7	0.1667	≈ 0.193	≈ 0.1048
9	0.125	≈ 0.061	≈ 0.0239
17	0.0625	≈ 0.0038	≈ 0.00111

Beobachtung:

- Halbierung von h verkleinert die *Schranke* exakt um $2^4 = 16$.
- Der tatsächliche Fehler muss bei groben Gittern nicht exakt diesem Faktor folgen (hier: ≈ 14 und ≈ 22): $O(h^4)$ beschreibt die *asymptotische* Rate.

Berechnet für den **eingespannten** Spline-Interpolanten aus dem Theorem, also mit $s'(0) = f'(0)$ und $s'(1) = f'(1)$.

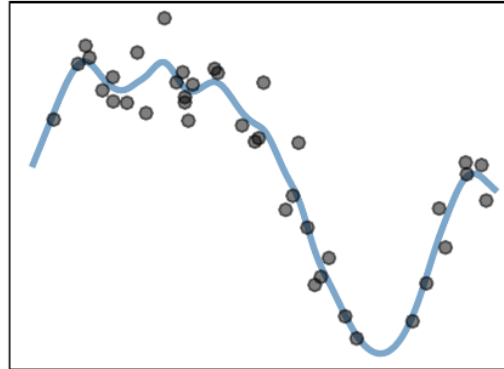
Approximation: Zusammenfassung

- (B-)Splines sind extrem mächtige Werkzeuge in der *Funktionsapproximation*.
- Sie haben hervorragende theoretische und praktische Eigenschaften:
 - Effiziente Berechnung (B-Splines)
 - Stabilität/Lokalität (B-Splines)
 - Minimale Krümmung des natürlichen kubischen Spline-Interpolanten
 - Approximationsfehler $O(h^4)$ für kubische Splines

Glättungs-/Regressionsprobleme

Das Problem

Glättung



Modell:

$$y_i = f(x_i) + \epsilon_i, \quad \epsilon_i \text{ i.i.d.} \quad \text{mit } \mathbb{E}[\epsilon_i | x_i] = 0.$$

Problem:

Finde eine Funktion $\hat{f}$ mit kleinem Fehler

$$\sum_{i=1}^n \left(y_i - \hat{f}(x_i) \right)^2.$$

Über **alle** Funktionen ist dieses Minimum 0 und wird von jedem Interpolanten angenommen (davon gibt es unendlich viele; siehe Kapitel Funktionsapproximation I) $\rightarrow$ entartetes Problem.

Wir brauchen eine Einschränkung des Suchraums!

Die Lösung

1. Nehme an, dass $\hat{f} \in \mathcal{F}_K$, d.h. $\hat{f}(x) = \sum_{k=1}^K a_k \phi_k(x)$ für

- **Basisfunktionen** $\phi_1, \dots, \phi_K$ (bekannt; z.B. **B-Spline-Basis**)
- **Basiskoeffizienten** $a_1, \dots, a_K$ (unbekannt)

2. Löse das KQ-Problem

$$\min_{\hat{f} \in \mathcal{F}_K} \sum_{i=1}^n \left(y_i - \hat{f}(x_i) \right)^2 = \min_{\mathbf{a}} \left\| \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} - \begin{pmatrix} \phi_1(x_1) & \cdots & \phi_K(x_1) \\ \vdots & \ddots & \vdots \\ \phi_1(x_n) & \cdots & \phi_K(x_n) \end{pmatrix} \begin{pmatrix} a_1 \\ \vdots \\ a_K \end{pmatrix} \right\|_2^2.$$

Unterschied zu Interpolation:

- **Interpolation**: $K = n$
- **Glättung**: $K < n$

Warum Glättung statt Interpolation?

Interpolation ($K = n$):

- Passt **alle** Datenpunkte exakt an:

$$\hat{f}(x_i) = y_i \quad \forall i,$$

sofern $\mathbf{B}$ invertierbar ist (mit ungünstig platzierten Knoten kann $\mathbf{B}$ auch für $K = n$ singulär sein)

- **Problem:**

$\hat{f}$ modelliert auch das Rauschen ε_i !

Glättung ($K < n$):

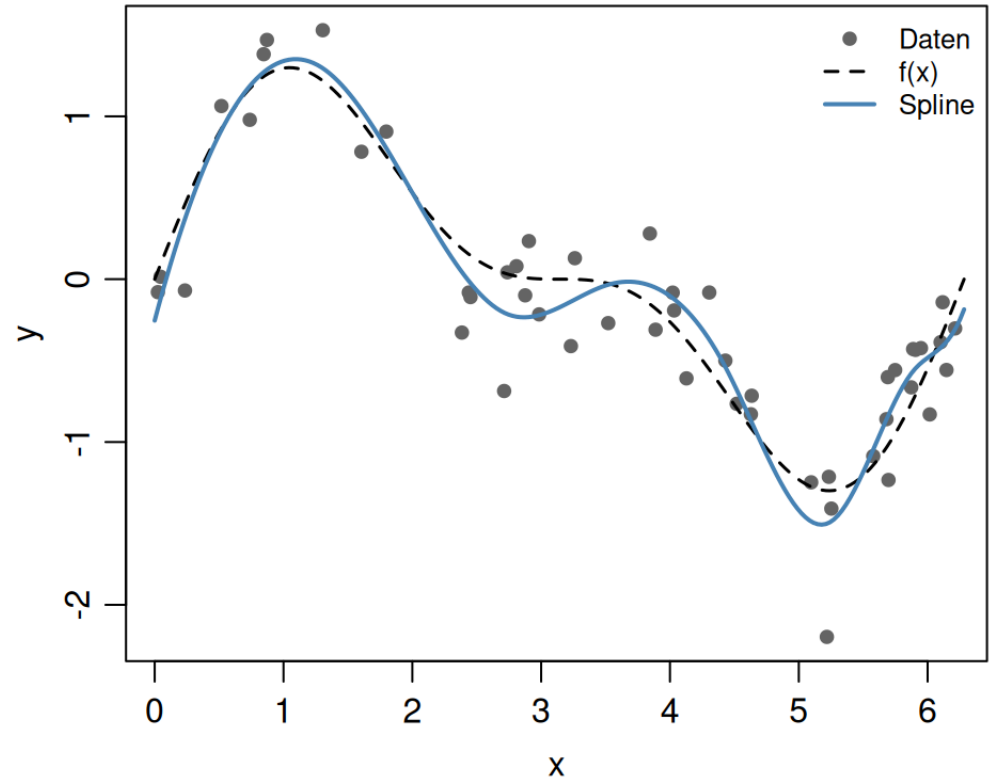
- Akzeptiert, dass $\hat{f}(x_i) \neq y_i$
- Erzielt eine **glattere, generalisierbarere** Funktion
- *Alternative:*
 - **Smoothing Splines** mit $K = n + \text{Glattheits-Penalty } \lambda \int |f''|^2$
(auch: **Penalized Splines** mit $K < n + \text{Glattheits-Penalty} \rightarrow \text{Additive Modelle}$)

Wann welche Methode?

Situation	Methode
Präzise, rauschfreie Daten	Interpolation
Verrauschte Messungen	Glättung
Vorhersage für neue Daten	Glättung

Beispiel

```
1 set.seed(42)
2 n <- 50
3 x <- sort(runif(n, 0, 2 * pi))
4 f_true <- function(x) sin(x) + 0.5 * sin(2 * x)
5 y <- f_true(x) + rnorm(n, sd = 0.3)
6
7 # Designmatrix: B-Spline-Basis, K = 10
8 B <- splines::bs(x, df = 10, intercept = TRUE,
9   Boundary.knots = c(0, 2 * pi))
10 a_hat <- qr.solve(B, y) # KQ via QR
11 #! äquivalent:
12 #!  $Q * R * a = y \rightarrow R * a = Q^T * y \rightarrow a = R^{-1} * Q^T * y$ 
13 #! qrB <- qr(B)
14 #! a_hat <- backsolve(qr.R(qrB), t(qr.Q(qrB)) %*% y)
15
16 # Auswertung auf neuen Punkten
17 x_new <- seq(0, 2 * pi, length = 200)
18 B_new <- splines::bs(x_new,
19   knots = attr(B, "knots"),
20   Boundary.knots = attr(B, "Boundary.knots"),
21   intercept = TRUE)
22 f_hat <- B_new %*% a_hat
```



Bias-Varianz Trade-off: Bias

Annahme: gleichmäßige Knoten auf $[a, b]$, $f \in \mathcal{C}^4$, kubische Splines

- Für Spline mit K Basisfunktionen:

$$h = \max_i |\xi_i - \xi_{i-1}| = O(1/K).$$

- Wäre $\epsilon_i = 0$, hätten wir

$$\max_{x \in [a, b]} |f(x) - \hat{f}(x)| = O(h^4) = O(K^{-4}).$$

- Der Approximationsfehler wird im statistischen Kontext auch **Bias** genannt.
- Für **kleinen Bias** müssen wir also K **möglichst groß** wählen.

Bias-Varianz Trade-off: Varianz

Annahme: $\mathbb{E}\text{Var}[\epsilon_i] = \sigma^2 \forall i$ (homoskedastisch)*

- Da $\boldsymbol{\epsilon} = (\epsilon_1, \dots, \epsilon_n)^\top$ zufällig ist, ist auch

$$\hat{\mathbf{a}} = \mathbf{B}^+ \mathbf{y} = \mathbf{B}^+ \mathbf{f} + \mathbf{B}^+ \boldsymbol{\epsilon}$$

zufällig.

$\mathbf{B}^+$ ist **Pseudoinverse** der Designmatrix $\mathbf{B} = [\phi_k(x_i)] \in \mathbb{R}^{n \times K}$ des KQ-Problems für $\mathbf{f} = (f(x_1), \dots, f(x_n))^\top$.

- Man kann zeigen, dass

$$\text{Var} [\hat{f}(x)] = \text{Var} \left[\sum_{k=1}^K \hat{a}_k \phi_k(x) \right] = O(K/n).$$

- Für **kleine Varianz** wollen wir also K **möglichst klein** wählen.

Exakt ist das im Mittel über die Designpunkte: $\frac{1}{n} \sum_{i=1}^n \text{Var} [\hat{f}(x_i)] = \sigma^2 K/n$ (Spur der Hatmatrix = K , falls $\mathbf{B}$ vollen Spaltenrang hat); punktwise braucht die Rate Zusatzannahmen an Knoten und Design.

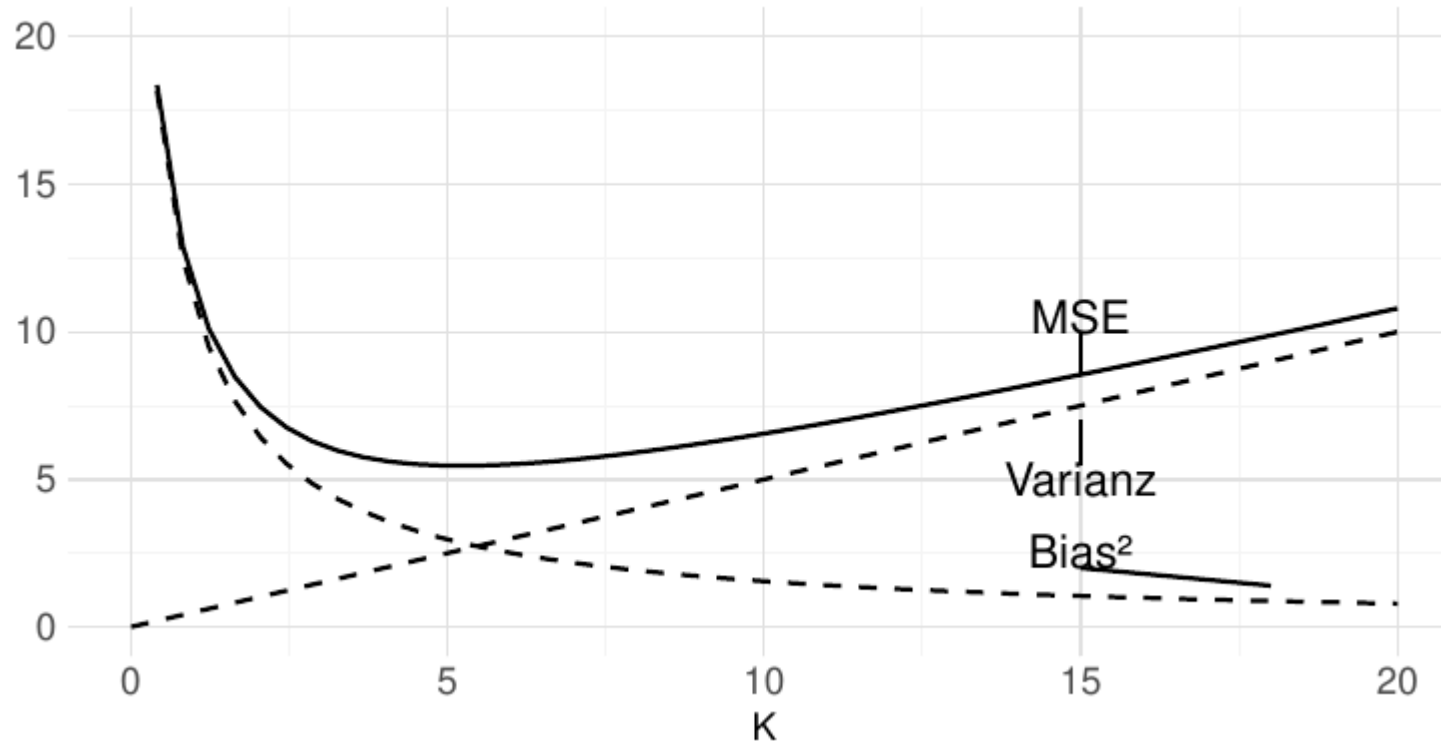
Vorhersage: Wenn K wächst — steigt Bias oder Varianz?

Think–Pair–Share (2 Minuten, mit Sitznachbar:in):

- Wenn K wächst: Was passiert mit dem **Bias**? Was mit der **Varianz**?
- Skizzieren Sie **Bias², Varianz und $MSE = \text{Bias}^2 + \text{Varianz}$** als Funktionen von K (auf Papier!).
- Wo liegt das optimale K — eher klein, eher groß, oder dazwischen? Woran erkennt man es in Ihrer Skizze?

→ Auflösung im folgenden Abschnitt zum Bias-Varianz-Trade-off.

Bias-Varianz Trade-off



- **Auflösung:**

Bias² fällt in K , Varianz steigt in K – der MSE setzt sich aus beiden zusammen:

$$\mathbb{E} \left[\left(\hat{f}(x) - f(x) \right)^2 \right] = \left(\mathbb{E} \left[\hat{f}(x) \right] - f(x) \right)^2 + \mathbb{V}\text{ar} \left[\hat{f}(x) \right] = O(K^{-8} + K/n).$$

- Das optimale K liegt *dazwischen*: am Minimum der MSE-Kurve, wo sich fallender Bias und steigende Varianz gerade ausbalancieren.

Die (theoretisch) optimale Wahl von K

$$\mathbb{E} \left[\left(\hat{f}(x) - f(x) \right)^2 \right] = O(K^{-8} + K/n)$$

- Die (theoretisch!) optimale Wahl $K \sim n^{1/9}$ balanciert beide Teile aus und es gilt dann:

$$\mathbb{E} \left[\left(\hat{f}(x) - f(x) \right)^2 \right] = O(n^{-8/9}).$$

- *Rein asymptotische Ordnung* — taugliches K in der Praxis deutlich größer, da Konstanten bei moderatem n dominieren.

Bias-Varianz: Beispiel

Setup:

$$f(x) = \sin(3x), n = 100, \sigma = 0.3$$

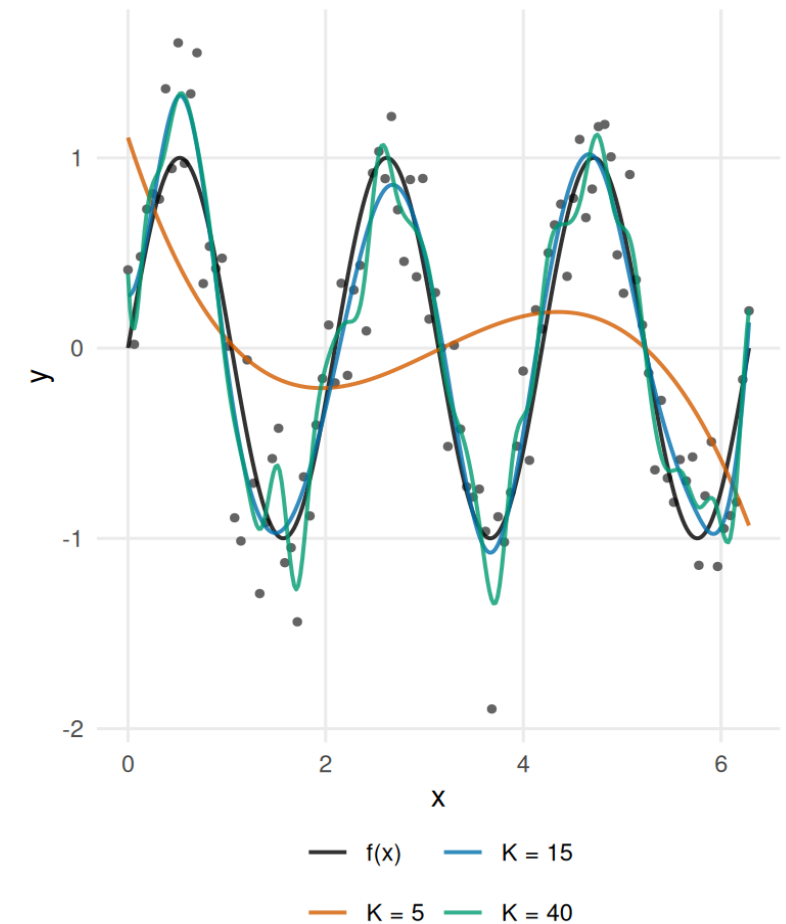
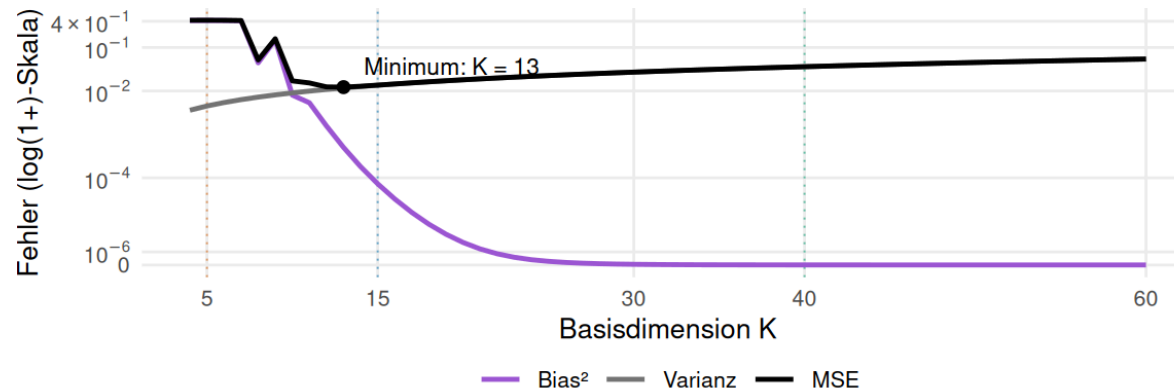
Modell	K	Bias ²	Var	MSE
Unteranpassung	5	0.416	0.005	0.421
Nahezu optimal	15	0.000	0.014	0.014
Überanpassung	40	0.000	0.036	0.036

(Bias² und Varianz exakt berechnet für den KQ-Fit, gemittelt über äquidistante Designpunkte;
 $\text{Var} = \sigma^2 K/n$)

Interpretation:

- $K = 5$: Spline zu starr $\implies$ hoher Bias
- $K = 40$: Spline passt Rauschen an $\implies$ hohe Varianz
- $K = 15$: Balanciert Bias und Varianz

Über $K \in 4, \dots, 60$



Wie wählt man K in der Praxis?

Problem:

Wir kennen f nicht, also wissen wir nicht, ob K optimal ist!

Mögliche Ansätze:

1. **Kreuzvalidierung**: Wähle K , das den Validierungsfehler minimiert
2. **Informationskriterien**: AIC und BIC balancieren Anpassung und Komplexität
3. **GCV**:
$$\text{GCV}(K) = \frac{\text{RSS}_K/n}{(1-K/n)^2}$$

Praktisch verwendeter Ansatz:

- wähle K “groß genug”
- steuere Glattheit der Funktionsschätzung über **Penalisierung**.
→ **P-Splines**, **Additive Modelle**
In R z.B. `mgcv : gam()`

Multivariate Funktionsapproximation

Multivariate Funktionsapproximation

- Zur Approximation von multivariaten $f: \mathbb{R}^p \rightarrow \mathbb{R}$ kann man eine **Tensor-Produkt-Basis** bilden (vgl. Tensorproduktbasen im Kapitel *Tensoren*):

$$\hat{f}(\mathbf{x}) = \sum_{j_1=1}^K \cdots \sum_{j_p=1}^K a_{j_1, \dots, j_p} (\phi_{j_1}(x_1) \times \cdots \times \phi_{j_p}(x_p))$$

mit Koeffizienten

$$\mathbf{A} = (a_{j_1, \dots, j_p})_{j_i=1, \dots, K, i=1, \dots, p} \in \mathbb{R}^{K \times \cdots \times K}.$$

- $\mathbf{A}$ kann wieder durch Lösung eines gewöhnlichen KQ-Problems gefunden werden.
- Aber: **Fluch der Dimensionalität**
 - $\hat{f}$ auswerten kostet $O(K^p)$ (mit B-Splines wegen der lokalen Träger nur $(q+1)^p$ Summanden – ebenfalls exponentiell in p).
 - $\text{MSE}(\hat{f}) = O(K^{-8} + K^p/n)$; mit “optimalem” $K \sim n^{1/(8+p)}$: $\text{MSE} = O(n^{-8/(8+p)})$
 $\implies$ sehr langsam für großes p .

Fluch der Dimensionalität

Szenario: $f: \mathbb{R}^p \rightarrow \mathbb{R}$ mit $K = 10$ Basisfunktionen pro Dimension

Speicher:

Dim. p	Koeffizienten K^p	Speicher
1	10	80 Bytes
2	100	800 Bytes
3	1 000	8 kB
5	100 000	800 kB
10	10^{10}	80 GB

Dezimale Präfixe: 1 kB = 10^3 Bytes, 1 GB = 10^9 Bytes.

⇒ Speicherbedarf und Datenbedarf wachsen **exponentiell** mit p !

Konvergenz (optimales $K \sim n^{1/(8+p)}$):

Dim. p	Konvergenzrate	n für $MSE \leq 0.01$
1	$n^{-8/9}$	≈ 180
2	$n^{-0.8}$	≈ 320
5	$n^{-8/13}$	$\approx 1\,800$
10	$n^{-8/18}$	$\approx$ 32 000

illustrative Werte – entscheidende Konstanten sind problemabhängig

GAMs: Lösung für multivariate Probleme

Problem:

Tensor-Produkt mit K^p Parametern für $p > 3$ unpraktikabel

Lösungsansatz:

Benutze *additive* Struktur statt vollem Tensor-Produkt:

Statt

$$\hat{f}(\mathbf{x}) = \sum_{j_1=1}^K \cdots \sum_{j_p=1}^K a_{j_1, \dots, j_p} \prod_{i=1}^p \phi_{j_i}(x_i) \quad (K^p \text{ Koeffizienten})$$

verwende **Generalisiertes Additives Modell (GAM)**:

$$\hat{f}(\mathbf{x}) = \beta_0 + \sum_{i=1}^p f_i(x_i) \quad (pK + 1 \text{ Koeffizienten})$$

Genauer:

$pK + 1$ Koeffizienten, wegen Identifizierbarkeit (f_i sind nur bis auf additive Konstanten bestimmt) nur $p(K - 1) + 1$ frei wählbar – Größenordnung $O(pK)$.

Aber:

rein additiv – *Interaktionseffekte* (z.B. $f(x_1, x_2) = x_1 \cdot x_2$) werden nicht erfasst; dafür: zusätzliche niedrigdimensionale Tensorprodukt-Terme.

Vorteile:

- *Skalierbar*: $O(pK)$ statt $O(K^p)$ Parameter
- *Interpretierbar*: Effekt jeder Variable einzeln visualisierbar
- **In R**: `mgcv::gam()` mit automatischer Glättungsparameterwahl

Self-Check & Zusammenfassung

Ausblick

Mehr zu

- theoretischer Analyse von statistischen Methoden
- Eigenschaften von Parameterschätzung
- Funktionsapproximation

in Thomas Naglers Vorlesungen:

- *Statistical Learning Theory* (MSc)
- *Mathematical Statistics* (BSc/MSc)

Mehr zu GAMs & co in *Statistik V: Konzepte Statistischer Modellierung* (BSc)

Self-Check

Frage 1: Unter allen Interpolanten $g \in \mathcal{C}^2[a, b]$ der Punkte $(x_i, y_i) \dots$

1. minimiert der natürliche kubische Spline $\int_a^b |g''(x)|^2 dx$.
2. ist der natürliche kubische Spline der *einzige* Interpolant, der $\sum_i (g(x_i) - y_i)^2$ minimiert.
3. hat jeder kubische Spline kleinere Krümmung als jedes Polynom.
4. minimiert die Parabel die Krümmung.

Frage 2: Wir verdoppeln die Anzahl der Basisfunktionen K (bei festem n). Dann gilt (asymptotisch):

1. Bias sinkt, Varianz steigt.
2. Bias steigt, Varianz sinkt.
3. Bias und Varianz sinken beide.
4. Bias und Varianz steigen beide.

Frage 3: Was bedeutet die “optimale Wahl” $K \sim n^{1/9}$?

1. In der Praxis ist $K = \lceil n^{1/9} \rceil$ immer die beste Wahl.
2. Rein asymptotische Ordnung — praktisch wählt man K deutlich größer, da die Konstanten dominieren.
3. K muss linear mit n wachsen.
4. Der MSE fällt dann mit Rate $n^{-1/9}$.

Wrap-up

Heute gelernt

- Eigenschaften kubischer Splines (*Minimale Krümmung* des natürlichen Spline-Interpolanten, Approximationsfehler $O(h^4)$)
- *Bias-Varianz Trade-off* bei der Glättung: $\text{MSE} = O(K^{-8} + K/n)$
- Praktische Modellwahl mit CV, AIC, GCV — oder K groß + Penalisierung
- *Fluch der Dimensionalität* und GAMs als Lösung

Finito!